



RTL Design Sherpa

STREAM Subsystem Micro-Architecture Specification 0.90

January 2, 2026

Table of Contents

1 Stream Index.....	62
2 Document Information.....	62
2.1 References.....	62
2.1.1 Related Documents.....	62
2.2 Terminology.....	63
2.3 Revision History.....	64
3 STREAM Architecture Overview.....	71
3.1 Introduction.....	71
3.1.1 Design Philosophy.....	71
3.2 System Architecture.....	72
3.2.1 High-Level Block Diagram.....	72
3.2.2 Figure 1.1.1: STREAM Architecture Overview.....	72
3.3 Key Architectural Concepts.....	72
3.3.1 1. Multi-Channel Design.....	72
3.3.2 2. Descriptor-Based Operation.....	73
3.3.3 3. Concurrent Read/Write.....	73
3.3.4 4. Space-Aware Flow Control.....	73
3.3.5 5. Priority-Based Arbitration.....	73
3.4 Data Flow Example.....	74
3.4.1 Single Channel Transfer.....	74
3.4.2 Figure 1.1.2: Single Channel Transfer Sequence.....	74
3.5 Component Hierarchy.....	74

3.5.1 Top-Level Integration (stream_top_ch8.sv).....	74
3.5.2 Core DMA Engine (stream_core.sv).....	75
3.6 Interface Summary.....	75
3.6.1 External Interfaces (stream_top_ch8.sv).....	75
3.6.2 Internal MonBus Interface (stream_core.sv).....	76
3.6.3 Configuration Registers (APB).....	76
3.7 Resource Utilization.....	76
3.7.1 Area Breakdown (Typical 512-bit Data Width).....	76
3.8 Performance Characteristics.....	77
3.8.1 Throughput.....	77
3.8.2 Latency.....	77
3.9 Design Decisions.....	77
3.9.1 Why These Simplifications?.....	77
3.9.2 Production Enhancements (Future).....	78
3.10 Testing Strategy.....	78
3.10.1 Verification Layers.....	78
3.10.2 Coverage Targets.....	78
3.11 Related Documentation.....	79
3.12 Revision History.....	79
4 STREAM Top-Level Port List.....	79
4.1 Overview.....	80
4.2 Clock and Reset.....	80
4.3 APB Programming Interface.....	80
4.4 Configuration Interface.....	81

4.4.1 Per-Channel Configuration.....	81
4.4.2 Global Scheduler Configuration.....	82
4.4.3 Descriptor Engine Configuration.....	82
4.4.4 AXI Monitor Configuration.....	83
4.4.5 AXI Transfer Configuration.....	84
4.4.6 Performance Profiler Configuration.....	84
4.5 Status Interface.....	85
4.5.1 Per-Channel Status.....	85
4.6 Performance Profiler Interface.....	86
4.7 AXI4 Master - Descriptor Fetch (256-bit).....	87
4.7.1 AR Channel (Read Address).....	87
4.7.2 R Channel (Read Data).....	88
4.8 AXI4 Master - Data Read (Parameterizable Width).....	88
4.8.1 AR Channel (Read Address).....	88
4.8.2 R Channel (Read Data).....	89
4.9 AXI4 Master - Data Write (Parameterizable Width).....	90
4.9.1 AW Channel (Write Address).....	90
4.9.2 W Channel (Write Data).....	91
4.9.3 B Channel (Write Response).....	91
4.10 Status/Debug Outputs.....	92
4.10.1 Descriptor AXI Monitor Status.....	92
4.10.2 Read Engine AXI Monitor Status.....	92
4.10.3 Write Engine AXI Monitor Status.....	93
4.11 Unified Monitor Bus Interface.....	93

4.12 Port Count Summary.....	94
4.13 Default Parameter Values.....	95
4.14 Related Documentation.....	96
4.15 Revision History.....	96
5 Clocks and Reset Specification.....	96
5.1 Overview.....	97
5.2 Clock Domain.....	97
5.2.1 Primary Clock: aclk.....	97
5.2.2 Secondary Clock: pclk (APB Clock).....	97
5.3 Reset.....	97
5.3.1 Primary Reset: aresetn.....	97
5.3.2 Secondary Reset: presetn.....	98
5.4 Reset Sequencing.....	98
5.4.1 Power-On Reset.....	98
5.4.2 Functional Reset.....	99
5.4.3 Reset Recovery.....	99
5.5 Clock Requirements by Module.....	99
5.5.1 Functional Unit Blocks (FUB).....	99
5.5.2 Integration Blocks (MAC).....	100
5.6 Timing Constraints.....	100
5.6.1 Setup and Hold Times.....	100
5.6.2 Critical Paths.....	100
5.7 Clock Domain Crossing (CDC).....	101
5.7.1 APB Configuration CDC.....	101

5.7.2 No CDC Required.....	102
5.8 Reset State Initialization.....	102
5.8.1 Register Reset Values.....	102
5.8.2 SRAM Reset.....	103
5.9 Clock Gating (Optional).....	103
5.9.1 Per-Channel Clock Gating.....	103
5.10 Verification Requirements.....	104
5.10.1 Clock Checks.....	104
5.10.2 Reset Checks.....	104
5.10.3 CDC Checks.....	104
5.11 Example Reset Testbench.....	104
5.12 Related Documentation.....	105
Top-Level: mac_04_stream_top.md - Clock/reset integration.....	105
5.13 Revision History.....	105
6 STREAM Core Specification.....	106
6.1 Overview.....	106
6.1.1 Key Features.....	106
6.1.2 Block Diagram.....	106
6.1.3 Figure 2.1.1: STREAM Core Block Diagram.....	107
6.2 Architecture.....	109
6.2.1 Component Hierarchy.....	109
6.2.2 Data Flow.....	109
6.3 Parameters.....	110
6.3.1 Primary Configuration.....	110

6.3.2 Monitor Control.....	111
6.3.3 Outstanding Transaction Limits.....	111
6.3.4 AXI Skid Buffer Depths.....	111
6.3.5 MonBus Agent IDs.....	112
6.4 Port List.....	112
6.4.1 Clock and Reset.....	112
6.4.2 APB Programming Interface.....	112
6.4.3 Configuration Interface.....	113
6.4.4 Status Interface.....	116
6.4.5 AXI4 Master - Descriptor Fetch (256-bit).....	117
6.4.6 AXI4 Master - Data Read (Parameterizable Width).....	119
6.4.7 AXI4 Master - Data Write (Parameterizable Width).....	120
6.4.8 Status/Debug Outputs.....	122
6.4.9 Unified Monitor Bus Interface.....	123
6.5 Operation.....	123
6.5.1 Transfer Initialization.....	123
6.5.2 Monitoring Transfer Progress.....	124
6.6 Timing Diagrams.....	124
6.6.1 Complete Data Flow Timing.....	124
6.7 Testing.....	127
6.8 Resource Utilization.....	127
6.9 Integration Example.....	128
6.10 Related Documentation.....	129

MonBus AXI-Lite Group: 16_monbus_axil_group.md - Monitor bus arbitration	130
6.11 Revision History.....	130
7 Scheduler Group Array.....	130
7.1 Overview.....	130
7.1.1 Key Features.....	130
7.1.2 Simplified from RAPIDS.....	131
7.2 Architecture.....	131
7.2.1 Component Hierarchy.....	131
7.2.2 Descriptor AXI Arbitration.....	131
7.2.3 Data Path Interfaces.....	132
7.3 Parameters.....	132
7.3.1 Monitor Bus Agent IDs.....	132
7.4 Port List.....	133
7.4.1 Clock and Reset.....	133
7.4.2 APB Programming Interface (Per-Channel).....	133
7.4.3 Configuration Interface (Per-Channel).....	133
7.4.4 Global Scheduler Configuration.....	134
7.4.5 Descriptor Engine Configuration.....	134
7.4.6 Status Interface (Per-Channel).....	135
7.4.7 Shared Descriptor AXI4 Master Read Interface.....	135
7.4.8 Shared Data Read Interface (Per-Channel Arrays).....	136
7.4.9 Shared Data Write Interface (Per-Channel Arrays).....	137
7.4.10 Unified Monitor Bus Interface.....	137

7.5 Operation.....	138
7.5.1 Descriptor Fetch Arbitration.....	138
7.5.2 Data Path Flow.....	138
7.5.3 MonBus Aggregation.....	138
7.6 Integration Example.....	138
7.7 Related Documentation.....	139
MonBus Arbiter: 16_monbus_axil_group.md - Monitor bus aggregation.....	140
7.8 Revision History.....	140
8 Scheduler Group.....	140
8.1 Overview.....	141
8.1.1 Key Features.....	141
8.1.2 Simplified from RAPIDS.....	141
8.2 Architecture.....	141
8.2.1 Component Hierarchy.....	141
8.2.2 Data Flow.....	141
8.3 Parameters.....	142
8.3.1 Monitor Bus Agent IDs.....	142
8.4 Port List.....	143
8.4.1 Clock and Reset.....	143
8.4.2 APB Programming Interface.....	143
8.4.3 Configuration Interface.....	143
8.4.4 Scheduler Configuration.....	144
8.4.5 Descriptor Engine Configuration.....	144
8.4.6 Status Interface.....	145

8.4.7 Descriptor AXI Interface (to external arbiter).....	145
8.4.8 Data Read Interface (to shared read engine).....	146
8.4.9 Data Write Interface (to shared write engine).....	146
8.4.10 Monitor Bus Interface.....	147
8.5 Operation.....	147
8.5.1 Transfer Sequence.....	147
8.5.2 MonBus Sources.....	148
8.6 Integration Example.....	148
8.7 Related Documentation.....	149
AXI Write Engine: 12_axi_write_engine.md - Shared write engine.....	149
8.8 Revision History.....	149
9 Scheduler Specification.....	150
9.1 Overview.....	150
9.1.1 Key Features.....	150
9.1.2 Block Diagram.....	151
9.1.3 Figure 2.4.1: Scheduler Block Diagram.....	151
9.2 CRITICAL: Concurrent Read/Write Design.....	151
9.3 Read-Ahead Descriptor Prefetch.....	152
9.4 Parameters.....	153
9.5 Port List.....	153
9.5.1 Clock and Reset.....	153
9.5.2 Configuration Interface.....	154
9.5.3 Status Interface.....	154
9.5.4 Descriptor Engine Interface.....	154

9.5.5 Data Read Interface.....	155
9.5.6 Data Write Interface.....	155
9.5.7 Error Signals.....	156
9.5.8 Monitor Bus Interface.....	156
9.6 Interface.....	157
9.6.1 Clock and Reset.....	157
9.6.2 Configuration Interface.....	157
9.6.3 Status Interface.....	157
9.6.4 Descriptor Engine Interface.....	157
9.6.5 Data Read Interface (to AXI Read Engine).....	158
9.6.6 Data Write Interface (to AXI Write Engine).....	158
9.6.7 Monitor Bus Interface.....	159
9.7 Descriptor Format.....	159
9.7.1 STREAM Descriptor (256-bit).....	159
9.8 FSM Operation.....	160
9.8.1 State Machine.....	160
9.8.2 Figure 2.4.2: Scheduler FSM.....	161
9.8.3 State Transitions.....	161
9.9 Beat Tracking.....	162
9.9.1 Independent Counters.....	162
9.9.2 Completion Detection.....	162
9.9.3 Multiple Requests per Descriptor.....	163
9.10 Address Management.....	164
9.10.1 Static Base Address.....	164

9.11 Timeout Detection (Recoverable).....	164
9.11.1 Timeout Counter.....	164
9.11.2 Consecutive-Window Strike Counter and Escalation.....	165
9.12 Error Handling.....	165
9.12.1 Error Sources.....	165
9.12.2 Sticky Error Flags.....	165
9.12.3 Error Recovery.....	166
9.13 Interrupt Generation.....	166
9.13.1 IRQ via MonBus.....	166
9.14 Descriptor Chaining.....	167
9.14.1 Chain Detection.....	167
9.14.2 Chain Termination.....	167
9.15 MonBus Integration.....	168
9.15.1 Event Types.....	168
9.15.2 MonBus Packet Format.....	168
9.16 Timing Diagrams.....	169
9.16.1 Normal Transfer (No Chaining).....	169
9.16.2 Figure 2.4.3: Scheduler Normal Transfer Timing.....	169
9.16.3 Descriptor Chaining.....	170
9.16.4 Figure 2.4.4: Scheduler Descriptor Chaining Timing.....	170
9.16.5 Single-Descriptor Transfer (Detailed).....	170
9.16.6 Chained Descriptors (Detailed).....	171
9.17 Testing.....	171
9.18 Performance Considerations.....	172

9.18.1 Concurrent Operation Benefit.....	172
9.18.2 Example Performance.....	172
9.19 Observation and Diagnostics (v0.92+).....	172
9.19.1 OBS_FLAGS Register Mapping.....	172
9.19.2 Observation Data Fields (OBS_DATA0 / OBS_DATA1).....	173
9.20 Related Documentation.....	173
9.21 Revision History.....	173
10 Descriptor Engine.....	175
10.1 Overview.....	175
10.1.1 Key Features.....	175
10.2 Architecture.....	176
10.2.1 Operating Flow.....	176
10.2.2 FSM States.....	176
10.2.3 Descriptor Format (256-bit STREAM).....	176
10.3 Parameters.....	177
10.3.1 Monitor Bus Parameters.....	178
10.4 Port List.....	178
10.4.1 Clock and Reset.....	178
10.4.2 APB Programming Interface.....	178
10.4.3 Scheduler Interface.....	179
10.4.4 Enhanced Control Outputs.....	179
10.4.5 AXI4 AR Channel.....	179
10.4.6 AXI4 R Channel (FIXED 256-bit).....	180
10.4.7 Configuration Interface.....	181

10.4.8 Status Interface.....	181
10.4.9 Monitor Bus Interface.....	181
10.5 Operation.....	182
10.5.1 APB Acceptance Policy.....	182
10.5.2 Autonomous Chaining Decision Tree.....	182
10.5.3 AXI Transaction.....	183
10.6 Timing Diagrams.....	183
10.6.1 APB Kick-off Sequence.....	183
10.7 Integration Example.....	185
10.8 Common Issues.....	186
10.8.1 Issue 1: APB Write Not Accepted.....	186
10.8.2 Issue 2: Descriptor Chain Stops Early.....	186
10.9 Monitoring (BOTH-End Descriptor Path Observability).....	186
10.9.1 Descriptor Fetch Monitoring (Master Side).....	186
10.9.2 Descriptor Consume Monitoring (Slave Side).....	186
10.9.3 Why Both Ends?.....	187
10.10 Related Documentation.....	187
APB Interface: 13_apbtodescr.md - APB to descriptor kick-off.....	187
10.11 Revision History.....	187
11 AXI Read Engine.....	188
11.1 Overview.....	188
11.1.1 Key Features.....	188
11.2 Architecture.....	189
11.2.1 Operation Flow.....	189

11.2.2 Key Design Decisions.....	189
11.2.3 System Idle Behavior.....	189
11.3 Parameters.....	189
11.3.1 Derived Parameters.....	190
11.4 Port List.....	190
11.4.1 Clock and Reset.....	190
11.4.2 Configuration Interface.....	190
11.4.3 Scheduler Interface (Per-Channel).....	191
11.4.4 Completion Interface (Per-Channel).....	191
11.4.5 SRAM Allocation Interface.....	191
11.4.6 SRAM Write Interface.....	192
11.4.7 AXI4 AR Channel.....	192
11.4.8 AXI4 R Channel.....	192
11.4.9 Error Interface.....	193
11.4.10 Debug Interface.....	193
11.5 Operation.....	193
11.5.1 Space-Aware Request Masking.....	193
11.5.2 Transfer Size Calculation.....	194
11.5.3 Outstanding Transaction Tracking.....	194
11.5.4 Completion Strobe.....	194
11.6 Timing Diagrams.....	194
11.6.1 Perfect Streaming - AXI Read Transaction.....	194
11.6.2 Multi-Channel Streaming.....	196
11.7 Integration Example.....	196

11.8 Bug Fixes (v0.92).....	198
11.8.1 w_arb_request Timing Closure (Commit 4e8f9e02).....	198
11.9 Related Documentation.....	198
Write Engine: 12_axi_write_engine.md - Complementary write datapath.....	198
11.10 Revision History.....	198
12 Stream Allocation Controller.....	199
12.1 Overview.....	199
12.1.1 What Makes This a “Virtual FIFO”.....	199
12.2 The Allocation Problem.....	199
12.2.1 Without Allocation Controller.....	199
12.2.2 With Allocation Controller.....	200
12.3 Architecture.....	201
12.3.1 Two-Pointer System.....	201
12.3.2 Figure 2.7.1: Stream Allocation Controller Block Diagram.....	201
12.3.3 CRITICAL: Confusing Naming Convention.....	203
12.4 Parameters.....	203
12.5 Port List.....	204
12.5.1 Clock and Reset.....	204
12.5.2 Write Interface (Allocation Requests).....	204
12.5.3 Read Interface (Actual Data Written).....	205
12.5.4 Status Outputs.....	205
12.6 Interfaces.....	205
12.6.1 Write Interface (Allocation Requests).....	205
12.6.2 Read Interface (Actual Data Written).....	206

12.6.3 Status Outputs.....	206
12.7 Operation.....	207
12.7.1 Allocation Flow.....	207
12.7.2 Pointer Arithmetic.....	207
12.8 Timing Behavior.....	208
12.8.1 Allocation Latency.....	208
12.8.2 Release Latency.....	208
12.9 Integration Example.....	209
12.9.1 In sram_controller_unit.sv.....	209
12.10 Debug Support.....	209
12.10.1 Display Statements.....	209
12.10.2 Waveform Analysis.....	210
12.11 Common Issues.....	210
12.11.1 Issue 1: Space Not Released.....	210
12.11.2 Issue 2: Overflow Despite Allocation.....	210
12.11.3 Issue 3: Allocation Pointer Overflow.....	211
12.12 Comparison with Drain Controller.....	211
12.13 Resource Utilization.....	211
12.14 Related Modules.....	212
12.15 Related Documentation.....	212
AXI Read Engine: 06_axi_read_engine.md.....	212
12.16 Revision History.....	212
13 SRAM Controller Specification.....	212
13.1 Overview.....	213

13.1.1 Key Features.....	213
13.1.2 Design Rationale.....	213
13.2 Architecture.....	214
13.2.1 Block Diagram.....	214
13.2.2 Figure 2.8.1: SRAM Controller Block Diagram.....	214
13.2.3 Per-Channel Architecture.....	214
13.3 Parameters.....	214
13.4 Port List.....	215
13.4.1 Clock and Reset.....	215
13.4.2 Allocation Interface.....	215
13.4.3 Write Interface.....	216
13.4.4 Drain Interface.....	216
13.4.5 Read Interface.....	216
13.4.6 Debug Interface.....	217
13.5 Interface.....	217
13.5.1 Clock and Reset.....	217
13.5.2 Allocation Interface (AXI Read Engine Flow Control).....	217
13.5.3 Write Interface (AXI Read Engine → FIFO).....	218
13.5.4 Drain Interface (AXI Write Engine Flow Control).....	218
13.5.5 Read Interface (FIFO → AXI Write Engine).....	219
13.5.6 Debug Interface.....	219
13.6 ID Decode Logic.....	220
13.6.1 Write Valid Decode.....	220
13.6.2 Read/Drain Decode.....	220

13.6.3 Allocation Decode.....	221
13.7 Per-Channel Unit.....	221
13.8 Operation Flows.....	222
13.8.1 Write Flow (AXI Read Data → FIFO).....	222
13.8.2 Read Flow (FIFO → AXI Write Data).....	223
13.9 Allocation vs. Drain Controllers.....	223
13.9.1 Why Separate Controllers?.....	223
13.9.2 Flow Control Comparison.....	224
13.10 Timing Diagrams.....	224
13.10.1 Combined Write and Read Timing.....	224
13.10.2 Write Path (R Data → FIFO).....	225
13.10.3 Read Path (FIFO → W Data).....	226
13.11 Error Conditions.....	226
13.11.1 Invalid Channel ID.....	226
13.11.2 FIFO Overflow.....	226
13.11.3 FIFO Underflow.....	227
13.12 Performance Considerations.....	227
13.12.1 Per-Channel FIFO Benefits.....	227
13.12.2 SRAM Resource Usage.....	227
13.13 Testing.....	227
13.14 Bug Fixes (v0.92).....	228
13.14.1 Registered Avail Outputs Reset (Commit b619eee9).....	228
13.15 Related Documentation.....	228
13.16 Revision History.....	228

14 SRAM Controller Unit.....	229
14.1 Overview.....	229
14.1.1 Key Features.....	230
14.2 Architecture.....	230
14.2.1 Block Diagram.....	230
14.2.2 Figure 1: SRAM Controller Unit Block Diagram.....	230
14.2.3 Component Hierarchy.....	230
14.2.4 Data Flow.....	230
14.2.5 Controller Naming Convention (CRITICAL).....	231
14.3 Parameters.....	231
14.3.1 Derived Parameters.....	231
14.4 Port List.....	231
14.4.1 Clock and Reset.....	231
14.4.2 Allocation Interface (Read Engine Flow Control).....	232
14.4.3 Write Interface (AXI Read Engine to FIFO).....	232
14.4.4 Drain Interface (Write Engine Flow Control).....	232
14.4.5 Read Interface (FIFO to AXI Write Engine).....	233
14.4.6 Debug Interface.....	233
14.5 Operation.....	233
14.5.1 Allocation Flow.....	233
14.5.2 Drain Flow.....	233
14.5.3 Data Available Calculation.....	234
14.6 Integration Example.....	234
14.7 Common Issues.....	235

14.7.1 Issue 1: Space Accounting Mismatch.....	235
14.7.2 Issue 2: Data Available Undercount.....	235
14.8 Related Documentation.....	235
Write Engine: 12_axi_write_engine.md - Data consumer.....	235
14.9 Revision History.....	235
15 Stream Latency Bridge.....	236
15.1 Overview.....	236
15.1.1 Key Features.....	236
15.2 Architecture.....	237
15.2.1 Block Diagram.....	237
15.2.2 Figure 2.10.1: Stream Latency Bridge Block Diagram.....	237
15.2.3 Operation Flow.....	237
15.2.4 Component Diagram.....	237
15.2.5 Backpressure Logic.....	238
15.3 Parameters.....	239
15.3.1 Derived Parameters.....	239
15.4 Port List.....	239
15.4.1 Clock and Reset.....	239
15.4.2 Upstream Interface (from registered FIFO).....	239
15.4.3 Downstream Interface (to consumer).....	240
15.4.4 Status Interface.....	240
15.4.5 Debug Interface.....	240
15.5 Operation.....	240
15.5.1 Glue Logic.....	240

15.5.2 Skid Buffer.....	241
15.5.3 Occupancy Calculation.....	241
15.6 Timing Diagrams.....	241
15.6.1 Backpressure Handling.....	241
15.7 Integration Example.....	242
15.8 Design Rationale.....	243
15.8.1 Why Use a Skid Buffer?.....	243
15.8.2 Why 4-Deep?.....	243
15.9 Related Documentation.....	243
Producer: 06_axi_read_engine.md - Feeds upstream FIFO.....	244
15.10 Revision History.....	244
16 Stream Drain Controller.....	244
16.1 Overview.....	244
16.1.1 Key Features.....	244
16.2 Architecture.....	245
16.2.1 Block Diagram.....	245
16.2.2 Figure 1: Stream Drain Controller Block Diagram.....	245
16.2.3 Virtual FIFO Concept.....	245
16.2.4 Pointer Operations.....	245
16.3 Parameters.....	246
16.3.1 Derived Parameters.....	246
16.4 Port List.....	246
16.4.1 Clock and Reset.....	246
16.4.2 Write Interface (Data Entry).....	246

16.4.3 Read Interface (Drain Requests).....	247
16.4.4 Status Outputs.....	247
16.5 Operation.....	247
16.5.1 Data Entry (Write Side).....	247
16.5.2 Drain Requests (Read Side).....	248
16.5.3 Status Generation.....	248
16.6 Comparison with stream_alloc_ctrl.....	249
16.7 Integration Example.....	249
16.8 Common Issues.....	250
16.8.1 Issue 1: Data Available Undercount.....	250
16.8.2 Issue 2: Drain Request Overflow.....	250
16.9 Bug Fixes (v0.92).....	250
16.9.1 Stale-View Race Closure (Commit a82627af).....	250
16.10 Related Documentation.....	250
Control Block: See fifo_control in rtl/amba/gaxi/.....	251
16.11 Revision History.....	251
17 AXI Write Engine.....	251
17.1 Overview.....	251
17.1.1 Key Features.....	251
17.2 Architecture.....	252
17.2.1 Block Diagram.....	252
17.2.2 Figure 2.12.1: AXI Write Engine Block Diagram.....	252
17.2.3 Operation Flow.....	252
17.2.4 Key Design Decisions.....	253

17.3 Parameters.....	253
17.3.1 Derived Parameters.....	254
17.4 Port List.....	254
17.4.1 Clock and Reset.....	254
17.4.2 Configuration Interface.....	254
17.4.3 Scheduler Interface (Per-Channel).....	255
17.4.4 Completion Interface (Per-Channel).....	255
17.4.5 SRAM Drain Interface.....	255
17.4.6 SRAM Read Interface.....	255
17.4.7 AXI4 AW Channel.....	256
17.4.8 AXI4 W Channel.....	256
17.4.9 AXI4 B Channel.....	257
17.4.10 Error Interface.....	257
17.4.11 Debug Interface.....	257
17.5 Operation.....	258
17.5.1 Data-Aware Request Masking.....	258
17.5.2 W-Phase Transaction FIFO.....	258
17.5.3 B-Phase Transaction FIFOs.....	258
17.5.4 Outstanding Transaction Tracking.....	259
17.6 Timing Diagrams.....	259
17.6.1 Perfect Streaming - AXI Write Transaction.....	259
17.6.2 Multi-Channel Streaming.....	260
17.7 Integration Example.....	261
17.8 Bug Fixes (v0.92).....	263

17.8.1 w_arb_request Timing Closure (Commit 4e8f9e02).....	263
17.8.2 W-Side Post-Flop Gating on wvalid (Commit a82627af).....	263
17.9 Related Documentation.....	263
Read Engine: 06_axi_read_engine.md - Complementary read datapath.....	263
17.10 Revision History.....	263
18 APB to Descriptor Router.....	264
18.1 Overview.....	264
18.1.1 Key Features.....	264
18.2 Architecture.....	265
18.2.1 Address Map.....	265
18.2.2 Block Diagram.....	265
18.2.3 Figure 2.13.1: APB to Descriptor Block Diagram.....	265
18.2.4 FSM Diagram.....	266
18.2.5 Figure 2.13.2: APB to Descriptor FSM.....	266
18.2.6 Write Flow.....	266
18.2.7 FSM States.....	267
18.3 Parameters.....	267
18.4 Port List.....	268
18.4.1 Clock and Reset.....	268
18.4.2 APB Slave CMD Interface.....	268
18.4.3 APB Slave RSP Interface.....	268
18.4.4 Descriptor Engine APB Ports.....	269
18.4.5 Integration Control.....	269
18.5 Operation.....	269

18.5.1 Address Decode.....	269
18.5.2 64-bit Address Assembly.....	269
18.5.3 Error Conditions.....	270
18.6 Timing Diagrams.....	270
18.6.1 Normal Write Sequence.....	270
18.6.2 Backpressure Handling.....	271
18.6.3 Channel Kick-off Examples.....	273
18.6.4 Multi-Channel Operation.....	274
18.7 Known Issues and Fixes.....	275
18.7.1 Issue: Multi-Channel Kick-off Address Routing (RESOLVED a3beb2eb)	275
18.8 Integration Example.....	275
18.9 Common Issues.....	276
18.9.1 Issue 1: APB Response Not Returning.....	276
18.9.2 Issue 2: Wrong Channel Kicked Off.....	276
18.10 Related Documentation.....	277
Register File: PeakRDL-generated stream_regs.sv.....	277
18.11 Revision History.....	277
19 APB Configuration Block.....	277
19.1 Overview.....	277
19.1.1 Key Features.....	278
19.2 Architecture.....	278
19.2.1 Block Diagram.....	278
19.2.2 Figure 1: Stream Config Block Diagram.....	278

19.2.3 Configuration Groups.....	279
19.3 Parameters.....	279
19.4 Port List.....	280
19.4.1 Clock and Reset.....	280
19.4.2 PeakRDL Register Inputs.....	280
19.4.3 Configuration Outputs.....	281
19.5 Operation.....	283
19.5.1 Global Enable Gating.....	283
19.5.2 Global Reset OR.....	283
19.5.3 Address Zero Extension.....	283
19.6 Integration Example.....	284
19.7 Related Documentation.....	284
Monitors: Monitor documentation for each AXI monitor.....	285
19.8 Revision History.....	285
20 Performance Profiler.....	285
20.1 Overview.....	285
20.1.1 Key Features.....	285
20.2 Architecture.....	286
20.2.1 Block Diagram.....	286
20.2.2 Figure 2.15.1: Performance Profiler Block Diagram.....	286
20.2.3 Profiling Modes.....	286
20.2.4 FIFO Entry Format (36-bit).....	287
20.3 Parameters.....	287
20.4 Port List.....	288

20.4.1 Clock and Reset.....	288
20.4.2 Channel Monitoring.....	288
20.4.3 Configuration Interface.....	288
20.4.4 FIFO Read Interface.....	289
20.5 Operation.....	289
20.5.1 Edge Detection.....	289
20.5.2 Timestamp Counter.....	289
20.5.3 Start Time Capture (Elapsed Mode).....	290
20.5.4 Two-Register Read Sequence.....	290
20.6 Integration Example.....	290
20.7 Software Usage.....	291
20.7.1 Example: Measure Channel 0 Transfer Time.....	291
20.8 Common Issues.....	292
20.8.1 Issue 1: FIFO Overflow.....	292
20.8.2 Issue 2: Timestamp Rollover.....	292
20.9 Related Documentation.....	292
Configuration: 14_apb_config.md - Config register mapping.....	292
20.10 Revision History.....	292
21 MonBus AXI-Lite Group.....	293
21.1 Overview.....	293
21.1.1 Key Features.....	293
21.1.2 Simplified from RAPIDS.....	294
21.2 Architecture.....	295
21.2.1 Block Diagram.....	295

21.2.2 Figure 2.16.1: MonBus AXI-Lite Group Block Diagram.....	295
21.2.3 Filter Decision Tree.....	296
21.3 Parameters.....	296
21.4 Port List.....	296
21.4.1 Clock and Reset.....	296
21.4.2 Monitor Bus Input.....	297
21.4.3 AXI-Lite Slave Read Interface.....	297
21.4.4 AXI-Lite Master Write Interface.....	297
21.4.5 Interrupt Output.....	298
21.4.6 Configuration Interface.....	298
21.4.7 Protocol Configuration (per protocol).....	299
21.4.8 Debug/Status.....	300
21.5 Operation.....	300
21.5.1 Packet Type Filtering.....	300
21.5.2 Master Write Address Generation.....	300
21.5.3 Master Write FSM.....	301
21.6 Integration Example.....	301
21.7 Common Issues.....	302
21.7.1 Issue 1: Missing Monitor Packets.....	302
21.7.2 Issue 2: Master Writes Stall.....	302
21.8 Related Documentation.....	302
Configuration: 14_apb_config.md - Register mapping.....	302
21.9 Revision History.....	302
22 Chapter 3: External Interfaces.....	303

22.1 Interface Specifications.....	303
22.1.1 01_axi4_interface_spec.md.....	303
22.1.2 02_axil4_interface_spec.md.....	303
22.1.3 03_apb_interface_spec.md.....	303
22.1.4 05_monbus_interface_spec.md.....	304
22.2 STREAM Interface Summary (stream_top_ch8.sv).....	304
23 AXI4 Interface Specification for STREAM.....	305
23.1 Overview.....	305
23.2 STREAM AXI4 Interface Summary.....	305
23.2.1 Number of Interfaces.....	305
23.2.2 Interface Parameters.....	306
23.2.3 Interface Types and Transfer Modes.....	306
23.2.4 Interface Group Parameter Settings.....	307
23.2.5 Interface Configuration Summary.....	308
23.3 Transfer Mode Specifications.....	309
23.3.1 Mode 1: Simplified Transfer Mode (Control Interfaces).....	309
23.3.2 Mode 2: Flexible Transfer Mode (Data Interfaces).....	309
23.4 Mode 1: Simplified Transfer Mode Specification.....	310
23.4.1 Assumption 1: Address Alignment to Data Bus Width.....	310
23.4.2 Assumption 2: Fixed Transfer Size.....	310
23.5 Mode 2: Flexible Transfer Mode Specification.....	311
23.5.1 Assumption 1: 4-Byte Address Alignment.....	311
23.5.2 Assumption 2: Multiple Transfer Sizes.....	311
23.5.3 Assumption 3: Progressive Alignment Strategy.....	312

23.5.4 Assumption 4: Chunk Enable Support.....	312
23.6 Common Protocol Assumptions (Both Modes).....	312
23.6.1 Assumption 1: Incrementing Bursts Only.....	312
23.6.2 Assumption 2: No Address Wraparound.....	313
23.7 Flexible Mode: Address Calculation Examples.....	313
23.7.1 Progressive Alignment Examples.....	313
23.7.2 Chunk Enable Pattern Examples.....	314
23.8 Master Read Interface Specification.....	315
23.8.1 Read Address Channel (AR).....	315
23.8.2 Read Data Channel (R).....	316
23.9 Master Write Interface Specification.....	316
23.9.1 Write Address Channel (AW).....	316
23.9.2 Write Data Channel (W).....	317
23.9.3 Write Response Channel (B).....	318
23.10 Address Calculation Rules.....	318
23.10.1 Simplified Mode Address Generation.....	318
23.10.2 Flexible Mode Address Generation.....	318
23.10.3 4KB Boundary Considerations (Both Modes).....	319
23.11 Write Strobe Generation.....	319
23.11.1 Simplified Mode Strobe Generation.....	319
23.11.2 Flexible Mode Strobe Generation.....	319
23.12 Response Codes.....	320
23.12.1 Response Code Specification.....	320
23.13 Implementation Benefits.....	321

23.13.1 Simplified Mode Benefits.....	321
23.13.2 Flexible Mode Benefits.....	321
23.13.3 Mode Selection Guidelines.....	321
23.14 Validation Requirements.....	322
23.14.1 Simplified Mode Validation.....	322
23.14.2 Flexible Mode Validation.....	322
23.14.3 Common Validation.....	323
23.15 Performance Characteristics.....	323
23.15.1 Simplified Mode Performance.....	323
23.15.2 Flexible Mode Performance.....	324
23.15.3 Performance Optimization Strategy.....	324
23.16 This dual-mode approach provides the best of both worlds: simplified, predictable operation for control interfaces and flexible, high-performance operation for data interfaces.....	324
23.17 Revision History.....	324
24 AXI4-Lite Interface Specification and Assumptions.....	325
24.1 Overview.....	325
24.2 Interface Summary.....	325
24.2.1 Number of Interfaces.....	325
24.2.2 Interface Parameters.....	325
24.3 Core Protocol Assumptions.....	325
24.3.1 Inherent AXI4-Lite Simplifications.....	325
24.3.2 Implementation Assumptions.....	326
24.4 Master Read Interface Specification.....	327

24.4.1 Read Address Channel (AR).....	327
24.4.2 Read Data Channel (R).....	328
24.4.3 AXI4-Lite Simplifications (Read).....	328
24.5 Master Write Interface Specification.....	329
24.5.1 Write Address Channel (AW).....	329
24.5.2 Write Data Channel (W).....	329
24.5.3 Write Response Channel (B).....	329
24.5.4 AXI4-Lite Simplifications (Write).....	330
24.6 Address Requirements.....	330
24.6.1 Address Alignment Rules.....	330
24.6.2 Address Validation Examples.....	331
24.7 Response Codes.....	331
24.7.1 Response Code Specification.....	331
24.7.2 Response Usage Guidelines.....	331
24.8 Protection Signal Usage.....	332
24.8.1 Protection Signal Encoding.....	332
24.8.2 Common Protection Patterns.....	332
24.9 Implementation Benefits.....	332
24.9.1 Simplified Control Register Interface.....	332
24.9.2 Address Decode Implementation.....	333
24.9.3 Error Generation Logic.....	333
24.10 Timing Requirements.....	334
24.10.1 Handshake Protocol.....	334
24.10.2 Channel Dependencies.....	334

24.10.3 Reset Behavior.....	334
24.11 Validation Requirements.....	335
24.11.1 Functional Validation.....	335
24.11.2 Timing Validation.....	335
24.11.3 Error Injection Testing.....	336
24.12 Example Transactions.....	336
24.12.1 64-bit Register Write.....	336
24.12.2 AW Transaction Flow.....	336
24.12.3 64-bit Register Read.....	337
24.12.4 AR Transaction Flow.....	337
24.12.5 Misaligned Address Example.....	337
24.13 Common Use Cases.....	338
24.13.1 Typical Applications.....	338
24.13.2 Performance Considerations.....	338
24.14 Revision History.....	339
25 APB Programming Interface Specification for STREAM.....	339
25.1 Overview.....	339
25.2 Interface Summary.....	339
25.2.1 STREAM Programming Interface.....	339
25.2.2 Interface Parameters.....	340
25.3 Core Protocol Assumptions.....	340
25.3.1 Inherent APB Simplifications.....	340
25.3.2 Implementation Assumptions.....	341
25.4 Interface Signal Specification.....	342

25.4.1 Clock and Reset Signals.....	342
25.4.2 Data Signals.....	343
25.4.3 Response Signals.....	343
25.5 Address Space Configuration.....	343
25.5.1 Address Range Specification.....	343
25.5.2 Address Decode Implementation.....	344
25.6 Transaction Protocol.....	344
25.6.1 2-Phase Transaction States.....	344
25.6.2 State Transitions.....	344
25.6.3 Transaction Timing Requirements.....	345
25.7 Advanced Features.....	345
25.7.1 Dynamic Clock Gating.....	345
25.7.2 Clock Domain Crossing (CDC) Handshaking.....	345
25.7.3 Buffering and Flow Control.....	346
25.8 Error Handling.....	346
25.8.1 Error Response Specification.....	346
25.8.2 Error Response Timing.....	347
25.9 Reset Behavior.....	347
25.9.1 Reset Requirements.....	347
25.10 Implementation Variants.....	347
25.10.1 Available Implementations.....	347
25.11 Performance Characteristics.....	348
25.11.1 Latency Specifications.....	348
25.11.2 Throughput Specifications.....	348

25.11.3 Power Consumption.....	348
25.12 Validation Requirements.....	349
25.12.1 Functional Validation.....	349
25.12.2 Timing Validation.....	349
25.13 Example Transactions.....	350
25.13.1 32-bit Register Write.....	350
25.13.2 32-bit Register Read with Wait State.....	350
25.14 Revision History.....	350
26 Monitor Bus Architecture and Event Code Organization.....	351
26.1 STREAM-Specific Context.....	351
26.2 Overview.....	351
26.3 Interface Summary.....	352
26.3.1 Number of Interfaces.....	352
26.3.2 Interface Parameters.....	352
26.4 Core Design Assumptions.....	353
26.4.1 Assumption 1: Hierarchical Event Organization.....	353
26.4.2 Assumption 2: Protocol Isolation.....	353
26.4.3 Assumption 3: Two-Tier Memory Architecture.....	353
26.4.4 Assumption 4: Configurable Packet Routing.....	354
26.5 Interface Signal Specification.....	354
26.5.1 Monitor Bus Output Interface.....	354
26.5.2 Protocol Input Interfaces.....	354
26.5.3 Control and Status Signals.....	355
26.6 Packet Format and Field Allocation.....	356

26.6.1 128-bit Monitor Bus Packet Structure.....	356
26.6.2 64-bit Side-Band Timestamp.....	357
26.6.3 Packet Type Definitions.....	357
26.7 Protocol-Specific Event Codes.....	358
26.7.1 AXI Protocol Events.....	358
26.7.2 APB Protocol Events.....	360
26.7.3 MNOC Protocol Events.....	364
26.7.4 ARB Protocol Events.....	367
26.7.5 CORE Protocol Events.....	371
26.8 Memory Architecture and Packet Routing.....	376
26.8.1 Two-Tier Memory Architecture.....	376
26.8.2 Routing Configuration.....	377
26.8.3 Address Space Management.....	378
26.8.4 Transaction State and Bus Transaction Structure.....	378
26.8.5 APB Transaction Phases.....	381
26.8.6 APB Protection Types.....	381
26.8.7 MNOC Payload Types.....	381
26.8.8 MNOC ACK Types.....	382
26.8.9 ARB State Types.....	382
26.8.10 CORE State Types.....	382
26.9 Configuration and Control.....	383
26.9.1 Monitor Configuration Registers.....	383
26.9.2 Protocol-Specific Configuration.....	384
26.10 Validation Requirements.....	386

26.10.1 Functional Validation.....	386
26.10.2 Performance Validation.....	387
26.10.3 Error Handling Validation.....	387
26.11 Usage Examples.....	388
26.11.1 Creating Monitor Packets.....	388
26.11.2 Packet Decoding.....	388
26.11.3 Monitor Bus Packet Helper Functions.....	388
26.12 Debug and Monitoring Signals.....	392
26.12.1 Essential Debug Signals.....	392
26.12.2 Performance Counters.....	392
26.13 Protocol Coverage Summary.....	392
26.13.1 Complete Protocol Event Matrix.....	392
26.14 Total Event Codes: 544 defined across all protocols and packet types....	393
26.15 Revision History.....	393
27 STREAM Register Map.....	394
27.1 Overview.....	394
27.2 Address Space Layout.....	394
27.3 Register Details.....	395
27.3.1 Channel Kick-off Registers (0x000 - 0x03F).....	395
27.3.2 Global Control and Status (0x100 - 0x11F).....	397
27.3.3 Per-Channel Control and Status (0x120 - 0x17F).....	398
27.3.4 Engine Completion and Error Status (0x170 - 0x17F).....	400
27.3.5 Monitor FIFO Status (0x1000 - 0x1007).....	401
27.3.6 Scheduler Configuration (0x200 - 0x21F).....	402

27.3.7 Descriptor Engine Configuration (0x220 - 0x23F).....	404
27.3.8 Descriptor AXI Monitor Configuration (0x10C0 - 0x10DF).....	405
27.3.9 Read Engine AXI Monitor Configuration (0x10E0 - 0x10FF).....	407
27.3.10 Write Engine AXI Monitor Configuration (0x1100 - 0x111F).....	410
27.3.11 AXI Transfer Configuration (0x2A0 - 0x2AF).....	412
27.3.12 Performance Profiler Configuration (0x2B0 - 0x2BF).....	413
27.4 Typical Usage Flow.....	413
27.4.1 Initialization.....	413
27.4.2 Start Transfer.....	414
27.4.3 Poll for Completion.....	414
27.4.4 Error Handling.....	414
27.5 Register Summary Table.....	415
27.6 PeakRDL Generation.....	415
27.6.1 Channel Observation Registers (0x2C0 - 0x2CF).....	416
28 Chapter 5: Programming Models.....	417
28.1 Contents.....	417
28.1.1 01_initialization.md.....	417
28.1.2 02_single_transfer.md.....	417
28.1.3 03_chained_transfers.md.....	418
28.1.4 04_multi_channel.md.....	418
28.1.5 05_error_handling.md.....	418
28.2 Planned (Future).....	418
28.2.1 06_performance_tuning.md.....	418
28.2.2 07_software_examples.md.....	418

29 Chapter 6: Configuration Reference.....	419
29.1 Overview.....	419
29.1.1 Configuration Categories.....	419
29.2 1. Channel Control Configuration.....	420
29.2.1 cfg_channel_enable[NUM_CHANNELS-1:0].....	420
29.2.2 cfg_channel_reset[NUM_CHANNELS-1:0].....	420
29.3 2. Scheduler Configuration.....	421
29.3.1 cfg_sched_timeout_cycles[15:0].....	421
29.3.2 cfg_sched_enable.....	422
29.3.3 cfg_sched_timeout_enable.....	422
29.3.4 cfg_sched_err_enable.....	422
29.3.5 cfg_sched_compl_enable.....	422
29.3.6 cfg_sched_perf_enable.....	422
29.3.7 cfg_rd_prefetch_enable.....	423
29.4 3. Descriptor Engine Configuration.....	423
29.4.1 cfg_desceng_enable.....	423
29.4.2 cfg_desceng_prefetch.....	423
29.4.3 cfg_desceng_fifo_thresh[3:0].....	423
29.4.4 cfg_desceng_addr0_base[31:0].....	424
29.4.5 cfg_desceng_addr0_limit[31:0].....	424
29.4.6 cfg_desceng_addr1_base[31:0].....	424
29.4.7 cfg_desceng_addr1_limit[31:0].....	424
29.5 4. AXI Monitor Configuration.....	424
29.5.1 4.1 Descriptor AXI Monitor (cfg_desc_mon_*).	425

29.5.2 4.2 Read Engine Monitor (cfg_rdeng_mon_*).	428
29.5.3 4.3 Write Engine Monitor (cfg_wreng_mon_*).	428
29.6 5. AXI Transfer Configuration.	429
29.6.1 cfg_axi_rd_xfer_beats[7:0].	429
29.6.2 cfg_axi_wr_xfer_beats[7:0].	429
29.7 6. Performance Profiler Configuration.	429
29.7.1 cfg_perf_enable.	429
29.7.2 cfg_perf_mode.	430
29.7.3 cfg_perf_clear.	430
29.8 7. Configuration Presets.	430
29.8.1 7.1 Minimal Configuration (Tutorial/Embedded).	430
29.8.2 7.2 Balanced Configuration (Typical FPGA).	431
29.8.3 7.3 High-Performance Configuration (ASIC/Datacenter).	432
29.8.4 7.4 Debug Configuration (Verbose Monitoring).	433
29.9 8. Configuration Best Practices.	434
29.9.1 8.1 Monitor Configuration Guidelines.	434
29.9.2 8.2 Timeout Configuration.	434
29.9.3 8.3 Prefetch Configuration.	435
29.9.4 8.4 Burst Size Selection.	435
29.10 9. Configuration Register Map Summary.	436
29.11 10. Software Configuration Examples.	437
29.11.1 10.1 C/C++ Initialization (Minimal).	437
29.11.2 10.2 C/C++ Initialization (Balanced).	438
29.12 11. Troubleshooting Configuration Issues.	438

29.12.1 Problem: No transfers occurring.....	438
29.12.2 Problem: Timeout errors.....	439
29.12.3 Problem: MonBus overflow.....	439
29.12.4 Problem: Low throughput.....	439
29.13 Related Documentation.....	439
30 Product Requirements Document (PRD).....	440
30.1 STREAM - Scatter-gather Transfer Rapid Engine for AXI Memory.....	440
30.2 1. Executive Summary.....	440
30.2.1 1.1 Quick Stats.....	440
30.2.2 1.2 Project Goals.....	440
30.3 2. Key Design Principles.....	440
30.3.1 2.1 Simplifications from RAPIDS.....	440
30.3.2 2.2 Tutorial-Friendly Features.....	441
30.4 3. Architecture Overview.....	441
30.4.1 3.1 Top-Level Block Diagram.....	441
30.4.2 3.2 Data Flow.....	442
30.5 4. Interfaces.....	442
30.5.1 4.1 External Interfaces.....	442
30.5.2 4.2 Descriptor Format.....	443
30.6 5. Key Components.....	444
30.6.1 5.1 Descriptor Engine (APB-Only for STREAM).....	444
30.6.2 5.2 Scheduler Group (Integration Wrapper).....	444
30.6.3 5.3 Scheduler (Simplified from RAPIDS).....	446
30.6.4 5.3 AXI Read Engine (Streaming Pipeline - NO FSM).....	446

30.6.5 5.4 AXI Write Engine (Streaming Pipeline - NO FSM).....	447
30.6.6 5.5 Simple SRAM.....	448
30.7 6. Configuration and Control.....	449
30.7.1 6.1 APB Register Map.....	449
30.7.2 6.2 Channel Configuration.....	449
30.8 7. Resource Sharing and Arbitration.....	449
30.8.1 7.1 Shared Resources.....	449
30.8.2 7.2 Arbitration Strategy.....	450
30.9 8. Error Detection and Recovery.....	450
30.9.1 8.1 Error Types.....	450
30.9.2 8.2 Error Recovery.....	450
30.10 9. MonBus Integration.....	451
30.10.1 9.1 Standard MonBus Format.....	451
30.10.2 9.2 STREAM Event Codes.....	451
30.10.3 9.3 Default Configuration.....	451
30.11 10. Design Constraints.....	451
30.11.1 10.1 Tutorial Constraints (Intentional Simplifications).....	451
30.11.2 10.2 Implementation Constraints.....	452
30.12 11. Verification Strategy.....	452
30.12.1 11.1 Test Organization.....	452
30.12.2 11.2 Test Levels.....	453
30.13 12. Performance Characteristics.....	453
30.13.1 12.1 Throughput by Engine Version.....	453
30.13.2 12.2 Latency.....	453

30.13.3 12.3 Resource Utilization by Engine Version.....	454
30.14 13. Development Roadmap.....	454
30.14.1 13.1 Phase 1: Foundation (Current).....	454
30.14.2 13.2 Phase 2: Core Blocks.....	454
30.14.3 13.3 Phase 3: Data Path.....	454
30.14.4 13.4 Phase 4: Integration.....	455
30.14.5 13.5 Phase 5: Multi-Channel.....	455
30.14.6 13.6 Phase 6: Advanced Engines (Future - V2/V3).....	455
30.15 14. Educational Value.....	455
30.15.1 14.1 Learning Objectives.....	455
30.15.2 14.2 Progression Path.....	456
30.16 15. Success Criteria.....	456
30.16.1 15.1 Functional.....	456
30.16.2 15.2 Quality.....	456
30.16.3 15.3 Performance.....	457
30.17 16. Open Questions (For Review).....	457
30.17.1 16.1 Descriptor Engine Adaptation.....	457
30.17.2 16.2 AXI Descriptor Master.....	457
30.17.3 16.3 Channel Arbitration.....	457
30.17.4 16.4 SRAM Partitioning.....	457
30.18 16. Attribution and Contribution Guidelines.....	457
30.18.1 16.1 Git Commit Attribution.....	457
30.19 16.2 PDF Generation Location.....	458
30.20 17. References.....	458

30.20.1 17.1 Internal Documentation.....	458
30.20.2 17.2 External References.....	458
30.21 Navigation.....	459
31 Claude Code Guide: STREAM Subsystem.....	459
31.1 Quick Context.....	459
31.2  Global Requirements Reference.....	459
31.3 Critical Rules for This Subsystem.....	460
31.3.1 Rule #0: Attribution Format for Git Commits.....	460
31.3.2 Rule #0.1: TUTORIAL FOCUS - Intentional Simplifications.....	460
31.3.3 Rule #0.1: Testbench Location and Test Structure (MANDATORY)....	460
31.3.4 Rule #0.2: Three Mandatory TB Methods (MANDATORY).....	461
31.3.5 Rule #1: REUSE from RAPIDS Where Appropriate.....	461
31.3.6 Rule #2: Descriptor Format is DIFFERENT from RAPIDS.....	462
31.3.7 Rule #3: Know the Shared Resources.....	462
31.4 Architecture Quick Reference.....	462
31.4.1 Block Organization.....	462
31.4.2 Module Status.....	463
31.5 Common User Questions and Responses.....	463
31.5.1 Q: “How is STREAM different from RAPIDS?”.....	463
31.5.2 Q: “How do I kick off a transfer?”.....	464
31.5.3 Q: “How many channels can I use?”.....	464
31.5.4 Q: “What’s the descriptor format?”.....	465
31.5.5 Q: “How do I run STREAM tests?”.....	465
31.6 Integration Patterns.....	466

31.6.1 Pattern 1: Basic STREAM Instantiation.....	466
31.6.2 Pattern 2: Descriptor Creation (Software Model).....	467
31.6.3 Pattern 3: MonBus Integration.....	468
31.7 Anti-Patterns to Catch.....	468
31.7.1 ✗ Anti-Pattern 1: Adding Alignment Fixup.....	468
31.7.2 ✗ Anti-Pattern 2: Using Length in Bytes.....	468
31.7.3 ✗ Anti-Pattern 3: Circular Buffer Descriptors.....	469
31.7.4 ✗ Anti-Pattern 4: Assuming Exclusive Channel Access.....	469
31.8 Debugging Workflow.....	469
31.8.1 Issue: Descriptor Not Fetched.....	469
31.8.2 Issue: Data Transfer Stalls.....	469
31.8.3 Issue: Chained Descriptors Not Following.....	470
31.9 Testing Guidance.....	470
31.9.1 Test Organization.....	470
31.9.2 Test Levels.....	470
31.10 Key Documentation Links.....	470
31.10.1 Always Reference These.....	470
31.11 Quick Commands.....	471
31.12 Remember.....	471
31.13 PDF Generation Location.....	471
32 STREAM Register Definitions.....	472
32.1 Directory Structure.....	472
32.2 Register Generation Workflow.....	473
32.2.1 Phase 1: Define Register Map (Future).....	473

32.2.2 Phase 2: PeakRDL-Generated Registers (Future).....	473
32.3 Register Map (Planned).....	474
32.3.1 Global Registers.....	474
32.3.2 Channel Registers (8 × 0x10 bytes).....	474
32.4 Integration Pattern.....	475
32.4.1 PeakRDL-Generated Implementation (Future):.....	475
32.5 Reference Examples.....	475
32.6 Status.....	475

List of Figures

Figure 1.1.1: STREAM Architecture Overview.....	72
Figure 1.1.2: Single Channel Transfer Sequence.....	74
Figure 2.1.1: STREAM Core Block Diagram.....	107
Figure 2.4.1: Scheduler Block Diagram.....	151
Figure 2.4.2: Scheduler FSM.....	161
Figure 2.4.3: Scheduler Normal Transfer Timing.....	169
Figure 2.4.4: Scheduler Descriptor Chaining Timing.....	170
Figure 2.7.1: Stream Allocation Controller Block Diagram.....	201
Figure 2.8.1: SRAM Controller Block Diagram.....	214
Figure 1: SRAM Controller Unit Block Diagram.....	230
Figure 2.10.1: Stream Latency Bridge Block Diagram.....	237
Figure 1: Stream Drain Controller Block Diagram.....	245
Figure 2.12.1: AXI Write Engine Block Diagram.....	252
Figure 2.13.1: APB to Descriptor Block Diagram.....	265
Figure 2.13.2: APB to Descriptor FSM.....	266
Figure 1: Stream Config Block Diagram.....	278
Figure 2.15.1: Performance Profiler Block Diagram.....	286
Figure 2.16.1: MonBus AXI-Lite Group Block Diagram.....	295

List of Tables

Table 1: Related Documents and Specifications.....	62
Table 2: STREAM MAS Document Revision History.....	64
Table 3: STREAM External Interfaces.....	75
Table 4: STREAM Internal MonBus Interface.....	76
Table 5: STREAM Resource Utilization Breakdown.....	76
Table 6: STREAM Architecture Overview Revision History.....	79
Table 7: Clock and Reset Signals.....	80
Table 8: APB Programming Interface Signals.....	81
Table 9: Per-Channel Configuration Signals.....	81
Table 10: Global Scheduler Configuration Signals.....	82
Table 11: Descriptor Engine Configuration Signals.....	82
Table 12: AXI Monitor Signal Prefixes.....	83
Table 13: AXI Monitor Configuration Signal Suffixes.....	83
Table 14: AXI Transfer Configuration Signals.....	84
Table 15: Performance Profiler Configuration Signals.....	84
Table 16: Per-Channel Status Signals.....	85
Table 17: Performance Profiler Interface Signals.....	86
Table 18: Descriptor AXI Master AR Channel Signals.....	87
Table 19: R Channel.....	88
Table 20: AR Channel.....	88
Table 21: R Channel.....	89
Table 22: AW Channel.....	90
Table 23: W Channel.....	91
Table 24: B Channel.....	91
Table 25: Descriptor AXI Monitor Status.....	92
Table 26: Read Engine AXI Monitor Status.....	92
Table 27: Write Engine AXI Monitor Status.....	93
Table 28: Unified Monitor Bus Interface.....	93
Table 29: Port Count Summary.....	94
Table 30: Default Parameter Values.....	95
Table 31: STREAM Top-Level Port List Revision History.....	96
Table 32: Reset Recovery.....	99
Table 33: Functional Unit Blocks.....	99
Table 34: Integration Blocks.....	100
Table 35: Clocks and Reset Specification Revision History.....	105
Table 36: Primary Configuration.....	110

Table 37: Monitor Control.....	111
Table 38: Outstanding Transaction Limits.....	111
Table 39: AXI Skid Buffer Depths.....	111
Table 40: MonBus Agent IDs.....	112
Table 41: Clock and Reset.....	112
Table 42: APB Programming Interface.....	113
Table 43: Configuration Interface.....	113
Table 44: Configuration Interface.....	113
Table 45: Configuration Interface.....	114
Table 46: Configuration Interface.....	114
Table 47: Status Interface.....	116
Table 48: Status Interface.....	116
Table 49: Status Interface.....	117
Table 50: AXI4 Master - Descriptor Fetch.....	117
Table 51: AXI4 Master - Descriptor Fetch.....	118
Table 52: AXI4 Master - Data Read.....	119
Table 53: AXI4 Master - Data Read.....	119
Table 54: AXI4 Master - Data Write.....	120
Table 55: AXI4 Master - Data Write.....	121
Table 56: AXI4 Master - Data Write.....	121
Table 57: Status/Debug Outputs.....	122
Table 58: Status/Debug Outputs.....	122
Table 59: Status/Debug Outputs.....	123
Table 60: Unified Monitor Bus Interface.....	123
Table 61: Resource Utilization.....	127
Table 62: STREAM Core Specification Revision History.....	130
Table 63: Parameters.....	132
Table 64: Monitor Bus Agent IDs.....	132
Table 65: Clock and Reset.....	133
Table 66: APB Programming Interface.....	133
Table 67: Configuration Interface.....	133
Table 68: Global Scheduler Configuration.....	134
Table 69: Descriptor Engine Configuration.....	134
Table 70: Status Interface.....	135
Table 71: Shared Descriptor AXI4 Master Read Interface.....	135
Table 72: Shared Descriptor AXI4 Master Read Interface.....	136
Table 73: Shared Data Read Interface.....	136
Table 74: Shared Data Write Interface.....	137

Table 75: Unified Monitor Bus Interface.....	137
Table 76: Scheduler Group Array Revision History.....	140
Table 77: Parameters.....	142
Table 78: Monitor Bus Agent IDs.....	142
Table 79: Clock and Reset.....	143
Table 80: APB Programming Interface.....	143
Table 81: Configuration Interface.....	143
Table 82: Scheduler Configuration.....	144
Table 83: Descriptor Engine Configuration.....	144
Table 84: Status Interface.....	145
Table 85: Descriptor AXI Interface.....	145
Table 86: Descriptor AXI Interface.....	146
Table 87: Data Read Interface.....	146
Table 88: Data Write Interface.....	146
Table 89: Monitor Bus Interface.....	147
Table 90: Scheduler Group Revision History.....	149
Table 91: Clock and Reset.....	153
Table 92: Configuration Interface.....	154
Table 93: Status Interface.....	154
Table 94: Descriptor Engine Interface.....	154
Table 95: Data Read Interface.....	155
Table 96: Data Read Interface.....	155
Table 97: Data Write Interface.....	155
Table 98: Data Write Interface.....	156
Table 99: Error Signals.....	156
Table 100: Monitor Bus Interface.....	156
Table 101: Revision History.....	173
Table 102: Descriptor Format.....	176
Table 103: Parameters.....	177
Table 104: Monitor Bus Parameters.....	178
Table 105: Clock and Reset.....	178
Table 106: APB Programming Interface.....	178
Table 107: Scheduler Interface.....	179
Table 108: Enhanced Control Outputs.....	179
Table 109: AXI4 AR Channel.....	179
Table 110: AXI4 R Channel.....	180
Table 111: Configuration Interface.....	181
Table 112: Status Interface.....	181

Table 113: Monitor Bus Interface.....	181
Table 114: Descriptor Engine Revision History.....	187
Table 115: Parameters.....	189
Table 116: Derived Parameters.....	190
Table 117: Clock and Reset.....	190
Table 118: Configuration Interface.....	190
Table 119: Scheduler Interface.....	191
Table 120: Completion Interface.....	191
Table 121: SRAM Allocation Interface.....	191
Table 122: SRAM Write Interface.....	192
Table 123: AXI4 AR Channel.....	192
Table 124: AXI4 R Channel.....	192
Table 125: Error Interface.....	193
Table 126: Debug Interface.....	193
Table 127: AXI Read Engine Revision History.....	198
Table 128: Parameters.....	203
Table 129: Clock and Reset.....	204
Table 130: Write Interface.....	204
Table 131: Read Interface.....	205
Table 132: Status Outputs.....	205
Table 133: Write Interface.....	205
Table 134: Read Interface.....	206
Table 135: Status Outputs.....	206
Table 136: Comparison with Drain Controller.....	211
Table 137: Stream Allocation Controller Revision History.....	212
Table 138: Clock and Reset.....	215
Table 139: Allocation Interface.....	215
Table 140: Write Interface.....	216
Table 141: Drain Interface.....	216
Table 142: Read Interface.....	217
Table 143: Debug Interface.....	217
Table 144: Revision History.....	228
Table 145: Parameters.....	231
Table 146: Derived Parameters.....	231
Table 147: Clock and Reset.....	231
Table 148: Allocation Interface.....	232
Table 149: Write Interface.....	232
Table 150: Drain Interface.....	232

Table 151: Read Interface.....	233
Table 152: Debug Interface.....	233
Table 153: SRAM Controller Unit Revision History.....	235
Table 154: Parameters.....	239
Table 155: Derived Parameters.....	239
Table 156: Clock and Reset.....	239
Table 157: Upstream Interface.....	239
Table 158: Downstream Interface.....	240
Table 159: Status Interface.....	240
Table 160: Debug Interface.....	240
Table 161: Stream Latency Bridge Revision History.....	244
Table 162: Parameters.....	246
Table 163: Derived Parameters.....	246
Table 164: Clock and Reset.....	246
Table 165: Write Interface.....	246
Table 166: Read Interface.....	247
Table 167: Status Outputs.....	247
Table 168: Comparison with stream_alloc_ctrl.....	249
Table 169: Stream Drain Controller Revision History.....	251
Table 170: Parameters.....	253
Table 171: Derived Parameters.....	254
Table 172: Clock and Reset.....	254
Table 173: Configuration Interface.....	254
Table 174: Scheduler Interface.....	255
Table 175: Completion Interface.....	255
Table 176: SRAM Drain Interface.....	255
Table 177: SRAM Read Interface.....	255
Table 178: AXI4 AW Channel.....	256
Table 179: AXI4 W Channel.....	256
Table 180: AXI4 B Channel.....	257
Table 181: Error Interface.....	257
Table 182: Debug Interface.....	257
Table 183: AXI Write Engine Revision History.....	263
Table 184: Parameters.....	267
Table 185: Clock and Reset.....	268
Table 186: APB Slave CMD Interface.....	268
Table 187: APB Slave RSP Interface.....	268
Table 188: Descriptor Engine APB Ports.....	269

Table 189: Integration Control.....	269
Table 190: APB to Descriptor Router Revision History.....	277
Table 191: Parameters.....	279
Table 192: Clock and Reset.....	280
Table 193: PeakRDL Register Inputs.....	280
Table 194: PeakRDL Register Inputs.....	280
Table 195: PeakRDL Register Inputs.....	281
Table 196: PeakRDL Register Inputs.....	281
Table 197: PeakRDL Register Inputs.....	281
Table 198: Configuration Outputs.....	282
Table 199: Configuration Outputs.....	282
Table 200: Configuration Outputs.....	282
Table 201: Configuration Outputs.....	283
Table 202: Configuration Outputs.....	283
Table 203: APB Configuration Block Revision History.....	285
Table 204: FIFO Entry Format.....	287
Table 205: Parameters.....	287
Table 206: Clock and Reset.....	288
Table 207: Channel Monitoring.....	288
Table 208: Configuration Interface.....	288
Table 209: FIFO Read Interface.....	289
Table 210: Performance Profiler Revision History.....	292
Table 211: Parameters.....	296
Table 212: Clock and Reset.....	296
Table 213: Monitor Bus Input.....	297
Table 214: AXI-Lite Slave Read Interface.....	297
Table 215: AXI-Lite Master Write Interface.....	297
Table 216: Interrupt Output.....	298
Table 217: Configuration Interface.....	298
Table 218: Protocol Configuration.....	299
Table 219: Protocol Configuration.....	299
Table 220: Protocol Configuration.....	299
Table 221: Debug/Status.....	300
Table 222: MonBus AXI-Lite Group Revision History.....	302
Table 223: Number of Interfaces.....	305
Table 224: Interface Parameters.....	306
Table 225: Interface Types and Transfer Modes.....	306
Table 226: Interface Group Parameter Settings.....	307

Table 227: Interface Configuration Summary.....	308
Table 228: Simplified Mode Assumptions.....	309
Table 229: Flexible Mode Assumptions.....	309
Table 230: Assumption 1: Address Alignment to Data Bus Width.....	310
Table 231: Assumption 2: Fixed Transfer Size.....	310
Table 232: Assumption 1: 4-Byte Address Alignment.....	311
Table 233: Assumption 2: Multiple Transfer Sizes.....	311
Table 234: Assumption 3: Progressive Alignment Strategy.....	312
Table 235: Assumption 4: Chunk Enable Support.....	312
Table 236: Assumption 1: Incrementing Bursts Only.....	312
Table 237: Assumption 2: No Address Wraparound.....	313
Table 238: Progressive Alignment Examples.....	313
Table 239: Progressive Alignment Examples.....	314
Table 240: Chunk Enable Pattern Examples.....	314
Table 241: Read Address Channel.....	315
Table 242: Read Data Channel.....	316
Table 243: Write Address Channel.....	316
Table 244: Write Data Channel.....	317
Table 245: Write Response Channel.....	318
Table 246: Simplified Mode Address Generation.....	318
Table 247: Flexible Mode Address Generation.....	318
Table 248: 4KB Boundary Considerations.....	319
Table 249: Simplified Mode Strobe Generation.....	319
Table 250: Flexible Mode Strobe Generation.....	320
Table 251: Response Code Specification.....	320
Table 252: Simplified Mode Benefits.....	321
Table 253: Flexible Mode Benefits.....	321
Table 254: Mode Selection Guidelines.....	321
Table 255: Simplified Mode Validation.....	322
Table 256: Flexible Mode Validation.....	322
Table 257: Common Validation.....	323
Table 258: Simplified Mode Performance.....	323
Table 259: Flexible Mode Performance.....	324
Table 260: AXI4 Interface Specification for STREAM Revision History.....	324
Table 261: Interface Parameters.....	325
Table 262: Inherent AXI4-Lite Simplifications.....	326
Table 263: Assumption 1: Address Alignment to Data Bus Width.....	326
Table 264: Assumption 2: Fixed Transfer Size.....	326

Table 265: Assumption 3: No Address Wraparound.....	327
Table 266: Assumption 4: Standard Protection Attributes.....	327
Table 267: Read Address Channel.....	327
Table 268: Read Data Channel.....	328
Table 269: AXI4-Lite Simplifications.....	328
Table 270: Write Address Channel.....	329
Table 271: Write Data Channel.....	329
Table 272: Write Response Channel.....	329
Table 273: AXI4-Lite Simplifications.....	330
Table 274: Address Alignment Rules.....	330
Table 275: Address Validation Examples.....	331
Table 276: Response Code Specification.....	331
Table 277: Response Usage Guidelines.....	331
Table 278: Protection Signal Encoding.....	332
Table 279: Common Protection Patterns.....	332
Table 280: Simplified Control Register Interface.....	332
Table 281: Address Decode Implementation.....	333
Table 282: Error Generation Logic.....	333
Table 283: Handshake Protocol.....	334
Table 284: Channel Dependencies.....	334
Table 285: Reset Behavior.....	334
Table 286: Functional Validation.....	335
Table 287: Timing Validation.....	335
Table 288: Error Injection Testing.....	336
Table 289: 64-bit Register Write.....	336
Table 290: AW Transaction Flow.....	336
Table 291: 64-bit Register Read.....	337
Table 292: AR Transaction Flow.....	337
Table 293: Misaligned Address Example.....	337
Table 294: Typical Applications.....	338
Table 295: Performance Considerations —.....	338
Table 296: AXI4-Lite Interface Specification and Assumptions Revision History	339
Table 297: STREAM Programming Interface.....	340
Table 298: Interface Parameters.....	340
Table 299: Assumption 1: Word-Aligned Access Only.....	341
Table 300: Assumption 2: Standard Transfer Sizes.....	341
Table 301: Assumption 3: Single-Cycle Default Operation.....	341
Table 302: Clock and Reset Signals ### Address and Control Signals.....	342

Table 303: Address and Control Signals.....	342
Table 304: Data Signals.....	343
Table 305: Response Signals.....	343
Table 306: Address Range Specification.....	343
Table 307: Address Decode Implementation.....	344
Table 308: 2-Phase Transaction States.....	344
Table 309: State Transitions.....	344
Table 310: Transaction Timing Requirements.....	345
Table 311: Dynamic Clock Gating.....	345
Table 312: Clock Domain Crossing (CDC) Handshaking.....	345
Table 313: Buffering and Flow Control.....	346
Table 314: Error Response Specification.....	346
Table 315: Error Response Timing.....	347
Table 316: Reset Requirements.....	347
Table 317: Available Implementations.....	347
Table 318: Latency Specifications.....	348
Table 319: Throughput Specifications.....	348
Table 320: Power Consumption.....	348
Table 321: Functional Validation.....	349
Table 322: Timing Validation.....	349
Table 323: 32-bit Register Write.....	350
Table 324: 32-bit Register Read with Wait State —.....	350
Table 325: APB Programming Interface Specification for STREAM Revision History.....	350
Table 326: Interface Parameters.....	352
Table 327: Assumption 1: Hierarchical Event Organization.....	353
Table 328: Assumption 2: Protocol Isolation.....	353
Table 329: Assumption 3: Two-Tier Memory Architecture.....	353
Table 330: Assumption 4: Configurable Packet Routing.....	354
Table 331: Monitor Bus Output Interface.....	354
Table 332: Protocol Input Interfaces.....	354
Table 333: Control and Status Signals.....	355
Table 334: 128-bit Monitor Bus Packet Structure.....	356
Table 335: 64-bit Side-Band Timestamp.....	357
Table 336: Packet Type Definitions.....	357
Table 337: Error Events.....	358
Table 338: Timeout Events.....	359
Table 339: Performance Events.....	359

Table 340: Error Events.....	360
Table 341: Timeout Events.....	361
Table 342: Completion Events.....	361
Table 343: Threshold Events.....	362
Table 344: Performance Events.....	362
Table 345: Debug Events.....	363
Table 346: Error Events.....	364
Table 347: Credit Events.....	365
Table 348: Channel Events.....	365
Table 349: Stream Events.....	366
Table 350: Error Events.....	367
Table 351: Timeout Events.....	368
Table 352: Completion Events.....	368
Table 353: Threshold Events.....	369
Table 354: Performance Events.....	369
Table 355: Debug Events.....	370
Table 356: Error Events.....	371
Table 357: Timeout Events.....	372
Table 358: Completion Events.....	373
Table 359: Threshold Events.....	373
Table 360: Performance Events.....	374
Table 361: Debug Events.....	375
Table 362: Local Error/Interrupt Memory.....	376
Table 363: Configurable External Memory.....	376
Table 364: Base and Limit Registers.....	377
Table 365: Routing Decision Logic.....	377
Table 366: Memory Layout Example.....	378
Table 367: Transaction State Enumeration.....	378
Table 368: Enhanced Transaction Structure.....	379
Table 369: Phase Completion Flags.....	379
Table 370: Protocol-Specific Tracking Fields.....	380
Table 371: APB Transaction Phases.....	381
Table 372: APB Protection Types.....	381
Table 373: MNOC Payload Types.....	381
Table 374: MNOC ACK Types.....	382
Table 375: ARB State Types.....	382
Table 376: CORE State Types.....	382
Table 377: Global Configuration.....	383

Table 378: Packet Type Enable Mapping.....	383
Table 379: AXI Monitor Configuration.....	384
Table 380: MNOC Monitor Configuration.....	385
Table 381: ARB Monitor Configuration.....	385
Table 382: CORE Monitor Configuration.....	386
Table 383: Functional Validation.....	386
Table 384: Performance Validation.....	387
Table 385: Error Handling Validation.....	387
Table 386: Creating Monitor Packets.....	388
Table 387: Packet Decoding.....	388
Table 388: Packet Field Extraction.....	388
Table 389: Packet Creation Function.....	389
Table 390: Event Code Creation Functions.....	389
Table 391: Validation Functions.....	391
Table 392: String Functions for Debugging.....	391
Table 393: Essential Debug Signals.....	392
Table 394: Performance Counters.....	392
Table 395: Complete Protocol Event Matrix.....	392
Table 396: Monitor Bus Architecture and Event Code Organization Revision History.....	393
Table 397: Channel Kick-off Registers.....	395
Table 398: GLOBAL_CTRL.....	397
Table 399: GLOBAL_STATUS.....	397
Table 400: VERSION.....	398
Table 401: CHANNEL_ENABLE.....	398
Table 402: CHANNEL_RESET.....	398
Table 403: CHANNEL_IDLE.....	399
Table 404: DESC_ENGINE_IDLE.....	399
Table 405: SCHEDULER_IDLE.....	399
Table 406: CH_STATE[0..7].....	399
Table 407: SCHED_ERROR.....	400
Table 408: AXI_RD_COMPLETE.....	401
Table 409: AXI_WR_COMPLETE.....	401
Table 410: MON_FIFO_STATUS.....	402
Table 411: MON_FIFO_COUNT.....	402
Table 412: SCHED_TIMEOUT_CYCLES.....	402
Table 413: SCHED_CONFIG.....	403
Table 414: SCHED_TIMEOUT_LIMIT.....	403

Table 415: DESCENG_CONFIG.....	404
Table 416: DESCENG_ADDR0_BASE.....	404
Table 417: DESCENG_ADDR0_LIMIT.....	404
Table 418: DESCENG_ADDR1_BASE.....	405
Table 419: DESCENG_ADDR1_LIMIT.....	405
Table 420: DAXMON_ENABLE.....	405
Table 421: DAXMON_TIMEOUT.....	405
Table 422: DAXMON_LATENCY_THRESH.....	406
Table 423: DAXMON_PKT_MASK.....	406
Table 424: DAXMON_ERR_CFG.....	406
Table 425: DAXMON_MASK1.....	406
Table 426: DAXMON_MASK2.....	407
Table 427: DAXMON_MASK3.....	407
Table 428: RDMON_ENABLE.....	407
Table 429: RDMON_TIMEOUT.....	408
Table 430: RDMON_LATENCY_THRESH.....	408
Table 431: RDMON_PKT_MASK.....	408
Table 432: RDMON_ERR_CFG.....	408
Table 433: RDMON_MASK1.....	409
Table 434: RDMON_MASK2.....	409
Table 435: RDMON_MASK3.....	409
Table 436: WRMON_ENABLE.....	410
Table 437: WRMON_TIMEOUT.....	410
Table 438: WRMON_LATENCY_THRESH.....	410
Table 439: WRMON_PKT_MASK.....	411
Table 440: WRMON_ERR_CFG.....	411
Table 441: WRMON_MASK1.....	411
Table 442: WRMON_MASK2.....	411
Table 443: WRMON_MASK3.....	412
Table 444: AXI_XFER_CONFIG.....	412
Table 445: PERF_CONFIG.....	413
Table 446: Register Summary Table.....	415
Table 447: Channel Observation Registers.....	416
Table 448: PeakRDL Generation.....	416
Table 449: Configuration Categories.....	420
Table 450: 9. Configuration Register Map Summary.....	436

List of Waveforms

Waveform 2.1.1: STREAM Core Data Flow.....	125
Waveform 2.5.1: Descriptor Engine APB Basic Kick-off.....	184
Waveform 2.6.1: AXI Read Engine - Perfect Streaming.....	195
Waveform 2.6.2: Datapath Read - Multi-Channel.....	196
Waveform 2.8.1: SRAM Controller Write/Read Operation.....	225
Waveform 2.10.1: Stream Latency Bridge Backpressure.....	242
Waveform 2.12.1: AXI Write Engine - Perfect Streaming.....	259
Waveform 2.12.2: Datapath Write - Multi-Channel.....	261
Waveform 2.13.1: APB Normal Write Sequence.....	271
Waveform 2.13.2: APB Write with Backpressure.....	272
Waveform 2.13.3: Channel 0 Kick-off.....	273
Waveform 2.13.4: Channel 7 Kick-off.....	274
Waveform 2.13.5: Multi-Channel Kick-off Sequence.....	275

1

Stream Index

Generated: 2026-07-17

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

2

Document Information

This document describes the STREAM subsystem, a Scatter-gather Transfer Rapid Engine for AXI Memory. STREAM provides descriptor-based DMA functionality with multi-channel support, enabling efficient memory-to-memory data transfers via AXI4 interfaces. The subsystem is designed as a tutorial-focused implementation with intentional simplifications for educational purposes.

2.1

References

2.1.1

Related Documents

Related Documents and Specifications

Source	Title	Version
RTL Design Sherpa	STREAM Product Requirements Document	1.0
RTL Design Sherpa	RAPIDS MAS (Parent Architecture)	0.90
ARM	AMBA AXI and ACE Protocol Specification	IHI0022H
ARM	AMBA APB Protocol Specification	IHI0024C

2.2 Terminology

AXI

Advanced eXtensible Interface. ARM's high-performance, high-frequency protocol for interconnect.

AXI4

Version 4 of the AXI protocol, supporting burst transfers up to 256 beats.

AXIL

AXI4-Lite. A simplified subset of AXI4 for low-throughput register access.

APB

Advanced Peripheral Bus. ARM's simple, low-power bus protocol for peripheral register access.

Beat

A single data transfer within an AXI burst. For STREAM with 512-bit data width, one beat = 64 bytes.

Burst

A group of consecutive data transfers (beats) initiated by a single address phase.

Channel

One of 8 independent DMA channels in STREAM. Each channel has its own descriptor chain and SRAM buffer allocation.

Descriptor

A 256-bit data structure containing transfer parameters: source address, destination address, length, and chain pointer.

Descriptor Chain

A linked list of descriptors where each descriptor points to the next, enabling scatter-gather operations.

DMA

Direct Memory Access. Hardware-controlled data movement between memory locations without CPU intervention.

FUB

Functional Unit Block. A self-contained RTL module with defined interfaces and testbench.

MAC

Macro. An integration-level block that instantiates and connects multiple FUBs.

MonBus

Monitor Bus. A 128-bit packet stream paired with a 64-bit side-band timestamp wire, used for performance monitoring and debug event reporting. Packet field layout is the canonical spec at docs/markdown/RTLAmba/includes/monitor_package_spec.md; the stream monbus_axil_group wraps the shared building block at rtl/amba/shared/monbus_axil_group.sv and emits 3-beat AXI-Lite records (pkt[63:0], pkt[127:64], ts[63:0]).

Outstanding Transaction

A transaction that has been issued but not yet completed. V2/V3 performance modes support multiple outstanding transactions.

Scatter-Gather

A DMA technique where non-contiguous memory regions are transferred using a descriptor chain.

SRAM

Static Random Access Memory. Used for internal data buffering between read and write datapaths.

V1/V2/V3

STREAM performance modes: - V1 (Low): Single outstanding transaction, tutorial-focused - V2 (Medium): Command pipelined, best area efficiency - V3 (High): Out-of-order completion, maximum throughput

2.3 Revision History

Revisions follow the convention x.y, where x is the major version and y is the minor version. Minor versions indicate incremental updates; major versions indicate significant architectural changes.

STREAM MAS Document Revision History

Rev	Date	Author	Notes
0.90	2025-11-22	seang	Initial STREAM MAS structure and block documentation
0.91	2026-05-14	seang	Sync to RTL

Rev	Date	Author	Notes
			changes since 2026-04-17 (commit be4e5a91); regenerate artifacts.
0.92	2026-06-05	seang	Sync to RTL state at 2026-06-05 (17 commits since be4e5a91). Adds (1) 128-bit monbus packet + 64-bit side-band timestamp documentation in ch02_blocks/16_monbus_axil_group.md and ch03_interfaces/05_monbus_interface_spec.md — stream wrapper now instantiates the shared building block from rtl/amba/shared/monbus_axil_group.sv, cfg_ts_append_* removed, m_axil records locked to 3 beats; (2) channel-observation mux documentation in ch02_blocks/14_apb_config.md plus ch04_registers/register_map.md (OBS_CTRL/FLAGS/DATA0/DATA1

Rev	Date	Author	Notes
			register set, fed by scheduler error stickies + timeout exposed in ch02_blocks/04_scheduler.md); (3) BOTH-end descriptor-path monitoring in ch02_blocks/05_descriptor_engine.md (distinct fetch-side / consume-side monitor wrappers); (4) Bug-fix notes for ch02_blocks/06_axi_read_engine.md + 12_axi_write_engine.md (registered w_arb_request, commit 4e8f9e02), 11_stream_drain_ctrl.md + 12_axi_write_engine.md (stale-view race + post-flop wvalid gate, commit a82627af), 08_sram_controller.md (reset on registered avail outputs, commit b619eee9).
0.93	2026-07-02	seang	Monitor/perf registers relocated into a separate stream_mon_regs

Rev	Date	Author	Notes
			regfile at 0x1000 (single APB slave, 13-bit/8 KB decode) in ch04_registers/re gister_map.md + ch03_interfaces/0 3_apb_interface_s pec.md; added SCHED_TIMEOUT_LIM IT @ 0x208 and documented the recoverable scheduler write- progress timeout + commit-gated completion + scheduler_idle now excluding CH_ERROR (ch02_blocks/04_s cheduler.md); documented functional descriptor prefetch throttle + deferred chaining (ch02_blocks/05_d escriptor_engine. md).
0.94	2026-07-07	seang	Documented the kick-burst fast-path programming model (ch05_programming /02_single_transf er.md + 04_multi_channel.

Rev	Date	Author	Notes
			md): program per-channel address registers, then one “go”-bit write fires N channel kicks back-to-back on a single clock cycle via the top-level i_kick_burst_* ports, versus the serial APB CHn_CTRL kick. Reference implementation = the NexysA7 char harness CH_KICK_ADDR shadow registers + KICK_GO bitmask CSR (host harness_kick.py:: batch_kick).
0.96	2026-07-17	seang	Documented always-on datapath performance meters: USE_AXI_MONITORS now gates only the heavy axi4_master*_mon CAM/reporter cones + latency histograms; the in-core axi_bus_meter buckets (RDMON/WRMON_PERF prod/bp/starv/idle)

Rev	Date	Author	Notes
			and the beat/byte/burst/window counters are always-on and the perf-window RUN bits pass through, so a monitors-off production build still measures datapath utilisation. Updated the USE_AXI_MONITORS = 0 note (ch02_blocks/01_stream_core.md) and the MON-block gating clause (ch04_registers/register_map.md). Also relocated stream + rapids under projects/components/dmas/ (path references throughout).
0.95	2026-07-09	seang	Documented read-ahead descriptor prefetch in ch02_blocks/04_scheduler.md: the r_rd_ahead flag (read flops load the descriptor-FIFO head WITHOUT draining it; write

Rev	Date	Author	Notes
			flops load it and drain the FIFO to catch up), the in-place CH_XFER_DATA advance that collapses the per-descriptor bubble to zero, and the registered launch/advance commit-accumulator addend that keeps the descriptor-FIFO output off the accumulator critical path for 100 MHz closure. New runtime field SCHED_CONFIG.RD_PREFETCH_EN @ 0x204[5] (default on) added to ch04_registers/register_map.md + ch06_configuration/config_reference.md.

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

3 STREAM Architecture Overview

Component: STREAM (Scatter-gather Transfer Rapid Engine for AXI Memory) **Version:** 0.90
Status: Pre-release

3.1 Introduction

STREAM is a high-performance, multi-channel descriptor-based DMA engine designed for memory-to-memory scatter-gather transfers. It provides an educational yet production-capable architecture demonstrating key DMA concepts while maintaining professional design practices.

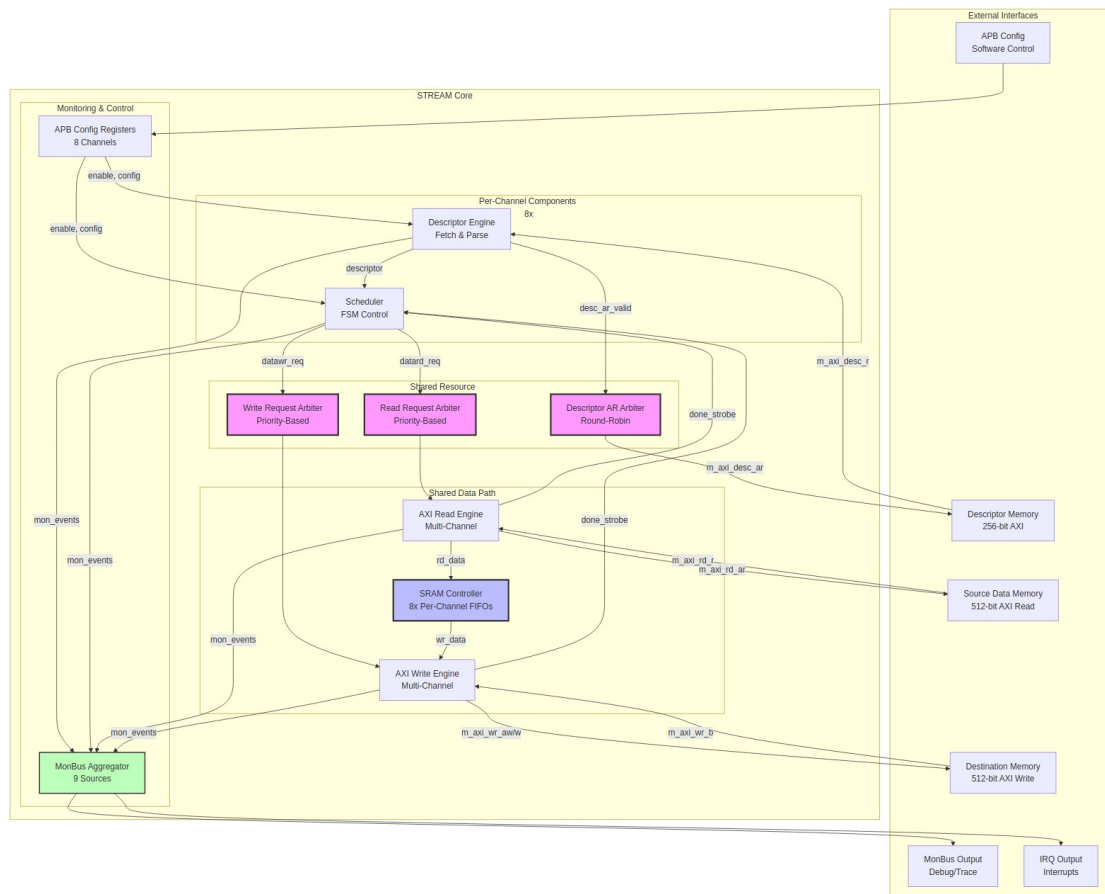
3.1.1 Design Philosophy

1. **Tutorial Focus** - Intentional simplifications for learning
 2. **Production Quality** - Industry-standard interfaces and verification
 3. **Scalability** - 8 independent channels with shared resource arbitration
 4. **Modularity** - Clear separation of concerns across functional blocks
-

3.2 System Architecture

3.2.1 High-Level Block Diagram

3.2.2 Figure 1.1.1: STREAM Architecture Overview



Diagram

3.3 Key Architectural Concepts

3.3.1 1. Multi-Channel Design

8 Independent Channels: - Each channel operates autonomously - Separate descriptor chains per channel - Independent FSM state per channel - Concurrent transfers across all channels

Resource Sharing: - Descriptor AXI master (shared via round-robin arbiter) - Data read AXI master (shared via priority arbiter) - Data write AXI master (shared via priority arbiter) - SRAM buffer (per-channel FIFOs, no arbitration)

3.3.2 2. Descriptor-Based Operation

Descriptor Format (256-bit):

[255:192] Reserved
[191:160] Next Descriptor Pointer
[159:128] Length (in beats)
[127:64] Destination Address
[63:0] Source Address

Descriptor Chain: - Single APB write kicks off chain - Automatic chaining via next_descriptor_ptr
- Explicit termination (ptr = 0 or last flag)

3.3.3 3. Concurrent Read/Write

CRITICAL Design Pattern:

STREAM uses concurrent read and write operations to handle transfers larger than the SRAM buffer:

Example: 100MB transfer with 64KB SRAM buffer

Sequential (WRONG):

Read 100MB → DEADLOCK at 64KB (SRAM full)

Concurrent (CORRECT):

Read fills SRAM → SRAM full → Read pauses

Write drains SRAM → SRAM space freed → Read resumes

Both continue until 100MB complete

Implementation: - Scheduler runs read and write FSMs concurrently in XFER_DATA state -
Independent beat counters for read vs write - Transfer completes when BOTH counters reach zero

3.3.4 4. Space-Aware Flow Control

Allocation Controller (Read Path): - Reserves SRAM space BEFORE issuing AXI AR - Prevents overflow due to AR/R latency gap - 2x space margin (accounts for in-flight allocations)

Drain Controller (Write Path): - Reserves SRAM data BEFORE issuing AXI AW - Prevents underflow due to AW/W latency gap - Includes latency bridge occupancy in count

3.3.5 5. Priority-Based Arbitration

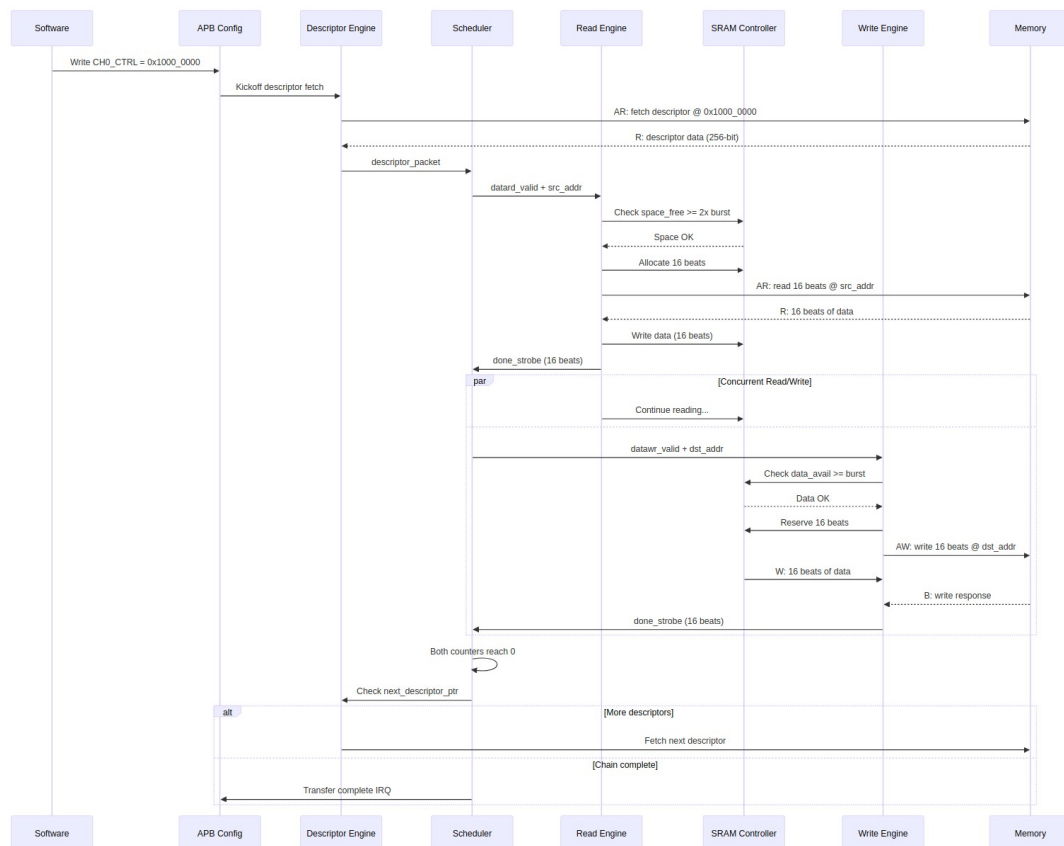
Descriptor Fetch: - Round-robin arbitration (fair access) - All channels equal priority

Data Read/Write: - Priority-based arbitration - Priority from descriptor - Round-robin within same priority tier - Timeout prevention for low-priority channels

3.4 Data Flow Example

3.4.1 Single Channel Transfer

3.4.2 Figure 1.1.2: Single Channel Transfer Sequence

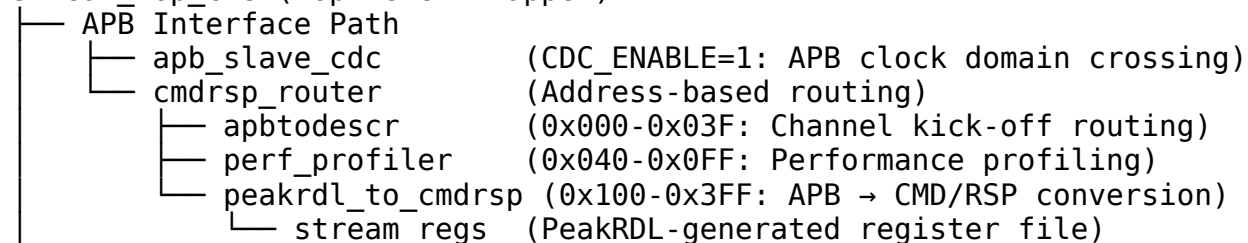


Diagram

3.5 Component Hierarchy

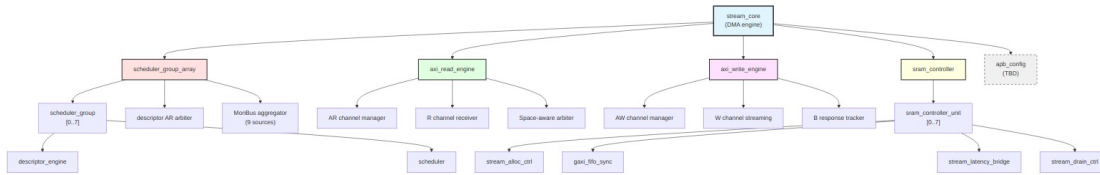
3.5.1 Top-Level Integration (stream_top_ch8.sv)

stream_top_ch8 (Top-Level Wrapper)



- └─ stream_config_block (Register → config signal mapping)
- └─ stream_core (Main DMA engine - see below)
- └─ monbus_axil_group (USE_AXI_MONITORS=1: MonBus → AXI-Lite)
 - └─ Error FIFO (s_axil_err_* slave read)
 - └─ Master Writer (m_axil_mon_* master write)

3.5.2 Core DMA Engine (stream_core.sv)



.1.3: Core DMA Engine Hierarchy

3.6 Interface Summary

3.6.1 External Interfaces (stream_top_ch8.sv)

STREAM External Interfaces

Interface	Type	Width	Purpose
APB	Slave	32-bit	Configuration, control, status (with optional CDC)
Descriptor AXI	Master	256-bit	Fetch descriptors from memory
Data Read AXI	Master	512-bit	Read source data
Data Write AXI	Master	512-bit	Write destination data
Error FIFO AXIL	Slave	32-bit	Read monitor error/interrupt FIFO
Monitor Write AXIL	Master	32-bit	Write monitor data to memory
IRQ	Output	1-bit	Interrupt (error FIFO not empty)

3.6.2 Internal MonBus Interface (stream_core.sv)

STREAM Internal MonBus Interface

Interface	Type	Width	Purpose
MonBus	Output	64-bit	Debug/trace event stream to monbus_axil_group

3.6.3 Configuration Registers (APB)

APB Address Map:

0x000-0x03F: Channel kick-off registers (apbtodescr routing)

0x040-0x0FF: Performance profiler interface

0x100-0x3FF: PeakRDL configuration registers

Key Register Groups: - 0x100: Global Control (enable, reset, version) - 0x120: Per-Channel Control (enable, reset) - 0x140: Per-Channel Status (idle, state, completion) - 0x180: Monitor FIFO Status - 0x200: Scheduler Configuration - 0x220: Descriptor Engine Configuration - 0x240-0x29F: AXI Monitor Configuration (DAXMON, RDMON, WRMON) - 0x2A0: AXI Transfer Configuration - 0x2B0: Performance Profiler Configuration

See: [Register Map](#) for complete documentation.

3.7 Resource Utilization

3.7.1 Area Breakdown (Typical 512-bit Data Width)

STREAM Resource Utilization Breakdown

Component	Quantity	Logic (LUTs)	Memory
Descriptor Engine	8	~300 each	Minimal
Scheduler	8	~400 each	Minimal
AXI Read Engine	1	~800	Minimal
AXI Write Engine	1	~800	Minimal
SRAM Controller	1	~600	8 × 64KB = 512KB

Component	Quantity	Logic (LUTs)	Memory
Arbiters	3	~150 each	Minimal
APB Config	1	~400	Register file
MonBus Aggregator	1	~200	Small FIFO
Total	-	~10K LUTs	~512KB RAM

Notes: - SRAM depth configurable (typical: 512 entries × 512 bits × 8 channels = 512KB) - Logic utilization scales with NUM_CHANNELS - Memory utilization scales with SRAM_DEPTH

3.8 Performance Characteristics

3.8.1 Throughput

Theoretical Maximum (512-bit @ 250MHz): - Single channel: 16 GB/s (512 bits × 250 MHz) - 8 channels concurrent: Limited by memory bandwidth, not DMA

Practical Performance: - Dependent on: - Memory controller latency - Burst sizes (larger = better efficiency) - Transfer alignment - Channel contention

Typical Real-World: - 8-12 GB/s sustained (single channel) - 50-75% of theoretical maximum

3.8.2 Latency

Descriptor Fetch Latency: - AR issue to R data: ~10-50 cycles (memory dependent) - Descriptor parsing: 1 cycle - **Total:** ~11-51 cycles per descriptor

Transfer Initiation: - APB write to first AR: ~5-10 cycles - AR to first R data: ~10-50 cycles - **Total:** ~15-60 cycles from kickoff to first data

Transfer Completion: - Last W beat to B response: ~5-20 cycles - B response to IRQ: 1 cycle - **Total:** ~6-21 cycles from last data to interrupt

3.9 Design Decisions

3.9.1 Why These Simplifications?

1. Aligned Addresses Only:

- Eliminates complex alignment fixup logic
- Clear data path with no byte shifting

- Focus: Core DMA operation, not edge cases
- 2. **Length in Beats:**
 - Direct mapping to AXI burst length
 - No unit conversion overhead
 - Focus: AXI protocol understanding
- 3. **No Credit Management:**
 - Simpler resource arbitration
 - Transaction limits via configuration
 - Focus: Arbitration basics, not complex flow control
- 4. **No Circular Buffers:**
 - Explicit chain termination
 - Clear end-of-transfer detection
 - Focus: Descriptor chaining, not circular logic

3.9.2 Production Enhancements (Future)

If extending STREAM for production use, consider: - ☐ Alignment fixup (byte-level granularity) - ☐ Length in bytes/chunks - ☐ Credit-based flow control - ☐ Circular buffer support - ☐ Advanced error recovery - ☐ Power management - ☐ Multiple SRAM segments

3.10 Testing Strategy

3.10.1 Verification Layers

1. Unit Tests (FUB Level): - Descriptor engine: Fetch, parse, chain - Scheduler: FSM states, concurrent read/write - AXI engines: AR/R and AW/W/B channels - SRAM controller: Allocation, drain, FIFO - Individual tests per module

2. Integration Tests (Multi-Block): - Scheduler + engines: Data flow - Descriptor + scheduler: Descriptor chaining - Full path: APB → Descriptor → Transfer → IRQ

3. System Tests (Full Core): - Single channel end-to-end - Multi-channel concurrent - Error injection and recovery - Performance validation - Long-duration stress tests

3.10.2 Coverage Targets

- **Code Coverage:** >95%
- **Functional Coverage:** >90%
- **Corner Cases:** 100% tested
- **Error Paths:** 100% tested

3.11 Related Documentation

Chapter 2 - Block Specifications: - [Scheduler](#) - Core FSM controller - [Descriptor Engine](#) - Descriptor fetch/parse - [AXI Read Engine](#) - Source data read - [AXI Write Engine](#) - Destination data write - [SRAM Controller](#) - Buffering and flow control - [Scheduler Group](#) - Per-channel wrapper - [Scheduler Group Array](#) - 8-channel array

Other Resources: - [STREAM PRD](#) - Product requirements - [Test Plan](#) - Verification strategy

Last Updated: 2025-12-01 **Document Version:** 0.91 —

3.12 Revision History

STREAM Architecture Overview Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 STREAM Top-Level Port List

Module: stream_core.sv **Location:** projects/components/dmas/stream/rtl/macro/ **Last Updated:** 2025-12-01

4.1 Overview

This document provides a complete reference for all top-level ports of the STREAM Core module, organized by interface type. All port names, directions, widths, and descriptions are extracted directly from the RTL implementation.

Note: This documents `stream_core.sv` which is instantiated inside `stream_top_ch8.sv`. The top-level wrapper adds: - APB slave interface (replaces `apb_valid/ready/addr` with standard APB protocol) - AXI-Lite interfaces for monitor bus (`s_axil_err_`, `m_axil_mon_`) - Optional APB CDC crossing (`CDC_ENABLE` parameter)

For top-level integration, see the [Architecture Overview](#).

Quick Navigation: - [Clock and Reset](#) - [APB Programming Interface](#) - [Configuration Interface](#) - [Status Interface](#) - [Performance Profiler Interface](#) - [AXI4 Master - Descriptor Fetch](#) - [AXI4 Master - Data Read](#) - [AXI4 Master - Data Write](#) - [Status/Debug Outputs](#) - [Unified Monitor Bus](#)

4.2 Clock and Reset

Clock and Reset Signals

Signal	Direction	Width	Description
<code>clk</code>	input	1	System clock (100-500 MHz typical)
<code>rst_n</code>	input	1	Active-low asynchronous reset

Notes: - All STREAM logic operates in the `clk` domain - Reset is asynchronous assert, synchronous deassert - All registers use reset macros from `reset_defs.svh`

4.3 APB Programming Interface

Per-channel descriptor kick-off interface. Each channel has independent valid/ready/address signals for descriptor chain start.

APB Programming Interface Signals

Signal	Direction	Width	Description
apb_valid[ch]	input	NUM_CHAN NELS	Channel descriptor address valid
apb_ready[ch]	output	NUM_CHAN NELS	Channel ready to accept descriptor address
apb_addr[ch]	input	NUM_CHAN NELS × ADDR_WIDT H	Descriptor address per channel (64-bit default)

Usage Pattern:

```
// Kick off channel 0 with descriptor at 0x1000_0000
apb_valid[0] = 1'b1;
apb_addr[0] = 64'h0000_0000_1000_0000;
// Wait for handshake
wait (apb_ready[0]);
apb_valid[0] = 1'b0;
```

Notes: - Standard valid/ready handshake protocol - Descriptor address must be aligned to descriptor size (32 bytes) - Channel starts operation immediately after handshake - Default: NUM_CHANNELS = 8, ADDR_WIDTH = 64

4.4 Configuration Interface

4.4.1 Per-Channel Configuration

Per-Channel Configuration Signals

Signal	Direction	Width	Description
cfg_channel_enable[ch]	input	NUM_CHAN NELS	Enable channel (0=disabled, 1=enabled)
cfg_channel_reset[ch]	input	NUM_CHAN NELS	Soft reset channel (FSM → IDLE state)

Notes: - Channel must be enabled before accepting descriptors - Soft reset clears channel FSM but preserves config

4.4.2 Global Scheduler Configuration

Global Scheduler Configuration Signals

Signal	Direction	Width	Description
cfg_sched_enable	input	1	Global scheduler enable
cfg_sched_timeout_cycles	input	16	Timeout threshold (clock cycles)
cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_sched_error_enable	input	1	Enable error event reporting
cfg_sched_completion_enable	input	1	Enable completion event reporting
cfg_sched_performance_enable	input	1	Enable performance event reporting

Notes: - cfg_sched_enable is master enable for all schedulers - Timeout measured from descriptor fetch to completion - Event enables control MonBus traffic

4.4.3 Descriptor Engine Configuration

Descriptor Engine Configuration Signals

Signal	Direction	Width	Description
cfg_desceng_enable	input	1	Enable descriptor engine
cfg_desceng_prefetch	input	1	Enable descriptor prefetch
cfg_desceng_fifo_thresh	input	4	FIFO threshold for prefetch trigger
cfg_desceng_addr0_base	input	ADDR_WIDTH H	Address range 0 base (protection)
cfg_desceng_addr0_limit	input	ADDR_WIDTH H	Address range 0 limit (protection)
cfg_desceng_addr1_base	input	ADDR_WIDTH H	Address range 1 base (protection)
cfg_desceng_addr1_limit	input	ADDR_WIDTH H	Address range 1 limit (protection)

Notes: - Descriptor engine shared across all channels - Prefetch improves latency for chained descriptors - Address ranges provide optional descriptor memory protection

4.4.4 AXI Monitor Configuration

Three identical sets of monitor config signals for descriptor, read, and write AXI masters:

AXI Monitor Signal Prefixes

Signal Prefix	Applies To	Description
cfg_desc_mon_*	Descriptor AXI	Descriptor fetch monitoring
cfg_rdeng_mon_*	Read AXI	Data read monitoring
cfg_wreng_mon_*	Write AXI	Data write monitoring

Each monitor has the following configuration signals:

AXI Monitor Configuration Signal Suffixes

Signal Suffix	Width	Description
_enable	1	Enable monitor
_err_enable	1	Enable error packet reporting
_perf_enable	1	Enable performance packet reporting
_timeout_enable	1	Enable timeout detection
_timeout_cycles	32	Timeout threshold (cycles)
_latency_thresh	32	Latency threshold for events
_pkt_mask	16	Packet type mask (filter events)
_err_select	4	Error type selector
_err_mask	8	Error event mask
_timeout_mask	8	Timeout event mask
_compl_mask	8	Completion event

Signal Suffix	Width	Description
		mask
_thresh_mask	8	Threshold event mask
_perf_mask	8	Performance event mask
_addr_mask	8	Address event mask
_debug_mask	8	Debug event mask

Example Signals:

```

cfg_desc_mon_enable           // Descriptor monitor enable
cfg_desc_mon_timeout_cycles  // Descriptor timeout threshold
cfg_rdeng_mon_err_enable     // Read engine error reporting enable
cfg_wreng_mon_perf_enable    // Write engine perf reporting enable

```

Notes: - Monitors generate MonBus packets for events - Masks control which event types are reported - Timeout measured from AR/AW valid to last R/B response

4.4.5 AXI Transfer Configuration

AXI Transfer Configuration Signals

Signal	Direction	Width	Description
cfg_axi_rd_xfer_beats	input	8	Read burst size (beats, 1-256)
cfg_axi_wr_xfer_beats	input	8	Write burst size (beats, 1-256)

Notes: - Configures AXI burst length for read/write engines - Larger bursts improve bandwidth but increase latency - Typical: 16 beats for DDR4, 8 beats for low-latency SRAM

4.4.6 Performance Profiler Configuration

Performance Profiler Configuration Signals

Signal	Direction	Width	Description
cfg_perf_enable	input	1	Enable profiler
cfg_perf_mode	input	1	Profiler mode (0=timestamp, 1=elapsed)
cfg_perf_clear	input	1	Clear profiler

Signal	Direction	Width	Description
r			counters and FIFO

Notes: - Timestamp mode: Records start/end timestamps (software calculates elapsed) - Elapsed mode: Hardware calculates elapsed time directly - Clear is a single-cycle pulse

4.5 Status Interface

4.5.1 Per-Channel Status

Per-Channel Status Signals

Signal	Direction	Width	Description
descriptor_engine_idle[ch]	output	NUM_CHAN NELS	Descriptor engine idle (no pending fetch)
scheduler_idle[ch]	output	NUM_CHAN NELS	Scheduler in IDLE state (waiting for descriptor)
scheduler_state[ch]	output	NUM_CHAN NELS × 7	Scheduler FSM state (ONE-HOT encoding)
sched_error[ch]	output	NUM_CHAN NELS	Scheduler error flag (sticky, cleared by reset)
axi_rd_all_complete[ch]	output	NUM_CHAN NELS	All read transactions complete for channel
axi_wr_all_complete[ch]	output	NUM_CHAN NELS	All write transactions complete for channel

Scheduler State Encoding (ONE-HOT):

```

7'b00000001 // CH_IDLE - Waiting for descriptor
7'b00000010 // CH_FETCH_DESC - Fetching descriptor
7'b00000100 // CH_XFER_DATA - Transfer in progress
7'b00010000 // CH_COMPLETE - Transfer complete
7'b00100000 // CH_NEXT_DESC - Chaining to next descriptor
7'b01000000 // CH_ERROR - Error occurred
7'b10000000 // (Reserved)

```

Transfer Complete Condition:

```
channel_complete = scheduler_idle[ch] &&
                    axi_rd_all_complete[ch] &&
                    axi_wr_all_complete[ch];
```

4.6 Performance Profiler Interface

Performance Profiler Interface Signals

Signal	Direction	Width	Description
perf_fifo_empty	output	1	Profiler FIFO empty flag
perf_fifo_full	output	1	Profiler FIFO full flag
perf_fifo_occupant	output	16	Profiler FIFO occupancy (0-256)
perf_fifo_read	input	1	Read profiler entry (pop FIFO)
perf_fifo_data_low	output	32	Profiler data [31:0] (timestamp or elapsed)
perf_fifo_data_high	output	32	Profiler data [63:32] (metadata)

FIFO Entry Format:

Low Word [31:0]: - Timestamp mode: Timestamp in clock cycles - Elapsed mode: Elapsed time in clock cycles

High Word [31:0]: - Bits [31:4]: Reserved (0) - Bit [3]: Event type (0=start, 1=end) - Bits [2:0]: Channel ID (0-7)

Read Sequence:

```
// Check FIFO not empty
if (!perf_fifo_empty) begin
    // Read 36-bit entry (two registers)
    perf_fifo_rd = 1'b1; // Pulse to pop FIFO
    @(posedge clk);
    perf_fifo_rd = 1'b0;

    // Sample data on next cycle
    timestamp = perf_fifo_data_low;
    metadata = perf_fifo_data_high;
    channel_id = metadata[2:0];
```

```

        event_type = metadata[3];
end

```

4.7 AXI4 Master - Descriptor Fetch (256-bit)

Fixed 256-bit width AXI4 master for descriptor fetch from memory.

4.7.1 AR Channel (Read Address)

Descriptor AXI Master AR Channel Signals

Signal	Direction	Width	Description
m_axi_desc_ar_id	output	AXI_ID_WIDTH	Transaction ID
m_axi_desc_ar_addr	output	ADDR_WIDTH	Read address
m_axi_desc_ar_len	output	8	Burst length - 1 (AXI encoding)
m_axi_desc_ar_size	output	3	Burst size = log2(bytes per beat)
m_axi_desc_ar_burst	output	2	Burst type (01=INCR)
m_axi_desc_ar_lock	output	1	Lock type (0=normal)
m_axi_desc_ar_cache	output	4	Cache attributes
m_axi_desc_ar_prot	output	3	Protection attributes
m_axi_desc_ar_qos	output	4	QoS value
m_axi_desc_ar_region	output	4	Region identifier
m_axi_desc_ar_user	output	CHAN_WIDTH	User signal (channel ID)
m_axi_desc_ar_valid	output	1	Address valid
m_axi_desc_ar	input	1	Address ready

Signal	Direction	Width	Description
ready			

4.7.2 R Channel (Read Data)

R Channel

Signal	Direction	Width	Description
m_axi_desc_r id	input	AXI_ID_WIDTH	Transaction ID
m_axi_desc_r data	input	256	Read data (FIXED 256-bit descriptor)
m_axi_desc_r resp	input	2	Response (00=OKAY, 01=EXOKAY, 10=SLVERR, 11=DECERR)
m_axi_desc_r last	input	1	Last beat of burst
m_axi_desc_r user	input	CHAN_WIDTH	User signal (channel ID)
m_axi_desc_r valid	input	1	Read data valid
m_axi_desc_r ready	output	1	Read data ready

Notes: - Data width is FIXED at 256 bits (descriptor size) - Typical burst: 1 beat (arlen=0) for single descriptor fetch - arsize = 3'b101 (32 bytes = 2⁵) - ID encodes channel number for response routing

4.8 AXI4 Master - Data Read (Parameterizable Width)

Parameterizable width AXI4 master for reading source data from memory. Default 512-bit.

4.8.1 AR Channel (Read Address)

AR Channel

Signal	Direction	Width	Description
m_axi_rd_arid	output	AXI_ID_WIDTH	Transaction ID
m_axi_rd_araddr	output	ADDR_WIDTH	Read address
m_axi_rd_arlen	output	8	Burst length - 1

Signal	Direction	Width	Description
n			
m_axi_rd_arsize	output	3	Burst size = log2(bytes per beat)
m_axi_rd_arburst	output	2	Burst type (01=INCR)
m_axi_rd_arlock	output	1	Lock type (0=normal)
m_axi_rd_arcache	output	4	Cache attributes
m_axi_rd_arprot	output	3	Protection attributes
m_axi_rd_arqos	output	4	QoS value
m_axi_rd_arregion	output	4	Region identifier
m_axi_rd_aruser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_rd_arvalid	output	1	Address valid
m_axi_rd_arready	input	1	Address ready

4.8.2 R Channel (Read Data)

R Channel

Signal	Direction	Width	Description
m_axi_rd_rid	input	AXI_ID_WIDTH	Transaction ID
m_axi_rd_rdata	input	DATA_WIDTH	Read data (default 512-bit)
m_axi_rd_rresp	input	2	Response
m_axi_rd_rlast	input	1	Last beat of burst
m_axi_rd_ruser	input	CHAN_WIDTH	User signal

Signal	Direction	Width	Description
r			(channel ID)
m_axi_rd_rval id	input	1	Read data valid
m_axi_rd_rrea dy	output	1	Read data ready

Notes: - Data width configurable via DATA_WIDTH parameter (default 512) - Burst length configured via cfg_axi_rd_xfer_beats (default 16) - arsize = log2(DATA_WIDTH/8) automatically calculated - AXI skid buffers on external interface for timing closure

4.9 AXI4 Master - Data Write (Parameterizable Width)

Parameterizable width AXI4 master for writing destination data to memory. Default 512-bit.

4.9.1 AW Channel (Write Address)

AW Channel

Signal	Direction	Width	Description
m_axi_wr_awid	output	AXI_ID_WIDTH	Transaction ID
m_axi_wr_awad dr	output	ADDR_WIDTH	Write address
m_axi_wr_awle n	output	8	Burst length - 1
m_axi_wr_aws ize	output	3	Burst size = log2(bytes per beat)
m_axi_wr_awbu rst	output	2	Burst type (01=INCR)
m_axi_wr_awlo ck	output	1	Lock type (0=normal)
m_axi_wr_awca che	output	4	Cache attributes
m_axi_wr_awpr ot	output	3	Protection attributes
m_axi_wr_awqo s	output	4	QoS value

Signal	Direction	Width	Description
m_axi_wr_awregion	output	4	Region identifier
m_axi_wr_awuser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_wr_awvalid	output	1	Address valid
m_axi_wr_awready	input	1	Address ready

4.9.2 W Channel (Write Data)

W Channel

Signal	Direction	Width	Description
m_axi_wr_wdata	output	DATA_WIDTH	Write data (default 512-bit)
m_axi_wr_wstrobe	output	DATA_WIDTH/8	Write strobes (byte enables, all 1s)
m_axi_wr_wlast	output	1	Last beat of burst
m_axi_wr_wuser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_wr_wvalid	output	1	Write data valid
m_axi_wr_wready	input	1	Write data ready

4.9.3 B Channel (Write Response)

B Channel

Signal	Direction	Width	Description
m_axi_wr_bid	input	AXI_ID_WIDTH	Transaction ID
m_axi_wr_bresp	input	2	Response (00=OKAY, 01=EXOKAY, 10=SLVERR, 11=DECERR)
m_axi_wr_buser	input	CHAN_WIDTH	User signal (channel ID)

Signal	Direction	Width	Description
m_axi_wr_bvalid	input	1	Response valid
m_axi_wr_bready	output	1	Response ready

Notes: - Data width configurable via DATA_WIDTH parameter (default 512) - Burst length configured via cfg_axi_wr_xfer_beats (default 16) - awsize = log2(DATA_WIDTH/8) automatically calculated - wstrb typically all 1s (full data width writes) - AXI skid buffers on external interface for timing closure

4.10 Status/Debug Outputs

4.10.1 Descriptor AXI Monitor Status

Descriptor AXI Monitor Status

Signal	Direction	Width	Description
cfg_sts_desc_mon_busy	output	1	Monitor busy processing transaction
cfg_sts_desc_mon_active_txns	output	8	Active transaction count (0-255)
cfg_sts_desc_mon_error_count	output	16	Cumulative error count
cfg_sts_desc_mon_txn_count	output	32	Total transaction count
cfg_sts_desc_mon_conflict_error	output	1	ID conflict detected (sticky)

4.10.2 Read Engine AXI Monitor Status

Read Engine AXI Monitor Status

Signal	Direction	Width	Description
cfg_sts_rden_g_skid_busy	output	1	Skid buffer busy (not empty)
cfg_sts_rden_g_mon_active_txns	output	8	Active transaction count

Signal	Direction	Width	Description
cfg_sts_rden g_mon_error_ count	output	16	Cumulative error count
cfg_sts_rden g_mon_txn_co unt	output	32	Total transaction count
cfg_sts_rden g_mon_confli ct_error	output	1	ID conflict detected (sticky)

4.10.3 Write Engine AXI Monitor Status

Write Engine AXI Monitor Status

Signal	Direction	Width	Description
cfg_sts_wren g_skid_busy	output	1	Skid buffer busy (not empty)
cfg_sts_wren g_mon_active _txns	output	8	Active transaction count
cfg_sts_wren g_mon_error_ count	output	16	Cumulative error count
cfg_sts_wren g_mon_txn_co unt	output	32	Total transaction count
cfg_sts_wren g_mon_confli ct_error	output	1	ID conflict detected (sticky)

Notes: - Status signals for debug and performance analysis - Transaction counts never roll over (use for profiling) - Error counts increment on any AXI error response - ID conflicts indicate internal RTL bug (should never occur)

4.11 Unified Monitor Bus Interface

Single output interface for all STREAM monitoring events.

Unified Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid

Signal	Direction	Width	Description
mon_ready	input	1	Monitor packet ready (from downstream consumer)
mon_packet	output	64	Monitor packet data (64-bit standard format)

MonBus Packet Format (64-bit):

[63:56] - Packet type (event code)
 [55:48] - Agent ID (source identifier)
 [47:40] - Unit ID (1 for STREAM)
 [39:32] - Channel ID (0-7)
 [31:0] - Event-specific data

MonBus Sources: - Descriptor engines: 8 sources, agent IDs 16-23 (0x10-0x17) - Schedulers: 8 sources, agent IDs 48-55 (0x30-0x37) - Descriptor AXI monitor: agent ID 8 (0x08) - Read AXI monitor: configurable agent ID - Write AXI monitor: configurable agent ID

Downstream Integration:

```

// Connect to MonBus FIFO for buffering
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(256)
) u_mon_fifo (
    .i_clk      (clk),
    .i_rst_n    (rst_n),
    .i_data     (mon_packet),
    .i_valid    (mon_valid),
    .o_ready    (mon_ready),
    // ... downstream connection
);

```

Notes: - Standard AMBA monitor bus protocol - Always buffer with FIFO to prevent backpressure to STREAM - Packet format documented in rtl/amba/includes/monitor_pkg.sv - Event codes defined in STREAM package

4.12 Port Count Summary

Port Count Summary

Interface Type	Input Ports	Output Ports	Bidirectional
Clock/Reset	2	0	0

Interface Type	Input Ports	Output Ports	Bidirectional
APB	NUM_CHANNE	NUM_CHANNE	0
Programming	LS × (1 + ADDR_WIDTH)	LS	
Configuration	~150	0	0
Status	1 (perf_fifo_rd)	NUM_CHANNE LS × 10 + 30	0
Perf Profiler	1	4	0
AXI Desc Master	7	14	0
AXI Read Master	7	14	0
AXI Write Master	10	19	0
MonBus	1	2	0

Approximate Total: ~350 ports (varies with NUM_CHANNELS and ADDR_WIDTH/DATA_WIDTH)

4.13 Default Parameter Values

Default Parameter Values

Parameter	Default	Description
NUM_CHANNELS	8	Number of DMA channels
CHAN_WIDTH	3	Channel ID width (log2(NUM_CHANNE LS))
ADDR_WIDTH	64	Address bus width
DATA_WIDTH	512	Data bus width
AXI_ID_WIDTH	8	AXI transaction ID width
FIFO_DEPTH	512	Per-channel FIFO depth
AR_MAX_OUTSTANDING	8	Max concurrent read

Parameter	Default	Description
AW_MAX_OUTSTANDING	8	requests Max concurrent write requests

4.14 Related Documentation

- [Stream Core Block Spec](#) - Detailed block documentation
- [Clocks and Reset](#) - Timing specifications
- [Architecture](#) - System architecture overview

Last Updated: 2025-12-01 Maintained By: STREAM Architecture Team —

4.15 Revision History

STREAM Top-Level Port List Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

5 Clocks and Reset Specification

Chapter: 01 Version: 0.90 Last Updated: 2025-11-22

5.1 Overview

STREAM operates in a single clock domain with a single asynchronous active-low reset. This chapter defines clock requirements, reset behavior, and timing constraints for the STREAM subsystem.

5.2 Clock Domain

5.2.1 Primary Clock: `aclk`

Specification: - **Name:** `aclk` (AXI clock) - **Type:** Synchronous, single-edge (rising edge) - **Frequency:** Configurable (typical: 100 MHz - 500 MHz) - **Duty Cycle:** 50% 5% - **Jitter:** < 100 ps peak-to-peak

Usage: - All STREAM internal logic - All AXI master interfaces - All AXIL interfaces - MonBus output - SRAM

5.2.2 Secondary Clock: `pclk` (APB Clock)

Specification: - **Name:** `pclk` (Peripheral clock) - **Type:** Synchronous, single-edge (rising edge) - **Frequency:** Configurable (typical: 50 MHz - 200 MHz) - **Relation to `aclk`:** May be asynchronous

Usage: - APB configuration interface only

Clock Domain Crossing (CDC): - If `pclk` `aclk`: CDC logic required in `apb_config.sv` - If `pclk` = `aclk`: Direct connection (no CDC)

CDC Implementation: - Use `apb_slave_cdc` wrapper (like HPET example) - `apb_slave_cdc` implements **handshake-based CDC** using `cdc_handshake` modules - One `cdc_handshake` for command interface (APB → core) - One `cdc_handshake` for response interface (core → APB) - Full req/ack handshake protocol (NOT async FIFO) - Works across all frequency ratios (slow-to-fast, fast-to-slow, any ratio)

5.3 Reset

5.3.1 Primary Reset: `aresetn`

Specification: - **Name:** `aresetn` (AXI reset, active-low) - **Type:** Asynchronous assertion, synchronous deassertion - **Polarity:** Active-low (0 = reset, 1 = normal operation) - **Assertion:** Asynchronous (can occur at any time) - **Deassertion:** Synchronous to `aclk` rising edge - **Duration:** Minimum 10 `aclk` cycles

Reset Behavior:

```

// Standard reset pattern for all STREAM modules
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        // Asynchronous reset assertion
        r_state <= IDLE;
        r_counter <= '0;
        r_valid <= 1'b0;
        // ... all registers to known state
    end else begin
        // Synchronous operation
        r_state <= w_next_state;
        // ... normal logic
    end
end

```

5.3.2 Secondary Reset: presetn

Specification: - **Name:** presetn (APB reset, active-low) - **Type:** Asynchronous assertion, synchronous deassertion - **Polarity:** Active-low (0 = reset, 1 = normal operation) - **Synchronization:** May be asynchronous to aresetn

Usage: - APB configuration interface only - Typically tied to aresetn if pclk = aclk

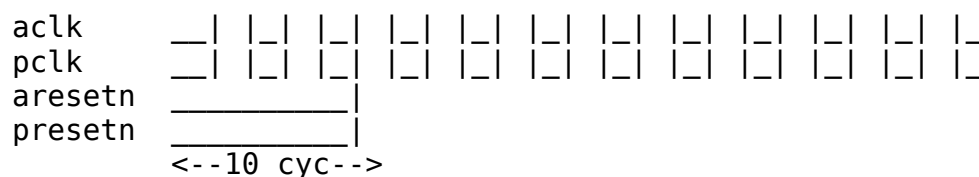
5.4 Reset Sequencing

5.4.1 Power-On Reset

Recommended sequence:

1. Assert aresetn (LOW)
2. Assert presetn (LOW)
3. Apply stable clocks (aclk, pclk)
4. Wait 10 aclk cycles
5. Deassert presetn (HIGH) on pclk rising edge
6. Deassert aresetn (HIGH) on aclk rising edge
7. Wait 5 aclk cycles for stabilization
8. Begin APB configuration

Timing diagram:



5.4.2 Functional Reset

Software-initiated reset (per channel):

```
// Reset specific channel via APB
write_apb(ADDR_GLOBAL_CTRL, CHANNEL_0_RESET); // Auto-clears after 1
cycle

// Hardware response:
// - Channel FSM returns to IDLE
// - Channel registers cleared
// - Outstanding transactions flushed
// - MonBus error packet generated (if mid-transfer)
```

5.4.3 Reset Recovery

After reset deassertion:

Reset Recovery

Cycle	Event
0	aresetn deasserted (rising edge)
1-5	Internal state stabilization
6+	Ready for APB configuration
10+	Ready for descriptor transfers

5.5 Clock Requirements by Module

5.5.1 Functional Unit Blocks (FUB)

Functional Unit Blocks

Module	Clock	Reset	Frequency	Notes
descriptor_engine	aclk	aresetn	100-500 MHz	AXI master timing
scheduler	aclk	aresetn	100-500 MHz	Single cycle FSM
axi_read_engine	aclk	aresetn	100-500 MHz	AXI master timing
axi_write_engine	aclk	aresetn	100-500 MHz	AXI master timing

Module	Clock	Reset	Frequency	Notes
simple_sram	acclk	aresetn	100-500 MHz	Synchronous SRAM

5.5.2 Integration Blocks (MAC)

Integration Blocks

Module	Clock(s)	Reset(s)	Frequency	Notes
channel_arbiter	acclk	aresetn	100-500 MHz	Single cycle arbitration
apb_config	pclk, (acclk)	presetn, (aresetn)	50-200 MHz (APB)	CDC if async
monbus_axil_group	acclk	aresetn	100-500 MHz	AXIL timing
stream_top	acclk, pclk	aresetn, presetn	Mixed	Top-level

5.6 Timing Constraints

5.6.1 Setup and Hold Times

Internal registers (relative to acclk): - Setup time: 0.5 ns (typical) - Hold time: 0.1 ns (typical) - Clock-to-Q: 0.3 ns (typical)

External interfaces: - AXI/AXIL: Per ARM IHI0022E specification - APB: Per ARM IHI0024C specification

5.6.2 Critical Paths

Identified critical paths:

1. **Arbiter -> Scheduler grant:**
 - Latency: 1 cycle
 - Path: Priority encoder -> One-hot grant
2. **AXI read -> SRAM write:**
 - Latency: 1 cycle
 - Path: R data -> SRAM write port

3. SRAM read -> AXI write:

- Latency: 1 cycle
- Path: SRAM read port -> W data

Maximum frequency estimation: - Typical FPGA (Xilinx 7-series): 250 MHz - High-end FPGA (UltraScale+): 400 MHz - ASIC (28nm): 500 MHz

5.7 Clock Domain Crossing (CDC)

5.7.1 APB Configuration CDC

When required: pclk aclk (asynchronous APB interface)

CDC Implementation:

```
// APB to STREAM domain (pclk -> aclk)
apb_slave_cdc #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .SYNC_STAGES(2) // Dual-flop synchronizer
) u_apb_cdc (
    // APB side (pclk domain)
    .s_pclk(pclk),
    .s_presetn(presetn),
    .s_paddr(paddr),
    .s_pwrite(pwrite),
    .s_pwdata(pwdata),
    .s_prdata(prdata),

    // STREAM side (aclk domain)
    .m_pclk(aclk),
    .m_presetn(aresetn),
    .m_paddr(paddr_sync),
    .m_pwrite(pwrite_sync),
    .m_pwdata(pwdata_sync),
    .m_prdata(prdata_sync)
);
```

CDC Mechanism: - apb_slave_cdc uses **handshake-based CDC** via cdc_handshake modules (NOT async FIFOs) - **Command path** (pclk → aclk): APB write/read commands cross via req/ack handshake - **Response path** (aclk → pclk): APB read data crosses back via req/ack handshake - **Internal synchronizers:** Dual-flop synchronizers (2-3 stages) for handshake signals - **ASYNC_REG attribute:** Applied to synchronizer stages for timing tools - **Timing constraints:** Proper constraints required in SDC/XDC for synchronizer paths

Handshake Protocol Benefits: - Works across **all frequency ratios** (slow-to-fast, fast-to-slow, arbitrary ratios) - Guaranteed data integrity (req/ack ensures data stability before sampling) - No FIFO depth management or gray code pointer complexity - Latency: 4-6 APB clock cycles for register access (handshake round-trip)

5.7.2 No CDC Required

Single clock domain: If pclk = aclk and presetn = aresetn:

```
// Direct connection (no CDC wrapper)
apb_config #(
    .NUM_CHANNELS(8)
) u_apb_config (
    .pclk(aclk),           // Same clock
    .presetn(aresetn),    // Same reset
    // ... direct APB signals
);
```

5.8 Reset State Initialization

5.8.1 Register Reset Values

All STREAM modules must initialize to known state on reset:

```
// Descriptor Engine
if (!aresetn) begin
    r_desc_fifo_wr_ptr <= '0;
    r_desc_valid <= 1'b0;
    r_desc_error <= 1'b0;
end

// Scheduler
if (!aresetn) begin
    r_current_state <= CH_IDLE;
    r_read_beats_remaining <= '0;
    r_write_beats_remaining <= '0;
    r_timeout_counter <= '0;
end

// AXI Engines
if (!aresetn) begin
    r_burst_counter <= '0;
    m_axi_arvalid <= 1'b0;
    m_axi_awvalid <= 1'b0;
end
```

```
// Arbiter
if (!aresetn) begin
    r_last_grant_id <= '0;
    r_grant_valid <= 1'b0;
end
```

5.8.2 SRAM Reset

SRAM contents: Undefined after reset (no initialization required)

SRAM pointers:

```
if (!aresetn) begin
    wr_ptr <= '0;
    rd_ptr <= '0;
end
```

5.9 Clock Gating (Optional)

For power optimization in ASIC implementations:

5.9.1 Per-Channel Clock Gating

```
// Clock gate when channel idle
clock_gate_ctrl u_ch0_clk_gate (
    .clk_in(aclk),
    .enable(ch0_enable),
    .clk_out(ch0_gated_clk)
);

// Use gated clock for channel logic
scheduler #(.CHANNEL_ID(0)) u_ch0_sched (
    .aclk(ch0_gated_clk), // Gated clock
    .aresetn(aresetn),
    // ...
);
```

Note: Clock gating typically not used in FPGA implementations (tutorial focus).

5.10 Verification Requirements

5.10.1 Clock Checks

Testbench must verify: - [Done] Clock period consistent - [Done] Clock duty cycle 50% tolerance - [Done] No glitches on clock - [Done] Setup/hold times met

5.10.2 Reset Checks

Testbench must verify: - [Done] All registers initialize to known state - [Done] Reset assertion clears FSMs to IDLE - [Done] Reset deassertion synchronous to clock - [Done] Minimum reset duration (10 cycles) enforced - [Done] Operations don't start until stabilization complete

5.10.3 CDC Checks

For APB CDC (if present): - [Done] No metastability violations - [Done] Data integrity across domains - [Done] Proper flag synchronization

5.11 Example Reset Testbench

```
# CocoTB testbench pattern
class StreamTB(TBBase):
    async def setup_clocks_and_reset(self):
        """Complete clock and reset initialization"""
        # Start clocks
        await self.start_clock('aclk', freq=10, units='ns') # 100 MHz
        await self.start_clock('pclk', freq=20, units='ns') # 50 MHz
    (async)

    # Assert reset
    await self.assert_reset()

    # Hold reset for 10 aclk cycles
    await self.wait_clocks('aclk', 10)

    # Deassert reset (synchronous to aclk)
    await self.deassert_reset()

    # Stabilization period
    await self.wait_clocks('aclk', 5)

    # Ready for operation

    async def assert_reset(self):
```

```

    """Assert both resets"""
    self.dut.aresetn.value = 0
    self.dut.presetn.value = 0

    async def deassert_reset(self):
        """Deassert both resets synchronously"""
        # Wait for rising edge of aclk
        await RisingEdge(self.dut.aclk)
        self.dut.aresetn.value = 1

        # Wait for rising edge of pclk
        await RisingEdge(self.dut.pclk)
        self.dut.presetn.value = 1

```

5.12 Related Documentation

- **Scheduler FSM:** [fub_02_scheduler.md](#) - Reset behavior
 - **APB Config:** [mac_02_apb_config.md](#) - CDC implementation
 - **Top-Level:** [mac_04_stream_top.md](#) - Clock/reset integration
-

5.13 Revision History

Clocks and Reset Specification Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice · [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 STREAM Core Specification

Module: stream_core.sv **Location:** projects/components/dmas/stream/rtl/macro/
Status: Implemented **Last Updated:** 2026-05-11

6.1 Overview

The STREAM Core is the top-level integration module that combines all STREAM components into a complete scatter-gather DMA engine. It provides 8 independent channels with descriptor-based memory-to-memory transfers.

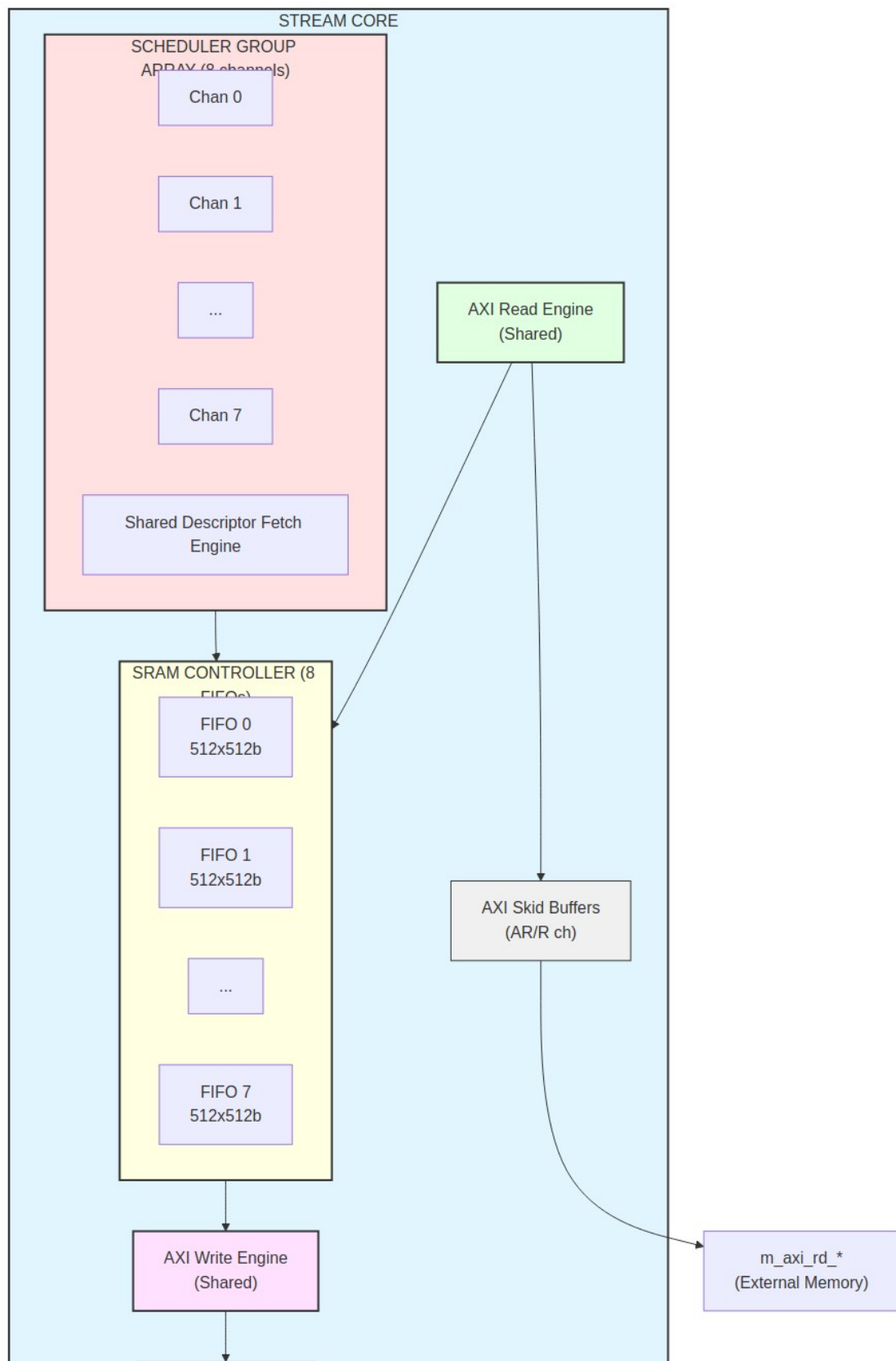
6.1.1 Key Features

- **8 Independent Channels:** Concurrent descriptor-based transfers
- **Shared AXI Masters:** Three shared AXI4 masters for efficiency
 - Descriptor fetch (256-bit fixed)
 - Data read (parameterizable, default 512-bit)
 - Data write (parameterizable, default 512-bit)
- **Per-Channel Buffering:** Independent FIFO per channel (512 entries default)
- **Performance Monitoring:** Integrated profiler with FIFO readout
- **AXI Skid Buffers:** Timing closure on all external interfaces
- **Unified MonBus:** Single monitor bus output for all events

6.1.2 Block Diagram

The STREAM Core integrates the following major components:

6.1.3 Figure 2.1.1: STREAM Core Block Diagram



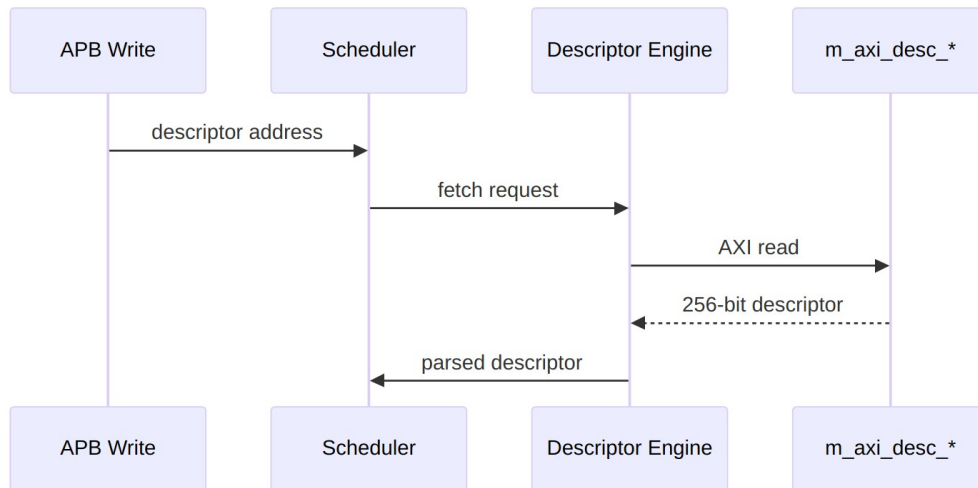
6.2 Architecture

6.2.1 Component Hierarchy

1. **scheduler_group_array** - Top scheduler layer
 - 8 × scheduler_group instances
 - Shared descriptor engine
 - Descriptor fetch arbitration
2. **sram_controller** - Buffering layer
 - 8 × independent FIFOs (gaxi_fifo_sync)
 - Per-channel allocation controllers
 - Per-channel drain controllers
3. **axi_read_engine** - Read datapath
 - Shared AXI master for all channels
 - ID-based routing to SRAM
 - Space allocation flow control
4. **axi_write_engine** - Write datapath
 - Shared AXI master for all channels
 - ID-based routing from SRAM
 - Drain reservation flow control
5. **perf_profiler** - Performance monitoring
 - Transaction counting
 - Bandwidth tracking
 - FIFO readout interface
6. **AXI Skid Buffers** - Timing closure
 - Descriptor AXI (AR/R)
 - Read AXI (AR/R)
 - Write AXI (AW/W/B)

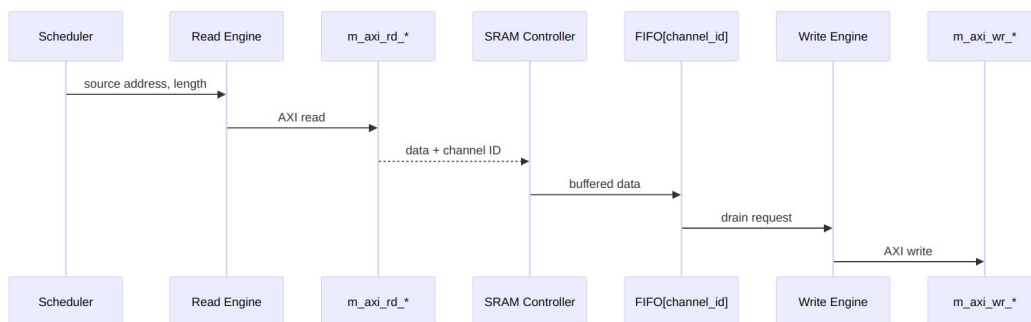
6.2.2 Data Flow

Descriptor Fetch Flow:



.1.2: Descriptor Fetch Flow

Data Transfer Flow:



.1.3: Data Transfer Flow

6.3 Parameters

6.3.1 Primary Configuration

Primary Configuration

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of independent DMA channels
CHAN_WIDTH	int	$\text{\$clog2}(\text{NUM_CHANNELS})$	Channel ID width (3 for 8 channels)
ADDR_WIDTH	int	64	Address bus width

Parameter	Type	Default	Description
DATA_WIDTH	int	512	Data bus width (must match memory interface)
AXI_ID_WIDTH	int	8	AXI transaction ID width
FIFO_DEPTH	int	512	Per-channel FIFO depth

6.3.2 Monitor Control

Monitor Control

Parameter	Type	Default	Description
USE_AXI_MONITORS	int	1	Enable (1) or disable (0) AXI transaction monitors

Note: USE_AXI_MONITORS gates only the **heavy** monitor logic. When USE_AXI_MONITORS = 0: - The axi4_master_*_mon transaction monitors (CAM + reporter cones) and the per-transaction latency histograms are gated off; their configuration inputs are tied off internally and MonBus output is present but inactive. - The **datapath performance meters stay active**: the in-core axi_bus_meter buckets (RDMON/WRMON_PERF productive / backpressure / starvation / idle) and the beat / byte / burst / window counters are always-on, and the perf-window RUN bits pass through. A production (monitors-off) build therefore still measures datapath utilisation, with no CAM / reporter / histogram area. - Reduces resource usage for production systems.

6.3.3 Outstanding Transaction Limits

Outstanding Transaction Limits

Parameter	Default	Description
AR_MAX_OUTSTANDING	8	Maximum concurrent read address requests
AW_MAX_OUTSTANDING	8	Maximum concurrent write address requests

6.3.4 AXI Skid Buffer Depths

AXI Skid Buffer Depths

Parameter	Default	Purpose
SKID_DEPTH_AR	2	AR channel timing closure

Parameter	Default	Purpose
SKID_DEPTH_R	4	R channel timing closure
SKID_DEPTH_AW	2	AW channel timing closure
SKID_DEPTH_W	4	W channel timing closure
SKID_DEPTH_B	2	B channel timing closure

Note: Deeper buffers on data channels (R/W) improve throughput.

6.3.5 MonBus Agent IDs

MonBus Agent IDs

Parameter	Default	Description
DESC_MON_BASE_AGENT_ID	16 (0x10)	Descriptor engines base (16-23)
SCHED_MON_BASE_AGENT_ID	48 (0x30)	Schedulers base (48-55)
DESC_AXI_MON_AGENT_ID	8 (0x08)	Descriptor AXI master monitor
MON_UNIT_ID	1 (0x1)	Unit ID for all STREAM events

6.4 Port List

6.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

6.4.2 APB Programming Interface

Per-channel descriptor kick-off interface:

APB Programming Interface

Signal	Direction	Width	Description
apb_valid[ch]	input	NUM_CHAN NELS	Channel descriptor address valid
apb_ready[ch]	output	NUM_CHAN NELS	Channel ready to accept descriptor address
apb_addr[ch]	input	NUM_CHAN NELS × ADDR_WIDT H	Descriptor address per channel

Usage:

```
// Kick off channel 0 with descriptor at 0x1000_0000
apb_valid[0] = 1'b1;
apb_addr[0] = 64'h0000_0000_1000_0000;
// Wait for handshake
wait (apb_ready[0]);
apb_valid[0] = 1'b0;
```

6.4.3 Configuration Interface

Per-Channel Configuration:

Configuration Interface

Signal	Direction	Width	Description
cfg_channel_enable[ch]	input	NUM_CHAN NELS	Enable channel
cfg_channel_reset[ch]	input	NUM_CHAN NELS	Soft reset channel (FSM → IDLE)

Global Scheduler Configuration:

Configuration Interface

Signal	Direction	Width	Description
cfg_sched_enable	input	1	Global scheduler enable
cfg_sched_timeout_cycles	input	16	Timeout threshold (cycles)
cfg_sched_timeout_detection	input	1	Enable timeout detection

Signal	Direction	Width	Description
meout_enable			
cfg_sched_err_enable	input	1	Enable error event reporting
cfg_sched_completion_enable	input	1	Enable completion event reporting
cfg_sched_performance_enable	input	1	Enable performance event reporting

Descriptor Engine Configuration:

Configuration Interface

Signal	Direction	Width	Description
cfg_desceng_enable	input	1	Enable descriptor engine
cfg_desceng_prefetch	input	1	Enable descriptor prefetch
cfg_desceng_fifo_thresh	input	4	FIFO threshold for prefetch
cfg_desceng_addr0_base	input	ADDR_WIDT H	Base address limit 0
cfg_desceng_addr0_limit	input	ADDR_WIDT H	Limit address limit 0
cfg_desceng_addr1_base	input	ADDR_WIDT H	Base address limit 1
cfg_desceng_addr1_limit	input	ADDR_WIDT H	Limit address limit 1

AXI Monitor Configuration:

Three identical sets of monitor config signals (descriptor, read, write):

Configuration Interface

Signal Prefix	Applies To	Description
cfg_desc_mon_*	Descriptor AXI	Descriptor fetch monitoring
cfg_rdeng_mon_*	Read AXI	Data read

Signal Prefix	Applies To	Description
cfg_wreng_mon_*	Write AXI	monitoring Data write monitoring

Each monitor has:

Signal Suffix	Width	Description
_enable	1	Enable monitor
_err_enable	1	Enable error reporting
_perf_enable	1	Enable performance reporting
_timeout_enable	1	Enable timeout detection
_timeout_cycles	32	Timeout threshold
_latency_thresh	32	Latency threshold for events
_pkt_mask	16	Packet type mask
_err_select	4	Error type selector
_err_mask	8	Error event mask
_timeout_mask	8	Timeout event mask
_compl_mask	8	Completion event mask
_thresh_mask	8	Threshold event mask
_perf_mask	8	Performance event mask
_addr_mask	8	Address event mask
_debug_mask	8	Debug event mask

AXI Transfer Configuration:

Signal	Direction	Width	Description
cfg_axi_rd_xfer_beats	input	8	Read burst size (beats)

Signal	Direction	Width	Description
cfg_axi_wr_xfer_beats	input	8	Write burst size (beats)

Performance Profiler Configuration:

Signal	Direction	Width	Description
cfg_perf_enable	input	1	Enable profiler
cfg_perf_mode	input	1	Profiler mode (0=count, 1=latency)
cfg_perf_clear	input	1	Clear profiler counters

6.4.4 Status Interface

System-Level Status:

Status Interface

Signal	Direction	Width	Description
system_idle	output	1	All channels idle (AND of all scheduler_idle)

Per-Channel Status:

Status Interface

Signal	Direction	Width	Description
descriptor_engine_idle[ch]	output	NUM_CHAN NELS	Descriptor engine idle
scheduler_idle[ch]	output	NUM_CHAN NELS	Scheduler in IDLE state
scheduler_state[ch]	output	NUM_CHAN NELS × 7	Scheduler FSM state (ONE-HOT)
sched_error[ch]	output	NUM_CHAN NELS	Scheduler error (sticky)
axi_rd_all_complete[ch]	output	NUM_CHAN NELS	All read transactions complete

Signal	Direction	Width	Description
axi_wr_all_complete[ch]	output	NUM_CHANNELS	All write transactions complete

Performance Profiler Status:

Status Interface

Signal	Direction	Width	Description
perf_fifo_empty	output	1	Profiler FIFO empty
perf_fifo_full	output	1	Profiler FIFO full
perf_fifo_count	output	16	Profiler FIFO occupancy
perf_fifo_rd	input	1	Read profiler entry
perf_fifo_data_low	output	32	Profiler data [31:0]
perf_fifo_data_high	output	32	Profiler data [63:32]

6.4.5 AXI4 Master - Descriptor Fetch (256-bit)

AR Channel:

AXI4 Master - Descriptor Fetch

Signal	Direction	Width	Description
m_axi_desc_ar_id	output	AXI_ID_WIDTH	Transaction ID
m_axi_desc_ar_addr	output	ADDR_WIDTH	Address
m_axi_desc_ar_len	output	8	Burst length - 1
m_axi_desc_ar_size	output	3	Burst size (log2 bytes)
m_axi_desc_ar_burst	output	2	Burst type (INCR)
m_axi_desc_ar_lock	output	1	Lock type

Signal	Direction	Width	Description
lock			
m_axi_desc_ar cache	output	4	Cache attributes
m_axi_desc_ar prot	output	3	Protection attributes
m_axi_desc_ar qos	output	4	QoS value
m_axi_desc_ar region	output	4	Region identifier
m_axi_desc_ar user	output	CHAN_WIDTH	User signal (channel ID)
m_axi_desc_ar valid	output	1	Address valid
m_axi_desc_ar ready	input	1	Address ready

R Channel:

AXI4 Master - Descriptor Fetch

Signal	Direction	Width	Description
m_axi_desc_r id	input	AXI_ID_WID TH	Transaction ID
m_axi_desc_r data	input	256	Read data (FIXED 256- bit)
m_axi_desc_r resp	input	2	Response (OKAY/EXOKAY/SLVERR/ DECERR)
m_axi_desc_r last	input	1	Last beat of burst
m_axi_desc_r user	input	CHAN_WIDT H	User signal (channel ID)
m_axi_desc_r valid	input	1	Read data valid
m_axi_desc_r ready	output	1	Read data ready

6.4.6 AXI4 Master - Data Read (Parameterizable Width)

AR Channel:

AXI4 Master - Data Read

Signal	Direction	Width	Description
m_axi_rd_arid	output	AXI_ID_WIDTH	Transaction ID
m_axi_rd_araddr	output	ADDR_WIDTH	Address
m_axi_rd_arlength	output	8	Burst length - 1
m_axi_rd_arsize	output	3	Burst size (log2 bytes)
m_axi_rd_arburst	output	2	Burst type (INCR)
m_axi_rd_arlock	output	1	Lock type
m_axi_rd_arcache	output	4	Cache attributes
m_axi_rd_arprot	output	3	Protection attributes
m_axi_rd_arqos	output	4	QoS value
m_axi_rd_arregion	output	4	Region identifier
m_axi_rd_aruser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_rd_arvalid	output	1	Address valid
m_axi_rd_arready	input	1	Address ready

R Channel:

AXI4 Master - Data Read

Signal	Direction	Width	Description
m_axi_rd_rid	input	AXI_ID_WIDTH	Transaction ID
m_axi_rd_rdata	input	DATA_WIDTH	Read data

Signal	Direction	Width	Description
			(default 512-bit)
m_axi_rd_rresp	input	2	Response
m_axi_rd_rlast	input	1	Last beat of burst
m_axi_rd_ruser	input	CHAN_WIDTH	User signal (channel ID)
m_axi_rd_rvalid	input	1	Read data valid
m_axi_rd_rready	output	1	Read data ready

6.4.7 AXI4 Master - Data Write (Parameterizable Width)

AW Channel:

AXI4 Master - Data Write

Signal	Direction	Width	Description
m_axi_wr_awid	output	AXI_ID_WIDTH	Transaction ID
m_axi_wr_awaddr	output	ADDR_WIDTH	Address
m_axi_wr_awlen	output	8	Burst length - 1
m_axi_wr_awsize	output	3	Burst size (log2 bytes)
m_axi_wr_awburst	output	2	Burst type (INCR)
m_axi_wr_awlock	output	1	Lock type
m_axi_wr_awcache	output	4	Cache attributes
m_axi_wr_awprotection	output	3	Protection attributes
m_axi_wr_awqos	output	4	QoS value

Signal	Direction	Width	Description
m_axi_wr_awregion	output	4	Region identifier
m_axi_wr_awuser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_wr_awvalid	output	1	Address valid
m_axi_wr_awready	input	1	Address ready

W Channel:

AXI4 Master - Data Write

Signal	Direction	Width	Description
m_axi_wr_wdata	output	DATA_WIDTH	Write data (default 512-bit)
m_axi_wr_wstrobe	output	DATA_WIDTH/8	Write strobes (byte enables)
m_axi_wr_wlast	output	1	Last beat of burst
m_axi_wr_wuser	output	CHAN_WIDTH	User signal (channel ID)
m_axi_wr_wvalid	output	1	Write data valid
m_axi_wr_wready	input	1	Write data ready

B Channel:

AXI4 Master - Data Write

Signal	Direction	Width	Description
m_axi_wr_bid	input	AXI_ID_WIDTH	Transaction ID
m_axi_wr_bresp	input	2	Response
m_axi_wr_buser	input	CHAN_WIDTH	User signal (channel ID)
m_axi_wr_bvalid	input	1	Response valid
m_axi_wr_bready	output	1	Response

Signal	Direction	Width	Description
dy			ready

6.4.8 Status/Debug Outputs

Descriptor AXI Monitor:

Status/Debug Outputs

Signal	Direction	Width	Description
cfg_sts_desc_mon_busy	output	1	Monitor busy
cfg_sts_desc_mon_active_txns	output	8	Active transaction count
cfg_sts_desc_mon_error_count	output	16	Error count
cfg_sts_desc_mon_txn_count	output	32	Total transaction count
cfg_sts_desc_mon_conflict_error	output	1	ID conflict detected

Read Engine AXI Monitor:

Status/Debug Outputs

Signal	Direction	Width	Description
cfg_sts_rden_g_skid_busy	output	1	Skid buffer busy
cfg_sts_rden_g_mon_active_txns	output	8	Active transaction count
cfg_sts_rden_g_mon_error_count	output	16	Error count
cfg_sts_rden_g_mon_txn_count	output	32	Total transaction count
cfg_sts_rden_g_mon_conflict_error	output	1	ID conflict detected

Write Engine AXI Monitor:

Status/Debug Outputs

Signal	Direction	Width	Description
cfg_sts_wren g_skid_busy	output	1	Skid buffer busy
cfg_sts_wren g_mon_active _txns	output	8	Active transaction count
cfg_sts_wren g_mon_error_ count	output	16	Error count
cfg_sts_wren g_mon_txn_co unt	output	32	Total transaction count
cfg_sts_wren g_mon_confli ct_error	output	1	ID conflict detected

6.4.9 Unified Monitor Bus Interface

Unified Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid
mon_ready	input	1	Monitor packet ready
mon_packet	output	64	Monitor packet data

MonBus Sources: - Descriptor engines (8 sources, agent IDs 16-23) - Schedulers (8 sources, agent IDs 48-55) - Descriptor AXI monitor (agent ID 8) - Read AXI monitor (configurable) - Write AXI monitor (configurable)

6.5 Operation

6.5.1 Transfer Initialization

Step 1: Configuration

1. Configure global settings (timeouts, monitors, transfer sizes)
2. Enable descriptor engine (cfg_desceng_enable = 1)

3. Enable scheduler (cfg_sched_enable = 1)
4. Enable target channel (cfg_channel_enable[ch] = 1)

Step 2: Descriptor Kick-off

1. Assert apb_valid[ch] = 1
2. Provide descriptor address on apb_addr[ch]
3. Wait for apb_ready[ch] handshake
4. Deassert apb_valid[ch]

Step 3: Automatic Transfer

1. Descriptor engine fetches descriptor via m_axi_desc_*
2. Scheduler receives descriptor, starts transfer
3. Read engine: memory → SRAM (via m_axi_rd_*)
4. Write engine: SRAM → memory (via m_axi_wr_*)
5. Scheduler reports completion via MonBus

6.5.2 Monitoring Transfer Progress

Check Scheduler State:

```
// Monitor scheduler state
case (scheduler_state[ch])
    7'b00000001: // CH_IDLE - waiting for descriptor
    7'b00000010: // CH_FETCH_DESC - fetching descriptor
    7'b00000100: // CH_XFER_DATA - transfer in progress
    7'b00001000: // CH_COMPLETE - transfer complete
    7'b00010000: // CH_NEXT_DESC - chaining to next
    7'b01000000: // CH_ERROR - error occurred
endcase
```

Check Completion:

```
// All complete when:
complete = scheduler_idle[ch] &&
            axi_rd_all_complete[ch] &&
            axi_wr_all_complete[ch];
```

6.6 Timing Diagrams

6.6.1 Complete Data Flow Timing

The following diagram shows the complete data flow through the STREAM core, from descriptor kick-off through data transfer completion:

6.6.1.1 Waveform 2.1.1: *STREAM* Core Data Flow

Source: [stream_core_data_flow.json](#)

6.7 Testing

Test Location: projects/components/dmas/stream/dv/tests/top/

Key Test Scenarios:

1. **Single channel transfer** - Basic end-to-end operation
2. **Multi-channel concurrent** - 2-8 channels simultaneously
3. **Descriptor chaining** - 2-5 descriptors linked
4. **FIFO overflow prevention** - Large transfer with small FIFO
5. **Error handling** - AXI errors, timeouts
6. **Performance profiling** - Bandwidth measurements
7. **MonBus event checking** - Verify all events reported

Test Configuration:

```
# Basic configuration
NUM_CHANNELS = 4
DATA_WIDTH = 128
FIFO_DEPTH = 512
cfg_axi_rd_xfer_beats = 16
cfg_axi_wr_xfer_beats = 16
```

6.8 Resource Utilization

Estimated Resources (8 channels, 512-bit data, 512-deep FIFOs):

Resource Utilization

Component	Quantity	Est. Size
Schedulers	8	8 × ~500 FFs
Descriptor Engine	1	~1000 FFs
SRAM FIFOs	8	8 × 512 × 512-bit = 256KB
AXI Engines	2	2 × ~2000 FFs

Component	Quantity	Est. Size
Skid Buffers	3 sets	~2000 FFs
Monitors	3	3 × ~1000 FFs
Total		~20K FFs + 256KB SRAM

Critical Paths: - AXI handshake paths (improved by skid buffers) - SRAM address decode - MonBus arbiter

6.9 Integration Example

```
stream_core #(
    .NUM_CHANNELS(8),
    .DATA_WIDTH(512),
    .FIFO_DEPTH(512)
) u_stream (
    .clk                (system_clk),
    .rst_n              (system_rst_n),

    // APB kick-off
    .apb_valid          (stream_apb_valid),
    .apb_ready          (stream_apb_ready),
    .apb_addr           (stream_apb_addr),

    // Configuration
    .cfg_channel_enable (stream_ch_enable),
    .cfg_sched_enable   (1'b1),
    .cfg_axi_rd_xfer_beats (8'd16),
    .cfg_axi_wr_xfer_beats (8'd16),
    // ... other config

    // AXI Descriptor Master
    .m_axi_desc_arid     (desc_arid),
    .m_axi_desc_araddr   (desc_araddr),
    // ... full AXI AR/R

    // AXI Read Master
    .m_axi_rd_arid       (rd_arid),
    .m_axi_rd_araddr     (rd_araddr),
    // ... full AXI AR/R

    // AXI Write Master
    .m_axi_wr_awid       (wr_awid),
```

```

        .m_axi_wr_awaddr      (wr_awaddr),
        // ... full AXI AW/W/B

        // MonBus
        .mon_valid            (stream_mon_valid),
        .mon_ready            (stream_mon_ready),
        .mon_packet           (stream_mon_packet)
    );

```

6.10 Related Documentation

- **Scheduler Group Array:** 02_scheduler_group_array.md - Multi-channel scheduler integration
- **Scheduler Group:** 03_scheduler_group.md - Single channel scheduler + descriptor engine
- **Scheduler:** 04_scheduler.md - Channel state machine and transfer coordination
- **Descriptor Engine:** 05_descriptor_engine.md - Descriptor fetch and parsing
- **AXI Read Engine:** 06_axi_read_engine.md - Read datapath
- **Stream Alloc Ctrl:** 07_stream_alloc_ctrl.md - Space allocation controller
- **SRAM Controller:** 08_sram_controller.md - Per-channel buffering
- **SRAM Controller Unit:** 09_sram_controller_unit.md - Single channel SRAM unit
- **Stream Latency Bridge:** 10_stream_latency_bridge.md - Timing bridge
- **Stream Drain Ctrl:** 11_stream_drain_ctrl.md - Drain flow controller
- **AXI Write Engine:** 12_axi_write_engine.md - Write datapath
- **APB to Descriptor:** 13_apbtodescr.md - APB configuration interface
- **Performance Profiler:** 15_perf_profiler.md - Performance monitoring

- **MonBus AXI-Lite Group: 16_monbus_axil_group.md - Monitor bus arbitration**
-

6.11 Revision History

STREAM Core Specification Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

7 Scheduler Group Array

Module: scheduler_group_array.sv **Location:** projects/components/dmas/stream/rtl/macro/ **Category:** MACRO (Top-level Integration)
Parent: stream_core.sv **Status:** Implemented **Last Updated:** 2025-11-30

7.1 Overview

The scheduler_group_array module is the top-level array of 8 scheduler_group instances that provides multi-channel DMA with shared resources. It serves as the scheduler layer of the STREAM engine, handling descriptor fetch arbitration and routing data path interfaces.

7.1.1 Key Features

- **8 Independent Channels:** Each channel has its own scheduler_group instance

- **Shared Descriptor AXI Master:** Round-robin arbitration for descriptor fetches
- **Per-Channel Data Interfaces:** Direct passthrough to AXI engines (no arbitration)
- **Unified MonBus Aggregation:** 9 sources (8 channels + descriptor AXI monitor)
- **ID-Based R Channel Routing:** Channel ID embedded in AXI ID for response demux

7.1.2 Simplified from RAPIDS

This module is intentionally simplified from RAPIDS for tutorial focus: - 8 channels (vs 32 in RAPIDS) - No control read/write engines - No program engine - No network interfaces (EOS completion) - No alignment bus - Direct data path interfaces (beats-based, no chunks)

7.2 Architecture

7.2.1 Component Hierarchy

```

scheduler_group_array
  scheduler_group[0..7]          # 8 channel instances
    scheduler                     # Channel FSM
    descriptor_engine             # Descriptor fetch
    arbiter_round_robin           # AR channel arbitration
    axi4_master_rd_mon            # Descriptor AXI monitor
    monbus_arbiter                # 9-source MonBus aggregation

```

7.2.2 Descriptor AXI Arbitration

AR Channel Flow:

Channel 0..7 AR Requests -> Round-Robin Arbiter -> Single AR to External AXI

|
Embed channel_id in AXI ID lower bits

R Channel Demux:

External R Response -> Extract channel_id from RID[CHAN_WIDTH-1:0] -> Route to correct channel

Critical: Channel ID is embedded in the lower bits of the AXI ID field:

```

// In AR mux
desc_axi_int_arid = {{(AXI_ID_WIDTH-CHAN_WIDTH){1'b0}}, ch[CHAN_WIDTH-

```

```
1:0]}};
```

```
// In R demux
```

```
desc_r_channel_id = desc_axi_int_rid[CHAN_WIDTH-1:0];
```

7.2.3 Data Path Interfaces

IMPORTANT: Data read and write interfaces are NOT arbitrated at this level!

- `sched_rd_*` - Per-channel arrays passed directly to `axi_read_engine`
- `sched_wr_*` - Per-channel arrays passed directly to `axi_write_engine`

The AXI engines themselves handle channel arbitration internally.

7.3 Parameters

Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of DMA channels
CHAN_WIDTH	int	$\$clog2(\text{NUM_CHANNELS})$	Channel ID width (3 bits for 8 channels)
ADDR_WIDTH	int	64	Address bus width
DATA_WIDTH	int	512	Data bus width
AXI_ID_WIDTH	int	8	AXI transaction ID width

7.3.1 Monitor Bus Agent IDs

Monitor Bus Agent IDs

Parameter	Default	Description
DESC_MON_BASE_AGENT_ID	16 (0x10)	Descriptor engine base (16-23)
SCHED_MON_BASE_AGENT_ID	48 (0x30)	Scheduler base (48-55)
DESC_AXI_MON_AGENT_ID	8 (0x08)	Descriptor AXI monitor
MON_UNIT_ID	1	Unit ID for all STREAM events
MON_MAX_TRANSACTIONS	16	Max outstanding transactions for monitor

7.4 Port List

7.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

7.4.2 APB Programming Interface (Per-Channel)

APB Programming Interface

Signal	Direction	Width	Description
apb_valid[ch]	input	NUM_CHAN NELS	Descriptor address valid
apb_ready[ch]	output	NUM_CHAN NELS	Ready to accept descriptor
apb_addr[ch]	input	NUM_CHAN NELS x ADDR_WIDTH	Descriptor address

7.4.3 Configuration Interface (Per-Channel)

Configuration Interface

Signal	Direction	Width	Description
cfg_channel_enable[ch]	input	NUM_CHANNELS	Enable channel
cfg_channel_reset[ch]	input	NUM_CHANNELS	Soft reset channel

7.4.4 Global Scheduler Configuration

Global Scheduler Configuration

Signal	Direction	Width	Description
cfg_sched_enable	input	1	Master scheduler enable
cfg_sched_timeout_cycles	input	16	Timeout threshold
cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_sched_err_enable	input	1	Enable error reporting
cfg_sched_completion_enable	input	1	Enable completion reporting
cfg_sched_performance_enable	input	1	Enable performance monitoring

7.4.5 Descriptor Engine Configuration

Descriptor Engine Configuration

Signal	Direction	Width	Description
cfg_desceng_enable	input	1	Enable descriptor engine
cfg_desceng_prefetch	input	1	Enable descriptor prefetch
cfg_desceng_fifo_thresh	input	4	FIFO threshold for prefetch
cfg_desceng_addr0_base	input	ADDR_WIDT H	Address range 0 base
cfg_desceng_addr0_limit	input	ADDR_WIDT H	Address range 0 limit
cfg_desceng_addr1_base	input	ADDR_WIDT H	Address range 1 base

Signal	Direction	Width	Description
cfg_desceng_addr1_limit	input	ADDR_WIDT H	Address range 1 limit

7.4.6 Status Interface (Per-Channel)

Status Interface

Signal	Direction	Width	Description
descriptor_engine_idle[ch]	output	NUM_CHAN NELS	Descriptor engine idle
scheduler_idle[ch]	output	NUM_CHAN NELS	Scheduler idle
scheduler_state[ch]	output	NUM_CHAN NELS x 7	Scheduler FSM state (ONE-HOT)
sched_error[ch]	output	NUM_CHAN NELS	Scheduler error (sticky)

7.4.7 Shared Descriptor AXI4 Master Read Interface

AR Channel:

Shared Descriptor AXI4 Master Read Interface

Signal	Direction	Width	Description
desc_axi_arvalid	output	1	Address valid
desc_axi_arready	input	1	Address ready
desc_axi_araddr	output	ADDR_WIDTH	Address
desc_axi_arlength	output	8	Burst length - 1
desc_axi_arsize	output	3	Burst size (log2 bytes)
desc_axi_arburst	output	2	Burst type (INCR)
desc_axi_arid	output	AXI_ID_WIDTH	Transaction ID
desc_axi_arlock	output	1	Lock type

Signal	Direction	Width	Description
desc_axi_arcache	output	4	Cache attributes
desc_axi_arprot	output	3	Protection attributes
desc_axi_arqos	output	4	QoS value
desc_axi_arregion	output	4	Region identifier

R Channel (FIXED 256-bit):

Shared Descriptor AXI4 Master Read Interface

Signal	Direction	Width	Description
desc_axi_rvalid	input	1	Read data valid
desc_axi_rready	output	1	Read data ready
desc_axi_rdata	input	256	Read data (FIXED 256-bit descriptor)
desc_axi_rresp	input	2	Response
desc_axi_rlast	input	1	Last beat of burst
desc_axi_rid	input	AXI_ID_WIDTH	Transaction ID

7.4.8 Shared Data Read Interface (Per-Channel Arrays)

Shared Data Read Interface

Signal	Direction	Width	Description
sched_rd_valid[ch]	output	NUM_CHANNELS	Read request valid
sched_rd_addr[ch]	output	NUM_CHANNELS x ADDR_WIDTH	Source address
sched_rd_beats[ch]	output	NUM_CHANNELS	Beats to read

Signal	Direction	Width	Description
		NELS x 32	
sched_rd_done_strobe[ch]	input	NUM_CHAN NELS	Read completion strobe
sched_rd_beats_done[ch]	input	NUM_CHAN NELS x 32	Beats completed
sched_rd_error[ch]	input	NUM_CHAN NELS	Read error

7.4.9 Shared Data Write Interface (Per-Channel Arrays)

Shared Data Write Interface

Signal	Direction	Width	Description
sched_wr_valid[ch]	output	NUM_CHAN NELS	Write request valid
sched_wr_ready[ch]	input	NUM_CHAN NELS	Write ready
sched_wr_address[ch]	output	NUM_CHAN NELS x ADDR_WIDTH	Destination address
sched_wr_beats[ch]	output	NUM_CHAN NELS x 32	Beats to write
sched_wr_done_strobe[ch]	input	NUM_CHAN NELS	Write completion strobe
sched_wr_beats_done[ch]	input	NUM_CHAN NELS x 32	Beats completed
sched_wr_error[ch]	input	NUM_CHAN NELS	Write error

7.4.10 Unified Monitor Bus Interface

Unified Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid

Signal	Direction	Width	Description
mon_ready	input	1	Monitor packet ready
mon_packet	output	64	Monitor packet data

7.5 Operation

7.5.1 Descriptor Fetch Arbitration

1. **Request Phase:** Channels assert desc_ar_valid[ch] when needing descriptor fetch
2. **Grant Phase:** Round-robin arbiter selects one channel
3. **AR Issue:** Granted channel's AR signals muxed to external interface
4. **ID Embedding:** Channel ID placed in lower bits of arid
5. **Grant ACK:** Arbiter acknowledged when arready received
6. **R Routing:** Response routed to correct channel based on rid[CHAN_WIDTH-1:0]

7.5.2 Data Path Flow

Data interfaces pass directly through without arbitration:

```
scheduler_group[ch].sched_rd_* -> sched_rd_[ch] -> axi_read_engine
(handles arbitration)
scheduler_group[ch].sched_wr_* -> sched_wr_[ch] -> axi_write_engine
(handles arbitration)
```

7.5.3 MonBus Aggregation

9 sources aggregated via monbus_arbiter: - Sources 0-7: Per-channel mon_* from scheduler_groups - Source 8: Descriptor AXI master monitor

7.6 Integration Example

```
scheduler_group_array #(
    .NUM_CHANNELS          (8),
    .ADDR_WIDTH            (64),
    .DATA_WIDTH            (512),
    .AXI_ID_WIDTH          (8)
) u_scheduler_group_array (
```

```

.clk                (clk),
.rst_n              (rst_n),

// APB kick-off
.apb_valid           (apb_valid),
.apb_ready           (apb_ready),
.apb_addr            (apb_addr),

// Configuration
.cfg_channel_enable  (cfg_channel_enable),
.cfg_channel_reset   (cfg_channel_reset),
.cfg_sched_enable     (cfg_sched_enable),
// ... other config

// Descriptor AXI master
.desc_axi_arvalid     (desc_axi_arvalid),
.desc_axi_arready     (desc_axi_arready),
// ... full AR/R interface

// Data read interface (to axi_read_engine)
.sched_rd_valid      (sched_rd_valid),
.sched_rd_addr        (sched_rd_addr),
.sched_rd_beats       (sched_rd_beats),
.sched_rd_done_strobe (rd_done_strobe),
.sched_rd_beats_done  (rd_beats_done),

// Data write interface (to axi_write_engine)
.sched_wr_valid       (sched_wr_valid),
.sched_wr_ready       (sched_wr_ready),
.sched_wr_addr         (sched_wr_addr),
.sched_wr_beats        (sched_wr_beats),
.sched_wr_done_strobe  (wr_done_strobe),
.sched_wr_beats_done   (wr_beats_done),

// Monitor bus
.mon_valid            (sga_mon_valid),
.mon_ready            (sga_mon_ready),
.mon_packet           (sga_mon_packet)
);

```

7.7 Related Documentation

- **Parent:** 01_stream_core.md - Top-level integration

- **Child:** 03_scheduler_group.md - Single channel scheduler + descriptor engine
 - **Scheduler:** 04_scheduler.md - Channel state machine
 - **Descriptor Engine:** 05_descriptor_engine.md - Descriptor fetch and parsing
 - **AXI Read Engine:** 06_axi_read_engine.md - Read datapath (receives sched_rd_*)
 - **AXI Write Engine:** 12_axi_write_engine.md - Write datapath (receives sched_wr_*)
 - **MonBus Arbiter:** 16_monbus_axil_group.md - Monitor bus aggregation
-

7.8 Revision History

Scheduler Group Array Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

8 Scheduler Group

Module: scheduler_group.sv **Location:** projects/components/dmas/stream/rtl/macro/
Category: MACRO (Channel Wrapper) **Parent:** scheduler_group_array.sv **Status:**
 Implemented **Last Updated:** 2025-11-30

8.1 Overview

The `scheduler_group` module is a wrapper combining the scheduler and `descriptor_engine` for a single STREAM channel. It provides a simplified interface compared to RAPIDS by removing unused features while maintaining the core descriptor-based DMA functionality.

8.1.1 Key Features

- **Single Channel Wrapper:** Combines scheduler + `descriptor_engine`
- **Simplified from RAPIDS:** No program engine, no control engines, no alignment bus
- **MonBus Aggregation:** Combines 2 internal sources (scheduler, `descriptor_engine`)
- **Direct Data Interfaces:** Per-channel read/write interfaces to shared engines

8.1.2 Simplified from RAPIDS

Removed from RAPIDS `scheduler_group`: - Program engine - Control read/write engines - Alignment bus - Network interfaces (EOS completion) - Credit management

8.2 Architecture

8.2.1 Component Hierarchy

```
scheduler_group
  scheduler          # Channel state machine
  descriptor_engine  # Descriptor fetch and parsing
  arbiter_round_robin_monbus # 2-source MonBus arbiter
```

8.2.2 Data Flow

```
APB kick-off -> descriptor_engine -> scheduler
                  |                     |
```

: Data Flow

```

                v                     v
Descriptor AXI  sched_rd_* / sched_wr_*
(to arbiter)    (to shared engines)
```

Descriptor Flow: 1. APB kick-off provides descriptor address 2. Descriptor engine fetches via shared AXI (arbitrated externally) 3. Parsed descriptor delivered to scheduler 4. Scheduler coordinates read/write phases

Data Path Flow: 1. Scheduler asserts sched_rd_valid with source address/beats 2. External read engine issues AXI reads, returns completion strobe 3. Scheduler asserts sched_wr_valid with destination address/beats 4. External write engine issues AXI writes, returns completion strobe 5. Scheduler advances to next descriptor or completes

8.3 Parameters

Parameters

Parameter	Type	Default	Description
CHANNEL_ID	int	0	Channel identifier (0-7)
NUM_CHANNELS	int	8	Total number of channels
CHAN_WIDTH	int	$\$clog2(\text{NUM_CHANNELS})$	Channel ID width
ADDR_WIDTH	int	64	Address bus width
DATA_WIDTH	int	512	Data bus width
AXI_ID_WIDTH	int	8	AXI transaction ID width

8.3.1 Monitor Bus Agent IDs

Monitor Bus Agent IDs

Parameter	Type	Default	Description
DESC_MON_AGENT_ID	8-bit	0x10+ch	Descriptor engine agent ID
SCHED_MON_AGENT_ID	8-bit	0x30+ch	Scheduler agent ID
MON_UNIT_ID	4-bit	0x1	Unit ID
MON_CHANNEL_ID	6-bit	ch	Channel ID in

Parameter	Type	Default	Description
D			packets

8.4 Port List

8.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

8.4.2 APB Programming Interface

APB Programming Interface

Signal	Direction	Width	Description
apb_valid	input	1	Descriptor address valid
apb_ready	output	1	Ready to accept descriptor
apb_addr	input	ADDR_WIDTH	Descriptor address

8.4.3 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_channel_enable	input	1	Enable this channel
cfg_channel_reset	input	1	Soft reset this channel

8.4.4 Scheduler Configuration

Scheduler Configuration

Signal	Direction	Width	Description
cfg_sched_timeout_cycles	input	16	Timeout threshold
cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_sched_error_enable	input	1	Enable error reporting
cfg_sched_completion_enable	input	1	Enable completion reporting
cfg_sched_performance_enable	input	1	Enable performance monitoring

8.4.5 Descriptor Engine Configuration

Descriptor Engine Configuration

Signal	Direction	Width	Description
cfg_desceng_prefetch	input	1	Enable descriptor prefetch
cfg_desceng_fifo_thresh	input	4	FIFO threshold for prefetch
cfg_desceng_addr0_base	input	ADDR_WIDT H	Address range 0 base
cfg_desceng_addr0_limit	input	ADDR_WIDT H	Address range 0 limit
cfg_desceng_addr1_base	input	ADDR_WIDT H	Address range 1 base
cfg_desceng_addr1_limit	input	ADDR_WIDT H	Address range 1 limit

8.4.6 Status Interface

Status Interface

Signal	Direction	Width	Description
descriptor_engine_idle	output	1	Descriptor engine idle
scheduler_idle	output	1	Scheduler in IDLE state
scheduler_state	output	7	Scheduler FSM state (ONE-HOT)
sched_error	output	1	Scheduler error (sticky)

8.4.7 Descriptor AXI Interface (to external arbiter)

AR Channel:

Descriptor AXI Interface

Signal	Direction	Width	Description
desc_ar_valid	output	1	Address valid
desc_ar_ready	input	1	Address ready
desc_ar_addr	output	ADDR_WIDTH	Address
desc_ar_len	output	8	Burst length - 1
desc_ar_size	output	3	Burst size
desc_ar_burst	output	2	Burst type
desc_ar_id	output	AXI_ID_WIDTH	Transaction ID
desc_ar_lock	output	1	Lock type
desc_ar_cache	output	4	Cache attributes
desc_ar_prot	output	3	Protection attributes
desc_ar_qos	output	4	QoS value
desc_ar_region	output	4	Region identifier

R Channel (256-bit fixed):

Descriptor AXI Interface

Signal	Direction	Width	Description
desc_r_valid	input	1	Read data valid
desc_r_ready	output	1	Read data ready
desc_r_data	input	256	Read data (256-bit descriptor)
desc_r_resp	input	2	Response
desc_r_last	input	1	Last beat
desc_r_id	input	AXI_ID_WIDTH	Transaction ID

8.4.8 Data Read Interface (to shared read engine)

Data Read Interface

Signal	Direction	Width	Description
sched_rd_valid	output	1	Read request valid
sched_rd_addr	output	ADDR_WIDTH	Source address
sched_rd_beats	output	32	Beats to read
sched_rd_done_strobe	input	1	Read completion strobe
sched_rd_beats_done	input	32	Beats completed
sched_rd_error	input	1	Read error

8.4.9 Data Write Interface (to shared write engine)

Data Write Interface

Signal	Direction	Width	Description
sched_wr_valid	output	1	Write request valid

Signal	Direction	Width	Description
sched_wr_ready	input	1	Write ready
sched_wr_addr	output	ADDR_WIDTH	Destination address
sched_wr_beats	output	32	Beats to write
sched_wr_done_strobe	input	1	Write completion strobe
sched_wr_beats_done	input	32	Beats completed
sched_wr_error	input	1	Write error

8.4.10 Monitor Bus Interface

Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid
mon_ready	input	1	Monitor packet ready
mon_packet	output	64	Monitor packet data

8.5 Operation

8.5.1 Transfer Sequence

1. **APB Kick-off:** apb_valid with descriptor address
2. **Descriptor Fetch:** descriptor_engine fetches via AXI
3. **Descriptor Parse:** 256-bit descriptor parsed into fields
4. **Read Phase:** scheduler asserts sched_rd_valid
5. **Read Completion:** engine returns sched_rd_done_strobe
6. **Write Phase:** scheduler asserts sched_wr_valid
7. **Write Completion:** engine returns sched_wr_done_strobe

8. **Chain Check:** if next_descriptor_ptr != 0, fetch next
9. **Complete:** scheduler returns to IDLE

8.5.2 MonBus Sources

The scheduler_group aggregates 2 MonBus sources: 1. **Descriptor Engine:** Fetch events, errors 2. **Scheduler:** State transitions, completions, errors

8.6 Integration Example

```
scheduler_group #(
    .CHANNEL_ID            (ch),
    .NUM_CHANNELS          (8),
    .ADDR_WIDTH            (64),
    .DATA_WIDTH            (512),
    .AXI_ID_WIDTH          (8),
    .DESC_MON_AGENT_ID     (8'h10 + ch),
    .SCHED_MON_AGENT_ID    (8'h30 + ch)
) u_scheduler_group (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // APB interface
    .apb_valid              (apb_valid[ch]),
    .apb_ready              (apb_ready[ch]),
    .apb_addr               (apb_addr[ch]),

    // Configuration
    .cfg_channel_enable     (cfg_channel_enable[ch]),
    .cfg_channel_reset      (cfg_channel_reset[ch]),
    // ... other config

    // Descriptor AXI (to arbiter)
    .desc_ar_valid          (desc_ar_valid[ch]),
    .desc_ar_ready          (desc_ar_ready[ch]),
    // ... full AR/R interface

    // Data interfaces (direct passthrough)
    .sched_rd_valid         (sched_rd_valid[ch]),
    .sched_rd_addr          (sched_rd_addr[ch]),
    .sched_rd_beats         (sched_rd_beats[ch]),
    .sched_rd_done_strobe   (sched_rd_done_strobe[ch]),
    .sched_rd_beats_done    (sched_rd_beats_done[ch]),
```



```

.sched_wr_valid      (sched_wr_valid[ch]),
.sched_wr_ready      (sched_wr_ready[ch]),
.sched_wr_addr       (sched_wr_addr[ch]),
.sched_wr_beats       (sched_wr_beats[ch]),
.sched_wr_done_strobe (sched_wr_done_strobe[ch]),
.sched_wr_beats_done  (sched_wr_beats_done[ch]),

// Monitor bus
.mon_valid           (mon_valid_ch[ch]),
.mon_ready           (mon_ready_ch[ch]),
.mon_packet          (mon_packet_ch[ch])
);

```

8.7 Related Documentation

- **Parent:** 02_scheduler_group_array.md - Multi-channel array
 - **Scheduler:** 04_scheduler.md - Channel state machine details
 - **Descriptor Engine:** 05_descriptor_engine.md - Descriptor fetch logic
 - **AXI Read Engine:** 06_axi_read_engine.md - Shared read engine
 - **AXI Write Engine:** 12_axi_write_engine.md - Shared write engine
-

8.8 Revision History

Scheduler Group Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

9 Scheduler Specification

Module: scheduler.sv **Location:** projects/components/dmas/stream/rtl/fub/ **Status:** Implemented **Last Updated:** 2025-11-30

9.1 Overview

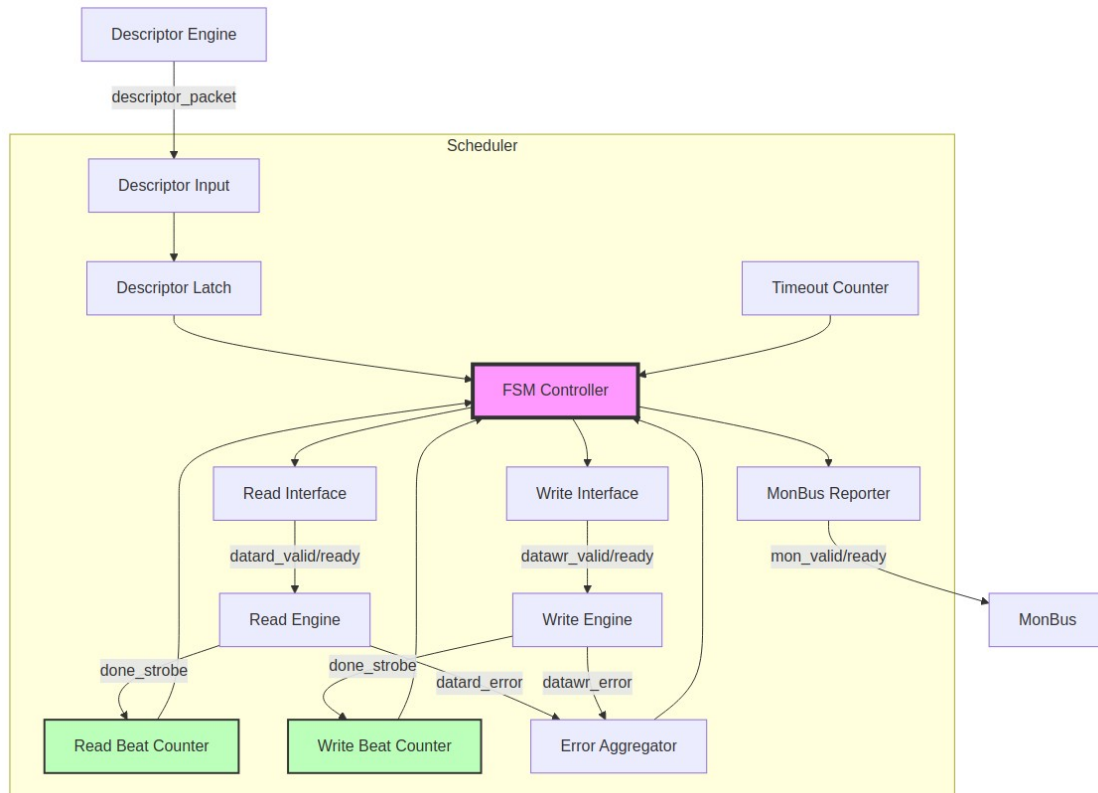
The Scheduler coordinates descriptor-based memory-to-memory DMA transfers for a single channel. It receives descriptors, manages concurrent read/write operations, and handles descriptor chaining.

9.1.1 Key Features

- **Concurrent read/write:** Read and write engines run simultaneously (prevents deadlock)
- **Read-ahead descriptor prefetch:** On a chained legacy descriptor the read side advances to the next descriptor while the write side still drains the current one, eliminating the per-descriptor boundary bubble (runtime SCHED_CONFIG.RD_PREFETCH_EN, default on)
- **Beat-based tracking:** Length in data width units (STREAM simplification)
- **Aligned addresses:** No alignment fixup logic (must be pre-aligned)
- **Descriptor chaining:** Follows next_descriptor_ptr for multi-buffer transfers
- **Interrupt generation:** MonBus IRQ event when gen_irq flag set
- **Commit-gated completion:** Channel completes only when all reads are done AND all writes are committed (B responses), not merely issued
- **Recoverable write-progress timeout:** Soft timeout re-arms each window and only escalates to a sticky CH_ERROR after cfg_sched_timeout_limit consecutive stalled windows (0 = never escalate)
- **Error handling:** Timeout detection, error aggregation from engines
- **MonBus integration:** State transition and IRQ event reporting

9.1.2 Block Diagram

9.1.3 Figure 2.4.1: Scheduler Block Diagram



Diagram

Source: [02_scheduler_block.mmd](#)

9.2 CRITICAL: Concurrent Read/Write Design

Why Concurrent Operation is Essential:

The scheduler runs read and write engines **CONCURRENTLY** in CH_XFER_DATA state. This prevents deadlock when transfer size exceeds SRAM buffer capacity:

Example: 100MB transfer with 2KB SRAM buffer

Sequential operation (WRONG):

1. Read 100MB → DEADLOCK at 2KB (SRAM full, can't complete read)

Concurrent operation (CORRECT):

1. Read starts filling SRAM → SRAM becomes full (2KB)
2. Read pauses (natural backpressure)
3. Write drains SRAM → SRAM has free space
4. Read resumes → Both continue until 100MB complete

Implementation: - Both `sched_rd_valid` and `sched_wr_valid` asserted in `CH_XFER_DATA` - Independent beat counters: `r_read_beats_remaining`, `r_write_beats_remaining` - Exit when **BOTH** counters reach zero

9.3 Read-Ahead Descriptor Prefetch

Runtime enable: `SCHED_CONFIG.RD_PREFETCH_EN` (default **on**). Applies to chained **legacy** descriptors only; `EXT` (row/col-major) descriptors always run lockstep.

Problem it solves. Without prefetch, when descriptor N's reads finish the FSM walks `CH_XFER_DATA` → `CH_COMPLETE` → `CH_NEXT_DESC` → `CH_FETCH_DESC` before the read engine can restart, and the read pipeline (AR → R) goes cold at every boundary. This is a per-descriptor **bubble** (~13 cycles measured) that caps efficiency on short-descriptor chains.

Mechanism. The descriptor FIFO (in the descriptor engine, already prefetched) is used as a one-entry holding buffer between the read and write sides. A single flag, `r_rd_ahead`, sequences the handoff:

- **SET** when the read flops load the next descriptor from the FIFO **head** *without draining the FIFO* — the read side runs one descriptor ahead, filling SRAM with descriptor N+1 while the write side still drains N.
- **CLEAR** when the write flops load that same descriptor and *drain the FIFO* (the write side catches up; read and write are lockstep again on N+1).

The SRAM sits between the two directions as an in-order FIFO, so racing the reads ahead is safe: descriptor N+1's data simply queues behind N's, and the write side counts beats to know where N ends and N+1 begins. The in-place advance collapses the four-state FSM walk into a single-cycle reload that stays in `CH_XFER_DATA`, and because the read pipeline never goes cold the AR→R refill latency is hidden.

Result. The per-descriptor bubble drops from ~13 cycles to 0 — chained descriptors stream at full rate; only a one-time fill/drain remains, independent of chain length. On-silicon A/B (16 x 64-beat descriptors, one bitstream): datapath utilisation 95.3% with prefetch on vs 76.1% off; longer descriptors reach 99-100%. See HAS Section 5.1.

Timing note. The write-commit accumulator's launch/advance addend is registered one cycle early (the add lands the following cycle; safe because commit B-responses always lag launches) so the descriptor-FIFO output and the launch/advance decode stay off the accumulator's combinational path — required to close 100 MHz timing with the added read-ahead fanout.

9.4 Parameters

```
parameter int CHANNEL_ID = 0;           // Channel identifier
parameter int NUM_CHANNELS = 8;         // Total channels in
system
parameter int CHAN_WIDTH = $clog2(NUM_CHANNELS); // Channel ID width
parameter int ADDR_WIDTH = 64;          // Address bus width
parameter int DATA_WIDTH = 512;        // Data bus width
(beats)

// Monitor Bus Parameters
parameter logic [7:0] MON_AGENT_ID = 8'h40; // STREAM Scheduler
Agent ID
parameter logic [3:0] MON_UNIT_ID = 4'h1;   // Unit identifier
parameter logic [5:0] MON_CHANNEL_ID = 6'h0; // Base channel ID

// Descriptor Width (FIXED at 256-bit for STREAM)
parameter int DESC_WIDTH = 256;

Validation:

// Scheduler only supports 256-bit STREAM descriptors
if (DESC_WIDTH != 256)
    $fatal("DESC_WIDTH must be 256 for STREAM scheduler");
```

9.5 Port List

9.5.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

9.5.2 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_channel_enable	input	1	Enable this channel
cfg_channel_reset	input	1	Channel soft reset (FSM → IDLE)
cfg_sched_timeout_cycles	input	32	Timeout threshold in clock cycles (32-bit, runtime config)
cfg_sched_timeout_enable	input	1	Enable timeout detection
cfg_sched_timeout_limit	input	8	Consecutive timeout windows before fatal escalation (0 = never escalate; from SCHED_TIMEOUT_LIMIT @ 0x208)

9.5.3 Status Interface

Status Interface

Signal	Direction	Width	Description
scheduler_idle	output	1	Scheduler idle flag
scheduler_state	output	7	Current FSM state (one-hot encoding)

9.5.4 Descriptor Engine Interface

Descriptor Engine Interface

Signal	Direction	Width	Description
descriptor_valid	input	1	Descriptor valid from descriptor engine
descriptor_ready	output	1	Scheduler ready to accept descriptor
descriptor_packet	input	256	256-bit STREAM descriptor (see format

Signal	Direction	Width	Description
descriptor_error	input	1	below) Error signal from descriptor engine

9.5.5 Data Read Interface

To AXI Read Engine:

Data Read Interface

Signal	Direction	Width	Description
sched_rd_val_id	output	1	Channel requests read
sched_rd_addr	output	ADDR_WIDTH	Source address (aligned, static during burst)
sched_rd_beats	output	32	Beats remaining to read

Completion from Read Engine:

Data Read Interface

Signal	Direction	Width	Description
sched_rd_done_strobe	input	1	Read burst completed (1-cycle pulse)
sched_rd_beats_done	input	32	Number of beats completed in burst
sched_rd_error	input	1	Read engine error (sticky)

9.5.6 Data Write Interface

To AXI Write Engine:

Data Write Interface

Signal	Direction	Width	Description
sched_wr_val_id	output	1	Channel requests write
sched_wr_ready	input	1	Engine ready for channel (completion handshake)

Signal	Direction	Width	Description
sched_wr_addr	output	ADDR_WIDTH	Destination address (aligned, static during burst)
sched_wr_beats	output	32	Beats remaining to write

Completion from Write Engine:

Data Write Interface

Signal	Direction	Width	Description
sched_wr_done_strobe	input	1	Write burst completed (1-cycle pulse)
sched_wr_beats_done	input	32	Number of beats completed in burst
sched_wr_error	input	1	Write engine error (sticky)

9.5.7 Error Signals

Error Signals

Signal	Direction	Width	Description
sched_error	output	1	Scheduler error output (aggregates rd/wr errors, sticky)

9.5.8 Monitor Bus Interface

Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid
mon_ready	input	1	Monitor bus ready
mon_packet	output	64	64-bit monitor bus packet

9.6 Interface

9.6.1 Clock and Reset

```
input logic          clk;
input logic          rst_n;      // Active-low
asynchronous reset
```

9.6.2 Configuration Interface

```
input logic          cfg_channel_enable;    //
Enable this channel
input logic          cfg_channel_reset;     //
Channel reset (soft reset)
input logic [31:0]   cfg_sched_timeout_cycles; //
Timeout threshold (runtime config)
input logic          cfg_sched_timeout_enable; //
Enable timeout detection
input logic [7:0]    cfg_sched_timeout_limit; //
Consecutive timeout windows before escalation (0=never)
```

Channel Reset Behavior: - `cfg_channel_reset` forces FSM to `CH_IDLE` immediately - Clears `descriptor_loaded` flag - Resets beat counters - Independent of global `rst_n`

Timeout Configuration: - Runtime configurable via `cfg_sched_timeout_cycles` (replaces compile-time `TIMEOUT_CYCLES` parameter) - Can be disabled/enabled dynamically via `cfg_sched_timeout_enable`

9.6.3 Status Interface

```
output logic         scheduler_idle;        //
Scheduler in CH_IDLE
output logic [6:0]   scheduler_state;      // Current
state (ONE-HOT)
```

State Encoding (ONE-HOT): - [0] = `CH_IDLE` - [1] = `CH_FETCH_DESC` - [2] = `CH_XFER_DATA` - [3] = `CH_COMPLETE` - [4] = `CH_NEXT_DESC` - [5] = `CH_ERROR` - [6] = Reserved

9.6.4 Descriptor Engine Interface

```
input logic          descriptor_valid;
output logic         descriptor_ready;
input logic [DESC_WIDTH-1:0] descriptor_packet; // 256-bit
STREAM descriptor
input logic          descriptor_error;        // Error
from descriptor engine
```

Descriptor Handshake: - `descriptor_ready` asserted in `CH_IDLE` or `CH_NEXT_DESC` - Descriptor captured when valid && ready - Supports descriptor chaining (`next_descriptor_ptr`)

9.6.5 Data Read Interface (to AXI Read Engine)

Request:

output logic <i>read access</i>	<code>sched_rd_valid;</code>	<i>// Request</i>
output logic [ADDR_WIDTH-1:0] <i>address (static base)</i>	<code>sched_rd_addr;</code>	<i>// Source</i>
output logic [31:0] <i>beats to read</i>	<code>sched_rd_beats;</code>	<i>// Total</i>

Completion:

input logic <i>engine completed beats</i>	<code>sched_rd_done_strobe;</code>	<i>// Read</i>
input logic [31:0] <i>of beats completed</i>	<code>sched_rd_beats_done;</code>	<i>// Number</i>

Error:

input logic <i>engine error</i>	<code>sched_rd_error;</code>	<i>// Read</i>
---	------------------------------	----------------

Address Management: - Scheduler provides **static base address** in `sched_rd_addr` - Read engine handles address increment internally - Scheduler does NOT update `sched_rd_addr` after each burst

Interface Timing Notes (November 2025 Updates): - Scheduler holds `sched_rd_valid` high while `sched_rd_beats > 0` - Engine reports completion via `sched_rd_done_strobe` (pulsed) - Scheduler decrements `r_read_beats_remaining` by `sched_rd_beats_done` - Process repeats until `r_read_beats_remaining == 0`

9.6.6 Data Write Interface (to AXI Write Engine)

Request:

output logic <i>write access</i>	<code>sched_wr_valid;</code>	<i>// Request</i>
input logic <i>grants access</i>	<code>sched_wr_ready;</code>	<i>// Engine</i>
output logic [ADDR_WIDTH-1:0] <i>Destination address (static base)</i>	<code>sched_wr_addr;</code>	<i>//</i>
output logic [31:0] <i>beats to write</i>	<code>sched_wr_beats;</code>	<i>// Total</i>

Completion:

input logic <i>engine completed beats</i>	<code>sched_wr_done_strobe;</code>	<i>// Write</i>
input logic [31:0] <i>of beats completed</i>	<code>sched_wr_beats_done;</code>	<i>// Number</i>

Error:

```
input logic                                sched_wr_error;           // Write
engine error
```

Interface Timing Notes (November 2025 Updates):

sched_wr_ready Timing: - REGISTERED OUTPUT from write engine - Not combinatorial from completion signals - Asserts 1 cycle after channel becomes ready to accept new request - Cleared 1 cycle after handshake (valid && ready) - **Do not expect immediate deassertion on handshake** - takes 1 cycle

Example Timing:

Cycle	sched_wr_valid	sched_wr_ready	sched_wr_done_strobe	Notes
0	0	0	0	Idle
1	1	0	0	Request
(not ready yet)				
2	1	1	0	
Handshake!				
3	1	0	0	Ready
cleared (registered)				
... (W-phase executes)				
50	1	0	1	B
response, done strobe				
51	1	1	0	Ready re-
asserts (next cycle)				

9.6.7 Monitor Bus Interface

```
output logic                                mon_valid;
input logic                                mon_ready;
output logic [63:0]                        mon_packet;
```

MonBus Events Generated: - State transitions (IDLE → FETCH_DESC, etc.) - IRQ event (when descriptor.gen_irq set) - Error events

9.7 Descriptor Format

9.7.1 STREAM Descriptor (256-bit)

```
typedef struct packed {
    logic [47:0] reserved;           // [255:208] Reserved
    logic [7:0] desc_priority;       // [207:200] Transfer
priority
    logic [3:0] channel_id;          // [199:196] Channel ID
(informational)
```

```

        logic          error;                // [195] Error flag
        logic          last;                // [194] Last in chain flag
        logic          gen_irq;             // [193] Generate interrupt
on completion
        logic          valid;               // [192] Valid descriptor
        logic [31:0]   next_descriptor_ptr; // [191:160] Next descriptor
address (0 = last)
        logic [31:0]   length;              // [159:128] Length in BEATS
        logic [63:0]   dst_addr;           // [127:64] Destination
address (aligned)
        logic [63:0]   src_addr;           // [63:0] Source address
(aligned)
} descriptor_t;

```

Field Constraints: - src_addr / dst_addr: Must be aligned to DATA_WIDTH (e.g., 64-byte aligned for 512-bit data) - length: Transfer size in **BEATS** (not bytes or chunks) - next_descriptor_ptr: 0 or address of next descriptor - valid: Must be 1 for descriptor to be accepted - last: Terminates chain (overrides next_descriptor_ptr) - gen_irq: Generates STREAM_EVENT_IRQ via MonBus when transfer completes

Descriptor Bit Positions:

```

DESC_SRC_ADDR:    [63:0]
DESC_DST_ADDR:    [127:64]
DESC_LENGTH:      [159:128]
DESC_NEXT_PTR:    [191:160]
DESC_VALID_BIT:   [192]
DESC_GEN_IRQ:     [193]
DESC_LAST:        [194]

```

9.8 FSM Operation

9.8.1 State Machine

States (ONE-HOT encoded):

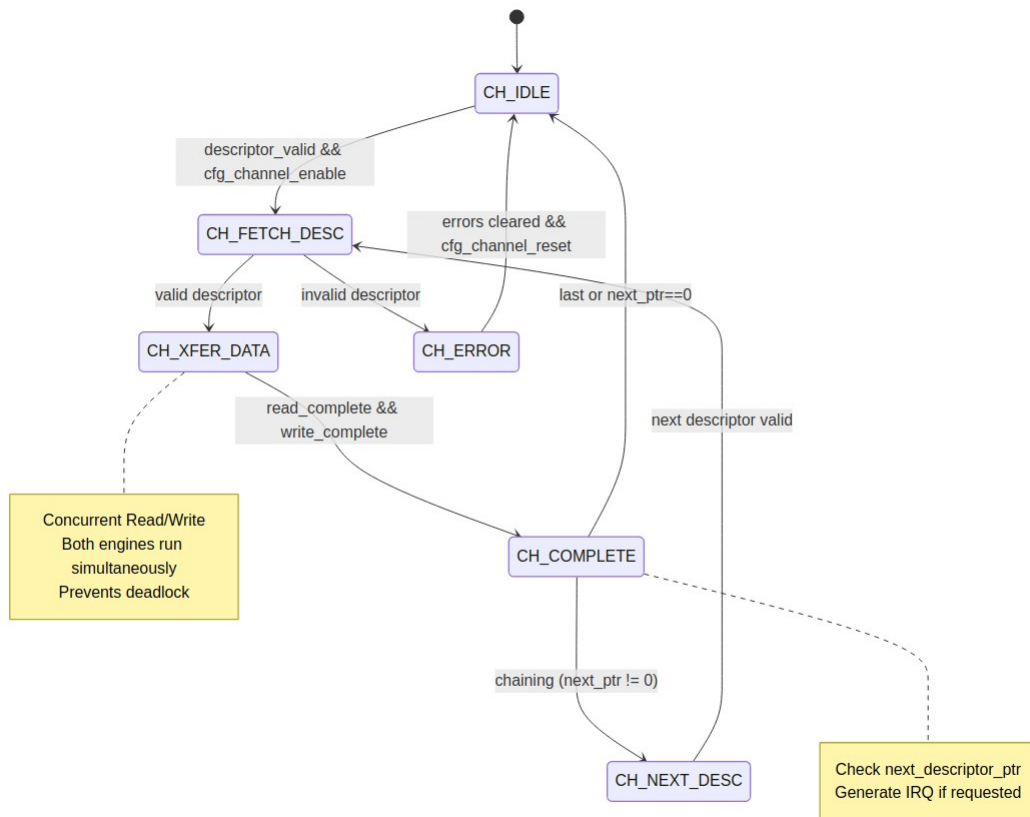
```

CH_IDLE           - Waiting for descriptor
CH_FETCH_DESC     - Latch and validate descriptor
CH_XFER_DATA      - Concurrent read/write transfer
CH_COMPLETE       - Transfer done, check chaining
CH_NEXT_DESC      - Fetch next chained descriptor
CH_ERROR          - Error condition

```

FSM Flow:

9.8.2 Figure 2.4.2: Scheduler FSM



Diagram

Source: [02_scheduler_block.mmd](#)

9.8.3 State Transitions

CH_IDLE: - **Wait for:** descriptor_valid && cfg_channel_enable - **Action:** Assert descriptor_ready - **Next:** CH_FETCH_DESC (when handshake occurs)

CH_FETCH_DESC: - **Action:** - Latch descriptor fields into r_descriptor - Initialize working registers (r_src_addr, r_dst_addr, r_*_beats_remaining) - Validate descriptor.valid bit - **Next:** - CH_XFER_DATA (if valid) - CH_ERROR (if invalid)

CH_XFER_DATA: - **Action:** - Assert **BOTH** sched_rd_valid and sched_wr_valid (concurrent operation!) - Decrement r_read_beats_remaining on sched_rd_done_strobe - Decrement r_write_beats_remaining on sched_wr_done_strobe - Monitor timeout counter - **Exit When:** r_read_beats_remaining == 0 && r_write_beats_remaining == 0 - **Next:** CH_COMPLETE

CH_COMPLETE: - **Action:** - Generate MonBus IRQ event (if gen_irq set) - Check next_descriptor_ptr and last flag - Clear descriptor_loaded flag - **Next:** - CH_NEXT_DESC (if chaining) - CH_IDLE (if last or no chain)

CH_NEXT_DESC: - **Wait for:** descriptor_valid (descriptor engine fetches next) - **Action:** Assert descriptor_ready - **Next:** CH_FETCH_DESC

CH_ERROR: - **STICKY STATE:** Once in error, stays here until channel reset - **Action:** Report error via MonBus - **Recovery:** Requires cfg_channel_reset assertion (or global reset) - **Note:** scheduler_idle does **NOT** assert in this state — a faulted channel must not report idle. The fault is surfaced via sched_error/CHANNEL_ERROR instead.

9.9 Beat Tracking

9.9.1 Independent Counters

Initialization (CH_FETCH_DESC):

```
r_read_beats_remaining <= r_descriptor.length;
r_write_beats_remaining <= r_descriptor.length;
```

Decrement (CH_XFER_DATA):

```
// Read progress (independent)
if (sched_rd_done_strobe) begin
    r_read_beats_remaining <= (r_read_beats_remaining >=
sched_rd_beats_done) ?
                                (r_read_beats_remaining -
sched_rd_beats_done) : 32'h0;
end

// Write progress (independent)
if (sched_wr_done_strobe) begin
    r_write_beats_remaining <= (r_write_beats_remaining >=
sched_wr_beats_done) ?
                                (r_write_beats_remaining -
sched_wr_beats_done) : 32'h0;
end
```

Saturation: - Counters saturate at 0 (prevent underflow) - Safety check for engine misbehavior

9.9.2 Completion Detection

Write completion is gated on COMMIT, not issue. Read completion tracks the read-beat counter, but write completion tracks a separate r_write_beats_to_commit counter that decrements only on write-data **commit** (B responses), not on AW issue. This prevents the channel from signalling done/interrupt while write data is still draining to memory.

Combinational Flags:

```

w_read_complete      = (r_read_beats_remaining == 0);
// Write is complete only when all issued beats have been COMMITTED (B
responses),
// not merely issued (r_write_beats_remaining).
w_write_complete     = (r_write_beats_to_commit == 0);
w_transfer_complete = w_read_complete && w_write_complete;

```

Write-request gating: sched_wr_valid is additionally gated on remaining *issue* beats (r_write_beats_remaining != 0). Once all beats are issued, the scheduler stops requesting new writes but stays in CH_XFER_DATA waiting for the outstanding commits (B responses) to retire r_write_beats_to_commit.

State Exit:

```

// In CH_XFER_DATA:
if (w_transfer_complete) begin // all reads done AND all writes
committed
    w_next_state = CH_COMPLETE;
end

```

9.9.3 Multiple Requests per Descriptor

For large transfers, scheduler issues multiple requests as engines complete work:

Descriptor: length = 256 beats (16KB @ 512-bit data)
Engine burst size: 16 beats (configured via cfg_axi_wr_xfer_beats)

Request sequence:

Request 1: sched_wr_valid=1, sched_wr_beats=256, ready → handshake
Engine executes 16-beat burst
sched_wr_done_strobe=1, sched_wr_beats_done=16
Scheduler updates: 256 - 16 = 240 beats remaining

Request 2: sched_wr_valid=1, sched_wr_beats=240, ready → handshake
Engine executes 16-beat burst
sched_wr_done_strobe=1, sched_wr_beats_done=16
Scheduler updates: 240 - 16 = 224 beats remaining

... (continues for 16 requests total)

Request 16: sched_wr_valid=1, sched_wr_beats=16, ready → handshake
Engine executes 16-beat burst
sched_wr_done_strobe=1, sched_wr_beats_done=16
Scheduler updates: 16 - 16 = 0 beats remaining →

COMPLETE

Key Point: Scheduler holds sched_wr_valid high and keeps reissuing requests as long as sched_wr_beats > 0. Engines handle each request independently.

9.10 Address Management

9.10.1 Static Base Address

Scheduler Provides:

```
assign sched_rd_addr = r_src_addr; // Static base, set in  
CH_FETCH_DESC  
assign sched_wr_addr = r_dst_addr; // Static base, set in  
CH_FETCH_DESC
```

Scheduler Does NOT: - Increment addresses after each burst - Calculate byte offsets - Handle alignment

Engine Responsibility: - Read engine: `m_axi_araddr = sched_rd_addr + (beats_issued << AXSIZE)` - Write engine: `m_axi_awaddr = sched_wr_addr + (beats_issued << AXSIZE)`

9.11 Timeout Detection (Recoverable)

The write-progress timeout is a **recoverable liveness fault**, not a hard error. A single expired window does NOT wedge the channel into CH_ERROR; the timeout only escalates to a fatal, sticky error after a configurable number of consecutive windows with no write progress.

9.11.1 Timeout Counter

Configuration: - Runtime configurable via `cfg_sched_timeout_cycles` (32-bit) - Can be enabled/disabled via `cfg_sched_timeout_enable` - Escalation limit via `cfg_sched_timeout_limit` (8-bit, from SCHED_TIMEOUT_LIMIT @ 0x208) - Replaces compile-time TIMEOUT_CYCLES parameter

Increment (counts stalled write cycles):

```
// In CH_XFER_DATA when the write request is stalled (valid high,  
ready low)  
if (cfg_sched_timeout_enable && sched_wr_valid && !sched_wr_ready)  
begin  
    r_timeout_counter <= r_timeout_counter + 1;  
end
```

Window Expiry (re-arming, one pulse per window):

```
// w_timeout_expired is a one-cycle PULSE per elapsed window (not a  
latched level).  
assign w_timeout_expired = cfg_sched_timeout_enable &&  
    (r_timeout_counter >=
```



```
cfg_sched_timeout_cycles);
// On expiry the counter re-arms (clears) so a new window can begin.
```

9.11.2 Consecutive-Window Strike Counter and Escalation

```
// r_timeout_strikes counts consecutive expired windows with NO write
progress.
// Any write progress (done or commit strobe) clears the strike
counter.
if (write_progress)           r_timeout_strikes <= 8'h0;
else if (w_timeout_expired)   r_timeout_strikes <=
r_timeout_strikes + 8'h1;

// Escalate to a fatal (sticky CH_ERROR) fault only after
cfg_sched_timeout_limit
// consecutive windows. cfg_sched_timeout_limit == 0 means never
escalate.
assign w_timeout_escalate = (cfg_sched_timeout_limit != 8'd0) &&
(r_timeout_strikes >=
cfg_sched_timeout_limit);
```

Action: - Each expired window emits a MonBus timeout event but leaves the channel running (soft timeout). - Any write progress (a sched_wr_done_strobe or a commit strobe) clears r_timeout_strikes. - Only w_timeout_escalate drives the FSM to the sticky CH_ERROR; recovery then requires cfg_channel_reset (or global reset). - With cfg_sched_timeout_limit == 0, the timeout never escalates (pure soft-timeout reporting).

9.12 Error Handling

9.12.1 Error Sources

External: - descriptor_error - Descriptor engine reports error - sched_rd_error - Read engine error (AXI RRESP != OKAY, etc.) - sched_wr_error - Write engine error (AXI BRESP != OKAY, etc.)

Internal: - w_timeout_escalate - Write-progress timeout persisted for cfg_sched_timeout_limit consecutive windows (a single w_timeout_expired window is recoverable and does NOT trigger CH_ERROR) - !r_descriptor.valid - Invalid descriptor in CH_FETCH_DESC

9.12.2 Sticky Error Flags

```
logic r_read_error_sticky; // Set on sched_rd_error, cleared in
CH_IDLE
logic r_write_error_sticky; // Set on sched_wr_error, cleared in
CH_IDLE
```

```
logic r_descriptor_error;    // Set on descriptor_error or validation failure
```

Set Condition:

```
if (sched_rd_error)
    r_read_error_sticky <= 1'b1;

if (sched_wr_error)
    r_write_error_sticky <= 1'b1;
```

Clear Condition:

```
if (r_current_state == CH_IDLE)
    r*_error_sticky <= 1'b0;
```

9.12.3 Error Recovery

Error Transition:

```
// Any state with a FATAL error condition (note: w_timeout_escalate,
// NOT the
// per-window w_timeout_expired pulse, which is a recoverable soft
// timeout).
if (descriptor_error || sched_rd_error || sched_wr_error ||
    r_read_error_sticky || r_write_error_sticky || w_timeout_escalate)
begin
    w_next_state = CH_ERROR;
end
```

CH_ERROR is STICKY:

```
CH_ERROR: begin
    // Error state - STICKY, stay here until reset
    // Once in error, only way out is through reset
    w_next_state = CH_ERROR;
end
```

Recovery: - CH_ERROR is a **sticky state** - does NOT auto-recover - Software **must** assert `cfg_channel_reset` (or global `rst_n`) - On channel reset: FSM → CH_IDLE, sticky flags cleared

9.13 Interrupt Generation

9.13.1 IRQ via MonBus

Trigger: - Descriptor completes (CH_COMPLETE state) - `r_descriptor.gen_irq` flag set

MonBus Event:

```

// In CH_COMPLETE state with gen_irq set
mon_packet = {
    MON_AGENT_ID,           // [63:56] Agent ID (0x40 = STREAM
Scheduler)
    MON_UNIT_ID,           // [55:52] Unit ID
    MON_CHANNEL_ID,        // [51:46] Channel ID
    STREAM_EVENT_IRQ,      // [45:40] Event code (IRQ)
    descriptor_fields      // [39:0] Descriptor info
};

```

No Separate IRQ Signal: - IRQ communicated via MonBus only - Software monitors MonBus for IRQ events - Event includes channel ID for routing

9.14 Descriptor Chaining

9.14.1 Chain Detection

In CH_COMPLETE:

```

if (r_descriptor.next_descriptor_ptr != 32'h0 && !r_descriptor.last)
begin
    w_next_state = CH_NEXT_DESC; // Chain to next descriptor
end else begin
    w_next_state = CH_IDLE;      // Complete (last or no chain)
end

```

9.14.2 Chain Termination

Explicit Termination: - next_descriptor_ptr == 0 → Stop - last == 1 → Stop (overrides next_descriptor_ptr)

Example Chain:

Descriptor 0 @ 0x1000:
 src_addr = 0x2000, dst_addr = 0x3000, length = 64
 next_descriptor_ptr = 0x1040, last = 0
 → Chains to next

Descriptor 1 @ 0x1040:
 src_addr = 0x2100, dst_addr = 0x3100, length = 32
 next_descriptor_ptr = 0x0000, last = 1
 → Last in chain

9.15 MonBus Integration

9.15.1 Event Types

State Transitions: - IDLE → FETCH_DESC: Descriptor fetch start - FETCH_DESC → XFER_DATA: Transfer start - XFER_DATA → COMPLETE: Transfer complete - COMPLETE → NEXT_DESC: Chain fetch - Any → ERROR: Error occurred

Special Events: - STREAM_EVENT_IRQ: Interrupt generation (gen_irq flag) - STREAM_EVENT_TIMEOUT: Timeout expired - STREAM_EVENT_ERROR: Error condition

9.15.2 MonBus Packet Format

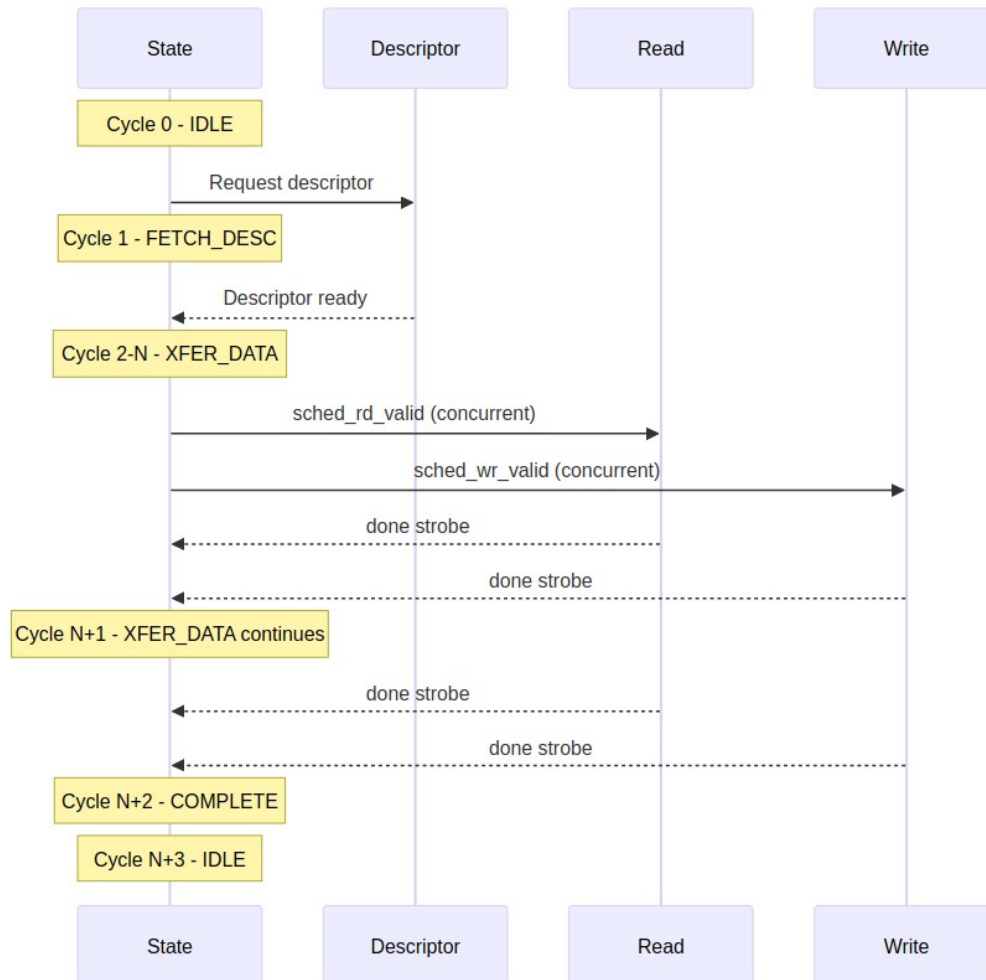
Generic Event:

[63:56] - MON_AGENT_ID (0x40)
[55:52] - MON_UNIT_ID
[51:46] - MON_CHANNEL_ID + CHANNEL_ID
[45:40] - Event code
[39:0] - Event-specific data

9.16 Timing Diagrams

9.16.1 Normal Transfer (No Chaining)

9.16.2 Figure 2.4.3: Scheduler Normal Transfer Timing



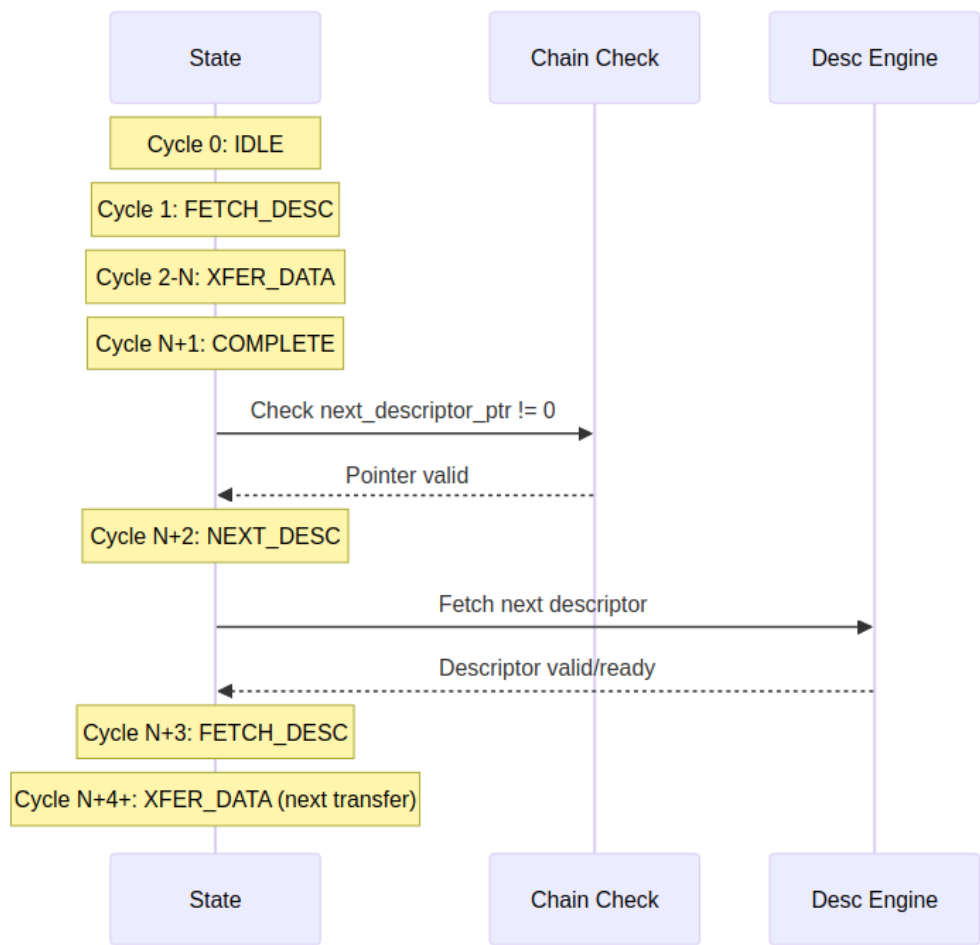
Scheduler Normal Transfer Timing

Source: [04_scheduler_normal_transfer.mmd](#)

Notes: - Both read and write run concurrently in XFER state - Independent done strobes decrement separate counters

9.16.3 Descriptor Chaining

9.16.4 Figure 2.4.4: Scheduler Descriptor Chaining Timing



Scheduler Descriptor Chaining Timing

Source: [04_scheduler_chaining.mmd](#)

9.16.5 Single-Descriptor Transfer (Detailed)

Cycle	State	sched_wr_valid	sched_wr_ready	sched_wr_beats
Notes				
-----	-----	-----	-----	-----

0	IDLE	0	0	0
Waiting				
1	FETCH_DESC	0	0	0
Request descriptor				
2	FETCH_DESC	0	0	0
(fetch latency)				

```

    3    XFER_DATA    0                0                0
Descriptor received
...    (read phase)
   50    XFER_DATA    1                0                256
Assert write request
   51    XFER_DATA    1                1                256
Engine ready, handshake!
   52    XFER_DATA    1                0                256
Ready cleared (registered)
...    (engine executes burst)
  100    XFER_DATA    1                0                256
done_strobe=1, beats_done=16
  101    XFER_DATA    1                1                240
Ready re-asserts, new beats_remaining
  102    XFER_DATA    1                0                240
Handshake again
...    (continues until beats_remaining == 0)

```

9.16.6 Chained Descriptors (Detailed)

Descriptor chain:

Desc 0: length=128, next_ptr=0x1000_0100

Desc 1: length=64, next_ptr=0

Cycle	State	Descriptor	sched_wr_beats	Notes
0	IDLE	-	0	Start
1	FETCH_DESC	Req @0x1000	0	Fetch first
3	XFER_DATA	Desc 0	128	Transfer desc 0
...	(8 bursts × 16 beats)			
200	COMPLETE	Desc 0	0	Transfer done
201	NEXT_DESC	Check ptr	0	next_ptr = 0x1000_0100
202	FETCH_DESC	Req @0x1100	0	Fetch second
204	XFER_DATA	Desc 1	64	Transfer desc 1
...	(4 bursts × 16 beats)			
400	COMPLETE	Desc 1	0	Transfer done
401	NEXT_DESC	Check ptr	0	next_ptr = 0 →
	terminate			
402	IDLE	-	0	Chain complete

9.17 Testing

Test Location: projects/components/dmas/stream/dv/tests/fub_tests/scheduler/

Key Test Scenarios:

1. **Single descriptor transfer** - Basic operation
 2. **Descriptor chaining** - 2-4 descriptors linked
 3. **Concurrent read/write** - Verify no deadlock with small SRAM
 4. **Large transfer (> SRAM)** - 100MB transfer with 2KB SRAM
 5. **IRQ generation** - gen_irq flag set
 6. **Error handling** - Descriptor, read, write errors
 7. **Timeout detection** - Engine stall scenarios
 8. **Channel reset** - cfg_channel_reset during transfer
 9. **Runtime timeout config** - Change cfg_sched_timeout_cycles dynamically
 10. **sched_rd_ready / sched_wr_ready timing validation** - Verify registered ready behavior
-

9.18 Performance Considerations

9.18.1 Concurrent Operation Benefit

Without Concurrency (Sequential): - Max transfer size = SRAM buffer size - Deadlock when transfer > buffer - Throughput = min(read_bw, write_bw)

With Concurrency: - No transfer size limit - Natural flow control via SRAM full/empty - Throughput = max(read_bw, write_bw) (pipeline overlap)

9.18.2 Example Performance

Configuration: - DATA_WIDTH = 512 bits (64 bytes/beat) - SRAM = 2KB (32 beats) - Transfer = 100MB (1,562,500 beats)

Sequential (hypothetical): - DEADLOCK at 2KB (can't complete read)

Concurrent: - Read fills SRAM (32 beats) - Write drains SRAM concurrently - Both engines sustain ~0.9 beats/cycle - Total time: ~1.7M cycles

9.19 Observation and Diagnostics (v0.92+)

9.19.1 OBS_FLAGS Register Mapping

The scheduler exposes two observation flags via the APB configuration block's OBS_FLAGS register:

Bit	Signal	Meaning
[0]	SCH_ERROR_STICKY	Error sticky flag (descriptor invalid, descriptor address invalid, AXI error, or timeout detected)
[1]	SCH_TIMEOUT	Timeout active flag (waiting on AXI master and timeout counter has elapsed)

Both flags are readable via APB OBS_FLAGS[CHANNEL_SEL] after selecting the channel via OBS_CTRL. The error sticky persists until cfg_channel_reset is asserted; the timeout flag clears when AXI activity resumes.

9.19.2 Observation Data Fields (OBS_DATA0 / OBS_DATA1)

Channel-specific diagnostic data is mirrored into the APB registers to enable software monitoring without stopping the scheduler. Contents may include current beat counter state, descriptor pointer, or FSM state depending on the channel's execution phase.

9.20 Related Documentation

- **Descriptor Engine:** 05_descriptor_engine.md - Descriptor fetch
- **AXI Read Engine:** 06_axi_read_engine.md - Source data read
- **AXI Write Engine:** 12_axi_write_engine.md - Destination data write
- **SRAM Controller:** 08_sram_controller.md - Buffer management
- **Scheduler Group:** 03_scheduler_group.md - Single-channel integration
- **Scheduler Group Array:** 02_scheduler_group_array.md - Multi-channel integration

9.21 Revision History

Revision History

Date	Version	Changes
2025-10-17	1.0	Initial documentation with old signal names (datard_, datawr_)
2025-11-16	1.5	Enhanced documentation

Date	Version	Changes
2025-11-21	2.0	with detailed sections Merged documentation:- Updated all signal names (sched_rd_, sched_wr_- Added runtime timeout configuration (cfg_sched_timeout_cycles/e nable)- Registered ready signal timing clarification- Added multiple requests per descriptor section- Enhanced beat tracking and error handling details- Updated all code examples and timing diagrams- Added timing examples for chained descriptors- Combined best content from multiple documentation sources
2025-11-30	2.1	RTL Sync Update:- CH_ERROR is now STICKY (requires reset to recover)- scheduler_idle asserts in CH_ERROR state- Updated related documentation references
2026-07-02	2.2	RTL Sync Update:- Added cfg_sched_timeout_limit port (SCHED_TIMEOUT_LIMIT @ 0x208)- Write completion now gated on B-response commit (r_write_beats_to_commit); sched_wr_valid gated on remaining issue beats-

Date	Version	Changes
		Write-progress timeout is now recoverable: re-arms per window, escalates to CH_ERROR only after <code>cfg_sched_timeout_limit</code> consecutive stalled windows- <code>scheduler_idle</code> now EXCLUDES CH_ERROR (a faulted channel is no longer reported idle) — corrects the 2.1 note

Last Updated: 2026-07-02 (matched to current RTL implementation)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

10 Descriptor Engine

Module: `descriptor_engine.sv` **Location:** `projects/components/dmas/stream/rtl/fub/`
Category: FUB (Functional Unit Block) **Parent:** `scheduler_group.sv` **Status:** Implemented **Last Updated:** 2025-11-30

10.1 Overview

The `descriptor_engine` module is an autonomous descriptor fetch engine with chaining support. It fetches 256-bit STREAM descriptors from memory via AXI4 master interface and provides two operating modes for kick-off.

10.1.1 Key Features

- **Two Operating Modes:**
 - APB-initiated: Software writes descriptor address, engine fetches

- Autonomous chaining: Engine automatically fetches next_descriptor_ptr
 - **Two-FIFO Architecture:**
 - Descriptor address FIFO: Holds addresses to fetch
 - Descriptor data FIFO: Buffers fetched descriptors for scheduler
 - **Address Range Validation:** Two configurable ranges for security
 - **MonBus Event Reporting:** Fetch complete and error events
-

10.2 Architecture

10.2.1 Operating Flow

APB Mode:

APB write -> skid buffer -> desc addr FIFO -> AXI fetch -> desc data FIFO -> scheduler

Chaining Mode:

Descriptor complete -> extract next_ptr -> validate -> desc addr FIFO -> AXI fetch -> desc data FIFO -> scheduler (repeat until last=1 or next_ptr=0)

10.2.2 FSM States

```
typedef enum logic [2:0] {
    RD_IDLE           = 3'b000, // Waiting for descriptor address
    RD_ISSUE_ADDR     = 3'b001, // Issue AXI AR transaction
    RD_WAIT_DATA      = 3'b010, // Wait for AXI R response
    RD_COMPLETE       = 3'b011, // Descriptor fetched, push to FIFO
    RD_ERROR          = 3'b100  // Error occurred
} read_engine_state_t;
```

10.2.3 Descriptor Format (256-bit STREAM)

Descriptor Format

Bits	Field	Description
[63:0]	src_addr	Source memory address (64-bit, must be aligned)
[127:64]	dst_addr	Destination memory address (64-bit, must be aligned)
[159:128]	length	Transfer length in BEATS (not bytes!)

Bits	Field	Description
[191:160]	next_descriptor_ptr	Address of next descriptor (32-bit, 0 = last)
[192]	valid	Descriptor valid flag
[193]	gen_irq	Generate interrupt on completion
[194]	last	Last descriptor in chain (explicit termination)
[195]	error	Descriptor error flag
[199:196]	channel_id	Channel identifier
[207:200]	priority	Descriptor priority
[255:208]	reserved	Reserved for future use

10.3 Parameters

Parameters

Parameter	Type	Default	Description
CHANNEL_ID	int	0	Channel identifier
NUM_CHANNELS	int	32	Total number of channels
CHAN_WIDTH	int	$\lceil \log_2(\text{NUM_CHANNELS}) \rceil$	Channel ID width
ADDR_WIDTH	int	64	Address bus width
AXI_ID_WIDTH	int	8	AXI transaction ID width
FIFO_DEPTH	int	8	Descriptor data FIFO depth
DESC_ADDR_FIFO_DEPTH	int	2	Descriptor address FIFO

Parameter	Type	Default	Description
TIMEOUT_CYCLE S	int	1000	depth AXI timeout threshold

10.3.1 Monitor Bus Parameters

Monitor Bus Parameters

Parameter	Type	Default	Description
MON_AGENT_ID	8-bit	0x10	Descriptor Engine Agent ID
MON_UNIT_ID	4-bit	0x1	Unit identifier
MON_CHANNEL_I D	6-bit	0x0	Base channel ID

10.4 Port List

10.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

10.4.2 APB Programming Interface

APB Programming Interface

Signal	Direction	Width	Description
apb_valid	input	1	Descriptor address valid
apb_ready	output	1	Ready to accept address
apb_addr	input	ADDR_WIDTH	Descriptor

Signal	Direction	Width	Description
			address

10.4.3 Scheduler Interface

Scheduler Interface

Signal	Direction	Width	Description
channel_idle	input	1	Scheduler idle (enables APB)
descriptor_valid	output	1	Descriptor available
descriptor_ready	input	1	Scheduler ready to accept
descriptor_packet	output	256	FIXED 256-bit descriptor
descriptor_error	output	1	Fetch error flag

10.4.4 Enhanced Control Outputs

Enhanced Control Outputs

Signal	Direction	Width	Description
descriptor_eos	output	1	End of Stream (future use)
descriptor_eol	output	1	End of Line (future use)
descriptor_eod	output	1	End of Data (future use)
descriptor_type	output	2	Packet type (future use)

10.4.5 AXI4 AR Channel

AXI4 AR Channel

Signal	Direction	Width	Description
ar_valid	output	1	Address valid
ar_ready	input	1	Address ready

Signal	Direction	Width	Description
ar_addr	output	ADDR_WIDTH	Address
ar_len	output	8	Burst length - 1
ar_size	output	3	Burst size (log2 bytes)
ar_burst	output	2	Burst type (INCR)
ar_id	output	AXI_ID_WIDTH	Transaction ID
ar_lock	output	1	Lock type
ar_cache	output	4	Cache attributes
ar_prot	output	3	Protection attributes
ar_qos	output	4	QoS value
ar_region	output	4	Region identifier

10.4.6 AXI4 R Channel (FIXED 256-bit)

AXI4 R Channel

Signal	Direction	Width	Description
r_valid	input	1	Read data valid
r_ready	output	1	Read data ready
r_data	input	256	Read data (FIXED 256-bit)
r_resp	input	2	Response
r_last	input	1	Last beat
r_id	input	AXI_ID_WIDTH	Transaction ID

10.4.7 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_prefetch_enable	input	1	Enable descriptor prefetch (functional): 0 = on-demand (≈ 1 buffered ahead), 1 = buffer up to cfg_fifo_threshold ahead
cfg_fifo_threshold	input	4	Max descriptors buffered ahead when prefetch enabled (min 1; 0 is clamped to 1)
cfg_addr0_base	input	ADDR_WIDTH	Address range 0 base
cfg_addr0_limit	input	ADDR_WIDTH	Address range 0 limit
cfg_addr1_base	input	ADDR_WIDTH	Address range 1 base
cfg_addr1_limit	input	ADDR_WIDTH	Address range 1 limit
cfg_channel_reset	input	1	Channel reset

10.4.8 Status Interface

Status Interface

Signal	Direction	Width	Description
descriptor_engine_idle	output	1	Engine idle

10.4.9 Monitor Bus Interface

Monitor Bus Interface

Signal	Direction	Width	Description
mon_valid	output	1	Monitor packet valid

Signal	Direction	Width	Description
mon_ready	input	1	Monitor packet ready
mon_packet	output	64	Monitor packet data

10.5 Operation

10.5.1 APB Acceptance Policy

APB writes are accepted ONLY when ALL conditions met: 1. `!r_channel_reset_active` - Not in channel reset 2. `w_desc_addr_fifo_empty` - No pending descriptor fetches 3. `channel_idle` - Scheduler completed all prior descriptors 4. `!r_apb_ip` - No APB transaction currently in progress 5. `apb_addr != 0` - Address 0 is invalid

10.5.2 Autonomous Chaining Decision Tree

Level 1: Basic Eligibility (`w_chain_condition`) - `next_descriptor_ptr != 0` (non-null pointer) - `descriptor.last == 0` (not explicitly marked as last) - `descriptor.valid == 1` (valid descriptor)

Level 2: Address Validation (`w_next_addr_valid`) - `next_addr` within `cfg_addr0_base..cfg_addr0_limit` OR - `next_addr` within `cfg_addr1_base..cfg_addr1_limit`

Level 3: Prefetch Throttle Gate (`w_should_chain`)

`cfg_prefetch_enable` and `cfg_fifo_threshold` are **functional** and bound how many descriptors the engine buffers ahead of the scheduler:

- **Prefetch disabled (`cfg_prefetch_enable = 0`):** on-demand mode. The prefetch limit is 1, so the engine keeps roughly one descriptor buffered ahead — it fetches the next descriptor only after the scheduler drains the current one.
- **Prefetch enabled (`cfg_prefetch_enable = 1`):** the engine buffers up to `cfg_fifo_threshold` descriptors ahead (minimum 1; a threshold of 0 is clamped to 1 to avoid stalling chaining).

An immediate chain push (`w_should_chain`) fires at descriptor commit only when Levels 1-2 hold, the descriptor FIFO has space (`w_desc_fifo_wr_ready`), and the throttle allows another fetch (`w_desc_fifo_count < prefetch_limit`).

Deferred chaining: A chain fetch that is eligible at commit but blocked by the throttle (or a full address FIFO) is **not dropped** — the engine latches the pending `next_descriptor_ptr`

(r_chain_pending / r_pending_chain_addr) and reissues it later, once the output descriptor FIFO drains below the prefetch limit.

10.5.3 AXI Transaction

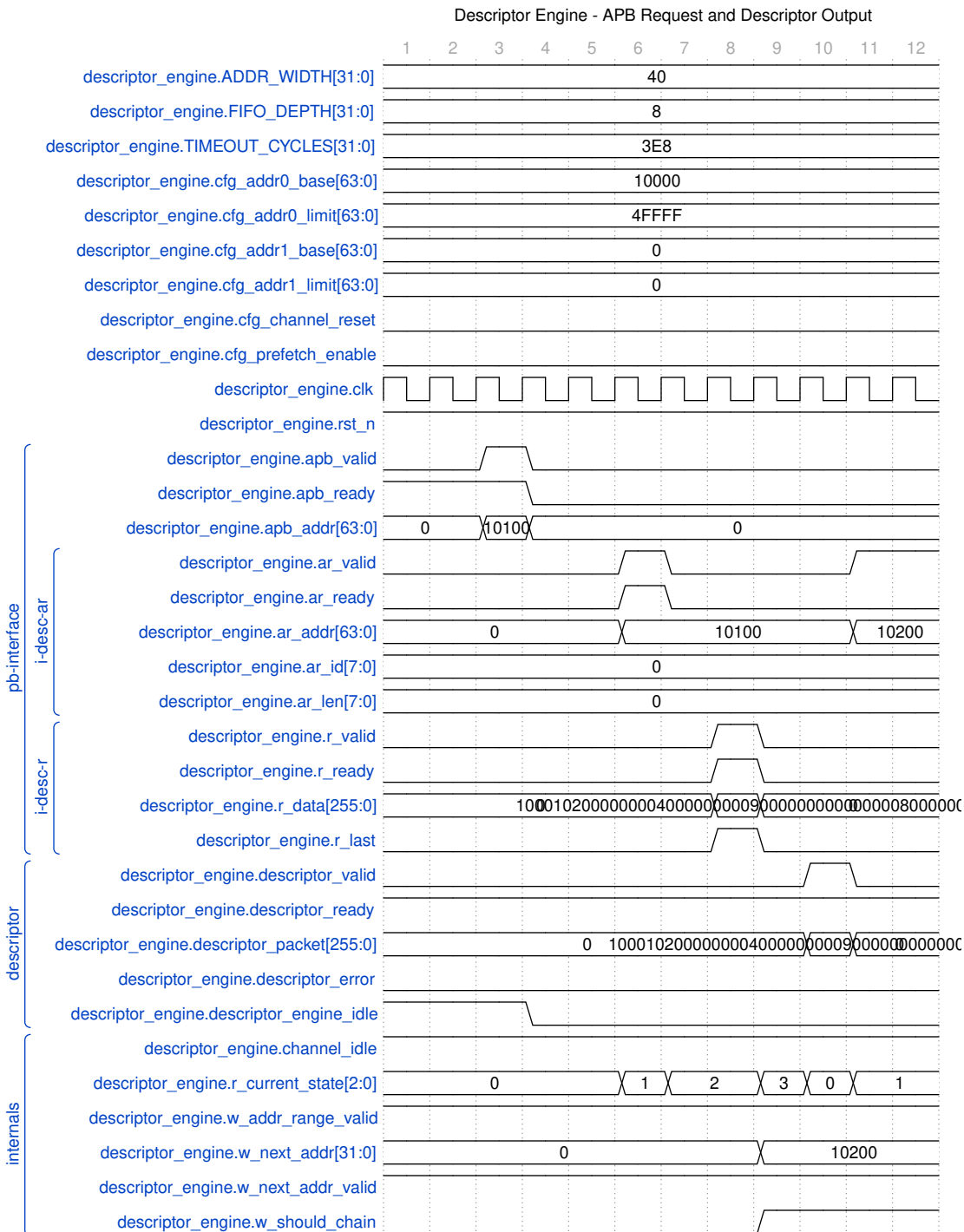
```
ar_len = 8'h00;           // Single beat transfer
ar_size = 3'b110;         // 64 bytes (512-bit for 256-bit descriptor)
ar_burst = 2'b01;         // INCR burst type
ar_id = {{padding}, CHANNEL_ID}; // Channel ID in lower bits
ar_cache = 4'b0010;       // Normal non-cacheable bufferable
ar_prot = 3'b000;         // Data, secure, unprivileged
```

10.6 Timing Diagrams

10.6.1 APB Kick-off Sequence

The following diagram shows the APB-initiated descriptor kick-off sequence:

10.6.1.1 Waveform 2.5.1: Descriptor Engine APB Basic Kick-off



Descriptor Engine APB Basic

Source: [descriptor_engine_apb_basic.json](#)

10.7 Integration Example

```
descriptor_engine #(
    .CHANNEL_ID          (ch),
    .NUM_CHANNELS        (8),
    .ADDR_WIDTH          (64),
    .AXI_ID_WIDTH        (8),
    .FIFO_DEPTH          (8),
    .MON_AGENT_ID        (8'h10 + ch)
) u_descriptor_engine (
    .clk                  (clk),
    .rst_n                (rst_n),

    // APB interface
    .apb_valid            (apb_valid),
    .apb_ready            (apb_ready),
    .apb_addr             (apb_addr),

    // Scheduler interface
    .channel_idle         (scheduler_idle),
    .descriptor_valid     (desc_valid),
    .descriptor_ready     (desc_ready),
    .descriptor_packet    (desc_packet),
    .descriptor_error     (desc_error),

    // AXI interface
    .ar_valid             (ar_valid),
    .ar_ready             (ar_ready),
    .ar_addr              (ar_addr),
    // ... full AR/R interface

    // Configuration
    .cfg_prefetch_enable  (cfg_desceng_prefetch),
    .cfg_addr0_base       (cfg_desceng_addr0_base),
    .cfg_addr0_limit      (cfg_desceng_addr0_limit),
    .cfg_addr1_base       (cfg_desceng_addr1_base),
    .cfg_addr1_limit      (cfg_desceng_addr1_limit),
    .cfg_channel_reset    (cfg_channel_reset),

    // Status
    .descriptor_engine_idle (desc_engine_idle),

    // Monitor bus
    .mon_valid            (desceng_mon_valid),
```

```
        .mon_ready          (desceng_mon_ready) ,  
        .mon_packet        (desceng_mon_packet)  
    );
```

10.8 Common Issues

10.8.1 Issue 1: APB Write Not Accepted

Symptom: apb_ready never asserts

Root Causes: 1. Channel not idle (channel_idle = 0) 2. Previous APB transaction still in progress (r_apb_ip = 1) 3. Descriptor address FIFO not empty 4. Channel reset active

Solution: Wait for channel to complete current work before next kick-off.

10.8.2 Issue 2: Descriptor Chain Stops Early

Symptom: Chaining stops before expected

Root Causes: 1. next_descriptor_ptr is 0 or out of valid range 2. last flag set in descriptor 3. AXI error during fetch

Debug: Check descriptor memory contents and address range configuration.

10.9 Monitoring (BOTH-End Descriptor Path Observability)

10.9.1 Descriptor Fetch Monitoring (Master Side)

The descriptor engine's AXI4 master interface is monitored by a dedicated AXI monitor wrapper with: - **Agent ID:** 8 (descriptor fetch monitor) - **Unit ID:** 1 (STREAM subsystem) - **Monitored Port:** m_axi_desc_* (AXI4 read burst for descriptor fetch)

This monitor tracks: - Descriptor fetch initiation (address phase) - Fetch completion (data phase) - Fetch errors or timeouts (AXI SLVERR, no response)

10.9.2 Descriptor Consume Monitoring (Slave Side)

The scheduler's consumption of descriptors (as a "slave" to the descriptor engine) is monitored separately with: - **Agent ID:** 48 + channel index (descriptor consume monitor, per-channel) - **Unit ID:** 1 (STREAM subsystem) - **Monitored Port:** descriptor_valid/descriptor_ready handshake

This monitor tracks: - When the scheduler pops descriptors from the engine’s output FIFO - Consume latency (time from engine ready to scheduler accept) - Consume stalls (scheduler not ready while descriptors pending)

10.9.3 Why Both Ends?

With dual-end monitoring, software can distinguish: - **Fetch latency** (master-side agent): Time from kick-off to descriptor received - **Consume stall** (slave-side agent): Time descriptor sits in FIFO awaiting scheduler

A slow transfer can be attributed to either: - Network/memory delay (fetch stall visible at master-side agent) - Scheduler queue full (consume stall visible at slave-side agent)

Both monitors emit MonBus packets with identical protocol-independent structure but distinct agent IDs for granular root-cause analysis.

10.10 Related Documentation

- **Parent:** 03_scheduler_group.md - Channel wrapper
- **Consumer:** 04_scheduler.md - Scheduler that receives descriptors
- **Arbiter:** 02_scheduler_group_array.md - Descriptor AXI arbitration
- **APB Interface:** 13_apbtodescr.md - APB to descriptor kick-off

10.11 Revision History

Descriptor Engine Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers
0.93	2026-07-02	seang	cfg_prefetch_enable / cfg_fifo_threshold are now functional; documented prefetch-throttle gate (on-demand vs. buffer-ahead) and deferred-chain (r_chain_pending)

Version	Date	Author	Description
			behavior

Last Updated: 2026-07-02

11 AXI Read Engine

Module: axi_read_engine.sv **Location:** projects/components/dmas/stream/rtl/fub/
Category: FUB (Functional Unit Block) **Parent:** stream_core.sv **Status:** Implemented **Last Updated:** 2025-11-30

11.1 Overview

The axi_read_engine module is a high-performance multi-channel AXI4 read engine with space-aware arbitration. It serves all 8 STREAM channels through a single AXI master interface with intelligent flow control.

11.1.1 Key Features

- **Round-Robin Arbitration:** Fair scheduling across channels
- **Space-Aware Masking:** Only arbitrate channels with sufficient SRAM space
- **Pre-Allocation Handshake:** Reserve SRAM space before data arrives
- **Streaming Data Path:** Direct passthrough to SRAM controller
- **Channel ID in AXI ID:** Enables per-channel response routing
- **Pipelined/Non-Pipelined Modes:** Configurable outstanding transaction depth

11.2 Architecture

11.2.1 Operation Flow

1. Scheduler Interface: Each channel can request read bursts
2. Space Checking: Mask channels without sufficient SRAM space
3. Arbitration: Round-robin arbiter selects next channel to service
4. AXI AR Issue: Issue read command to AXI, assert rd_alloc to SRAM controller
5. AXI R Response: Stream read data directly to SRAM controller

11.2.2 Key Design Decisions

Combinational AR Outputs: AR outputs are driven combinationally from the arbiter to avoid 1-cycle delay. When axi_rd_alloc_space_free goes to 0, arvalid drops in the same cycle.

No Internal Buffering: The engine is a streaming pipeline with no internal data storage. Data flows directly from AXI R channel to SRAM controller.

11.2.3 System Idle Behavior

When ALL channels complete their work (`w_arb_request == 0`), the engine becomes idle until new requests arrive. This is NOT a bubble - it's legitimate system idle.

Why This Matters for Testing: - With few active channels and short bursts, there are brief periods when all channels are in their WRITE phase - During these periods, `m_axi_arvalid = 0` (no R channel activity) - This is EXPECTED behavior, not a performance bug - The `dbg_arb_request` signal exposes `w_arb_request` to help testbenches distinguish: - **TRUE BUBBLE:** `arvalid=0` while `arb_request!=0` (channels waiting, arbiter stalled) - **SYSTEM IDLE:** `arvalid=0` while `arb_request==0` (no channels need service)

11.3 Parameters

Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of channels
ADDR_WIDTH	int	64	AXI address width
DATA_WIDTH	int	512	AXI data width
ID_WIDTH	int	8	AXI ID width
SEG_COUNT_WIDTH	int	8	Width of space/count signals
PIPELINE	int	0	0: non-pipelined, 1:

Parameter	Type	Default	Description
AR_MAX_OUTSTANDING	int	8	pipelined Maximum outstanding AR requests (PIPELINE=1)
STROBE EVERY_BEAT	int	0	0: strobe on last beat, 1: strobe every beat

11.3.1 Derived Parameters

Derived Parameters

Parameter	Derivation	Description
NC	NUM_CHANNELS	Short alias
AW	ADDR_WIDTH	Short alias
DW	DATA_WIDTH	Short alias
IW	ID_WIDTH	Short alias
SCW	SEG_COUNT_WIDTH	Segment count width
CIW	$\$clog2(NC)$	Channel ID width (min 1 bit)

11.4 Port List

11.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

11.4.2 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_axi_rd_xfer_beats	input	8	Transfer size in beats (all

Signal	Direction	Width	Description
			channels)

11.4.3 Scheduler Interface (Per-Channel)

Scheduler Interface

Signal	Direction	Width	Description
sched_rd_valid[ch]	input	NC	Channel requests read
sched_rd_addr[ch]	input	NC x AW	Source addresses
sched_rd_beats[ch]	input	NC x 32	Beats remaining to read

11.4.4 Completion Interface (Per-Channel)

Completion Interface

Signal	Direction	Width	Description
sched_rd_done_strobe[ch]	output	NC	Burst completed (1 cycle pulse)
sched_rd_beats_done[ch]	output	NC x 32	Number of beats completed

11.4.5 SRAM Allocation Interface

SRAM Allocation Interface

Signal	Direction	Width	Description
axi_rd_alloc_req	output	1	Request space allocation
axi_rd_alloc_size	output	8	Beats to reserve
axi_rd_alloc_id	output	IW	Transaction ID (channel)
axi_rd_alloc_space_free[ch]	input	NC x SCW	Free space per channel

11.4.6 SRAM Write Interface

SRAM Write Interface

Signal	Direction	Width	Description
axi_rd_sram_valid	output	1	Read data valid
axi_rd_sram_ready	input	1	Ready to accept data
axi_rd_sram_id	output	IW	Transaction ID (channel)
axi_rd_sram_data	output	DW	Read data payload

11.4.7 AXI4 AR Channel

AXI4 AR Channel

Signal	Direction	Width	Description
m_axi_arvalid	output	1	Address valid
m_axi_arready	input	1	Address ready
m_axi_arid	output	IW	Transaction ID
m_axi_araddr	output	AW	Address
m_axi_arlen	output	8	Burst length - 1
m_axi_arsize	output	3	Burst size (log2 bytes)
m_axi_arburst	output	2	Burst type (INCR)

11.4.8 AXI4 R Channel

AXI4 R Channel

Signal	Direction	Width	Description
m_axi_rvalid	input	1	Read data valid
m_axi_rready	output	1	Read data ready
m_axi_rid	input	IW	Transaction ID

Signal	Direction	Width	Description
m_axi_rdata	input	DW	Read data
m_axi_rresp	input	2	Response
m_axi_rlast	input	1	Last beat of burst

11.4.9 Error Interface

Error Interface

Signal	Direction	Width	Description
sched_rd_error[ch]	output	NC	Sticky error flag per channel

11.4.10 Debug Interface

Debug Interface

Signal	Direction	Width	Description
dbg_rd_all_complete[ch]	output	NC	All reads complete
dbg_r_beats_received	output	32	Total R beats received
dbg_sram_writes	output	32	Total writes to SRAM
dbg_arb_request[ch]	output	NC	Arbiter request vector

11.5 Operation

11.5.1 Space-Aware Request Masking

```
// Only request arbitration if:
// 1. Scheduler is requesting (sched_rd_valid)
// 2. Sufficient SRAM space available (w_space_ok)
// 3. Below outstanding limit (w_below_outstanding_limit)
w_arb_request[i] = sched_rd_valid[i] && w_space_ok[i] &&
w_below_outstanding_limit[i];
```

11.5.2 Transfer Size Calculation

```
// Calculate actual transfer size for this channel
// Min of remaining beats or configured max
w_transfer_size[i] = (sched_rd_beats[i] <= cfg_axi_rd_xfer_beats +
1) ?
    (sched_rd_beats[i] - 1) : cfg_axi_rd_xfer_beats;
```

11.5.3 Outstanding Transaction Tracking

PIPELINE=0 (Non-Pipelined): - Binary flag per channel (0 or 1 outstanding) - Set when AR issues for channel - Clear when R last arrives for channel

PIPELINE=1 (Pipelined): - Counter per channel (0 to AR_MAX_OUTSTANDING) - Increment on AR handshake - Decrement on R last handshake

11.5.4 Completion Strobe

Strobes fire when AR transaction is accepted (not when R completes):

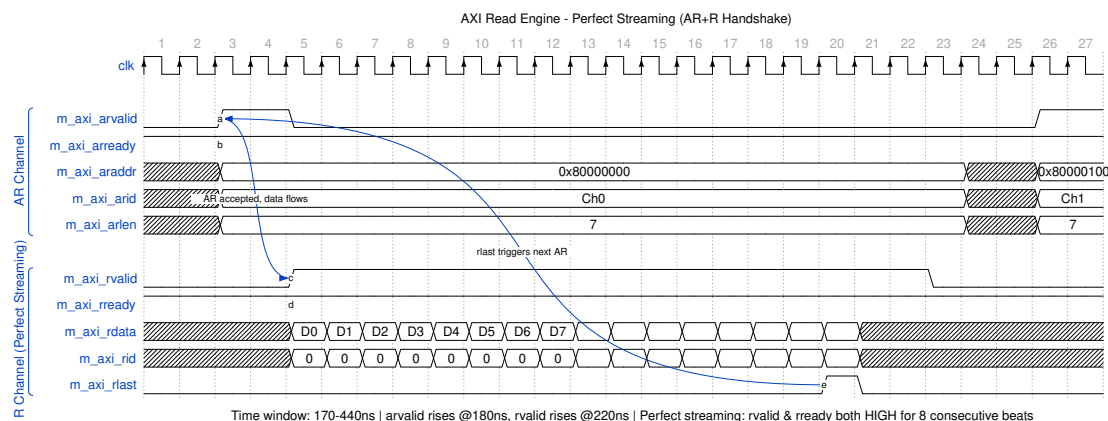
```
// Pulse when AR transaction is accepted (arvalid && arready)
// This tells scheduler that request was issued to AXI bus
if (m_axi_arvalid && m_axi_arready) begin
    r_done_strobe[w_arb_grant_id] <= 1'b1;
    r_beats_done[w_arb_grant_id] <= w_transfer_size[w_arb_grant_id] +
1;
end
```

11.6 Timing Diagrams

11.6.1 Perfect Streaming - AXI Read Transaction

The following timing diagram shows the AXI read engine operating at maximum throughput with **perfect streaming** - where both rvalid and rready remain HIGH for consecutive clock cycles, achieving one data beat per cycle.

11.6.1.1 Waveform 2.6.1: AXI Read Engine - Perfect Streaming



AXI Read Engine - Perfect Streaming

Transaction Flow:

1. AR Channel Handshake (Address Phase)

- m_axi_arvalid rises to initiate a read request
- m_axi_arready is already HIGH (slave ready)
- Address (araddr), ID (arid), and length (arlen=7 for 8 beats) are captured
- Handshake completes in a single cycle when both valid and ready are HIGH

2. R Channel Streaming (Data Phase)

- After AR acceptance, the AXI slave begins returning data
- m_axi_rvalid rises and **stays HIGH** for all 8 beats
- m_axi_rready remains HIGH throughout (no backpressure from SRAM controller)
- **Perfect streaming:** One data beat transferred every clock cycle
- m_axi_rlast rises on the final beat (beat 7) to mark burst completion

3. Next Transaction

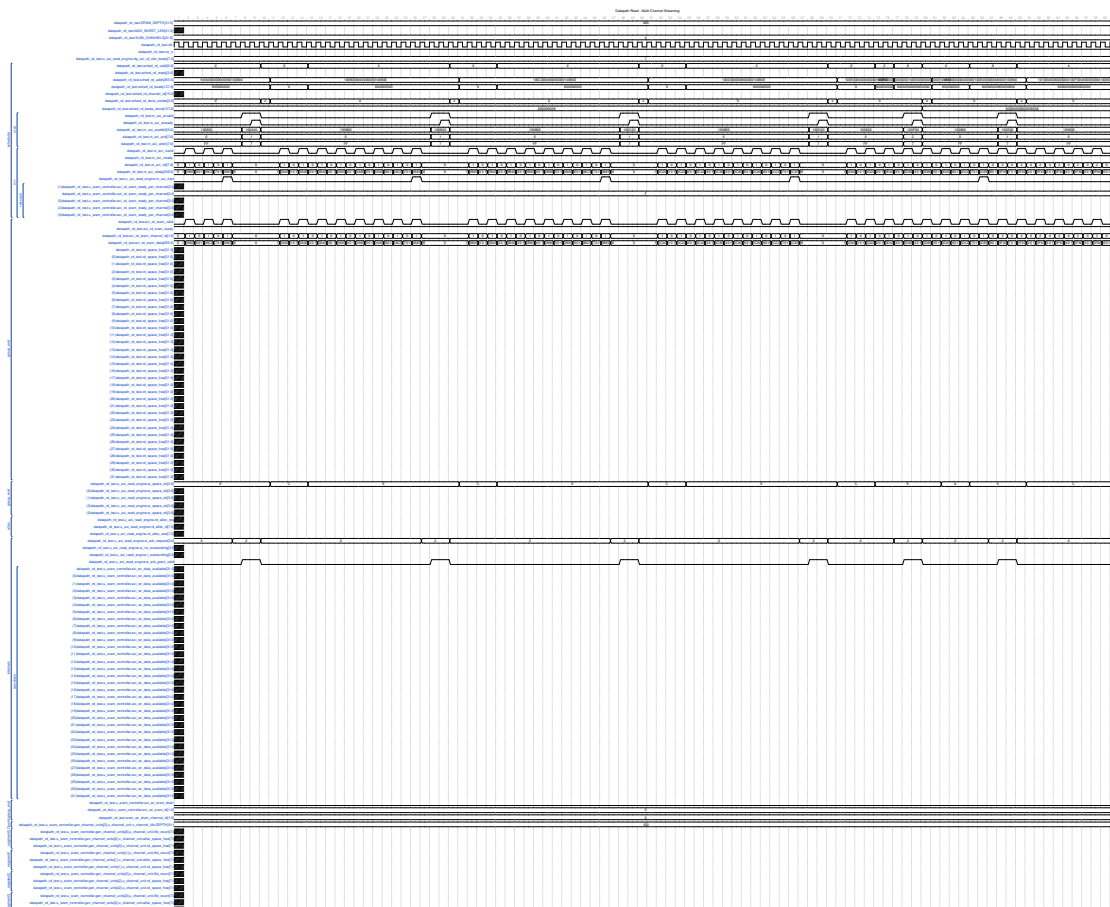
- After rlast, the engine can immediately issue the next AR request
- The cycle repeats for the next channel or next burst

Key Performance Indicators: - **No bubbles:** rvalid and rready both HIGH during data phase - **Full bandwidth:** Data width (256/512 bits) x clock frequency - **Zero-wait states:** SRAM controller accepts data at line rate

11.6.2 Multi-Channel Streaming

For multi-channel operation showing channel switching while maintaining streaming performance, see:

11.6.2.1 Waveform 2.6.2: Datapath Read - Multi-Channel



Datapath Read - Multi-Channel

This diagram shows how the engine arbitrates between channels while maintaining high throughput.

11.7 Integration Example

```
axi_read_engine #(
    .NUM_CHANNELS      (8),
    .ADDR_WIDTH        (64),
    .DATA_WIDTH        (512),
    .ID_WIDTH          (8),
    .SEG_COUNT_WIDTH   (10),
```



```

        .PIPELINE                (0),
        .AR_MAX_OUTSTANDING (8)
    ) u_axi_read_engine (
        .clk                      (clk),
        .rst_n                   (rst_n),

        // Configuration
        .cfg_axi_rd_xfer_beats    (cfg_axi_rd_xfer_beats),

        // Scheduler interface
        .sched_rd_valid          (sched_rd_valid),
        .sched_rd_addr           (sched_rd_addr),
        .sched_rd_beats          (sched_rd_beats),

        // Completion interface
        .sched_rd_done_strobe     (sched_rd_done_strobe),
        .sched_rd_beats_done      (sched_rd_beats_done),

        // SRAM allocation interface
        .axi_rd_alloc_req         (axi_rd_alloc_req),
        .axi_rd_alloc_size        (axi_rd_alloc_size),
        .axi_rd_alloc_id          (axi_rd_alloc_id),
        .axi_rd_alloc_space_free  (axi_rd_alloc_space_free),

        // SRAM write interface
        .axi_rd_sram_valid        (axi_rd_sram_valid),
        .axi_rd_sram_ready        (axi_rd_sram_ready),
        .axi_rd_sram_id           (axi_rd_sram_id),
        .axi_rd_sram_data         (axi_rd_sram_data),

        // AXI master
        .m_axi_arvalid            (m_axi_rd_arvalid),
        .m_axi_arready            (m_axi_rd_arready),
        .m_axi_arid               (m_axi_rd_arid),
        .m_axi_araddr             (m_axi_rd_araddr),
        .m_axi_arlen              (m_axi_rd_arlen),
        .m_axi_arsize             (m_axi_rd_arsize),
        .m_axi_arburst            (m_axi_rd_arburst),

        .m_axi_rvalid             (m_axi_rd_rvalid),
        .m_axi_rready             (m_axi_rd_rready),
        .m_axi_rid                (m_axi_rd_rid),
        .m_axi_rdata              (m_axi_rd_rdata),
        .m_axi_rresp              (m_axi_rd_rresp),
        .m_axi_rlast              (m_axi_rd_rlast),

```

```

// Error and debug
.sched_rd_error      (sched_rd_error),
.dbg_rd_all_complete (dbg_rd_all_complete),
.dbg_arb_request      (dbg_arb_request)
);

```

11.8 Bug Fixes (v0.92)

11.8.1 w_arb_request Timing Closure (Commit 4e8f9e02)

At 8 channels, the arbiter request combinational path from scheduler to AXI master became critical. The w_arb_request signal is now **registered** to add a single pipeline stage, closing timing at the cost of 1 cycle of latency on the scheduler→engine request path.

Impact: ~1% throughput penalty in worst case, 100% timing closure margin gain. Highly beneficial for high-frequency designs.

11.9 Related Documentation

- **Parent:** 01_stream_core.md - Top-level integration
 - **Scheduler Array:** 02_scheduler_group_array.md - Provides sched_rd_* signals
 - **Allocation Controller:** 07_stream_alloc_ctrl.md - Space tracking for read engine
 - **SRAM Controller:** 08_sram_controller.md - Receives read data
 - **Write Engine: 12_axi_write_engine.md - Complementary write datapath**
-

11.10 Revision History

AXI Read Engine Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure

Version	Date	Author	Description
			numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

12 Stream Allocation Controller

Module: stream_alloc_ctrl.sv **Location:** projects/components/dmas/stream/rtl/fub/
Category: FUB (Functional Unit Block) **Parent:** sram_controller_unit.sv **Status:** Implemented
Last Updated: 2025-11-30

12.1 Overview

The stream_alloc_ctrl module is a **virtual FIFO without data storage** that tracks space allocation and availability using FIFO pointer logic. It provides pre-allocation support for AXI read engines to prevent race conditions between burst requests and data arrival.

12.1.1 What Makes This a “Virtual FIFO”

Virtual FIFO: - Has write pointer (allocation pointer) - Has read pointer (actual write pointer) - Calculates full/empty/space_free - **NO data storage** - only pointer arithmetic

Use Case: Reserve FIFO space BEFORE AXI read data arrives

12.2 The Allocation Problem

12.2.1 Without Allocation Controller

Problem: Race condition between space check and data arrival

Cycle 0: Read engine checks FIFO space
space_free = 100 beats → OK to issue 16-beat burst

Cycle 5: AR handshake completes, AXI read starts

Cycle 10: Meanwhile, write engine drains 90 beats from FIFO

space_free = 10 beats (NOT ENOUGH!)

Cycle 15: AXI read data starts arriving (16 beats)
→ OVERFLOW! Only 10 beats of space available

12.2.2 With Allocation Controller

Solution: Reserve space when issuing AR, not when data arrives

Cycle 0: Read engine checks space_free = 100 beats
Allocate 16 beats:
 rd_alloc_req = 1, rd_alloc_size = 16
 → space_free = 84 beats (reserved!)

Cycle 5: AR handshake completes

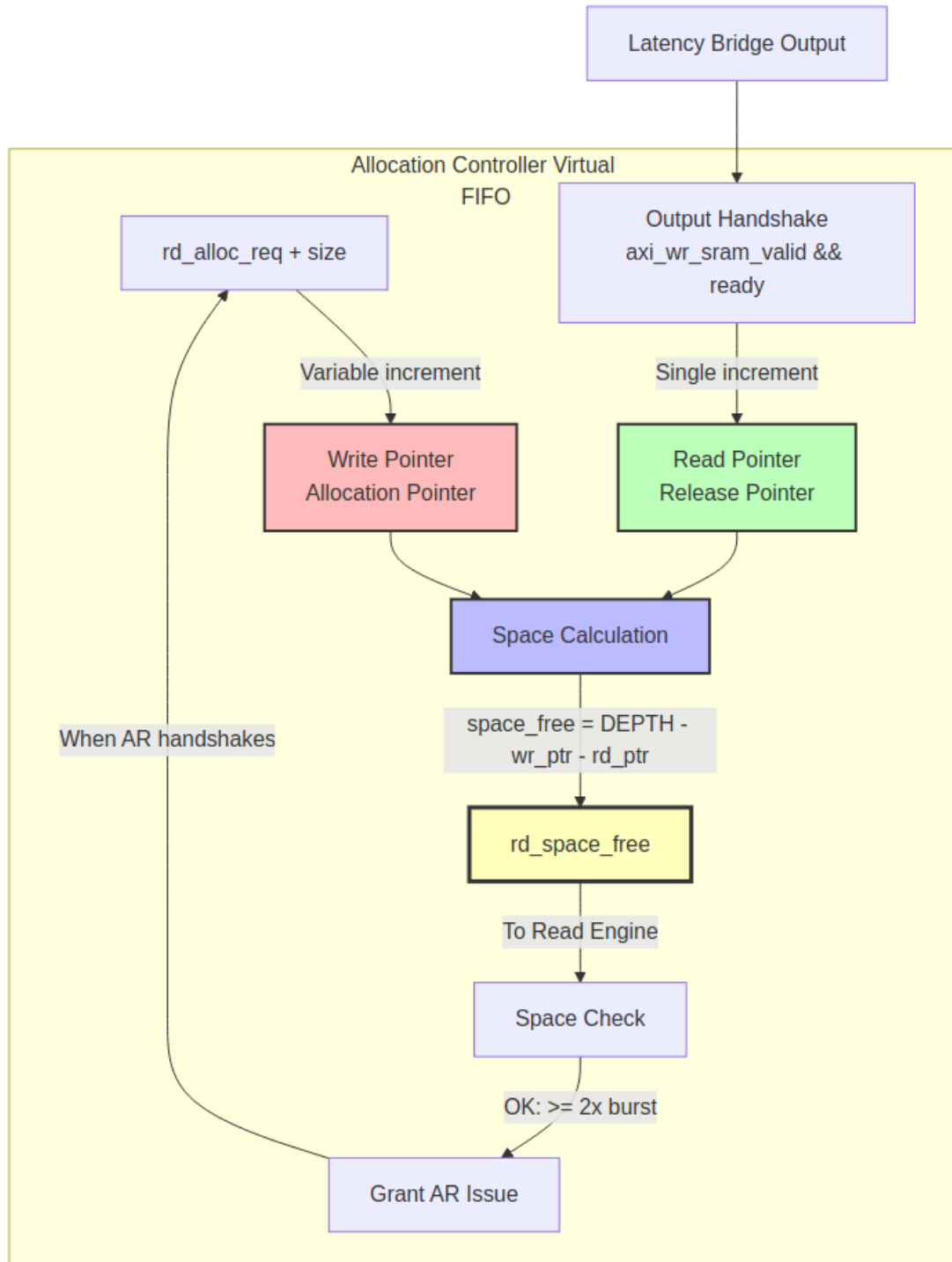
Cycle 10: Write engine checks space_free = 84 beats
→ Sees reduced space, won't overdrain

Cycle 15: AXI read data arrives, enters FIFO
→ Space was reserved, guaranteed to fit

12.3 Architecture

12.3.1 Two-Pointer System

12.3.2 Figure 2.7.1: Stream Allocation Controller Block Diagram



Diagram

→

Write Pointer (Allocation Pointer): - Advances when space is **allocated** (burst request) - Variable-size increment (rd_alloc_size) - Represents “reserved space”

Read Pointer (Actual Write Pointer): - Advances when data **exits the controller** (output handshake) - Single-beat increment - Represents “released space”

Space Calculation:

$$\text{space_free} = \text{DEPTH} - (\text{wr_ptr} - \text{rd_ptr})$$

12.3.3 CRITICAL: Confusing Naming Convention

WARNING: Allocation controller uses OPPOSITE naming from normal FIFO!

Normal FIFO: | Signal | Meaning | |——|——| | wr_* | Write data → Consumes space → space_free decreases | | rd_* | Read data → Frees space → space_free increases |

: CRITICAL: Confusing Naming Convention

Allocation Controller: | Signal | Meaning | |——|——| | wr_* | **ALLOCATE** space → Reserves space → space_free **decreases** | | rd_* | **RELEASE** space → Data exits controller → space_free **increases** |

: CRITICAL: Confusing Naming Convention

Why This Matters:

```
// "Write" side = ALLOCATE (reserve space for upcoming burst)
// Read engine says "I need 16 beats"
.wr_valid (rd_alloc_req),
.wr_size  (rd_alloc_size), // Variable size (16 in this example)

// "Read" side = RELEASE (data exits controller, free space)
// Must monitor OUTPUT handshake (after latency bridge!)
.rd_valid (axi_wr_sram_valid && axi_wr_sram_ready), // Output side!
```

Key Insight: The “read” side of allocation controller connects to the OUTPUT of the FIFO + latency bridge, NOT the FIFO input!

12.4 Parameters

Parameters

Parameter	Type	Default	Description
DEPTH	int	512	Virtual FIFO depth (must match physical FIFO)

Parameter	Type	Default	Description
ALMOST_WR_MARGIN	int	1	Almost full threshold
ALMOST_RD_MARGIN	int	1	Almost empty threshold
REGISTERED	int	1	Register outputs for timing

12.5 Port List

12.5.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	System clock
axi_aresetn	input	1	Active-low asynchronous reset

12.5.2 Write Interface (Allocation Requests)

Write Interface

Signal	Direction	Width	Description
wr_valid	input	1	Allocate space request
wr_size	input	8	Number of entries to allocate
wr_ready	output	1	Space available (!wr_full)

12.5.3 Read Interface (Actual Data Written)

Read Interface

Signal	Direction	Width	Description
rd_valid	input	1	Data exits controller (release space)
rd_ready	output	1	Not empty (!rd_empty)

12.5.4 Status Outputs

Status Outputs

Signal	Direction	Width	Description
space_free	output	AW+1	Available space (beats)
wr_full	output	1	Full flag (no space available)
wr_almost_full	output	1	Almost full flag
rd_empty	output	1	Empty flag (no allocations)
rd_almost_empty	output	1	Almost empty flag

12.6 Interfaces

12.6.1 Write Interface (Allocation Requests)

Write Interface

Signal	Direction	Width	Description
wr_valid	Input	1	Allocate space
wr_size	Input	8	Number of entries to allocate
wr_ready	Output	1	Space

Signal	Direction	Width	Description
			available (! wr_full)

Usage:

```
// Read engine allocates space before issuing AR
rd_alloc_req = (space_check_ok && !ar_pending);
rd_alloc_size = cfg_axi_rd_xfer_beats;
```

12.6.2 Read Interface (Actual Data Written)

Read Interface

Signal	Direction	Width	Description
rd_valid	Input	1	Data exits controller
rd_ready	Output	1	Not empty (! rd_empty)

Usage:

```
// Connect to OUTPUT handshake (after latency bridge)
assign rd_valid = (axi_wr_sram_valid && axi_wr_sram_ready);
```

12.6.3 Status Outputs

Status Outputs

Signal	Direction	Width	Description
space_free	Output	AW+1	Available unreserved space
wr_full	Output	1	No space available
wr_almost_full	Output	1	Almost full
rd_empty	Output	1	No allocations pending
rd_almost_empty	Output	1	Almost empty

Note: space_free is the most important output - used by read engine for space checking.

12.7 Operation

12.7.1 Allocation Flow

Step 1: Check Space

```
// Read engine checks space availability  
if (space_free >= (cfg_axi_rd_xfer_beats << 1)) begin // 2x margin  
    // Space OK, proceed to allocation  
end
```

Step 2: Allocate Space

```
rd_alloc_req = 1'b1;  
rd_alloc_size = 8'd16; // Reserve 16 beats  
  
// Next cycle: wr_ptr advances  
r_wr_ptr_bin <= r_wr_ptr_bin + 16;  
space_free decreases by 16
```

Step 3: AXI Read Executes

```
// Some cycles later, AXI AR handshake completes  
m_axi_arvalid = 1, m_axi_arready = 1  
// AXI read starts
```

Step 4: Data Arrives at FIFO

```
// Many cycles later, AXI read data arrives  
axi_rd_sram_valid = 1, axi_rd_sram_ready = 1  
// Data enters FIFO (allocation controller doesn't see this directly)
```

Step 5: Data Exits Controller (After Latency Bridge)

```
// Eventually, data traverses FIFO + latency bridge  
axi_wr_sram_valid = 1, axi_wr_sram_ready = 1  
  
// THIS triggers allocation controller release!  
rd_valid = 1  
r_rd_ptr_bin <= r_rd_ptr_bin + 1  
space_free increases by 1
```

12.7.2 Pointer Arithmetic

Write Pointer (Allocation):

```
// Variable-size increment  
if (w_write && !r_wr_full) begin  
    r_wr_ptr_bin <= r_wr_ptr_bin + (AW+1)'(wr_size);  
end
```

Read Pointer (Release):

```
// Single-beat increment (uses counter_bin utility)
counter_bin #(
    .WIDTH (AW + 1),
    .MAX    (D)
) read_pointer_inst (
    .clk      (axi_aclk),
    .rst_n    (axi_aresetn),
    .enable   (w_read && !r_rd_empty),
    .counter_bin_curr (r_rd_ptr_bin),
    .counter_bin_next (w_rd_ptr_bin_next)
);
```

Space Calculation:

```
// Occupancy = wr_ptr - rd_ptr
w_count = w_wr_ptr_bin_next - w_rd_ptr_bin_next;

// Space free = total depth - occupancy
space_free = (AW+1)'(D) - w_count;
```

12.8 Timing Behavior

12.8.1 Allocation Latency

Allocation is IMMEDIATE (combinational + 1 cycle):

```
Cycle N:   rd_alloc_req = 1, rd_alloc_size = 16
Cycle N+1: r_wr_ptr_bin = old_value + 16
           space_free = old_value - 16
```

Read engine sees updated space_free on next cycle.

12.8.2 Release Latency

Release is IMMEDIATE (combinational + 1 cycle):

```
Cycle N:   axi_wr_sram_valid = 1, axi_wr_sram_ready = 1
           → rd_valid = 1
Cycle N+1: r_rd_ptr_bin = old_value + 1
           space_free = old_value + 1
```

Write engine sees updated space_free on next cycle.

12.9 Integration Example

12.9.1 In sram_controller_unit.sv

```
stream_alloc_ctrl #(
    .DEPTH(SD),
    .REGISTERED(1)
) u_alloc_ctrl (
    .axi_aclk          (clk),
    .axi_aresetn       (rst_n),

    // ALLOCATE (reserve space for upcoming burst)
    .wr_valid          (rd_alloc_req),      // From read engine
    .wr_size           (rd_alloc_size),     // Burst size
    .wr_ready          (),                  // Unused (space check
uses space_free)

    // RELEASE (data exits controller - OUTPUT handshake!)
    .rd_valid          (axi_wr_sram_valid && axi_wr_sram_ready), //
After latency bridge!
    .rd_ready          (),                  // Unused

    // Space tracking
    .space_free        (alloc_space_free),  // To read engine (via
register)

    // Unused status
    .wr_full           (),
    .wr_almost_full    (),
    .rd_empty          (),
    .rd_almost_empty   ()
);

// Register output to break long paths
`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        rd_space_free <= SCW'(SD); // Full space on reset
    end else begin
        rd_space_free <= alloc_space_free;
    end
end
);
```

12.10 Debug Support

12.10.1 Display Statements

Allocation:

```

$display("ALLOC @ %t: allocated %0d beats, wr_ptr: %0d -> %0d,
space_free will be %0d",
    $time, wr_size, r_wr_ptr_bin, r_wr_ptr_bin + wr_size,
    D - (r_wr_ptr_bin + wr_size - r_rd_ptr_bin));

```

Release (Drain):

```

$display("DRAIN @ %t: drained 1 beat, rd_ptr: %0d -> %0d, space_free
will be %0d",
    $time, r_rd_ptr_bin, w_rd_ptr_bin_next,
    D - (r_wr_ptr_bin - w_rd_ptr_bin_next));

```

12.10.2 Waveform Analysis

Key Signals to Monitor: - r_wr_ptr_bin - Allocation pointer - r_rd_ptr_bin - Release pointer - space_free - Available space - rd_alloc_req, rd_alloc_size - Allocation requests - axi_wr_sram_valid, axi_wr_sram_ready - Release handshake

12.11 Common Issues

12.11.1 Issue 1: Space Not Released

Symptom: space_free decreases but never increases

Root Cause: rd_valid not connected to output handshake

Wrong:

```

.rd_valid (axi_rd_sram_valid && axi_rd_sram_ready) // FIFO input -
WRONG!

```

Correct:

```

.rd_valid (axi_wr_sram_valid && axi_wr_sram_ready) // After latency
bridge - CORRECT!

```

12.11.2 Issue 2: Overflow Despite Allocation

Symptom: FIFO overflows even with allocation controller

Root Cause: Read engine uses wrong space value

Wrong:

```

if (space_free >= cfg_axi_rd_xfer_beats) begin // Exact match -
WRONG!
    allocate();
end

```

Correct:

```

if (space_free >= (cfg_axi_rd_xfer_beats << 1)) begin // 2x margin -
CORRECT!
    allocate();
end

```

Why 2x: Accounts for in-flight allocations and pipeline delays.

12.11.3 Issue 3: Allocation Pointer Overflow

Symptom: r_wr_ptr_bin wraps at unexpected value

Root Cause: Pointer width insufficient

Check:

```

parameter int AW = $clog2(D); // Address width
logic [AW:0] r_wr_ptr_bin; // AW+1 bits (for full detection)

```

Extra bit allows distinguishing full (wr_ptr = rd_ptr + DEPTH) from empty (wr_ptr = rd_ptr).

12.12 Comparison with Drain Controller

Comparison with Drain Controller

Aspect	Allocation Controller	Drain Controller
Purpose	Reserve FIFO space	Reserve FIFO data
User	AXI read engine	AXI write engine
Write side	Allocation request (reserve)	Data enters FIFO (increment)
Read side	Data exits controller (release)	Drain request (reserve)
Naming	OPPOSITE of normal FIFO	SAME as normal FIFO
Output	space_free	data_available

12.13 Resource Utilization

Per Instance: - $2 \times (AW+1)$ -bit counters (wr_ptr, rd_ptr) - $1 \times (AW+1)$ -bit space_free calculation - FIFO control block (full/empty logic) - ~50-100 flip-flops total

Example (DEPTH=512, AW=9): - 2×10 -bit counters = 20 FFs - Control logic = ~30 FFs - **Total:** ~50 FFs

Very lightweight - pointer logic only, no data storage.

12.14 Related Modules

- **Counterpart:** `stream_drain_ctrl.sv` - Drain-side flow control
- **Parent:** `sram_controller_unit.sv` - Instantiates allocation controller
- **User:** `axi_read_engine.sv` - Checks `rd_space_free` before issuing AR

12.15 Related Documentation

- **Drain Controller:** `11_stream_drain_ctrl.md`
 - **SRAM Controller Unit:** `09_sram_controller_unit.md`
 - **SRAM Controller:** `08_sram_controller.md`
 - **AXI Read Engine:** `06_axi_read_engine.md`
-

12.16 Revision History

Stream Allocation Controller Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 SRAM Controller Specification

Module: `sram_controller.sv` **Location:** `projects/components/dmas/stream/rtl/fub/`
Status: Implemented **Last Updated:** 2025-11-30

13.1 Overview

The SRAM Controller provides per-channel buffering between AXI read and write engines using independent FIFO structures. Each channel has its own FIFO with dedicated allocation controller (write side) and drain controller (read side).

13.1.1 Key Features

- **Per-channel FIFOs:** Independent gaxi_fifo_sync per channel (no segmentation complexity)
- **ID-based routing:** Transaction ID selects channel for write/read operations
- **Allocation controller:** Reserves space before AXI read data arrives
- **Drain controller:** Manages data availability for AXI write engine
- **Latency bridge:** Aligns FIFO read latency (registered output)
- **Saturating counters:** 8-bit space/count reporting per channel

13.1.2 Design Rationale

Why Per-Channel FIFOs Instead of Segmented SRAM?

The implementation uses **independent FIFOs** rather than a monolithic SRAM divided into segments:

Advantages of Per-Channel FIFOs: - **Simpler logic** - FIFOs are standard, well-tested components - **Better isolation** - Channel failures don't affect others - **Easier timing** - No cross-channel paths or arbitration - **Modular** - Easy to add/remove channels - **No pointer arithmetic** - FIFO handles internally

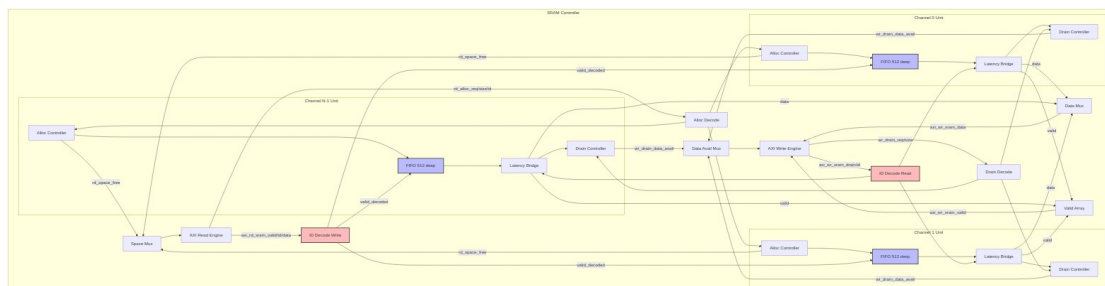
Trade-offs: - More SRAM resources ($NC \times DEPTH$ vs. shared pool) - Potential waste if channels idle (unused FIFO space)

Decision: Simplicity and isolation outweigh SRAM efficiency for STREAM's tutorial focus.

13.2 Architecture

13.2.1 Block Diagram

13.2.2 Figure 2.8.1: SRAM Controller Block Diagram



Diagram

Source: [05_sram_controller_block.mmd](#)

13.2.3 Per-Channel Architecture

Each channel contains three components (in `sram_controller_unit`):

- 1. Allocation Controller (`stream_alloc_ctrl`):**
 - Receives `rd_alloc_req` from read engine
 - Tracks reserved vs. committed space
 - Provides `rd_space_free` to read engine
- 2. FIFO (`gaxi_fifo_sync`):**
 - Stores data between read and write engines
 - Depth = `SRAM_DEPTH` parameter
 - Standard valid/ready handshaking
- 3. Drain Controller + Latency Bridge (`stream_drain_ctrl`):**
 - Receives `wr_drain_req` from write engine
 - Provides `wr_drain_data_avail` to write engine
 - Latency bridge aligns FIFO read latency

13.3 Parameters

```
parameter int NUM_CHANNELS = 8;           // Number of
independent channels
parameter int DATA_WIDTH = 512;         // Data width in bits
parameter int SRAM_DEPTH = 512;         // Depth per channel
```

```

FIFO
parameter int SEG_COUNT_WIDTH = $clog2(SRAM_DEPTH) + 1; // Width of
count signals

// Short aliases (internal use)
parameter int NC = NUM_CHANNELS;
parameter int DW = DATA_WIDTH;
parameter int SD = SRAM_DEPTH;
parameter int SCW = SEG_COUNT_WIDTH; // FIFO depth counter
width
parameter int CIW = (NC > 1) ? $clog2(NC) : 1; // Channel ID width
(min 1 bit)

```

Note: “SEG_COUNT_WIDTH” refers to the FIFO depth counter width (historical name from segmented design consideration).

13.4 Port List

13.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

13.4.2 Allocation Interface

AXI Read Engine Flow Control (Space Reservation):

Allocation Interface

Signal	Direction	Width	Description
axi_rd_alloc_req	input	1	Allocation request (single request with ID)
axi_rd_alloc_size	input	8	Number of beats to allocate
axi_rd_alloc_id	input	CIW	Channel ID for allocation
axi_rd_alloc_space_free[	output	NUM_CHAN	Free space per channel

Signal	Direction	Width	Description
ch]		NELS × SEG_COUNT _WIDTH	FIFO

13.4.3 Write Interface

AXI Read Engine → FIFO (ID-based routing):

Write Interface

Signal	Direction	Width	Description
axi_rd_sram_ valid	input	1	Write data valid (single valid with ID)
axi_rd_sram_ ready	output	1	Ready (muxed from selected channel)
axi_rd_sram_ id	input	CIW	Channel ID select for write
axi_rd_sram_ data	input	DATA_WIDT H	Write data (common bus)

13.4.4 Drain Interface

Write Engine Flow Control (Data Availability):

Drain Interface

Signal	Direction	Width	Description
axi_wr_drain_ data_avail[ch]	output	NUM_CHAN NELS × SEG_COUNT _WIDTH	Available data after reservations per channel
axi_wr_drain_ req[ch]	input	NUM_CHAN NELS	Per-channel drain request
axi_wr_drain_ size[ch]	input	NUM_CHAN NELS × 8	Per-channel drain size (beats to reserve)

13.4.5 Read Interface

FIFO → AXI Write Engine (ID-based routing):

Read Interface

Signal	Direction	Width	Description
axi_wr_sram_valid[ch]	output	NUM_CHAN NELS	Per-channel valid (data available)
axi_wr_sram_drain	input	1	Drain request (consumer ready)
axi_wr_sram_id	input	CIW	Channel ID select for read
axi_wr_sram_data	output	DATA_WIDT H	Read data from selected channel (muxed)

13.4.6 Debug Interface

Debug Interface

Signal	Direction	Width	Description
dbg_bridge_pending[ch]	output	NUM_CHAN NELS	Latency bridge pending per channel
dbg_bridge_output_valid[ch]	output	NUM_CHAN NELS	Latency bridge output valid per channel

13.5 Interface

13.5.1 Clock and Reset

```
input logic clk;
input logic rst_n;    // Active-low asynchronous reset
```

13.5.2 Allocation Interface (AXI Read Engine Flow Control)

Space Reservation:

```
input logic axi_rd_alloc_req;    // Single allocation request
input logic [7:0] axi_rd_alloc_size;    // Beats to allocate
input logic [CIW-1:0] axi_rd_alloc_id;    // Channel ID for allocation
output logic [NC-1:0][SCW-1:0] axi_rd_alloc_space_free;    // Free space per channel
```

Allocation Protocol: 1. Read engine issues AR transaction 2. **Before R data arrives:** Engine asserts `axi_rd_alloc_req` with size and ID 3. Allocation controller reserves space in selected channel 4. Engine tracks reserved space, prevents over-issuing AR commands 5. Space commits when actual data arrives (FIFO write)

Why Allocation is Critical:

Without pre-allocation, multiple AR commands could be issued before data arrives, causing FIFO overflow:

Problem without allocation:

```
Cycle 0: Issue AR (16 beats) - FIFO has 32 free
Cycle 1: Issue AR (16 beats) - FIFO still shows 32 free (data hasn't
arrived!)
Cycle 2: Issue AR (16 beats) - FIFO still shows 32 free
Cycle 10: R data starts arriving (48 beats total)
→ OVERFLOW! Only 32 beats of space
```

Solution with allocation:

```
Cycle 0: Issue AR (16 beats), allocate 16 - FIFO shows 16 free
(reserved)
Cycle 1: Issue AR (16 beats), allocate 16 - FIFO shows 0 free
(reserved)
Cycle 2: Cannot issue AR - no space available
Cycle 10: R data arrives - space is guaranteed
```

13.5.3 Write Interface (AXI Read Engine → FIFO)

ID-Based Write:

```
input logic                                axi_rd_sram_valid;      // Single
valid for all channels
input logic [CIW-1:0]                     axi_rd_sram_id;        // Channel ID
select
output logic                               axi_rd_sram_ready;      // Ready
(muxed from selected channel)
input logic [DW-1:0]                     axi_rd_sram_data;      // Shared data
bus
```

Write Protocol: 1. Read engine asserts `axi_rd_sram_valid` with data 2. `axi_rd_sram_id` selects which channel FIFO receives data 3. Controller decodes ID to per-channel `valid_decoded[id]` 4. Selected channel's FIFO ready muxed to `axi_rd_sram_ready`

13.5.4 Drain Interface (AXI Write Engine Flow Control)

Data Availability:

```
input logic [NC-1:0]                     axi_wr_drain_req;      //
Per-channel drain request
input logic [NC-1:0][7:0]                 axi_wr_drain_size;    //
```

Per-channel drain size

```
output logic [NC-1:0][SCW-1:0]                axi_wr_drain_data_avail;  
// Available data after reservations
```

Drain Protocol: 1. Write engine checks `axi_wr_drain_data_avail[ch]` for available data 2. Engine asserts `axi_wr_drain_req[ch]` to reserve data for W burst 3. Drain controller updates available count (subtracts reserved) 4. Data commits when actually read from FIFO

Why Drain Reservation is Critical:

Similar to allocation, drain prevents under-reporting data availability:

Problem without drain reservation:

```
Cycle 0: FIFO has 32 beats available  
Cycle 1: Issue AW (16 beats) - FIFO still shows 32 available  
Cycle 2: Issue AW (16 beats) - FIFO still shows 32 available  
Cycle 3: Issue AW (16 beats) - FIFO still shows 32 available  
Cycle 10: W bursts start draining (48 beats expected)  
→ UNDERFLOW! Only 32 beats available
```

Solution with drain reservation:

```
Cycle 0: FIFO has 32 beats available  
Cycle 1: Issue AW (16 beats), drain 16 - shows 16 available  
(reserved)  
Cycle 2: Issue AW (16 beats), drain 16 - shows 0 available  
(reserved)  
Cycle 3: Cannot issue AW - no data available  
Cycle 10: W bursts drain - data is guaranteed
```

13.5.5 Read Interface (FIFO → AXI Write Engine)

ID-Based Read:

```
output logic [NC-1:0]                axi_wr_sram_valid;        // Per-channel  
valid (data available)  
input  logic                        axi_wr_sram_drain;         // Single  
drain (consumer requests data)  
input  logic [CIW-1:0]              axi_wr_sram_id;           // Channel ID  
select  
output logic [DW-1:0]              axi_wr_sram_data;          // Data from  
selected channel
```

Read Protocol: 1. Write engine checks `axi_wr_sram_valid[ch]` (per-channel) 2. Engine asserts `axi_wr_sram_drain` with `axi_wr_sram_id` 3. Controller decodes ID to per-channel `drain_decoded[id]` 4. Selected channel's data muxed to `axi_wr_sram_data`

13.5.6 Debug Interface

```
output logic [NC-1:0]                dbg_bridge_pending;      // Latency  
bridge pending per channel
```

```
output logic [NC-1:0]          dbg_bridge_out_valid;    // Latency
bridge output valid per channel
```

Purpose: Monitor latency bridge state to catch bugs in read timing.

13.6 ID Decode Logic

13.6.1 Write Valid Decode

Decode `axi_rd_sram_id` to per-channel valid:

```
always_comb begin
    axi_rd_sram_valid_decoded = '0;
    if (axi_rd_sram_valid && axi_rd_sram_id < NC) begin
        axi_rd_sram_valid_decoded[axi_rd_sram_id] = 1'b1;
    end
end
```

Mux ready from selected channel:

```
always_comb begin
    if (axi_rd_sram_id < NC) begin
        axi_rd_sram_ready =
axi_rd_sram_ready_per_channel[axi_rd_sram_id];
    end else begin
        axi_rd_sram_ready = 1'b0;    // Invalid ID → not ready
    end
end
```

13.6.2 Read/Drain Decode

Decode `axi_wr_sram_id` to per-channel drain:

```
always_comb begin
    axi_wr_sram_drain_decoded = '0;
    if (axi_wr_sram_drain && axi_wr_sram_id < NC) begin
        axi_wr_sram_drain_decoded[axi_wr_sram_id] = 1'b1;
    end
end
```

Mux data from selected channel:

```
always_comb begin
    if (axi_wr_sram_id < NC) begin
        axi_wr_sram_data =
axi_wr_sram_data_per_channel[axi_wr_sram_id];
    end else begin
        axi_wr_sram_data = '0;    // Invalid ID → zero data
    end
end
```



```
end
end
```

13.6.3 Allocation Decode

Decode `axi_rd_alloc_id` to per-channel allocation:

```
always_comb begin
    axi_rd_alloc_req_decoded = '0;
    if (axi_rd_alloc_req && axi_rd_alloc_id < NC) begin
        axi_rd_alloc_req_decoded[axi_rd_alloc_id] = 1'b1;
    end
end
```

13.7 Per-Channel Unit

Each channel instantiates `sram_controller_unit` (separate module):

```
sram_controller_unit #(
    .DATA_WIDTH(DW),
    .SRAM_DEPTH(SRAM_DEPTH),
    .SEG_COUNT_WIDTH(SEG_COUNT_WIDTH)
) u_channel_unit (
    .clk                (clk),
    .rst_n              (rst_n),

    // Write interface (decoded valid from ID)
    .axi_rd_sram_valid  (axi_rd_sram_valid_decoded[i]),
    .axi_rd_sram_ready  (axi_rd_sram_ready_per_channel[i]),
    .axi_rd_sram_data    (axi_rd_sram_data),           // SHARED

    // Read interface (decoded drain)
    .axi_wr_sram_valid  (axi_wr_sram_valid[i]),
    .axi_wr_sram_ready  (axi_wr_sram_drain_decoded[i]),
    .axi_wr_sram_data    (axi_wr_sram_data_per_channel[i]),

    // Allocation interface (decoded req from ID)
    .rd_alloc_req       (axi_rd_alloc_req_decoded[i]),
    .rd_alloc_size       (axi_rd_alloc_size),           // SHARED
    .rd_space_free       (axi_rd_alloc_space_free[i]),

    // Drain interface (per-channel)
    .wr_drain_req        (axi_wr_drain_req[i]),
    .wr_drain_size        (axi_wr_drain_size[i]),
    .wr_drain_data_avail (axi_wr_drain_data_avail[i]),
```

```

// Debug
.dbg_bridge_pending      (dbg_bridge_pending[i]),
.dbg_bridge_out_valid    (dbg_bridge_out_valid[i])
);

```

Key Points: - axi_rd_sram_data is **shared** - all units see same data bus - Only unit with valid_decoded[i] = 1 writes to its FIFO - rd_alloc_size is **shared** - all see same size value - Only unit with alloc_req_decoded[i] = 1 reserves space

13.8 Operation Flows

13.8.1 Write Flow (AXI Read Data → FIFO)

Allocation Phase (before data arrives):

1. Read engine issues AR command (araddr, arlen, arid)
2. Engine extracts channel ID from arid[CIW-1:0]
3. Engine asserts:
 - axi_rd_alloc_req = 1
 - axi_rd_alloc_id = channel_id
 - axi_rd_alloc_size = arlen + 1 (burst size in beats)
4. Allocation controller (in sram_controller_unit):
 - Checks rd_space_free[channel_id] >= alloc_size
 - Reserves space (doesn't commit yet)
5. Engine proceeds with AR transaction

Data Arrival Phase:

1. AXI R data arrives:
 - m_axi_rvalid = 1
 - m_axi_rid = transaction_id (contains channel_id in lower bits)
 - m_axi_rdata = data
2. Read engine forwards to SRAM controller:
 - axi_rd_sram_valid = 1
 - axi_rd_sram_id = rid[CIW-1:0] (extract channel ID)
 - axi_rd_sram_data = rdata
3. SRAM controller decodes:
 - axi_rd_sram_valid_decoded[channel_id] = 1
4. Selected channel FIFO:
 - Writes data if ready
 - Commits 1 beat of reserved space
 - Decrements reserved counter

5. Ready mux:
 `axi_rd_sram_ready = fifo_ready[channel_id]`

13.8.2 Read Flow (FIFO → AXI Write Data)

Drain Reservation Phase (before W data drains):

1. Write engine checks `drain_data_avail[channel_id]`
2. If sufficient data available:
 `axi_wr_drain_req[channel_id] = 1`
 `axi_wr_drain_size[channel_id] = burst_size`
3. Drain controller:
 - Reserves data (decrements available count)
 - Prevents other channels from seeing this data

Data Drain Phase:

1. Write engine ready to consume data:
 `axi_wr_sram_drain = 1`
 `axi_wr_sram_id = channel_id`
 2. SRAM controller decodes:
 `axi_wr_sram_drain_decoded[channel_id] = 1`
 3. Selected channel FIFO:
 - Reads data (1-cycle latency through bridge)
 - Commits 1 beat of reserved data
 - Decrements drain reservation
 4. Data mux:
 `axi_wr_sram_data = fifo_data[channel_id]`
 5. Valid output:
 `axi_wr_sram_valid[channel_id] = bridge_valid`
-

13.9 Allocation vs. Drain Controllers

13.9.1 Why Separate Controllers?

Allocation Controller (Write Side): - Problem: AXI read AR issued before R data arrives -
Solution: Pre-allocate space when AR issues, commit when R arrives - Prevents: Over-issuing AR commands when FIFO full

Drain Controller (Read Side): - Problem: AXI write AW issued before W data drains from FIFO -
Solution: Reserve data when AW issues, commit when W drains - Prevents: Under-reporting
available data when burst in progress

13.9.2 Flow Control Comparison

Read Engine (uses allocation):

```
// Check space BEFORE issuing AR  
if (rd_space_free[ch] >= burst_size) begin  
    issue_ar_command();  
    assert_rd_alloc_req(); // Reserve space  
end  
// Later: R data arrives → commits reservation
```

Write Engine (uses drain):

```
// Check data availability BEFORE issuing AW  
if (wr_drain_data_avail[ch] >= burst_size) begin  
    issue_aw_command();  
    assert_wr_drain_req(); // Reserve data  
end  
// Later: W beats drain → commits reservation
```

13.10 Timing Diagrams

13.10.1 Combined Write and Read Timing

The following diagram shows the complete SRAM controller write and read flow:

13.10.1.1 Waveform 2.8.1: SRAM Controller Write/Read Operation



SRAM Controller Write/Read

Source: [sram_controller_wr_rd.json](#)

13.10.2 Write Path (R Data → FIFO)

Cycle: 0 1 2 3 4 5

: Write Path

Alloc: [REQ][---grant---]
(ch2, size=4)

R Data: [V][V][V][V]
R ID: [2][2][2][2]

: Write Path

Decode: [1][1][1][1] (channel 2)
 FIFO: [W][W][W][W]
 Ready: [1][1][1][1][1][1][1]...

- Notes:
- Allocation at cycle 0 reserves 4 beats
 - R data arrives cycles 3-6
 - Each beat commits 1 reserved entry

13.10.3 Read Path (FIFO → W Data)

Cycle:	0	1	2	3	4	5	6

: Read Path
 Drain: [REQ][---reserve---]
 (ch2, size=2)

Drain:	[D][D]
Drain ID:	[2][2]

: Read Path
 Decode: [1][1] (channel 2)
 Bridge: [-][V][V] (1-cycle latency)
 W Data: [D0][D1]

- Notes:
- Drain request at cycle 0 reserves 2 beats
 - Actual drain at cycles 3-4
 - Latency bridge adds 1 cycle (data at 4-5)

13.11 Error Conditions

13.11.1 Invalid Channel ID

Condition: axi_rd_sram_id >= NUM_CHANNELS or axi_wr_sram_id >= NUM_CHANNELS

Handling: - Write: axi_rd_sram_ready = 0 (not ready, data dropped) - Read:
 axi_wr_sram_data = 0 (zero data output) - **No error signal** - engines should not generate
 invalid IDs

13.11.2 FIFO Overflow

Condition: Write when FIFO full

Prevention: - Allocation controller tracks reserved + committed space - Read engine checks `rd_space_free` before issuing AR - Only issues AR when sufficient space available

Should never happen if read engine follows protocol!

13.11.3 FIFO Underflow

Condition: Read when FIFO empty

Prevention: - Drain controller tracks available - reserved data - Write engine checks `wr_drain_data_avail` before issuing AW - Only issues AW when sufficient data available

Should never happen if write engine follows protocol!

13.12 Performance Considerations

13.12.1 Per-Channel FIFO Benefits

Isolation: - Channel failures don't affect others - No cross-channel contention - Simplified debug (each channel independent)

Timing: - No shared arbitration delays - Predictable latency per channel - Easier timing closure

Scalability: - Easy to add/remove channels - Parameterizable depth per channel - No global SRAM redesign needed

13.12.2 SRAM Resource Usage

Total SRAM: $\text{NUM_CHANNELS} \times \text{SRAM_DEPTH} \times \text{DATA_WIDTH}$

Example (8 channels, 512 depth, 512-bit data): - Per channel: $512 \times 512 \text{ bits} = 32\text{KB}$ - Total: $8 \times 32\text{KB} = 256\text{KB}$

Comparison to Segmented Approach: - Segmented: $1 \times (8 \times 512) \times 512 \text{ bits} = 256\text{KB}$ (same total) - But segmented requires complex pointer arithmetic and arbitration - Per-channel FIFOs trade complexity for slight area increase (FIFO overhead)

13.13 Testing

Test Location:

`projects/components/dmas/stream/dv/tests/fub_tests/sram_controller/`

Key Test Scenarios:

1. **Single channel fill/drain** - Basic FIFO operation

2. **All channels concurrent** - Independent operation
 3. **ID decode** - Correct channel selection
 4. **Allocation before data** - Space reserved correctly
 5. **Drain before read** - Data reserved correctly
 6. **Latency bridge** - 1-cycle delay verified
 7. **Invalid ID** - Graceful handling
 8. **Back-to-back** - No bubbles between bursts
-

13.14 Bug Fixes (v0.92)

13.14.1 Registered Avail Outputs Reset (Commit b619eee9)

The registered availability outputs (`w_alloc_avail`, `w_drain_avail`) were missing explicit reset, causing: - X propagation in simulation (metastability) - Unknown post-reset state in synthesis (unmapped flops)

Fix: Added explicit reset to both registered avail outputs. They now initialize to '0 (no space available, no data available) on reset, preventing garbage values.

13.15 Related Documentation

- **SRAM Controller Unit:** `09_sram_controller_unit.md` - Per-channel implementation
 - **Allocation Controller:** `07_stream_alloc_ctrl.md` - Write-side flow control
 - **Drain Controller:** `11_stream_drain_ctrl.md` - Read-side flow control
 - **AXI Read Engine:** `06_axi_read_engine.md` - Write interface usage
 - **AXI Write Engine:** `12_axi_write_engine.md` - Read interface usage
 - **Stream Core:** `01_stream_core.md` - Top-level integration
-

13.16 Revision History

Revision History

Date	Version	Changes
2025-10-18	0.5	Initial draft documenting segmented SRAM approach

Date	Version	Changes
2025-11-16	1.0	Updated to document per-channel FIFO implementation (actual RTL)
2025-11-21	1.5	Merged documentation:- Consolidated duplicate files- Added design rationale section- Enhanced allocation/drain explanations with examples- Clarified per-channel FIFO architecture- Added performance comparison- Verified all content matches current RTL implementation
2025-11-30	1.6	Updated related documentation references

Last Updated: 2025-11-30 (verified against RTL implementation)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

14 SRAM Controller Unit

Module: `sram_controller_unit.sv` **Location:** `projects/components/dmas/stream/rtl/fub/` **Category:** FUB (Functional Unit Block) **Parent:** `sram_controller.sv` **Status:** Implemented **Last Updated:** 2025-11-30

14.1 Overview

The `sram_controller_unit` module is a single-channel SRAM controller unit containing allocation controller, FIFO buffer, and latency bridge. It handles one channel's data flow from AXI read engine through buffering to AXI write engine with proper flow control.

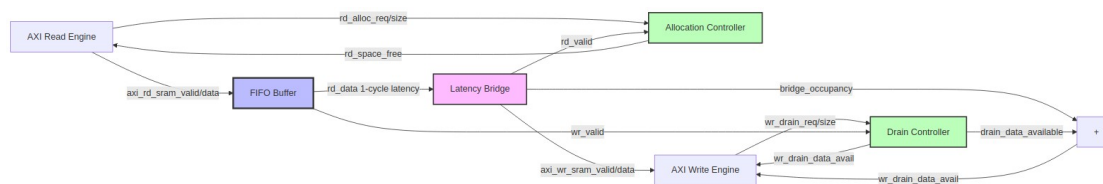
14.1.1 Key Features

- **Three-Component Architecture:**
 - Allocation controller (stream_alloc_ctrl) - Space tracking for reads
 - FIFO buffer (gaxi_fifo_sync) - Physical data storage
 - Latency bridge (stream_latency_bridge) - Timing compensation
- **Drain Controller:** Tracks data availability for write engine
- **Virtual FIFO Pattern:** Pointer arithmetic without data storage for flow control
- **Registered Outputs:** Breaks combinatorial paths for timing closure

14.2 Architecture

14.2.1 Block Diagram

14.2.2 Figure 1: SRAM Controller Unit Block Diagram



SRAM Controller Unit Block Diagram

Source: [06_sram_controller_unit_block.mmd](#)

14.2.3 Component Hierarchy

```
sram_controller_unit
  stream_alloc_ctrl      # Allocation tracking (space availability)
  stream_drain_ctrl      # Drain tracking (data availability)
  gaxi_fifo_sync         # Physical data storage FIFO
  stream_latency_bridge  # 1-cycle latency compensation
```

14.2.4 Data Flow

AXI Read Engine → FIFO Write Port → FIFO Storage → FIFO Read Port →
 Latency Bridge → AXI Write Engine

Allocation Controller (space tracking)

Drain Controller (data tracking)

14.2.5 Controller Naming Convention (CRITICAL)

Allocation Controller Perspective: - wr side = ALLOCATE (reserve space, advance wr_ptr) - rd side = FULFILL (data arrives, advance rd_ptr, FREE space)

Drain Controller Perspective: - wr side = DATA WRITTEN (increment occupancy) - rd side = DRAIN REQUEST (reserve data for write burst)

This is OPPOSITE of normal FIFO naming conventions!

14.3 Parameters

Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	512	Data width in bits
SRAM_DEPTH	int	512	FIFO depth in entries
SEG_COUNT_WIDTH	int	$\lceil \log_2(\text{SRAM_DEPTH}) \rceil + 1$	Width for count signals

14.3.1 Derived Parameters

Derived Parameters

Parameter	Derivation	Description
DW	DATA_WIDTH	Short alias
SD	SRAM_DEPTH	Short alias
SCW	SEG_COUNT_WIDTH	Segment count width
ADDR_WIDTH	$\lceil \log_2(\text{SD}) \rceil$	FIFO address width

14.4 Port List

14.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous

Signal	Direction	Width	Description
			reset

14.4.2 Allocation Interface (Read Engine Flow Control)

Allocation Interface

Signal	Direction	Width	Description
axi_rd_alloc_req	input	1	Request space allocation
axi_rd_alloc_size	input	8	Beats to reserve
axi_rd_alloc_space_free	output	SCW	Free space available

14.4.3 Write Interface (AXI Read Engine to FIFO)

Write Interface

Signal	Direction	Width	Description
axi_rd_sram_valid	input	1	Write data valid
axi_rd_sram_ready	output	1	Ready to accept data
axi_rd_sram_data	input	DW	Write data payload

14.4.4 Drain Interface (Write Engine Flow Control)

Drain Interface

Signal	Direction	Width	Description
axi_wr_drain_data_avail	output	SCW	Data available for drain
axi_wr_drain_req	input	1	Request to drain data
axi_wr_drain_size	input	8	Beats to drain

14.4.5 Read Interface (FIFO to AXI Write Engine)

Read Interface

Signal	Direction	Width	Description
axi_wr_sram_valid	output	1	Read data valid
axi_wr_sram_ready	input	1	Ready to accept data
axi_wr_sram_data	output	DW	Read data payload

14.4.6 Debug Interface

Debug Interface

Signal	Direction	Width	Description
dbg_bridge_pending	output	1	Data in flight in bridge
dbg_bridge_output_valid	output	1	Bridge output valid

14.5 Operation

14.5.1 Allocation Flow

1. **Space Check:** Read engine checks axi_rd_alloc_space_free
2. **Reservation:** Read engine asserts axi_rd_alloc_req with burst size
3. **Space Decrement:** Allocation controller decrements space_free
4. **Data Arrival:** Data enters FIFO via axi_rd_sram_* interface
5. **Space Release:** When data exits (output handshake), space_free increments

14.5.2 Drain Flow

1. **Data Check:** Write engine checks axi_wr_drain_data_avail
2. **Reservation:** Write engine asserts axi_wr_drain_req with burst size
3. **Data Decrement:** Drain controller decrements data_available
4. **Data Consumption:** Write engine reads via axi_wr_sram_* interface

14.5.3 Data Available Calculation

```
// Total data available = drain controller data + latency bridge
occupancy
assign axi_wr_drain_data_avail = drain_data_available +
SCW'(bridge_occupancy);
```

The latency bridge can hold 1 beat in flight plus up to 4 beats in its skid buffer, which must be accounted for in the data availability count.

14.6 Integration Example

```
sram_controller_unit #(
    .DATA_WIDTH      (512),
    .SRAM_DEPTH      (512),
    .SEG_COUNT_WIDTH(10)
) u_sram_controller_unit (
    .clk              (clk),
    .rst_n            (rst_n),

    // Allocation interface
    .axi_rd_alloc_req  (axi_rd_alloc_req[ch]),
    .axi_rd_alloc_size (axi_rd_alloc_size),
    .axi_rd_alloc_space_free(axi_rd_alloc_space_free[ch]),

    // Write interface (from AXI Read Engine)
    .axi_rd_sram_valid  (axi_rd_sram_valid[ch]),
    .axi_rd_sram_ready  (axi_rd_sram_ready[ch]),
    .axi_rd_sram_data    (axi_rd_sram_data),

    // Drain interface
    .axi_wr_drain_data_avail(axi_wr_drain_data_avail[ch]),
    .axi_wr_drain_req      (axi_wr_drain_req[ch]),
    .axi_wr_drain_size     (axi_wr_drain_size[ch]),

    // Read interface (to AXI Write Engine)
    .axi_wr_sram_valid  (axi_wr_sram_valid[ch]),
    .axi_wr_sram_ready  (axi_wr_sram_ready[ch]),
    .axi_wr_sram_data    (axi_wr_sram_data[ch]),

    // Debug
    .dbg_bridge_pending  (dbg_bridge_pending[ch]),
    .dbg_bridge_out_valid (dbg_bridge_out_valid[ch])
);
```

14.7 Common Issues

14.7.1 Issue 1: Space Accounting Mismatch

Symptom: Read engine sees space but FIFO overflows

Root Causes: 1. Allocation controller not receiving fulfillment signals 2. Output handshake not connected to allocation controller rd_valid

Solution: Ensure axi_wr_sram_valid && axi_wr_sram_ready connects to allocation controller.

14.7.2 Issue 2: Data Available Undercount

Symptom: Write engine stalls despite data in FIFO

Root Causes: 1. Bridge occupancy not included in data_available calculation 2. Drain controller not receiving write handshakes

Solution: Verify $\text{axi_wr_drain_data_avail} = \text{drain_data_available} + \text{bridge_occupancy}$.

14.8 Related Documentation

- **Parent:** 08_sram_controller.md - Multi-channel SRAM controller
 - **Allocation Controller:** 07_stream_alloc_ctrl.md - Space tracking details
 - **Drain Controller:** 11_stream_drain_ctrl.md - Data tracking details
 - **Latency Bridge:** 10_stream_latency_bridge.md - Timing compensation
 - **Read Engine:** 06_axi_read_engine.md - Data producer
 - **Write Engine:** 12_axi_write_engine.md - Data consumer
-

14.9 Revision History

SRAM Controller Unit Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table

Version	Date	Author	Description
			captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

15 Stream Latency Bridge

Module: stream_latency_bridge.sv **Location:** projects/components/dmas/stream/rtl/fub/ **Category:** FUB (Functional Unit Block) **Parent:** sram_controller_unit.sv **Status:** Implemented **Last Updated:** 2025-11-30

15.1 Overview

The stream_latency_bridge module is a simple latency-1 bridge that compensates for registered FIFO output latency. It uses glue logic plus a skid buffer to handle backpressure from downstream consumers.

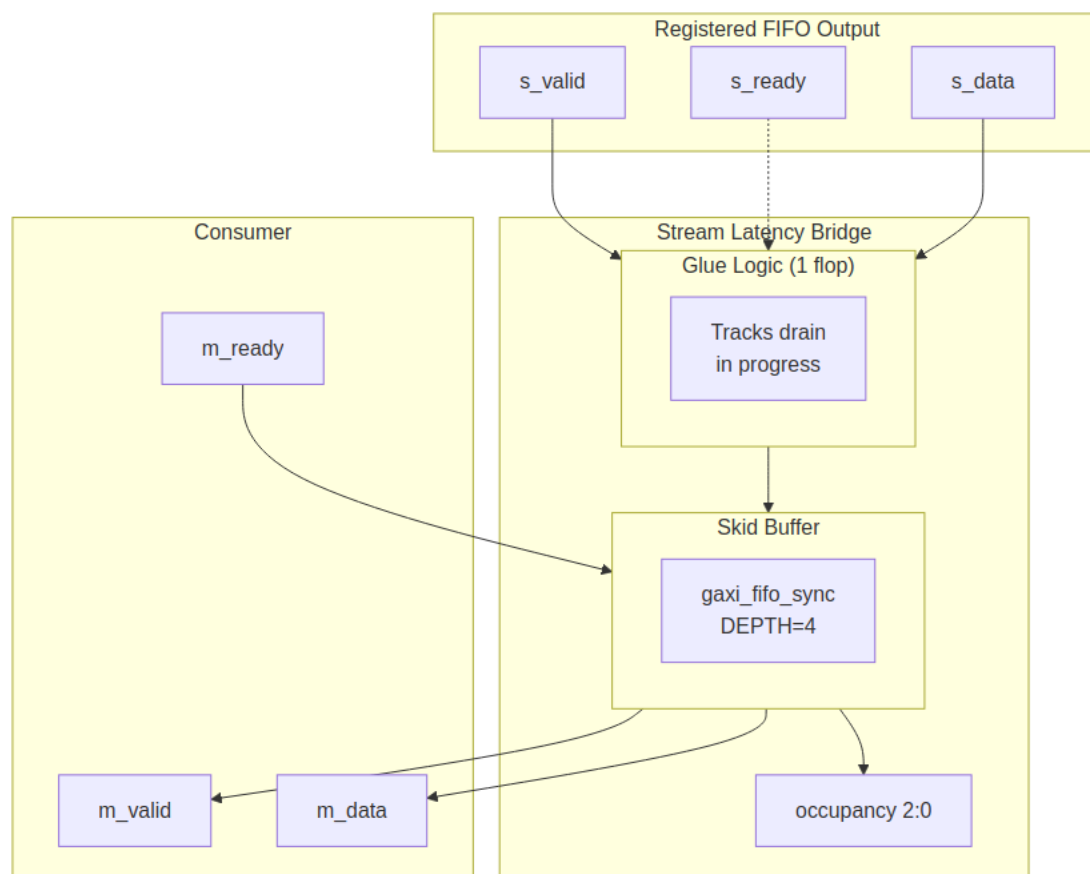
15.1.1 Key Features

- **1-Cycle Latency Compensation:** Bridges registered FIFO output to consumer
- **Skid Buffer Architecture:** 4-deep FIFO absorbs consumer backpressure
- **Full Throughput:** Maintains back-to-back transfers when consumer is ready
- **Simple Design:** No complex ready calculations - skid buffer handles it all
- **Occupancy Tracking:** Reports total beats held for flow control

15.2 Architecture

15.2.1 Block Diagram

15.2.2 Figure 2.10.1: Stream Latency Bridge Block Diagram



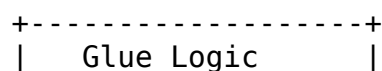
Stream Latency Bridge Block Diagram

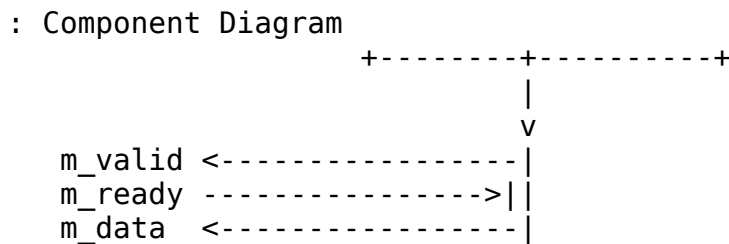
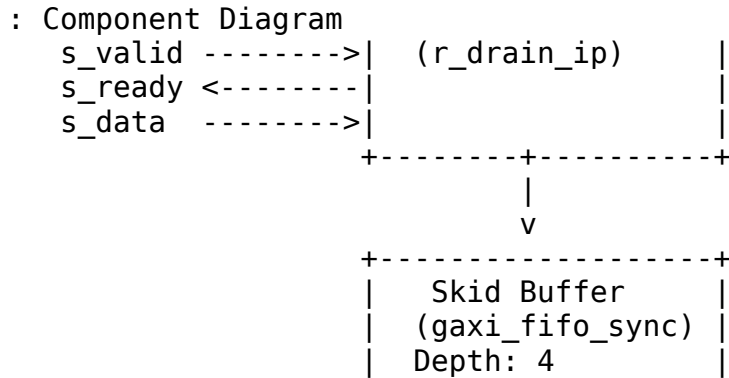
Source: [06_stream_latency_bridge_block.mmd](#)

15.2.3 Operation Flow

Cycle 0: Drain FIFO (`s_valid` && `skid_ready`) $\emptyset$ `r_drain_ip` = 1
Cycle 1: Data arrives from FIFO $\emptyset$ Push to skid (`skid_valid` = `r_drain_ip`)
Cycle N: Consumer drains skid buffer at its own pace

15.2.4 Component Diagram





15.2.5 Backpressure Logic

The bridge uses a conservative backpressure approach: - Track stalled writes (wr_valid && ! wr_ready) - Calculate pending count = skid_count + stalled - Allow accept when pending < SKID_DEPTH OR consumer is draining

Truth Table (SKID_DEPTH=4):

count	wr_valid	wr_ready	stalled	pending	room	s_ready	
0	0	1	0	0	1	1	0
Empty							
0	1	1	0	0	1	1	0
Writing, completes							
3	0	1	0	3	1	1	0 Room
for 1							
3	1	1	0	3	1	1	0
Writing, will be 4							
3	1	0	1	4	0	0	0
Write stalled							
4	0	0	0	4	0	0	0 Full

15.3 Parameters

Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	64	Data width in bits
SKID_DEPTH	int	4	Skid buffer depth (2-4 recommended)

15.3.1 Derived Parameters

Derived Parameters

Parameter	Derivation	Description
DW	DATA_WIDTH	Short alias

15.4 Port List

15.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

15.4.2 Upstream Interface (from registered FIFO)

Upstream Interface

Signal	Direction	Width	Description
s_valid	input	1	Upstream valid (FIFO not empty)
s_ready	output	1	Ready to accept from FIFO
s_data	input	DW	Data from FIFO (valid 1 cycle after handshake)

15.4.3 Downstream Interface (to consumer)

Downstream Interface

Signal	Direction	Width	Description
m_valid	output	1	Data valid to consumer
m_ready	input	1	Consumer ready
m_data	output	DW	Data to consumer

15.4.4 Status Interface

Status Interface

Signal	Direction	Width	Description
occupancy	output	3	Beats in bridge (0-5: 1 in flight + 4 in skid)

15.4.5 Debug Interface

Debug Interface

Signal	Direction	Width	Description
dbg_r_pending	output	1	Data in flight (r_drain_ip)
dbg_r_out_val_id	output	1	Output valid (m_valid)

15.5 Operation

15.5.1 Glue Logic

The glue logic consists of a single flop (r_drain_ip) that tracks when a FIFO drain is in progress:

```
// Drain FIFO when upstream has data AND we can accept
wire w_drain_fifo = s_valid && s_ready;

// Flop the drain signal to track data in flight
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_drain_ip <= 1'b0;
```

```

    end else begin
        r_drain_ip <= w_drain_fifo;
    end
end

// When r_drain_ip asserted, data arrived from FIFO 0 push to skid
assign skid_wr_valid = r_drain_ip;
assign skid_wr_data = s_data;

```

15.5.2 Skid Buffer

A 4-deep gaxi_fifo_sync instance with: - Non-registered mode (REGISTERED=0) for minimum latency - Auto memory style selection

15.5.3 Occupancy Calculation

```

// Total occupancy = data in skid buffer
// Note: r_drain_ip not counted separately (conservative)
assign occupancy = skid_count;

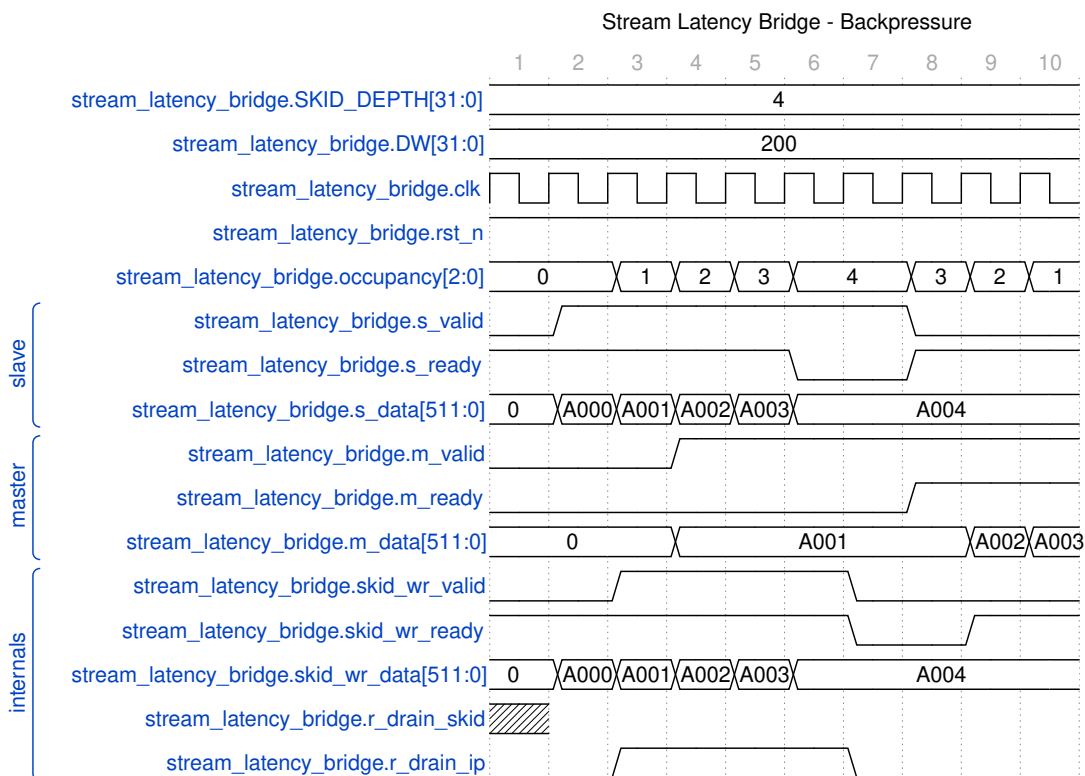
```

15.6 Timing Diagrams

15.6.1 Backpressure Handling

The following diagram shows how the latency bridge handles downstream backpressure:

15.6.1.1 Waveform 2.10.1: Stream Latency Bridge Backpressure



Latency Bridge Backpressure

Source: [latency_bridge_backpressure.json](#)

15.7 Integration Example

```
stream_latency_bridge #(
    .DATA_WIDTH (512),
    .SKID_DEPTH (4)
) u_latency_bridge (
    .clk          (clk),
    .rst_n        (rst_n),

    // Slave (from FIFO)
    .s_data       (fifo_rd_data),
    .s_valid      (fifo_rd_valid),
    .s_ready      (fifo_rd_ready),

    // Master (to AXI Write Engine)
    .m_data       (axi_wr_sram_data),
```

```

.m_valid      (axi_wr_sram_valid),
.m_ready      (axi_wr_sram_ready),

// Status
.occupancy     (bridge_occupancy),

// Debug
.dbg_r_pending (dbg_bridge_pending),
.dbg_r_out_valid(dbg_bridge_out_valid)
);

```

15.8 Design Rationale

15.8.1 Why Use a Skid Buffer?

Previous designs attempted complex ready signal calculations to handle backpressure. This led to:

- Timing issues from deep combinatorial paths
- Edge cases with multi-cycle backpressure
- Difficult debugging of flow control bugs

The skid buffer approach:

- Simple glue logic (single flop)
- FIFO handles all backpressure complexity
- Proven, reusable component
- Easy to verify and debug

15.8.2 Why 4-Deep?

- **2-deep:** Minimum for back-to-back (1 in flight + 1 buffered)
- **4-deep:** Allows 2-3 cycles of consumer stall without upstream backpressure
- **8-deep:** Overkill for most use cases

4-deep provides good balance between:

- Latency tolerance (handles brief stalls)
- Resource usage (small FIFO)
- Throughput (back-to-back capable)

15.9 Related Documentation

- **Parent:** 09_sram_controller_unit.md - Single-channel SRAM controller
- **Consumer:** 12_axi_write_engine.md - Receives bridge output

- **Producer: 06_axi_read_engine.md - Feeds upstream FIFO**

15.10 Revision History

Stream Latency Bridge Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

16 Stream Drain Controller

Module: stream_drain_ctrl.sv **Location:** projects/components/dmas/stream/rtl/fub/
Category: FUB (Functional Unit Block) **Parent:** sram_controller_unit.sv **Status:** Implemented
Last Updated: 2025-11-30

16.1 Overview

The stream_drain_ctrl module is a virtual FIFO for write engine flow control. It tracks data availability using FIFO pointer logic without storing any data. This allows the write engine to pre-reserve data before issuing AXI write commands.

16.1.1 Key Features

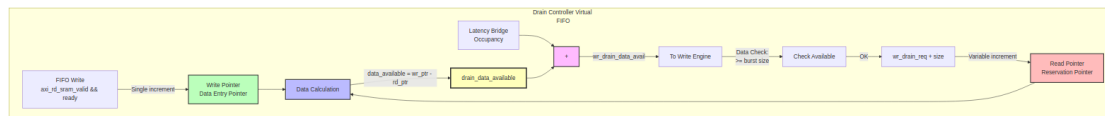
- **Virtual FIFO Pattern:** Pointer arithmetic only, no data storage
- **Pre-Reservation:** Write engine reserves data before AXI AW command
- **Variable Size Drains:** Supports burst-sized reservations (not single beat)

- **Registered Outputs:** Optional output registration for timing
- **Underflow Prevention:** Ensures data exists before write engine commits

16.2 Architecture

16.2.1 Block Diagram

16.2.2 Figure 1: Stream Drain Controller Block Diagram



Stream Drain Controller Block Diagram

Source: [09_stream_drain_ctrl_block.mmd](#)

16.2.3 Virtual FIFO Concept

Unlike a real FIFO that stores data, the drain controller only tracks pointers:

Real FIFO:

Data[0]
Data[1]
Data[2]
...

Virtual FIFO (Drain Controller):

wr_ptr: 5 Tracks entries written
rd_ptr: 2 Tracks entries drained
available:3 Difference

Actual data stored in gaxi_fifo_sync (separate component)

16.2.4 Pointer Operations

Write Side (Data Entry):

When FIFO receives data:

```
wr_ptr += 1
data_available += 1
```

Read Side (Drain Request):

When write engine requests drain:

```
rd_ptr += rd_size (variable burst size!)
data_available -= rd_size
```

16.3 Parameters

Parameters

Parameter	Type	Default	Description
DEPTH	int	512	Virtual FIFO depth
ALMOST_WR_MARGIN	int	1	Almost full margin
ALMOST_RD_MARGIN	int	1	Almost empty margin
REGISTERED	int	1	Registered outputs

16.3.1 Derived Parameters

Derived Parameters

Parameter	Derivation	Description
D	DEPTH	Short alias
AW	$\lceil \log_2(D) \rceil$	Address width

16.4 Port List

16.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	System clock
axi_aresetn	input	1	Active-low asynchronous reset

16.4.2 Write Interface (Data Entry)

Write Interface

Signal	Direction	Width	Description
wr_valid	input	1	Data written to FIFO

Signal	Direction	Width	Description
wr_ready	output	1	Not full (can accept)

16.4.3 Read Interface (Drain Requests)

Read Interface

Signal	Direction	Width	Description
rd_valid	input	1	Request to drain data
rd_size	input	8	Number of entries to drain
rd_ready	output	1	Data available (not empty)

16.4.4 Status Outputs

Status Outputs

Signal	Direction	Width	Description
data_available	output	AW+1	Available data count
wr_full	output	1	Full (no space)
wr_almost_full	output	1	Almost full
rd_empty	output	1	Empty (no data)
rd_almost_empty	output	1	Almost empty

16.5 Operation

16.5.1 Data Entry (Write Side)

Data entry is single-beat increments, tracking each beat written to the physical FIFO:

```
// Write pointer uses counter_bin for single-beat increments
counter_bin #(
    .WIDTH (AW + 1),
```

```

        .MAX      (D)
    ) write_pointer_inst (
        .clk      (axi_aclk),
        .rst_n    (axi_aresetn),
        .enable    (w_write && !r_wr_full),
        .counter_bin_curr (r_wr_ptr_bin),
        .counter_bin_next (w_wr_ptr_bin_next)
    );

```

16.5.2 Drain Requests (Read Side)

Drain requests support variable-size bursts:

```

// Read pointer advances by rd_size on each drain request
always_ff @(posedge axi_aclk or negedge axi_aresetn) begin
    if (!axi_aresetn) begin
        r_rd_ptr_bin <= '0;
    end else begin
        if (w_read && !r_rd_empty) begin
            r_rd_ptr_bin <= r_rd_ptr_bin + (AW+1)'(rd_size);
        end
    end
end

```

16.5.3 Status Generation

Uses fifo_control block for full/empty/almost flags:

```

fifo_control #(
    .DEPTH          (D),
    .ADDR_WIDTH     (AW),
    .ALMOST_RD_MARGIN (ALMOST_RD_MARGIN),
    .ALMOST_WR_MARGIN (ALMOST_WR_MARGIN),
    .REGISTERED     (REGISTERED)
) fifo_control_inst (
    // ... pointer inputs
    .count          (w_count),
    .wr_full        (r_wr_full),
    .wr_almost_full (r_wr_almost_full),
    .rd_empty       (r_rd_empty),
    .rd_almost_empty (r_rd_almost_empty)
);

```

16.6 Comparison with stream_alloc_ctrl

Comparison with stream_alloc_ctrl

Aspect	stream_alloc_ctrl	stream_drain_ctrl
Purpose	Track space for reads	Track data for writes
Write Side	Reservation (burst)	Data entry (single beat)
Read Side	Fulfillment (single beat)	Drain request (burst)
Consumer	AXI Read Engine	AXI Write Engine

Both use virtual FIFO pattern but with opposite semantics for read/write operations.

16.7 Integration Example

```
stream_drain_ctrl #(
    .DEPTH          (512),
    .REGISTERED     (1)
) u_drain_ctrl (
    .axi_aclk       (clk),
    .axi_aresetn    (rst_n),

    // Write interface: Connected to FIFO write handshake
    .wr_valid       (fifo_wr_valid && fifo_wr_ready),
    .wr_ready       (), // Not used (tracks, doesn't control)

    // Read interface: Connected to write engine drain requests
    .rd_valid       (axi_wr_drain_req),
    .rd_size        (axi_wr_drain_size),
    .rd_ready       (), // Not used (polling interface)

    // Status
    .data_available (drain_data_available),
    .wr_full        (),
    .wr_almost_full (),
    .rd_empty       (),
    .rd_almost_empty ()
);
```

16.8 Common Issues

16.8.1 Issue 1: Data Available Undercount

Symptom: Write engine sees 0 available when FIFO has data

Root Causes: 1. `wr_valid` not connected to FIFO write handshake 2. Bridge occupancy not added to `data_available`

Solution: Ensure connection: `wr_valid = fifo_wr_valid && fifo_wr_ready`

16.8.2 Issue 2: Drain Request Overflow

Symptom: `rd_ptr` advances past `wr_ptr`

Root Causes: 1. Write engine requesting more than available 2. Multiple simultaneous drain requests

Solution: Write engine must check `data_available >= requested_size` before draining.

16.9 Bug Fixes (v0.92)

16.9.1 Stale-View Race Closure (Commit a82627af)

The drain controller was sampling a registered signal one cycle stale, producing `wvalid` before SRAM data had settled. This occurred because:

1. SRAM read data changes on clock edge
2. Drain controller samples that data combinatorially in the same cycle
3. Data fed to write engine, `wvalid` asserted → write engine consumes stale data

Fix: A post-flop register is inserted after the drain mux (SRAM data path), and `wvalid` is only asserted after this register. This guarantees that data has settled before `wvalid` assertion.

Matching half: Write engine (`12_axi_write_engine.md`) implements the post-flop gating on `wvalid` in coordination with this fix.

16.10 Related Documentation

- **Parent:** `09_sram_controller_unit.md` - Integration context
- **Counterpart:** `07_stream_alloc_ctrl.md` - Space tracking (read side)

- **Consumer:** `12_axi_write_engine.md` - Uses drain interface
- **Control Block:** See `fifo_control` in `rtl/amba/gaxi/`

16.11 Revision History

Stream Drain Controller Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

17 AXI Write Engine

Module: `axi_write_engine.sv` **Location:** `projects/components/dmas/stream/rtl/fub/`
Category: FUB (Functional Unit Block) **Parent:** `stream_core.sv` **Status:** Implemented **Last Updated:** 2025-11-30

17.1 Overview

The `axi_write_engine` module is a high-performance multi-channel AXI4 write engine with data-aware arbitration. It serves all 8 STREAM channels through a single AXI master interface with intelligent flow control.

17.1.1 Key Features

- **Round-Robin Arbitration:** Fair scheduling across channels

17.2.4 Key Design Decisions

FIFO-Based W Tracking (No FSM): Instead of a state machine, uses a single shared FIFO to track pending W-phase transactions. This preserves the AW command order which is critical for W-phase correctness.

Separate W and B Phase FIFOs: - **W-Phase FIFO:** Single shared FIFO (in-order with AW) - **B-Phase FIFOs:** Per-channel FIFOs (out-of-order responses allowed by AXI4)

ID-Based SRAM Interface: The write engine drives `axi_wr_sram_id` to select which channel's data to drain. The SRAM controller muxes the appropriate channel's data to `axi_wr_sram_data`.

17.3 Parameters

Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of channels
ADDR_WIDTH	int	64	AXI address width
DATA_WIDTH	int	512	AXI data width
ID_WIDTH	int	8	AXI ID width
USER_WIDTH	int	8	AXI USER width
SEG_COUNT_WIDTH	int	8	Width of space/count signals
PIPELINE	int	0	0: non-pipelined, 1: pipelined
AW_MAX_OUTSTANDING	int	8	Maximum outstanding AW requests (PIPELINE=1)
W_PHASE_FIFO_DEPTH	int	64	W-phase transaction FIFO depth
B_PHASE_FIFO_DEPTH	int	16	B-phase transaction FIFO depth per channel

17.3.1 Derived Parameters

Derived Parameters

Parameter	Derivation	Description
NC	NUM_CHANNELS	Short alias
AW	ADDR_WIDTH	Short alias
DW	DATA_WIDTH	Short alias
IW	ID_WIDTH	Short alias
UW	USER_WIDTH	Short alias
SCW	SEG_COUNT_WIDTH	Segment count width
CIW	$\text{\$clog2}(\text{NC})$	Channel ID width (min 1 bit)
AXSIZE	$\text{\$clog2}(\text{DW}/8)$	AXI burst size

17.4 Port List

17.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

17.4.2 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_axi_wr_xfer_beats	input	8	Transfer size in beats (all channels)

17.4.3 Scheduler Interface (Per-Channel)

Scheduler Interface

Signal	Direction	Width	Description
sched_wr_val id[ch]	input	NC	Channel requests write
sched_wr_rea dy[ch]	output	NC	Engine ready (descriptor complete)
sched_wr_add r[ch]	input	NC x AW	Destination addresses
sched_wr_bea ts[ch]	input	NC x 32	Beats remaining to write
sched_wr_bur st_len[ch]	input	NC x 8	Requested burst length

17.4.4 Completion Interface (Per-Channel)

Completion Interface

Signal	Direction	Width	Description
sched_wr_don e_strobe[ch]	output	NC	Burst completed (1 cycle pulse)
sched_wr_bea ts_done[ch]	output	NC x 32	Number of beats completed

17.4.5 SRAM Drain Interface

SRAM Drain Interface

Signal	Direction	Width	Description
axi_wr_drain _req[ch]	output	NC	Request to reserve data
axi_wr_drain _size[ch]	output	NC x 8	Beats to reserve
axi_wr_drain _data_avail[ch]	input	NC x SCW	Data available per channel

17.4.6 SRAM Read Interface

SRAM Read Interface

Signal	Direction	Width	Description
axi_wr_sram_v	input	NC	Per-channel

Signal	Direction	Width	Description
alid[ch]			valid
axi_wr_sram_drain	output	1	Drain request
axi_wr_sram_id	output	CIW	Channel ID select
axi_wr_sram_data	input	DW	Muxed data from selected channel

17.4.7 AXI4 AW Channel

AXI4 AW Channel

Signal	Direction	Width	Description
m_axi_awvalid	output	1	Address valid
m_axi_awready	input	1	Address ready
m_axi_awid	output	IW	Transaction ID
m_axi_awaddr	output	AW	Address
m_axi_awlen	output	8	Burst length - 1
m_axi_awsz	output	3	Burst size (log2 bytes)
m_axi_awburst	output	2	Burst type (INCR)

17.4.8 AXI4 W Channel

AXI4 W Channel

Signal	Direction	Width	Description
m_axi_wvalid	output	1	Write data valid
m_axi_wready	input	1	Write data ready
m_axi_wdata	output	DW	Write data
m_axi_wstrb	output	DW/8	Write strobes
m_axi_wlast	output	1	Last beat of burst

Signal	Direction	Width	Description
m_axi_wuser	output	UW	Channel ID for tracking

17.4.9 AXI4 B Channel

AXI4 B Channel

Signal	Direction	Width	Description
m_axi_bvalid	input	1	Response valid
m_axi_bready	output	1	Response ready
m_axi_bid	input	IW	Transaction ID
m_axi_bresp	input	2	Response

17.4.10 Error Interface

Error Interface

Signal	Direction	Width	Description
sched_wr_error[ch]	output	NC	Sticky error flag per channel

17.4.11 Debug Interface

Debug Interface

Signal	Direction	Width	Description
dbg_wr_all_complete[ch]	output	NC	All writes complete
dbg_aw_transactions	output	32	Total AW transactions issued
dbg_w_beats	output	32	Total W beats written to AXI

17.5 Operation

17.5.1 Data-Aware Request Masking

```
// Only request arbitration if:  
// 1. Scheduler is requesting (sched_wr_valid)  
// 2. Sufficient SRAM data available (w_data_ok)  
// 3. Below outstanding limit (w_no_outstanding)  
w_arb_request[i] = sched_wr_valid[i] && w_data_ok[i] &&  
w_no_outstanding[i];
```

```
// Data check includes final burst handling  
w_data_ok[i] = w_has_data[i] || w_final_burst[i];
```

17.5.2 W-Phase Transaction FIFO

Single shared FIFO preserves AW command order:

```
typedef struct packed {  
    logic [7:0]    beats;           // Number of beats for this W  
    transaction  
    logic [CIW-1:0] channel_id;    // Channel ID for this transaction  
} w_phase_txn_t;
```

```
// Push on AW handshake (preserves order)  
if (m_axi_awvalid && m_axi_awready) begin  
    w_phase_txn_fifo_wr = 1'b1;  
    w_phase_txn_fifo_din.beats = m_axi_awlen + 8'd1;  
    w_phase_txn_fifo_din.channel_id = r_aw_channel_id;  
end
```

17.5.3 B-Phase Transaction FIFOs

Per-channel FIFOs track beats and last flag for completion:

```
typedef struct packed {  
    logic [7:0]    beats;           // Number of beats in this transaction  
    logic          last;           // Is this the last transfer for  
    descriptor?  
} b_phase_txn_t;
```

```
// last flag determines when sched_wr_ready asserts  
if (m_axi_bvalid && m_axi_bready) begin  
    if (b_phase_txn_fifo_dout[ch_id].last) begin  
        r_sched_ready[ch_id] <= 1'b1;  
    end  
end
```

17.5.4 Outstanding Transaction Tracking

PIPELINE=0 (Non-Pipelined): - Boolean flag per channel (0 or 1 outstanding) - Set when AW issues, clear when B arrives

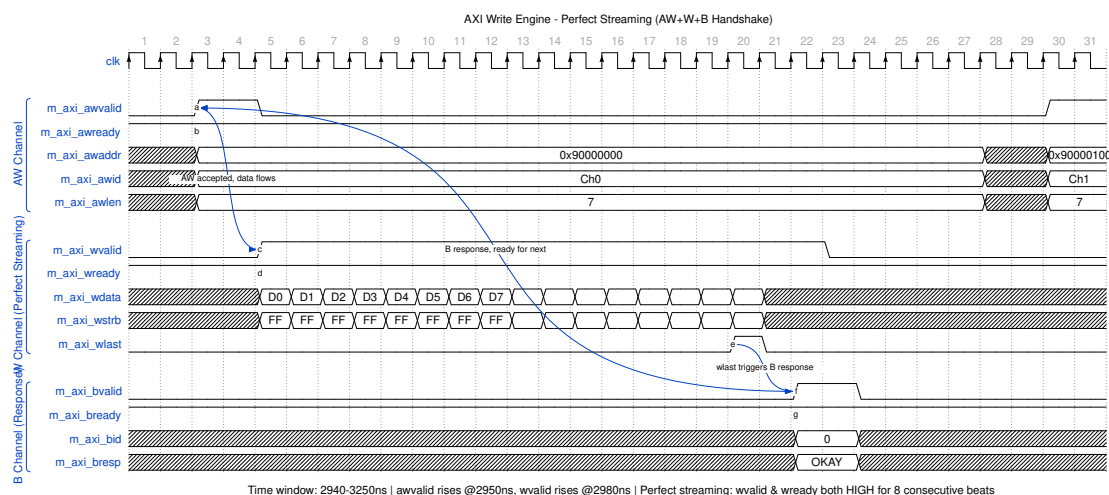
PIPELINE=1 (Pipelined): - Counter per channel (0 to AW_MAX_OUTSTANDING) - Increment on AW, decrement on B

17.6 Timing Diagrams

17.6.1 Perfect Streaming - AXI Write Transaction

The following timing diagram shows the AXI write engine operating at maximum throughput with **perfect streaming** - where both `wvalid` and `wready` remain HIGH for consecutive clock cycles, achieving one data beat per cycle.

17.6.1.1 Waveform 2.12.1: AXI Write Engine - Perfect Streaming



AXI Write Engine - Perfect Streaming

Transaction Flow:

1. AW Channel Handshake (Address Phase)

- `m_axi_awvalid` rises to initiate a write request
- `m_axi_awready` is already HIGH (slave ready)
- Address (`awaddr`), ID (`awid`), and length (`awlen=7` for 8 beats) are captured
- Handshake completes in a single cycle when both valid and ready are HIGH

- Simultaneously, axi_wr_drain_req reserves data from SRAM controller

2. W Channel Streaming (Data Phase)

- After AW acceptance, the engine begins streaming data from SRAM
- m_axi_wvalid rises and **stays HIGH** for all 8 beats
- m_axi_wready remains HIGH throughout (no backpressure from AXI slave)
- **Perfect streaming:** One data beat transferred every clock cycle
- m_axi_wlast rises on the final beat (beat 7) to mark burst completion
- wstrb is all-ones (0xFF) indicating full data width valid

3. B Channel Response (Completion Phase)

- After wlast, the AXI slave returns a write response
- m_axi_bvalid rises with bid matching the transaction ID
- m_axi_bready is HIGH (engine always ready for responses)
- bresp=OKAY indicates successful write
- Response triggers sched_wr_done_strobe to notify scheduler

4. Next Transaction

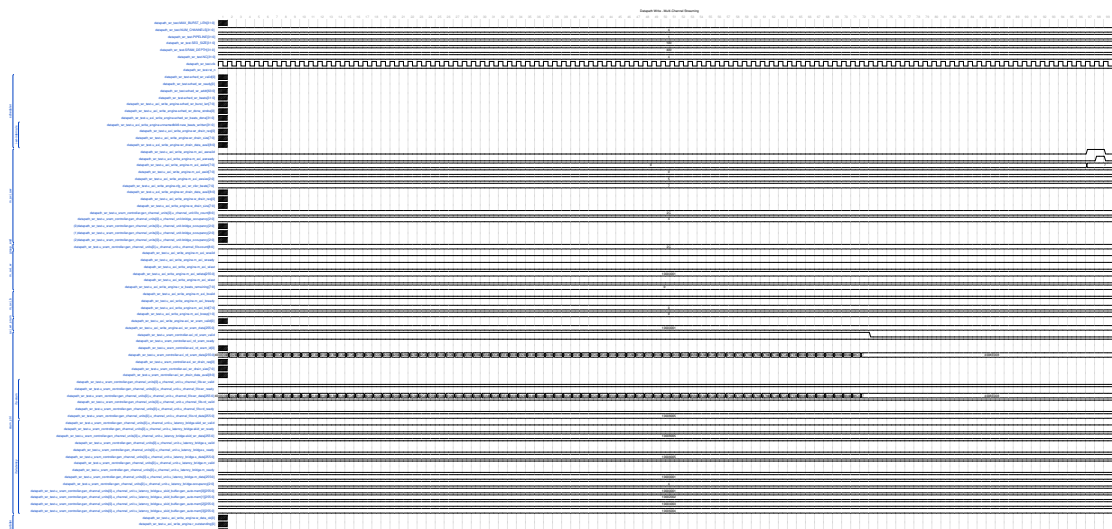
- After B response, the engine can immediately issue the next AW request
- The cycle repeats for the next channel or next burst

Key Performance Indicators: - **No bubbles:** wvalid and wready both HIGH during data phase - **Full bandwidth:** Data width (256/512 bits) x clock frequency - **Zero-wait states:** SRAM controller provides data at line rate - **Minimal latency:** B response arrives shortly after wlast

17.6.2 Multi-Channel Streaming

For multi-channel operation showing channel switching while maintaining streaming performance, see:

17.6.2.1 Waveform 2.12.2: Datapath Write - Multi-Channel



Datapath Write - Multi-Channel

This diagram shows how the engine arbitrates between channels while maintaining high throughput. Note how: - Different channel IDs (awid) appear in sequence - W-phase FIFO preserves AW order for correct data association - B responses can arrive out-of-order (per AXI4 spec)

17.7 Integration Example

```
axi_write_engine #(
    .NUM_CHANNELS          (8),
    .ADDR_WIDTH            (64),
    .DATA_WIDTH            (512),
    .ID_WIDTH              (8),
    .SEG_COUNT_WIDTH       (10),
    .PIPELINE              (0),
    .AW_MAX_OUTSTANDING    (8),
    .W_PHASE_FIFO_DEPTH    (64),
    .B_PHASE_FIFO_DEPTH    (16)
) u_axi_write_engine (
    .clk                    (clk),
    .rst_n                  (rst_n),

    // Configuration
    .cfg_axi_wr_xfer_beats  (cfg_axi_wr_xfer_beats),

    // Scheduler interface
    .sched_wr_valid         (sched_wr_valid),
```

```

.sched_wr_ready      (sched_wr_ready),
.sched_wr_addr       (sched_wr_addr),
.sched_wr_beats       (sched_wr_beats),
.sched_wr_burst_len   (sched_wr_burst_len),

// Completion interface
.sched_wr_done_strobe (sched_wr_done_strobe),
.sched_wr_beats_done   (sched_wr_beats_done),

// SRAM drain interface
.axi_wr_drain_req      (axi_wr_drain_req),
.axi_wr_drain_size     (axi_wr_drain_size),
.axi_wr_drain_data_avail(axi_wr_drain_data_avail),

// SRAM read interface
.axi_wr_sram_valid     (axi_wr_sram_valid),
.axi_wr_sram_drain     (axi_wr_sram_drain),
.axi_wr_sram_id        (axi_wr_sram_id),
.axi_wr_sram_data      (axi_wr_sram_data),

// AXI master
.m_axi_awvalid         (m_axi_wr_awvalid),
.m_axi_awready         (m_axi_wr_awready),
.m_axi_awid            (m_axi_wr_awid),
.m_axi_awaddr          (m_axi_wr_awaddr),
.m_axi_awlen           (m_axi_wr_awlen),
.m_axi_awsz            (m_axi_wr_awsz),
.m_axi_awburst         (m_axi_wr_awburst),

.m_axi_wvalid          (m_axi_wr_wvalid),
.m_axi_wready          (m_axi_wr_wready),
.m_axi_wdata           (m_axi_wr_wdata),
.m_axi_wstrb           (m_axi_wr_wstrb),
.m_axi_wlast           (m_axi_wr_wlast),
.m_axi_wuser           (m_axi_wr_wuser),

.m_axi_bvalid          (m_axi_wr_bvalid),
.m_axi_bready          (m_axi_wr_bready),
.m_axi_bid             (m_axi_wr_bid),
.m_axi_bresp           (m_axi_wr_bresp),

// Error and debug
.sched_wr_error        (sched_wr_error),
.dbg_wr_all_complete   (dbg_wr_all_complete),
.dbg_aw_transactions   (dbg_aw_transactions),

```

```

        .dbg_w_beats                (dbg_w_beats)
    );

```

17.8 Bug Fixes (v0.92)

17.8.1 w_arb_request Timing Closure (Commit 4e8f9e02)

At 8 channels, the arbiter request combinational path became critical. The `w_arb_request` signal is now **registered** to add a single pipeline stage, closing timing. See `06_axi_read_engine.md` for details.

17.8.2 W-Side Post-Flop Gating on wvalid (Commit a82627af)

The write engine now uses **post-flop gating on wvalid** to defeat a stale-view race in the drain controller. Previously, the `wvalid` signal was generated before SRAM data had settled through the drain path's register, causing the drain controller to sample data one cycle stale.

Fix: A register is placed after the W-path data multiplexer, and `wvalid` is only asserted after this flop. This ensures data stability before `wvalid` assertion, preventing premature consumption.

Cross-reference: `11_stream_drain_ctrl.md` documents the matching half of this fix (invalidating the stale-view acknowledgment).

17.9 Related Documentation

- **Parent:** `01_stream_core.md` - Top-level integration
 - **Scheduler Array:** `02_scheduler_group_array.md` - Provides `sched_wr_*` signals
 - **SRAM Controller:** `08_sram_controller.md` - Data source
 - **Drain Controller:** `11_stream_drain_ctrl.md` - Data availability tracking
 - **Read Engine:** `06_axi_read_engine.md` - Complementary read datapath
-

17.10 Revision History

AXI Write Engine Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block

Version	Date	Author	Description
0.91	2026-01-02	seang	specification Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

18 APB to Descriptor Router

Module: `apbtodescr.sv` **Location:** `projects/components/dmas/stream/rtl/fub/` **Category:** FUB (Functional Unit Block) **Parent:** `stream_top.sv` **Status:** Implemented **Last Updated:** 2025-11-30

18.1 Overview

The `apbtodescr` module is an APB-to-descriptor engine router that handles address-based routing of APB writes to descriptor engine kick-off ports. It decodes the APB address to determine the target channel and routes the 64-bit descriptor address using a two-write sequence.

18.1.1 Key Features

- **Address-Based Routing:** APB address bits select target channel
- **64-bit Address Assembly:** Two 32-bit APB writes combine into 64-bit descriptor address
- **Backpressure Handling:** Delays APB response if descriptor engine busy
- **Address Range Checking:** Error on out-of-range addresses
- **Integration Signal:** `apb_descriptor_kickoff_hit` for response muxing

18.2 Architecture

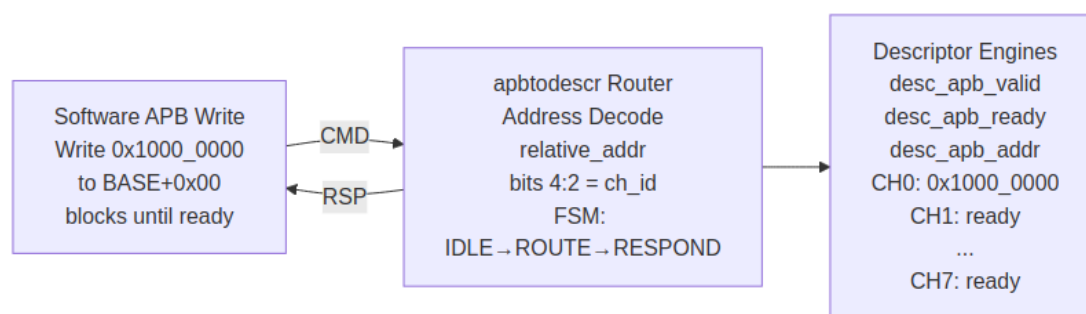
18.2.1 Address Map

Relative to BASE_ADDR:

BASE + 0x00: Channel 0 descriptor address LOW [31:0]
BASE + 0x04: Channel 0 descriptor address HIGH [63:32]
BASE + 0x08: Channel 1 descriptor address LOW [31:0]
BASE + 0x0C: Channel 1 descriptor address HIGH [63:32]
...
BASE + 0x38: Channel 7 descriptor address LOW [31:0]
BASE + 0x3C: Channel 7 descriptor address HIGH [63:32]

18.2.2 Block Diagram

18.2.3 Figure 2.13.1: APB to Descriptor Block Diagram

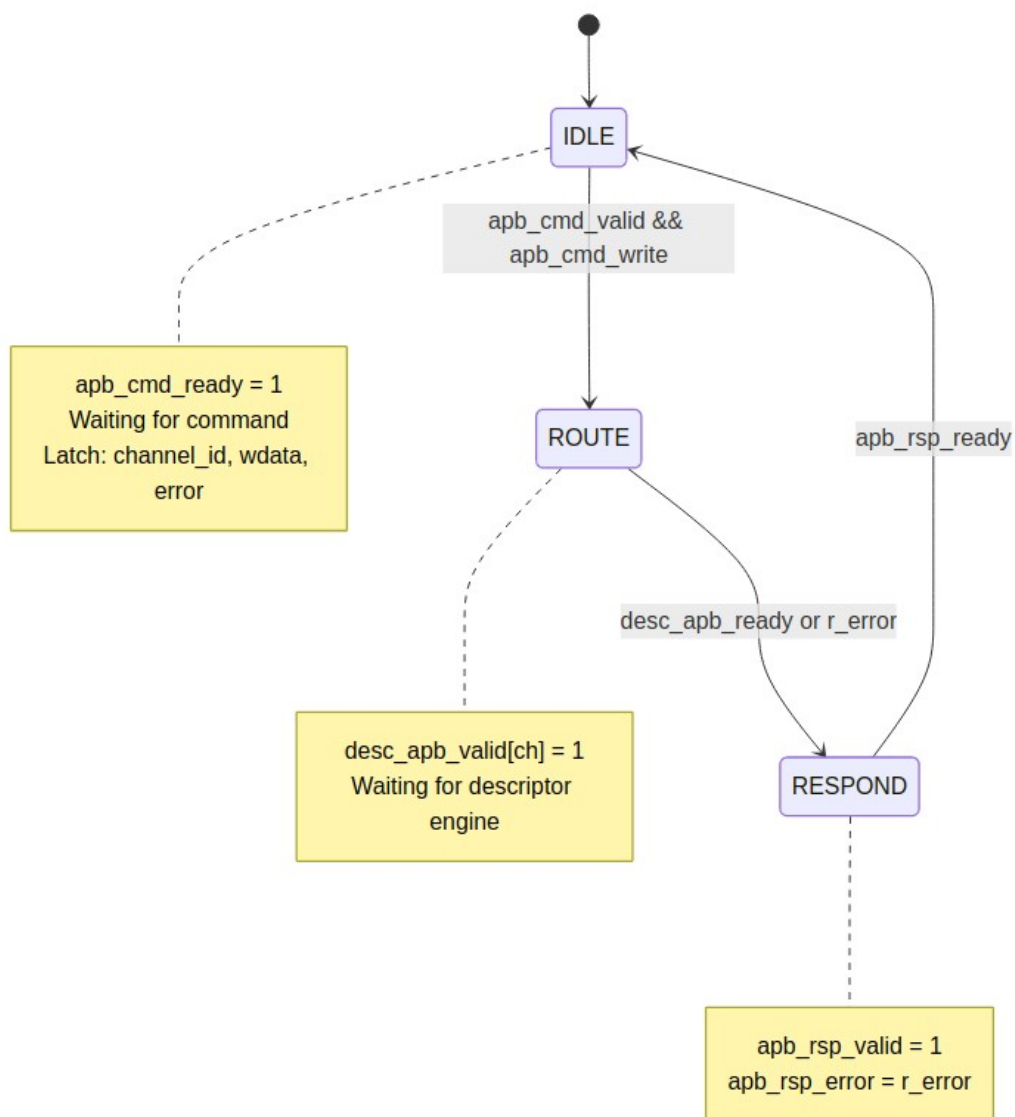


APB to Descriptor Block Diagram

Source: [02_apbtodescr_block.mmd](#)

18.2.4 FSM Diagram

18.2.5 Figure 2.13.2: APB to Descriptor FSM



APB to Descriptor FSM

Source: [02_apbtodescr_fsm.mmd](#)

18.2.6 Write Flow

1. Software writes LOW word to CHx_CTRL_LOW register
2. PeakRDL APB slave presents write on cmd/rsp interface
3. Module captures LOW word, responds with OKAY
4. Software writes HIGH word to CHx_CTRL_HIGH register

5. Module assembles 64-bit address from LOW + HIGH
6. Module asserts desc_apb_valid[channel_id]
7. Module drives desc_apb_addr[channel_id]
8. Waits for desc_apb_ready[channel_id]
9. Completes APB transaction (asserts apb_rsp_valid)

18.2.7 FSM States

```
typedef enum logic [2:0] {
    IDLE           = 3'b000, // Waiting for APB command (LOW write)
    RESPOND_LOW    = 3'b001, // Sending response after LOW write
    WAIT_HIGH      = 3'b010, // Waiting for HIGH write
    ROUTE          = 3'b011, // Routing to descriptor engine
    RESPOND_HIGH   = 3'b100  // Sending final response after HIGH
write
} state_t;
```

18.3 Parameters

Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB bus address width (for address decoding only)
DATA_WIDTH	int	32	APB data width
NUM_CHANNELS	int	8	Number of DMA channels
DESC_ADDR_WIDTH	int	32	Descriptor address output width (independent of ADDR_WIDTH)

Important: DESC_ADDR_WIDTH is separate from ADDR_WIDTH. The APB address bus is used for register decode (typically 12 bits), while descriptor addresses are wide (typically 32 or 64 bits). The two-word programming protocol assembles a full-width descriptor address and truncates to DESC_ADDR_WIDTH bits when routing to descriptor engines.

18.4 Port List

18.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

18.4.2 APB Slave CMD Interface

APB Slave CMD Interface

Signal	Direction	Width	Description
apb_cmd_valid	input	1	Command valid
apb_cmd_ready	output	1	Ready to accept command
apb_cmd_addr	input	ADDR_WIDTH	Command address
apb_cmd_wdata	input	DATA_WIDTH	Write data
apb_cmd_write	input	1	Write enable (1=write, 0=read)

18.4.3 APB Slave RSP Interface

APB Slave RSP Interface

Signal	Direction	Width	Description
apb_rsp_valid	output	1	Response valid
apb_rsp_ready	input	1	Response ready
apb_rsp_rdata	output	DATA_WIDTH	Read data (always 0)
apb_rsp_error	output	1	Error flag

18.4.4 Descriptor Engine APB Ports

Descriptor Engine APB Ports

Signal	Direction	Width	Description
desc_apb_val id[ch]	output	NUM_CHAN NELS	Per-channel valid
desc_apb_rea dy[ch]	input	NUM_CHAN NELS	Per-channel ready
desc_apb_add r[ch]	output	NUM_CHAN NELS x 64	64-bit descriptor addresses

18.4.5 Integration Control

Integration Control

Signal	Direction	Width	Description
apb_descript or_kickoff_h it	output	1	Indicates kick-off in progress

18.5 Operation

18.5.1 Address Decode

```
// Extract channel ID from address (dword-aligned: addr[5:3])
// Address bits [1:0] are ignored (word-aligned)
// Address bit [2] selects LOW (0) or HIGH (1) register
// Address bits [5:3] select channel 0-7
assign channel_id = apb_cmd_addr[5:3];
assign r_is_high_write = apb_cmd_addr[2];

// Valid range: 0x00 to 0x3F (8 channels x 8 bytes)
assign addr_in_range = ({20'h0, apb_cmd_addr[11:0]} < (NUM_CHANNELS *
8));
```

18.5.2 64-bit Address Assembly

```
// LOW write captured in IDLE state
if (r_state == IDLE && apb_cmd_valid) begin
    r_wdata_low <= apb_cmd_wdata;
end

// HIGH write captured in WAIT_HIGH state
if (r_state == WAIT_HIGH && apb_cmd_valid) begin
```

```
        r_wdata_high <= apb_cmd_wdata;
end

// Assembled 64-bit address
desc_apb_addr[ch] = {r_wdata_high, r_wdata_low};
```

18.5.3 Error Conditions

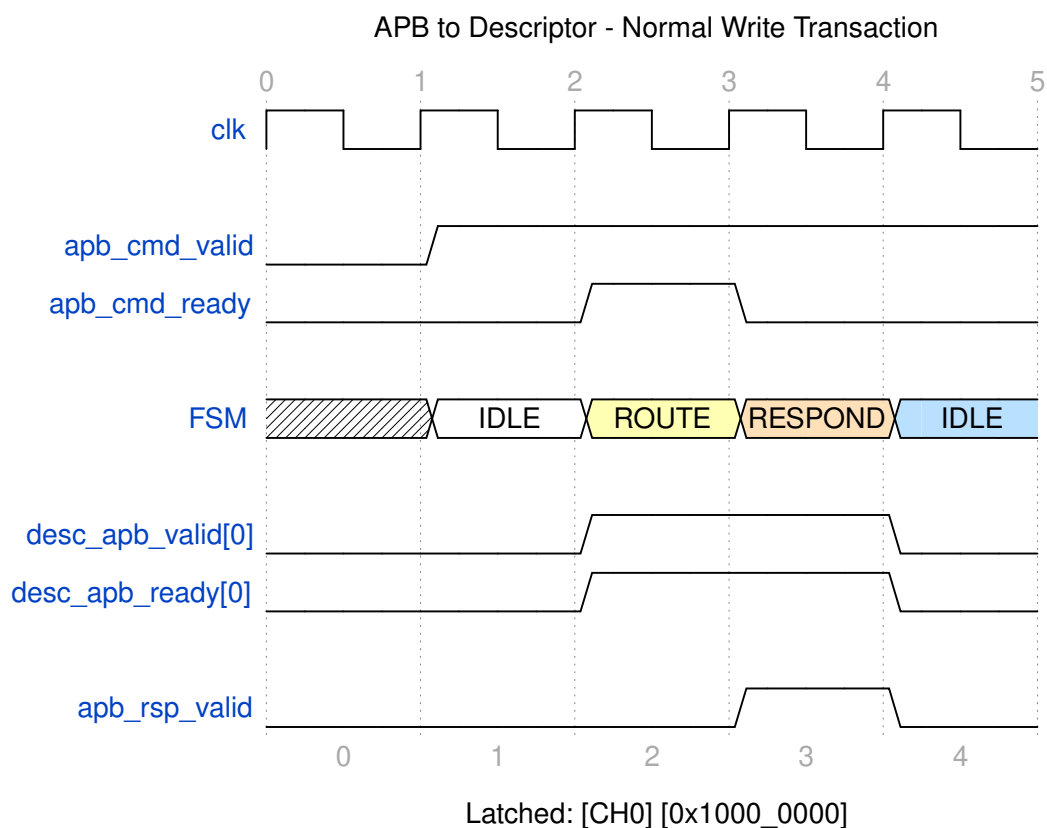
1. **Read Operation:** APB reads not supported
 2. **Address Out of Range:** Beyond channel address space
 3. **HIGH Before LOW:** HIGH write without preceding LOW write
 4. **Wrong Channel:** HIGH write to different channel than LOW
-

18.6 Timing Diagrams

18.6.1 Normal Write Sequence

The following diagram shows the normal write flow where a LOW word followed by HIGH word kicks off a channel:

18.6.1.1 Waveform 2.13.1: APB Normal Write Sequence



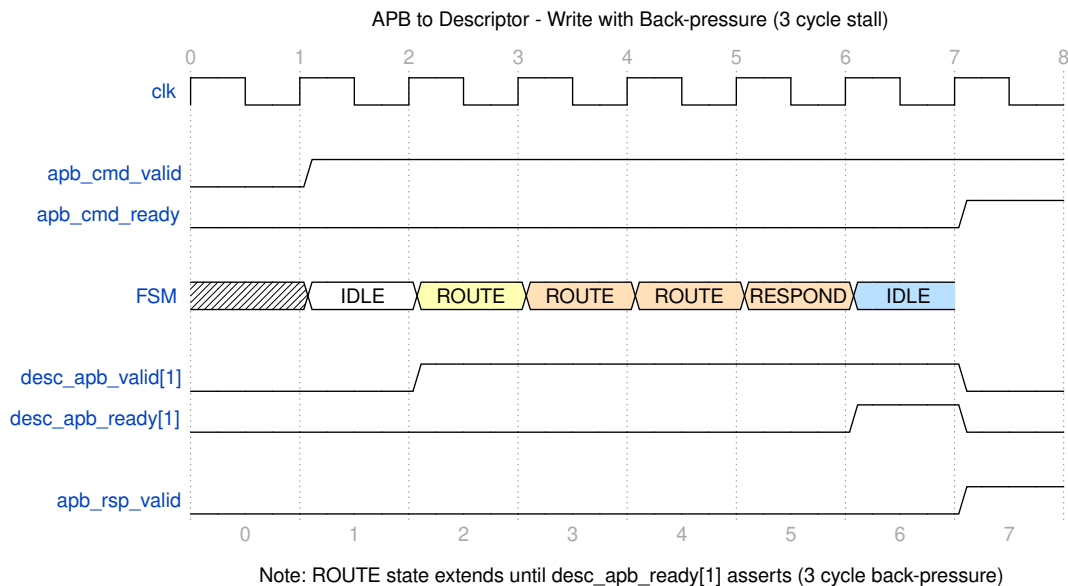
APB Normal Write

Source: [apbtodescr_normal_write.json](#)

18.6.2 Backpressure Handling

When the descriptor engine is busy, the APB response is delayed until the engine accepts the kick-off:

18.6.2.1 Waveform 2.13.2: APB Write with Backpressure

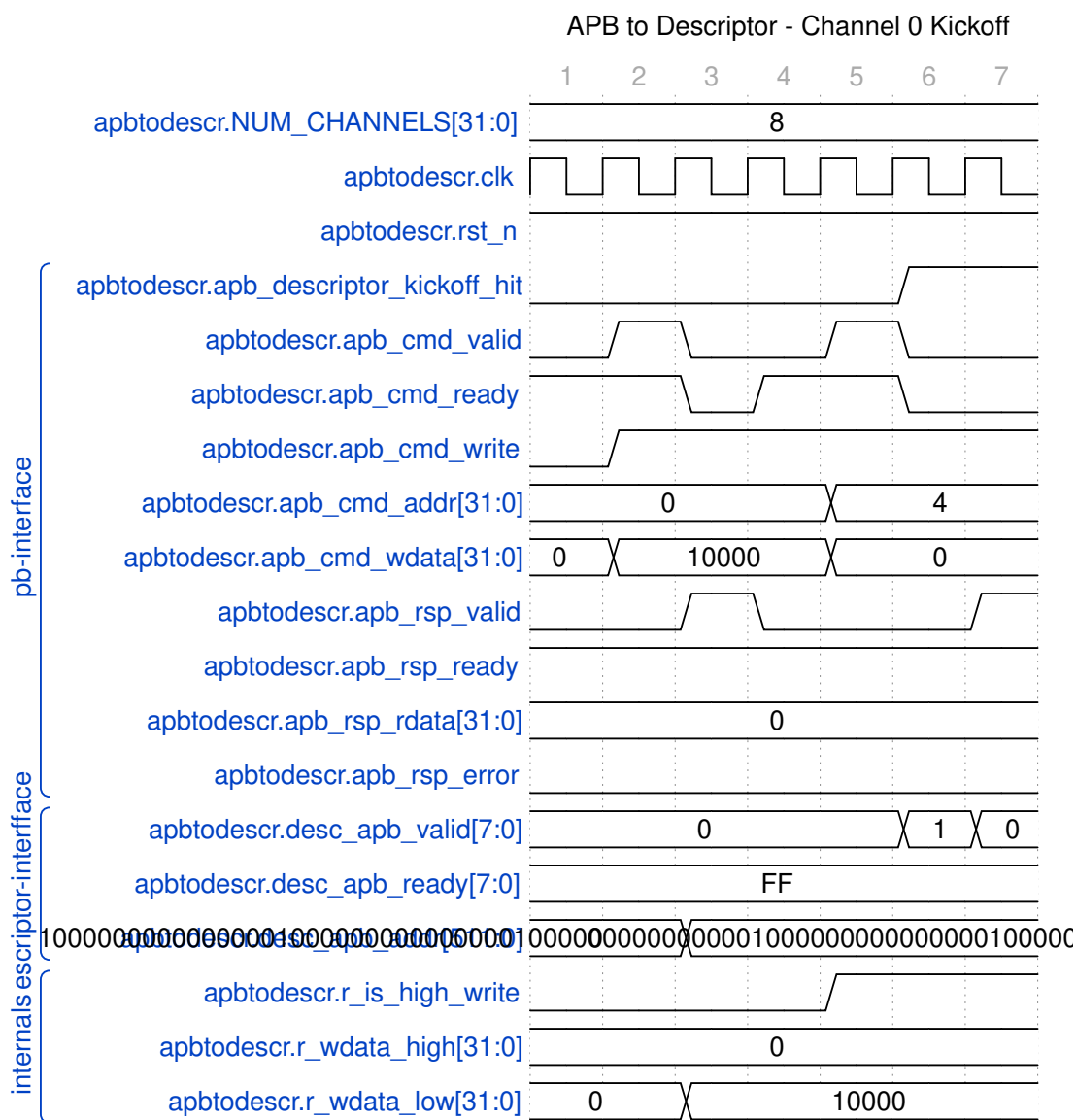


APB Backpressure Write

Source: [apbtodescr_backpressure_write.json](#)

18.6.3 Channel Kick-off Examples

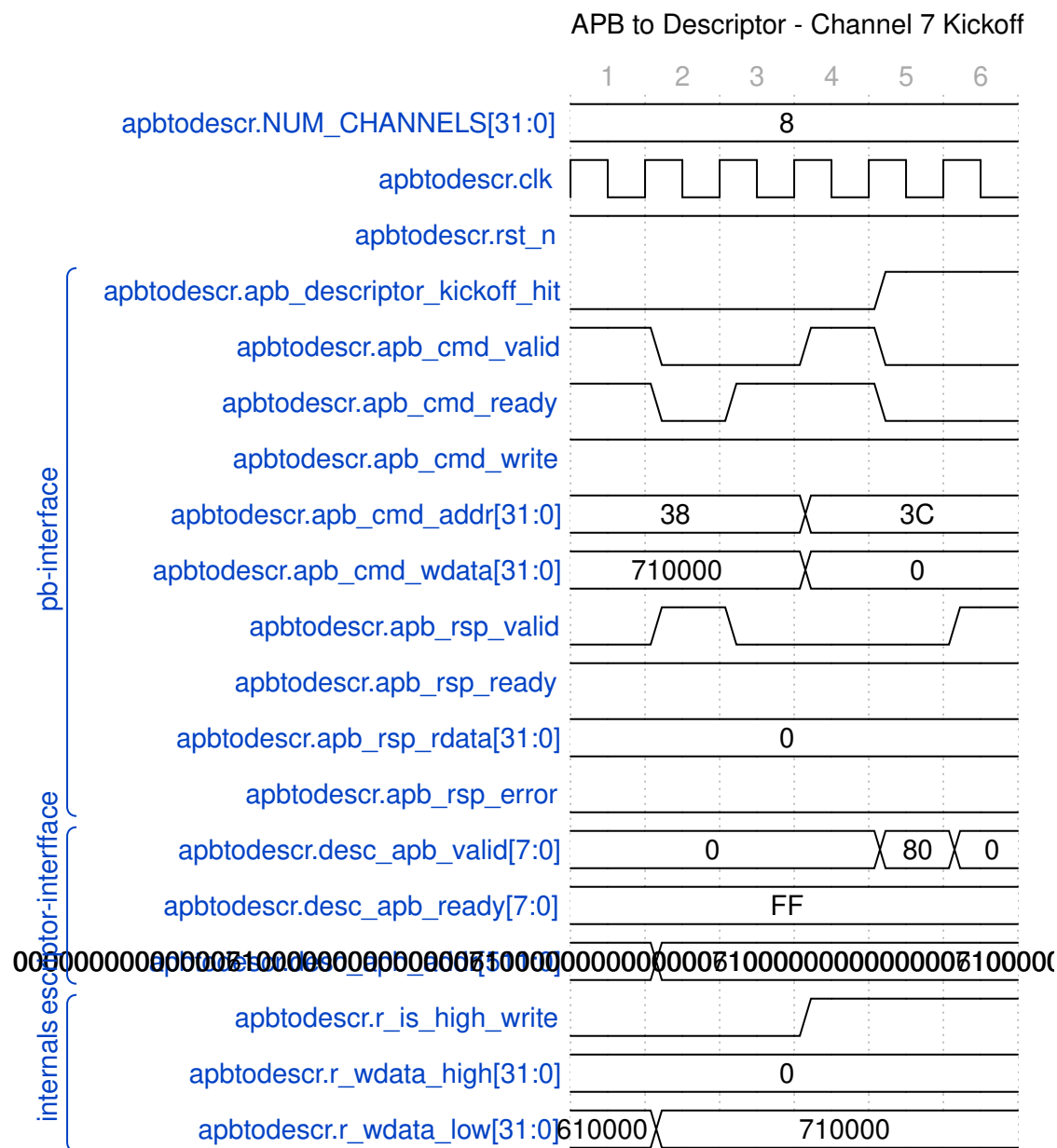
18.6.3.1 Waveform 2.13.3: Channel 0 Kick-off



Channel 0 Kick-off

Source: [apbtodescr_ch0_kickoff.json](#)

18.6.3.2 Waveform 2.13.4: Channel 7 Kick-off



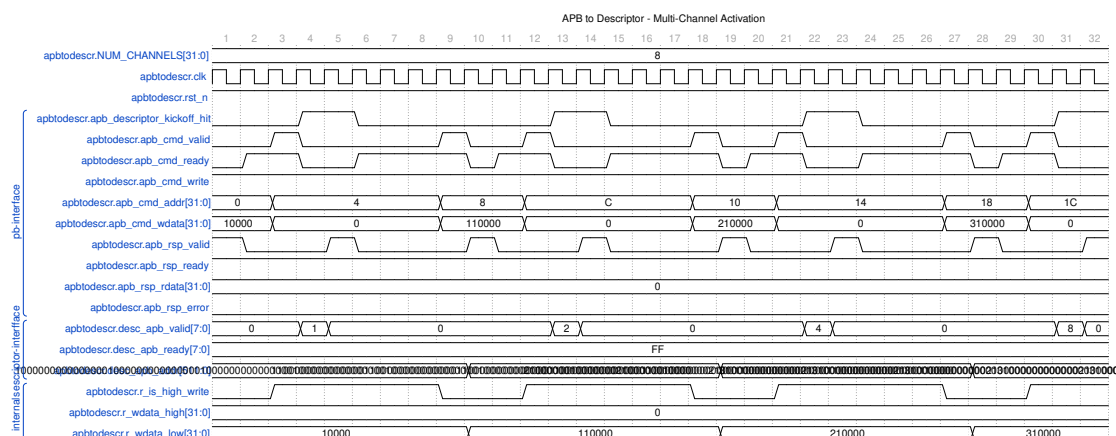
Channel 7 Kick-off

Source: [apbtodescr_ch7_kickoff.json](#)

18.6.4 Multi-Channel Operation

The following diagram shows multiple channels being kicked off in sequence:

18.6.4.1 Waveform 2.13.5: Multi-Channel Kick-off Sequence



Multi-Channel Kick-off

Source: [apbtodescr_multi_channel.json](#)

18.7 Known Issues and Fixes

18.7.1 Issue: Multi-Channel Kick-off Address Routing (RESOLVED a3beb2eb)

Symptom: In multi-channel operations, channels 1-7 received corrupted descriptor addresses (bit-shuffled) while channel 0 worked correctly.

Root Cause: Parameter mismatch between `apbtodescr` and parent modules. The `ADDR_WIDTH` parameter (12 bits, for APB decode) was incorrectly used as the descriptor address output width. Parent modules expected 32-bit descriptor addresses but received 12-bit values routed to 32-bit wires, causing SystemVerilog to re-slice the bits and place the HIGH word (all-0s) on odd channels.

Fix (Commit a3beb2eb): Separated `DESC_ADDR_WIDTH` as an independent parameter (default 32). The two-word programming protocol still assembles a full descriptor address, but now correctly truncates to `DESC_ADDR_WIDTH` bits when routing to all channels uniformly.

Status: RESOLVED (a3beb2eb - 2026-04-28)

18.8 Integration Example

```
apbtodescr #(
    .ADDR_WIDTH      (32),
    .DATA_WIDTH      (32),
    .NUM_CHANNELS    (8),
    .DESC_ADDR_WIDTH (32) // Descriptor address output width
```

```

) u_apbtodescr (
    .clk                (clk),
    .rst_n              (rst_n),

    // APB CMD/RSP from PeakRDL
    .apb_cmd_valid      (apb_kickoff_cmd_valid),
    .apb_cmd_ready      (apb_kickoff_cmd_ready),
    .apb_cmd_addr       (apb_kickoff_cmd_addr),
    .apb_cmd_wdata      (apb_kickoff_cmd_wdata),
    .apb_cmd_write      (apb_kickoff_cmd_write),

    .apb_rsp_valid      (apb_kickoff_rsp_valid),
    .apb_rsp_ready      (apb_kickoff_rsp_ready),
    .apb_rsp_rdata      (apb_kickoff_rsp_rdata),
    .apb_rsp_error      (apb_kickoff_rsp_error),

    // To descriptor engines
    .desc_apb_valid     (desc_apb_valid),
    .desc_apb_ready     (desc_apb_ready),
    .desc_apb_addr      (desc_apb_addr),

    // Integration
    .apb_descriptor_kickoff_hit (kickoff_hit)
);

```

18.9 Common Issues

18.9.1 Issue 1: APB Response Not Returning

Symptom: APB write hangs without response

Root Causes: 1. Descriptor engine not asserting ready 2. FSM stuck in ROUTE state 3. apb_rsp_ready not being asserted

Solution: Check descriptor engine acceptance and FSM state.

18.9.2 Issue 2: Wrong Channel Kicked Off

Symptom: Different channel starts than expected

Root Cause: HIGH write sent to different channel than LOW

Solution: Ensure software writes LOW then HIGH to same channel address.

18.10 Related Documentation

- **Parent:** 01_stream_core.md - Top-level integration
 - **Consumer:** 05_descriptor_engine.md - Receives kick-off
 - **Register File:** PeakRDL-generated stream_regs.sv
-

18.11 Revision History

APB to Descriptor Router Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

19 APB Configuration Block

Module: stream_config_block.sv **Location:** projects/components/dmas/stream/rtl/top/ **Category:** TOP (Integration) **Parent:** stream_top.sv **Status:** Implemented **Last Updated:** 2025-11-30

19.1 Overview

The stream_config_block module maps PeakRDL-generated register outputs to STREAM core configuration inputs. It provides a clean interface layer between the register file and the STREAM DMA engine core, handling field extraction, width conversion, and global enable gating.

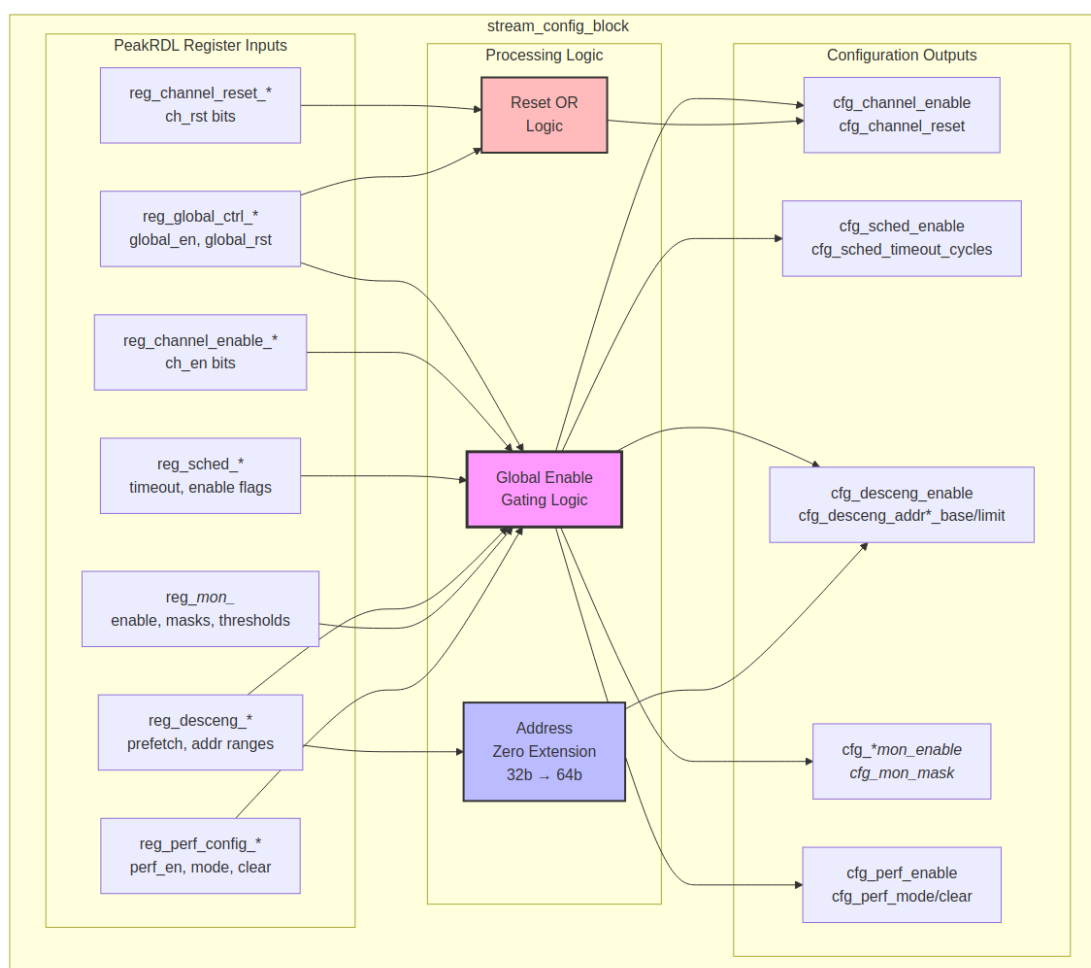
19.1.1 Key Features

- **Field Extraction:** Extracts fields from packed register values
 - **Width Conversion:** Converts register fields to configuration signal widths
 - **Global Enable Gating:** Gates enables by global enable register
 - **Address Extension:** Zero-extends 32-bit register addresses to 64-bit
 - **Clean Interface:** Isolates register naming from core logic
-

19.2 Architecture

19.2.1 Block Diagram

19.2.2 Figure 1: Stream Config Block Diagram



Stream Config Block Diagram

Source: [14_stream_config_block.mmd](#)

19.2.3 Configuration Groups

```
stream_config_block
  Global and Channel Control
    cfg_channel_enable (gated by global_en)
    cfg_channel_reset (OR'd with global_rst)
  Scheduler Configuration
    cfg_sched_enable (gated by global_en)
    cfg_sched_timeout_cycles
    cfg_sched_*_enable flags
  Descriptor Engine Configuration
    cfg_desceng_enable (gated by global_en)
    cfg_desceng_prefetch
    cfg_desceng_addr*_base/limit (zero-extended)
  Monitor Configurations (3 sets)
    Descriptor AXI Monitor (cfg_desc_mon_*)
    Read Engine Monitor (cfg_rdeng_mon_*)
    Write Engine Monitor (cfg_wreng_mon_*)
  AXI Transfer Configuration
    cfg_axi_rd_xfer_beats
    cfg_axi_wr_xfer_beats
  Performance Profiler
    cfg_perf_enable (gated by global_en)
    cfg_perf_mode
    cfg_perf_clear
```

19.3 Parameters

Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of DMA channels
ADDR_WIDTH	int	64	Address width for config outputs

19.4 Port List

19.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

19.4.2 PeakRDL Register Inputs

Global Control:

PeakRDL Register Inputs

Signal	Direction	Width	Description
reg_global_ctl_global_en	input	1	Global enable
reg_global_ctl_global_rst	input	1	Global reset
reg_channel_enable_ch_en	input	8	Per-channel enable bits
reg_channel_reset_ch_rst	input	8	Per-channel reset bits

Scheduler Configuration:

PeakRDL Register Inputs

Signal	Direction	Width	Description
reg_sched_timeout_cycles_*	input	16	Timeout threshold
reg_sched_config_*	input	1	Various enables

Descriptor Engine Configuration:

PeakRDL Register Inputs

Signal	Direction	Width	Description
reg_desceng_c onfig_*	input	varies	Enable and threshold
reg_desceng_a ddr*_*	input	32	Address range limits

Monitor Configurations (3 sets):

Each monitor (DAXMON, RDMON, WRMON) has: - reg_*_enable_* - Enable flags - reg_*_timeout_* - Timeout configuration - reg_*_latency_thresh_* - Latency threshold - reg_*_pkt_mask_* - Packet type mask - reg_*_err_cfg_* - Error configuration - reg_*_mask*_* - Event masks

AXI Transfer Configuration:

PeakRDL Register Inputs

Signal	Direction	Width	Description
reg_axi_xfer_ config_rd_xfe r_beats	input	8	Read burst size
reg_axi_xfer_ config_wr_xfe r_beats	input	8	Write burst size

Performance Profiler:

PeakRDL Register Inputs

Signal	Direction	Width	Description
reg_perf_conf ig_perf_en	input	1	Profiler enable
reg_perf_conf ig_perf_mode	input	1	Profiling mode
reg_perf_conf ig_perf_clear	input	1	Clear profiler

19.4.3 Configuration Outputs

Global and Channel:

Configuration Outputs

Signal	Direction	Width	Description
cfg_channel_enable	output	NUM_CHAN NELS	Gated channel enables
cfg_channel_reset	output	NUM_CHAN NELS	Combined channel resets

Scheduler:

Configuration Outputs

Signal	Direction	Width	Description
cfg_sched_enable	output	1	Gated scheduler enable
cfg_sched_timeout_cycles	output	16	Timeout value
cfg_sched_*_enable	output	1	Feature enables

Descriptor Engine:

Configuration Outputs

Signal	Direction	Width	Description
cfg_desceng_enable	output	1	Gated desceng enable
cfg_desceng_prefetch	output	1	Prefetch enable
cfg_desceng_fifo_thresh	output	4	FIFO threshold
cfg_desceng_addr*_base	output	ADDR_WIDT H	Zero-extended address
cfg_desceng_addr*_limit	output	ADDR_WIDT H	Zero-extended address

Monitor Outputs (3 sets):

Each monitor outputs ~16 configuration signals covering: - Enable flags - Timeout values - Thresholds - Masks

AXI Transfer:

Configuration Outputs

Signal	Direction	Width	Description
cfg_axi_rd_xfer_beats	output	8	Read burst size
cfg_axi_wr_xfer_beats	output	8	Write burst size

Performance Profiler:

Configuration Outputs

Signal	Direction	Width	Description
cfg_perf_enable	output	1	Gated profiler enable
cfg_perf_mode	output	1	Profiling mode
cfg_perf_clear	output	1	Clear signal

19.5 Operation

19.5.1 Global Enable Gating

```
// Gate all channel enables by global enable
assign cfg_channel_enable = reg_channel_enable_ch_en &
{NUM_CHANNELS{reg_global_ctrl_global_en}};

// Gate scheduler enable
assign cfg_sched_enable = reg_sched_config_sched_en &
reg_global_ctrl_global_en;

// Gate monitor enables
assign cfg_desc_mon_enable = reg_daxmon_enable_mon_en &
reg_global_ctrl_global_en;
```

19.5.2 Global Reset OR

```
// Channel resets are OR'd with global reset
assign cfg_channel_reset = reg_channel_reset_ch_rst |
{NUM_CHANNELS{reg_global_ctrl_global_rst}};
```

19.5.3 Address Zero Extension

```
// Zero-extend 32-bit register addresses to ADDR_WIDTH (typically 64-bit)
```

```

assign cfg_desceng_addr0_base = {{(ADDR_WIDTH-32){1'b0}},
reg_desceng_addr0_base_addr0_base};
assign cfg_desceng_addr0_limit = {{(ADDR_WIDTH-32){1'b0}},
reg_desceng_addr0_limit_addr0_limit};

```

19.6 Integration Example

```

stream_config_block #(
    .NUM_CHANNELS    (8),
    .ADDR_WIDTH      (64)
) u_config_block (
    .clk              (clk),
    .rst_n            (rst_n),

    // From PeakRDL register file
    .reg_global_ctrl_global_en    (regs_global_ctrl_global_en),
    .reg_global_ctrl_global_rst  (regs_global_ctrl_global_rst),
    .reg_channel_enable_ch_en    (regs_channel_enable_ch_en),
    // ... more register inputs

    // To stream_core
    .cfg_channel_enable          (cfg_channel_enable),
    .cfg_channel_reset           (cfg_channel_reset),
    .cfg_sched_enable            (cfg_sched_enable),
    // ... more config outputs
);

```

19.7 Related Documentation

- **Parent:** 01_stream_core.md - Receives configuration signals
- **Register File:** PeakRDL-generated stream_regs.sv
- **Scheduler:** 04_scheduler.md - Uses sched_* config
- **Descriptor Engine:** 05_descriptor_engine.md - Uses desceng_* config

- **Monitors: Monitor documentation for each AXI monitor**
-

19.8 Revision History

APB Configuration Block Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

20 Performance Profiler

Module: perf_profiler.sv **Location:** projects/components/dmas/stream/rtl/fub/
Category: FUB (Functional Unit Block) **Parent:** stream_top.sv **Status:** Implemented **Last Updated:** 2026-05-11

20.1 Overview

The perf_profiler module captures timing information for channel activity to enable performance analysis and bottleneck identification. It monitors channel idle signals and records either timestamps or elapsed times to a FIFO for software retrieval.

20.1.1 Key Features

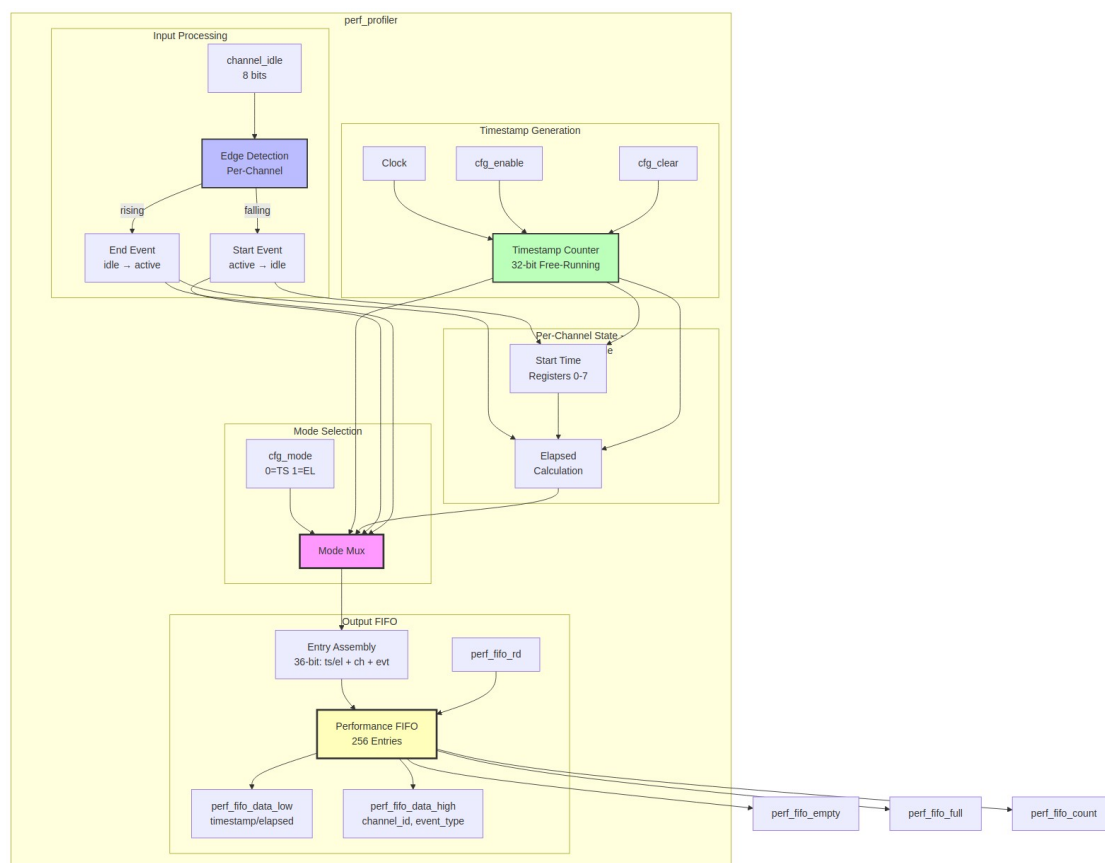
- **Two Profiling Modes:**
 - Mode 0 (TIMESTAMP): Captures timestamps on idle signal edges
 - Mode 1 (ELAPSED): Captures elapsed time directly

- **Per-Channel Profiling:** Tracks all 8 channels independently
- **256-Entry FIFO:** Buffers performance data for software polling
- **Channel ID Tagging:** Each entry includes source channel
- **Configurable via APB:** Enable, mode, and clear controls

20.2 Architecture

20.2.1 Block Diagram

20.2.2 Figure 2.15.1: Performance Profiler Block Diagram



Performance Profiler Block Diagram

Source: [15_perf_profiler_block.mmd](#)

20.2.3 Profiling Modes

Mode 0 - TIMESTAMP:

Records timestamp when:

- Channel transitions idle to active (start)
- Channel transitions active to idle (end)

Software calculates: $\text{elapsed} = \text{timestamp_end} - \text{timestamp_start}$

Advantages:

- Simpler hardware
- More flexible analysis
- Can detect partial transfers

Mode 1 - ELAPSED:

Records elapsed time when:

- Channel returns to idle

$\text{elapsed_time} = \text{current_timestamp} - \text{start_timestamp}$

Advantages:

- Hardware calculates duration
- Simpler software processing
- Direct performance metric

20.2.4 FIFO Entry Format (36-bit)

FIFO Entry Format

Bits	Field	Description
[31:0]	timestamp/elapsed	Timestamp or elapsed time
[34:32]	channel_id	Source channel (0-7)
[35]	event_type	0=start, 1=end (timestamp mode)

20.3 Parameters

Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of channels to monitor
CHANNEL_WIDTH	int	$\lceil \log_2(\text{NUM_C}) \rceil$	Channel ID

Parameter	Type	Default	Description
		HANNELS)	width
TIMESTAMP_WIDTH	int	32	Timestamp counter width
FIFO_DEPTH	int	256	Performance FIFO depth
FIFO_ADDR_WIDTH	int	$\$clog2(\text{FIFO_DEPTH})$	FIFO address width

20.4 Port List

20.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
clk	input	1	System clock
rst_n	input	1	Active-low asynchronous reset

20.4.2 Channel Monitoring

Channel Monitoring

Signal	Direction	Width	Description
channel_idle	input	NUM_CHANNELS	Per-channel idle signals

20.4.3 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_enable	input	1	Enable profiling
cfg_mode	input	1	0=timestamp, 1=elapsed
cfg_clear	input	1	Clear FIFO and

Signal	Direction	Width	Description
			counters

20.4.4 FIFO Read Interface

FIFO Read Interface

Signal	Direction	Width	Description
perf_fifo_rd	input	1	Read strobe (pops FIFO)
perf_fifo_data_low	output	32	Timestamp/elapsed [31:0]
perf_fifo_data_high	output	32	{28'b0, event_type, channel_id}
perf_fifo_empty	output	1	FIFO empty flag
perf_fifo_full	output	1	FIFO full flag
perf_fifo_counter	output	16	Number of entries

20.5 Operation

20.5.1 Edge Detection

```
// Detect idle signal transitions
assign w_idle_rising = channel_idle & ~r_idle_prev; // Active → Idle
(end)
assign w_idle_falling = ~channel_idle & r_idle_prev; // Idle → Active
(start)
```

20.5.2 Timestamp Counter

```
// Free-running 32-bit counter
// Increments every cycle when profiling enabled
// Wraps at 2^32 - 1 (software handles rollover)
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_timestamp_counter <= '0;
    end else if (cfg_clear) begin
        r_timestamp_counter <= '0;
    end else if (cfg_enable) begin
        r_timestamp_counter <= r_timestamp_counter + 1'b1;
    end
end
```

20.5.3 Start Time Capture (Elapsed Mode)

```
// Per-channel start time tracking
generate
    for (ch = 0; ch < NUM_CHANNELS; ch++) begin
        always_ff @(posedge clk) begin
            if (cfg_enable && cfg_mode == MODE_ELAPSED) begin
                if (w_idle_falling[ch]) begin
                    r_start_time[ch] <= r_timestamp_counter;
                    r_channel_active[ch] <= 1'b1;
                end
                else if (w_idle_rising[ch]) begin
                    r_channel_active[ch] <= 1'b0;
                end
            end
        end
    end
endgenerate
```

20.5.4 Two-Register Read Sequence

Software reads FIFO via APB:

1. Check perf_fifo_empty == 0
 2. Read PERF_FIFO_DATA_LOW:
 - Triggers perf_fifo_rd strobe
 - Pops FIFO, latches 36-bit entry
 - Returns timestamp/elapsed [31:0]
 3. Read PERF_FIFO_DATA_HIGH:
 - Returns {28'b0, event_type, channel_id}
 - No FIFO pop
 4. Parse data and repeat
-

20.6 Integration Example

```
perf_profiler #(
    .NUM_CHANNELS          (8),
    .TIMESTAMP_WIDTH      (32),
    .FIFO_DEPTH            (256)
) u_perf_profiler (
    .clk                   (clk),
    .rst_n                 (rst_n),

    // From schedulers
    .channel_idle          (scheduler_idle),

    // Configuration
    .cfg_enable            (cfg_perf_enable),
```

```

        .cfg_mode          (cfg_perf_mode),
        .cfg_clear         (cfg_perf_clear),

        // FIFO read interface (to APB)
        .perf_fifo_rd      (perf_fifo_rd),
        .perf_fifo_data_low(perf_fifo_data_low),
        .perf_fifo_data_high(perf_fifo_data_high),
        .perf_fifo_empty   (perf_fifo_empty),
        .perf_fifo_full    (perf_fifo_full),
        .perf_fifo_count   (perf_fifo_count)
    );

```

20.7 Software Usage

20.7.1 Example: Measure Channel 0 Transfer Time

```

// Enable timestamp mode
write_reg(PERF_CONFIG, PERF_EN | MODE_TIMESTAMP);

// Start transfer on channel 0
write_reg(CH0_CTRL_LOW, desc_addr_low);
write_reg(CH0_CTRL_HIGH, desc_addr_high);

// Wait for completion
while (!read_reg(CH0_STATUS) & COMPLETE);

// Read performance data
while (!(read_reg(PERF_STATUS) & FIFO_EMPTY)) {
    uint32_t low  = read_reg(PERF_FIFO_DATA_LOW); // Pops FIFO
    uint32_t high = read_reg(PERF_FIFO_DATA_HIGH); // Reads latched

    uint32_t timestamp = low;
    uint8_t  channel   = high & 0x7;
    uint8_t  event      = (high >> 3) & 0x1;

    if (channel == 0) {
        if (event == 0) // Start
            start_time = timestamp;
        else           // End
            elapsed = timestamp - start_time;
    }
}

```

20.8 Common Issues

20.8.1 Issue 1: FIFO Overflow

Symptom: Performance data lost

Root Cause: Software not polling fast enough

Solution: - Poll more frequently - Use larger FIFO depth - Consider interrupt on half-full

20.8.2 Issue 2: Timestamp Rollover

Symptom: Negative elapsed times calculated

Root Cause: 32-bit counter wrapped during measurement

Solution: Handle rollover in software:

```
if (end_time < start_time)
    elapsed = (0xFFFFFFFF - start_time) + end_time + 1;
```

20.9 Related Documentation

- **Parent:** 01_stream_core.md - Top-level integration
 - **Data Source:** 04_scheduler.md - Provides idle signals
 - **Configuration:** 14_apb_config.md - Config register mapping
-

20.10 Revision History

Performance Profiler Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

21 MonBus AXI-Lite Group

Module: monbus_axil_group.sv **Location:** projects/components/dmas/stream/rtl/macro/ **Category:** MACRO (Integration) **Parent:** stream_top.sv **Status:** Implemented **Last Updated:** 2025-11-30

21.1 Overview

The monbus_axil_group module is the canonical shared monitor bus processing block (RTL source: rtl/amba/shared/monbus_axil_group.sv) instantiated by STREAM for packet filtering and routing. It receives 128-bit monitor bus packets paired with 64-bit timestamps (192-bit atomic group through AXI-Lite skid), filters them per protocol/packet type, and routes to either error/interrupt FIFO or external memory.

21.1.1 Key Features

- **128-bit Monitor Packets + 64-bit Timestamps:** Carried atomically through 192-bit AXI-Lite record (3 beats: [127:0] pkt, [63:0] ts, [63:0] spare)
- **Per-Protocol Filtering:** Configurable packet type masking per AXI/AXIS/CORE protocol
- **Dual Output Paths:**
 - Error/Interrupt FIFO (64 entries) - generates interrupt when not empty
 - Master Write FIFO (32 entries) - writes packets to configurable memory range
- **Fixed AXI-Lite Record Format:** 3 beats per record (no configurable cfg_ts_append_* settings)
- **Shared Building Block:** Source of truth at rtl/amba/shared/monbus_axil_group.sv; STREAM instantiates this with zero modifications (drift area 1, commit 13706d10)

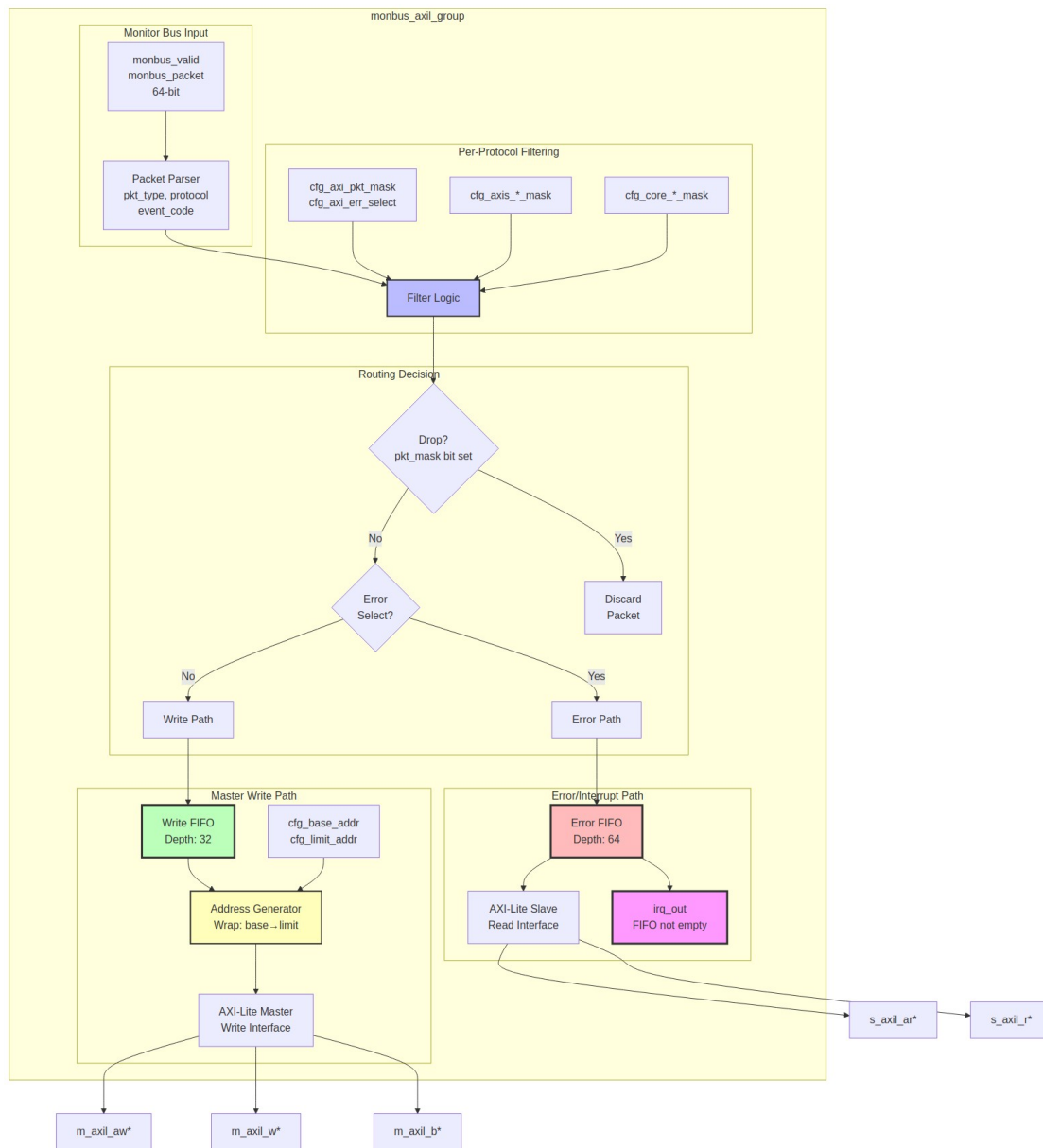
21.1.2 Simplified from RAPIDS

RAPIDS has source + sink data paths requiring TWO monitor buses. STREAM has memory-to-memory only, so this module has ONE monitor bus input (no arbitration needed).

21.2 Architecture

21.2.1 Block Diagram

21.2.2 Figure 2.16.1: MonBus AXI-Lite Group Block Diagram



MonBus AXI-Lite Group Block Diagram

Source: [16_monbus_axil_group_block.mmd](#)

21.2.3 Filter Decision Tree

For each incoming packet:

1. Extract: `pkt_type`, `pkt_protocol`, `pkt_event_code`
 2. Check protocol-specific masks:
 - `pkt_mask`: Drop if `bit[pkt_type] = 1`
 - `err_select`: Route to error FIFO if `bit[pkt_type] = 1`
 - `event_mask`: Drop if `bit[event_code] = 1`
 3. Route:
 - Dropped packets: Consumed silently
 - Error-selected: To error/interrupt FIFO
 - Remaining: To master write FIFO
-

21.3 Parameters

Parameters

Parameter	Type	Default	Description
FIFO_DEPTH_ERROR	int	64	Error/interrupt FIFO depth
FIFO_DEPTH_WRITE	int	32	Master write FIFO depth
ADDR_WIDTH	int	32	AXI address width
DATA_WIDTH	int	32	AXI data width
NUM_PROTOCOLS	int	3	AXI, AXIS, CORE

21.4 Port List

21.4.1 Clock and Reset

Clock and Reset

Signal	Direction	Width	Description
<code>axi_aclk</code>	input	1	AXI clock
<code>axi_aresetn</code>	input	1	AXI active-low reset

21.4.2 Monitor Bus Input

Monitor Bus Input

Signal	Direction	Width	Description
monbus_valid	input	1	Packet valid
monbus_ready	output	1	Ready to accept
monbus_packet	input	128	128-bit monitor packet (protocol-independent structure)
monbus_ts	input	64	64-bit side-band timestamp (cycle count at packet capture)

21.4.3 AXI-Lite Slave Read Interface

AXI-Lite Slave Read Interface

Signal	Direction	Width	Description
s_axil_arvalid	input	1	Read address valid
s_axil_arready	output	1	Read address ready
s_axil_araddr	input	ADDR_WIDTH	Read address
s_axil_arprot	input	3	Protection bits
s_axil_rvalid	output	1	Read data valid
s_axil_rready	input	1	Read data ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response

21.4.4 AXI-Lite Master Write Interface

AXI-Lite Master Write Interface

Signal	Direction	Width	Description
m_axil_awvalid	output	1	Write address valid

Signal	Direction	Width	Description
m_axil_awready	input	1	Write address ready
m_axil_awaddr	output	ADDR_WIDTH	Write address
m_axil_awprot	output	3	Protection bits
m_axil_wvalid	output	1	Write data valid
m_axil_wready	input	1	Write data ready
m_axil_wdata	output	DATA_WIDTH	Write data
m_axil_wstrb	output	DATA_WIDTH/8	Write strobes
m_axil_bvalid	input	1	Write response valid
m_axil_bready	output	1	Write response ready
m_axil_bresp	input	2	Write response

21.4.5 Interrupt Output

Interrupt Output

Signal	Direction	Width	Description
irq_out	output	1	Interrupt (error FIFO not empty)

21.4.6 Configuration Interface

Configuration Interface

Signal	Direction	Width	Description
cfg_base_addr	input	ADDR_WIDTH	Base for master writes
cfg_limit_addr	input	ADDR_WIDTH	Limit for master writes

21.4.7 Protocol Configuration (per protocol)

AXI Protocol:

Protocol Configuration

Signal	Direction	Width	Description
cfg_axi_pkt_mask	input	16	Drop mask for packet types
cfg_axi_err_select	input	16	Error FIFO select
cfg_axi_*_mask	input	16	Per-event-type masks

AXIS Protocol:

Protocol Configuration

Signal	Direction	Width	Description
cfg_axis_pkt_mask	input	16	Drop mask
cfg_axis_err_select	input	16	Error FIFO select
cfg_axis_*_mask	input	16	Per-event-type masks

CORE Protocol:

Protocol Configuration

Signal	Direction	Width	Description
cfg_core_pkt_mask	input	16	Drop mask
cfg_core_err_select	input	16	Error FIFO select
cfg_core_*_mask	input	16	Per-event-type masks

21.4.8 Debug/Status

Debug/Status

Signal	Direction	Width	Description
err_fifo_full	output	1	Error FIFO full
write_fifo_full	output	1	Write FIFO full
err_fifo_count	output	8	Error FIFO count
write_fifo_count	output	8	Write FIFO count

21.5 Operation

21.5.1 Packet Type Filtering

```
// Protocol-specific filtering (example: AXI)
case (pkt_protocol)
  3'b000: begin // AXI
    // Check if packet type is dropped
    pkt_drop = cfg_axi_pkt_mask[pkt_type];

    // Check if packet goes to error FIFO
    pkt_to_err_fifo = cfg_axi_err_select[pkt_type] && !pkt_drop;

    // Check individual event masking
    case (pkt_type)
      PktTypeError:    pkt_event_masked =
cfg_axi_error_mask[pkt_event_code];
      PktTypeTimeout:  pkt_event_masked =
cfg_axi_timeout_mask[pkt_event_code];
      // ... more packet types
    endcase
  end
endcase
```

21.5.2 Master Write Address Generation

```
// Address wraps within configured range
always_comb begin
  next_write_addr = current_write_addr + (DATA_WIDTH == 64 ? 8 : 4);
  if (next_write_addr > cfg_limit_addr) begin
    next_write_addr = cfg_base_addr;
  end
end
```



```
end
end
```

21.5.3 Master Write FSM

```
typedef enum logic [2:0] {
    WRITE_IDLE      = 3'b000,
    WRITE_ADDR      = 3'b001,
    WRITE_DATA_LOW  = 3'b010,
    WRITE_DATA_HIGH = 3'b011, // For 32-bit data width only
    WRITE_RESP      = 3'b100
} write_state_t;
```

21.6 Integration Example

```
monbus_axil_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (32),
    .ADDR_WIDTH          (32),
    .DATA_WIDTH          (32),
    .NUM_PROTOCOLS       (3)
) u_monbus_axil_group (
    .axi_aclk             (clk),
    .axi_aresetn          (rst_n),

    // Monitor bus input
    .monbus_valid         (stream_mon_valid),
    .monbus_ready         (stream_mon_ready),
    .monbus_packet        (stream_mon_packet),

    // AXI-Lite slave read (for error FIFO access)
    .s_axil_arvalid       (axil_slave_arvalid),
    .s_axil_arready       (axil_slave_arready),
    // ... more slave signals

    // AXI-Lite master write (for logging)
    .m_axil_awvalid       (axil_master_awvalid),
    .m_axil_awready       (axil_master_awready),
    // ... more master signals

    // Interrupt
    .irq_out              (monbus_interrupt),

    // Configuration
    .cfg_base_addr        (cfg_monbus_base_addr),
    .cfg_limit_addr       (cfg_monbus_limit_addr),
```

```

        .cfg_axi_pkt_mask    (cfg_axi_pkt_mask),
        // ... more protocol configuration
    );

```

21.7 Common Issues

21.7.1 Issue 1: Missing Monitor Packets

Symptom: Expected events not appearing in FIFO

Root Causes: 1. Packet type masked in `cfg_pkt_mask` 2. Event masked in `cfg*_mask` 3. FIFO full (packets dropped)

Solution: Check mask configuration and FIFO status.

21.7.2 Issue 2: Master Writes Stall

Symptom: Write FIFO fills up, backpressure to monitor bus

Root Causes: 1. AXI-Lite master target not responding 2. Address range exhausted (wrapping too fast)

Solution: Increase address range or reduce logging rate via masks.

21.8 Related Documentation

- **Parent:** `01_stream_core.md` - Provides monitor bus packets
 - **Packet Format:** Monitor Bus Protocol documentation
 - **Configuration:** `14_apb_config.md` - Register mapping
-

21.9 Revision History

MonBus AXI-Lite Group Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure

Version	Date	Author	Description
			numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

22 Chapter 3: External Interfaces

This chapter documents the external interfaces of the STREAM DMA engine (stream_top_ch8.sv).

22.1 Interface Specifications

STREAM has seven primary external interfaces:

22.1.1 01_axi4_interface_spec.md

- **AXI4 Master Interfaces (3 total)**
 - Descriptor fetch (256-bit, read-only)
 - Data read (parameterizable, default 512-bit)
 - Data write (parameterizable, default 512-bit)
- Protocol specifications and assumptions
- Transfer modes and alignment requirements
- Timing and performance characteristics

22.1.2 02_axil4_interface_spec.md

- **AXI4-Lite Interfaces (2 total, in monbus_axil_group)**
 - Error FIFO slave read (s_axil_err_*): 32-bit, read-only
 - Monitor data master write (m_axil_mon_*): 32-bit, write-only
- Used for monitor bus packet access and logging
- Part of monbus_axil_group integration (USE_AXI_MONITORS=1)

22.1.3 03_apb_interface_spec.md

- **APB4 Slave Interface**
- Full APB4 protocol with PSEL/PENABLE

- Optional CDC for asynchronous clock domains (CDC_ENABLE parameter)
- Address map: 0x000-0x03F kick-off, 0x100-0x3FF registers

22.1.4 05_monbus_interface_spec.md

- **MonBus Internal Protocol**
- Unified 64-bit monitoring bus (stream_core → monbus_axil_group)
- Event packet format and encoding
- Agent ID assignments for STREAM components
- Converted to AXI-Lite at top level

22.2 STREAM Interface Summary (stream_top_ch8.sv)

Interface	Type	Direction	Width	Purpose
APB4	Slave	Input	32-bit data, 12-bit addr	Configuration and kick-off
AXI4 Descriptor	Master Read	Output	256-bit	Descriptor fetch
AXI4 Data Read	Master Read	Output	512-bit (param)	Source data read
AXI4 Data Write	Master Write	Output	512-bit (param)	Destination data write
AXIL Error FIFO	Slave Read	Input	32-bit	Monitor error/interrupt FIFO
AXIL Monitor Write	Master Write	Output	32-bit	Monitor data to memory
IRQ	Output	Output	1-bit	Interrupt (error FIFO not empty)

Note: AXIL interfaces are active when USE_AXI_MONITORS=1. MonBus is internal (stream_core → monbus_axil_group).

Last Updated: 2025-12-01

23 AXI4 Interface Specification for STREAM

23.1 Overview

This document defines the AXI4 interface specification for the STREAM DMA engine. STREAM uses three AXI4 master interfaces for memory access:

- 1. **Descriptor Fetch Master** - 256-bit read-only interface for fetching descriptors
- 2. **Data Read Master** - Parameterizable width (default 512-bit) for reading source data
- 3. **Data Write Master** - Parameterizable width (default 512-bit) for writing destination data

Note: This document focuses on STREAM-specific implementation details. For generic AXI4 protocol information, refer to the AMBA AXI4 Protocol Specification v4.0.

23.2 STREAM AXI4 Interface Summary

23.2.1 Number of Interfaces

STREAM implements **3 AXI4 Master Interfaces**:

Number of Interfaces

Interface	Type	Width	Channels	Purpose
Descriptor Fetch	Master Read	256-bit (fixed)	AR, R	Fetch descriptors from memory
Data Read	Master Read	512-bit (param)	AR, R	Read source data to SRAM

Interface	Type	Width	Channels	Purpose
Data Write	Master Write	512-bit (param)	AW, W, B	Write SRAM data to destination

23.2.2 Interface Parameters

Interface Parameters

Parameter	Description	Valid Values	Default
DATA_WIDTH	AXI data bus width in bits	32, 64, 128, 256, 512, 1024	32
ADDR_WIDTH	AXI address bus width in bits	32, 64	37
ID_WIDTH	AXI ID tag width in bits	1-16	8
USER_WIDTH	AXI user signal width in bits (optional)	0-16	1

23.2.3 Interface Types and Transfer Modes

Interface Types and Transfer Modes

Interface Group	Channels	Transfer Mode	Address Alignment	Monitor	DCG	Notes
AXI4 Master Read-Split	AR, R	Flexible	4-byte	Yes	Yes	Data interface with chunk enables
AXI4 Master Write-Split	AW, W, B	Flexible	4-byte	Yes	Yes	Data interface

Interface Group	Channels	Transfer Mode	Address Alignment	Monitor	DCG	Notes
						rfaces with chunk enables
AXI4 Master Read	AR, R	Simplified	Bus-width	No	Yes	Control interfaces, fully aligned
AXI4 Master Write	AW, W, B	Simplified	Bus-width	Yes	Yes	Control interfaces, fully aligned

23.2.4 Interface Group Parameter Settings

Interface Group Parameter Settings

Interface Group	Data Width	Address Width	ID Width	User Width	Transfer Mode
AXI4 Master Read-Split	512 bits	37 bits	8 bits	1 bit	Flexible
AXI4 Master Write-Split	512 bits	37 bits	8 bits	1 bit	Flexible

Interface Group	Data Width	Address Width	ID Width	User Width	Transfer Mode
AXI4 Master Read	32 bits	37 bits	8 bits	1 bit	Simplified
AXI4 Master Write	32 bits	37 bits	8 bits	1 bit	Simplified

23.2.5 Interface Configuration Summary

Interface Configuration Summary

Interface Type	Interface Group	Transfer Mode	Alignment	Notes
Descriptor Sink	AXI4 Master Read	Simplified	32-bit aligned	Control interface
Descriptor Source	AXI4 Master Read	Simplified	32-bit aligned	Control interface
Data Source	AXI4 Master Read-Split	Flexible	4-byte aligned	High-bandwidth data
Data Sink	AXI4 Master Write-Split	Flexible	4-byte aligned	High-bandwidth data
Program Sink	AXI4 Master Write	Simplified	32-bit aligned	Control interface
Program Source	AXI4 Master Write	Simplified	32-bit aligned	Control interface

Interface Type	Interface Group	Transfer Mode	Alignment	Notes
Flag Sink	AXI4 Master Read	Simplified	32-bit aligned	Control interface
Flag Source	AXI4 Master Read	Simplified	32-bit aligned	Control interface

23.3 Transfer Mode Specifications

This specification defines two distinct transfer modes to optimize different interface types:

23.3.1 Mode 1: Simplified Transfer Mode (Control Interfaces)

Used for control interfaces (descriptors, programs, flags) that prioritize simplicity and predictable timing.

23.3.1.1 *Simplified Mode Assumptions*

Simplified Mode Assumptions

Aspect	Requirement
Address Alignment	All addresses aligned to full data bus width
Transfer Size	All transfers use maximum size equal to bus width
Burst Type	Incrementing bursts only (AxBURST = 2'b01)
Transfer Complexity	Maximum simplicity for predictable operation

23.3.2 Mode 2: Flexible Transfer Mode (Data Interfaces)

Used for high-bandwidth data interfaces that need to handle arbitrary address alignment while maintaining efficiency.

23.3.2.1 *Flexible Mode Assumptions*

Flexible Mode Assumptions

Aspect	Requirement
Address Alignment	4-byte aligned addresses (minimum alignment)
Transfer Sizes	Multiple sizes supported: 4, 8, 16, 32, 64 bytes

Aspect	Requirement
Burst Type	Incrementing bursts only (AxBURST = 2'b01)
Alignment Strategy	Progressive alignment to optimize bus utilization

23.4 Mode 1: Simplified Transfer Mode Specification

23.4.1 Assumption 1: Address Alignment to Data Bus Width

Assumption 1: Address Alignment to Data Bus Width

Aspect	Requirement
Alignment Rule	All AXI transactions aligned to data bus width
32-bit bus (4 bytes)	Address[1:0] must be 2'b00
64-bit bus (8 bytes)	Address[2:0] must be 3'b000
128-bit bus (16 bytes)	Address[3:0] must be 4'b0000
256-bit bus (32 bytes)	Address[4:0] must be 5'b00000
512-bit bus (64 bytes)	Address[5:0] must be 6'b000000
1024-bit bus (128 bytes)	Address[6:0] must be 7'b0000000
Rationale	Maximizes bus efficiency and eliminates unaligned access complexity

23.4.2 Assumption 2: Fixed Transfer Size

Assumption 2: Fixed Transfer Size

Aspect	Requirement
Transfer Size Rule	All transfers use maximum size equal to bus width
32-bit bus	AxSIZE = 3'b010 (4 bytes)
64-bit bus	AxSIZE = 3'b011 (8 bytes)
128-bit bus	AxSIZE = 3'b100 (16 bytes)
256-bit bus	AxSIZE = 3'b101 (32 bytes)
512-bit bus	AxSIZE = 3'b110 (64 bytes)
1024-bit bus	AxSIZE = 3'b111 (128 bytes)
Rationale	Maximizes bus utilization and simplifies address

Aspect	Requirement
	alignment

23.5 Mode 2: Flexible Transfer Mode Specification

23.5.1 Assumption 1: 4-Byte Address Alignment

Assumption 1: 4-Byte Address Alignment

Aspect	Requirement
Alignment Rule	All AXI transactions aligned to 4-byte boundaries
Address Constraint	Address[1:0] must be 2'b00
Rationale	Balances flexibility with AXI protocol requirements
Benefit	Supports arbitrary data placement while maintaining AXI compliance

23.5.2 Assumption 2: Multiple Transfer Sizes

Assumption 2: Multiple Transfer Sizes

Transfer Size	AxSIZE Value	Use Case
4 bytes	3'b010	Initial alignment, small transfers
8 bytes	3'b011	Progressive alignment
16 bytes	3'b100	Progressive alignment
32 bytes	3'b101	Progressive alignment
64 bytes	3'b110	Optimal full-width transfers
128 bytes	3'b111	Maximum efficiency (1024-bit bus)

23.5.3 Assumption 3: Progressive Alignment Strategy

Assumption 3: Progressive Alignment Strategy

Aspect	Requirement
Alignment Goal	Align to 64-byte boundaries for optimal bus utilization
Alignment Sequence	Use progressive sizes: 4 → 8 → 16 → 32 → 64 bytes
Optimization	Choose largest possible transfer size at each step
Example	Address 0x1004: 4-byte transfer → aligned to 0x1008, then larger transfers

23.5.4 Assumption 4: Chunk Enable Support

Assumption 4: Chunk Enable Support

Aspect	Requirement
Chunk Granularity	16 chunks of 32-bits each (512-bit bus)
Write Strobes	Generated from chunk enables for precise byte control
Alignment Transfers	Chunk patterns optimized for alignment sequences
Benefits	Precise data validity, optimal memory utilization

23.6 Common Protocol Assumptions (Both Modes)

23.6.1 Assumption 1: Incrementing Bursts Only

Assumption 1: Incrementing Bursts Only

Aspect	Requirement
Burst Type	All AXI bursts use incrementing address mode (AxBURST = 2'b01)
Excluded Types	No FIXED (2'b00) or WRAP (2'b10) bursts supported
Rationale	Simplifies address generation logic and covers most use cases

Aspect	Requirement
Benefit	Eliminates wrap boundary calculations and fixed address handling

23.6.2 Assumption 2: No Address Wraparound

Assumption 2: No Address Wraparound

Aspect	Requirement
Wraparound Rule	Transactions never wrap around top of address space
Example	No 0xFFFFFFFF → 0x00000000 transitions
Rationale	Real systems never allow this due to memory layout
Benefit	Dramatically simplified boundary crossing detection logic

23.7 Flexible Mode: Address Calculation Examples

23.7.1 Progressive Alignment Examples

Example 1: Address 0x1004 → 0x1040 (64-byte boundary)

Progressive Alignment Examples

Step	Address	Size	AxSIZE	Length	Bytes Transferred	Notes
1	0x1004	4 bytes	3'b010	1 beat	4	Initial alignment
2	0x1008	8 bytes	3'b011	1 beat	8	Progressive alignment
3	0x1010	16 bytes	3'b100	1 beat	16	Progressive alignment
4	0x1020	32	3'b101	1 beat	32	Progre

Step	Address	Size	AxSIZE	Length	Bytes Transferred	Notes
		bytes				ssive alignm ent
5	0x1040	64 bytes	3'b110	N beats	64×N	Optimal transf ers

Example 2: Address 0x1010 → 0x1040 (64-byte boundary)

Progressive Alignment Examples

Step	Address	Size	AxSIZE	Length	Bytes Transferred	Notes
1	0x1010	16 bytes	3'b100	1 beat	16	Optim al initial size
2	0x1020	32 bytes	3'b101	1 beat	32	Progre ssive alignm ent
3	0x1040	64 bytes	3'b110	N beats	64×N	Optim al transf ers

23.7.2 Chunk Enable Pattern Examples

512-bit Bus with 16×32-bit chunks

Chunk Enable Pattern Examples

Transfer Size	Address Offset	Chunk Pattern	Description
4 bytes	0x04	16'h0002	Chunk 1 only
8 bytes	0x08	16'h000C	Chunks 2-3
16 bytes	0x10	16'h00F0	Chunks 4-7
32 bytes	0x20	16'hFF00	Chunks 8-15
64 bytes	0x00	16'hFFFF	All chunks

23.8 Master Read Interface Specification

23.8.1 Read Address Channel (AR)

Read Address Channel

Signal	Width	Direction	Simplified Mode	Flexible Mode	Description
ar_add r	ADDR_ WIDTH	Master→Sl ave	Bus-width aligned	4-byte aligned	Read address
ar_len	8	Master→Sl ave	0-255	0-255	Burst length - 1
ar_siz e	3	Master→Sl ave	Fixed per bus	Variable: 4-64 bytes	Transfer size
ar_bur st	2	Master→Sl ave	2'b01 (INCR only)	2'b01 (INCR only)	Burst type
ar_id	ID_WI DTH	Master→Sl ave	Any	Any	Transaction ID
ar_loc k	1	Master→Sl ave	1'b0	1'b0	Lock type (normal)
ar_cac he	4	Master→Sl ave	Implementation specific	4'b0011	Cache attributes
ar_pro t	3	Master→Sl ave	Implementation specific	3'b000	Protection attributes
ar_qos	4	Master→Sl ave	4'b0000	4'b0000	Quality of Service
ar_reg ion	4	Master→Sl ave	4'b0000	4'b0000	Region identifier
ar_use r	USER_ WIDTH	Master→Sl ave	Optional	Optional	User-defined
ar_val id	1	Master→Sl ave	0 or 1	0 or 1	Address valid
ar_rea dy	1	Slave→Ma ster	0 or 1	0 or 1	Address ready

23.8.2 Read Data Channel (R)

Read Data Channel

Signal	Width	Direction	Description
r_data	DATA_WIDTH	Slave→Master	Read data
r_id	ID_WIDTH	Slave→Master	Transaction ID
r_resp	2	Slave→Master	Read response
r_last	1	Slave→Master	Last transfer in burst
r_user	USER_WIDTH	Slave→Master	User-defined (optional)
r_valid	1	Slave→Master	Read data valid
r_ready	1	Master→Slave	Read data ready

23.9 Master Write Interface Specification

23.9.1 Write Address Channel (AW)

Write Address Channel

Signal	Width	Direction	Simplified Mode	Flexible Mode	Description
aw_addr	ADDR_WIDTH	Master→Slave	Bus-width aligned	4-byte aligned	Write address
aw_len	8	Master→Slave	0-255	0-255	Burst length - 1
aw_size	3	Master→Slave	Fixed per bus	Variable: 4-64 bytes	Transfer size
aw_burst	2	Master→Slave	2'b01 (INCR only)	2'b01 (INCR only)	Burst type
aw_id	ID_WIDTH	Master→Slave	Any	Any	Transaction ID
aw_lock	1	Master→Slave	1'b0	1'b0	Lock type (normal)
aw_cac	4	Master→Slave	Implementation	4'b0011	Cache

Signal	Width	Direction	Simplified Mode	Flexible Mode	Description
he		ave	specific		attributes
aw_prot	3	Master→Slave	Implementation specific	3'b000	Protection attributes
aw_qos	4	Master→Slave	4'b0000	4'b0000	Quality of Service
aw_region	4	Master→Slave	4'b0000	4'b0000	Region identifier
aw_user	USER_WIDTH	Master→Slave	Optional	Optional	User-defined
aw_valid	1	Master→Slave	0 or 1	0 or 1	Address valid
aw_ready	1	Slave→Master	0 or 1	0 or 1	Address ready

23.9.2 Write Data Channel (W)

Write Data Channel

Signal	Width	Direction	Simplified Mode	Flexible Mode	Description
w_data	DATA_WIDTH	Master→Slave	Write data	Write data	Write data
w_strb	DATA_WIDTH/8	Master→Slave	All 1's	From chunk enables	Write strobes
w_last	1	Master→Slave	Last transfer	Last transfer	Last transfer in burst
w_user	USER_WIDTH	Master→Slave	Optional	Optional	User-defined
w_valid	1	Master→Slave	0 or 1	0 or 1	Write data valid
w_ready	1	Slave→Master	0 or 1	0 or 1	Write data ready

23.9.3 Write Response Channel (B)

Write Response Channel

Signal	Width	Direction	Description
b_id	ID_WIDTH	Slave→Master	Transaction ID
b_resp	2	Slave→Master	Write response
b_user	USER_WIDTH	Slave→Master	User-defined (optional)
b_valid	1	Slave→Master	Response valid
b_ready	1	Master→Slave	Response ready

23.10 Address Calculation Rules

23.10.1 Simplified Mode Address Generation

Simplified Mode Address Generation

Parameter	Formula	Description
First Address	Must be bus-width aligned	Starting address
Address N	$\text{First_Address} + (N \times \text{Bus_Width_Bytes})$	Address for beat N
Alignment Check	$(\text{Address} \% \text{Bus_Width_Bytes}) == 0$	Must always be true

23.10.2 Flexible Mode Address Generation

Flexible Mode Address Generation

Parameter	Formula	Description
First Address	Must be 4-byte aligned	Starting address
Address N	$\text{First_Address} + (N \times \text{Transfer_Size})$	Address for beat N
Alignment Check	$(\text{Address} \% 4) == 0$	Must always be true
Progressive Alignment	Choose largest size	Optimization strategy

Parameter	Formula	Description
	$\leq$ bytes_to_boundary	

23.10.3 4KB Boundary Considerations (Both Modes)

4KB Boundary Considerations

Validation Rule	Formula	Description
4KB Boundary	Bursts cannot cross 4KB (0x1000) boundaries	AXI specification
Max Burst Calculation	Max_Beats = (4KB - (Start_Address % 4KB)) / Transfer_Size	Burst limit
Boundary Check	Verify no 4KB crossings in burst	Mandatory validation

23.11 Write Strobe Generation

23.11.1 Simplified Mode Strobe Generation

Simplified Mode Strobe Generation

Bus Width	Strobe Pattern	Description
32-bit	4'b1111	All bytes valid
512-bit	64'hFFFFFFFFFFFFFF FFF	All bytes valid

23.11.2 Flexible Mode Strobe Generation

From Chunk Enables (512-bit bus example):

```
// Convert 16x32-bit chunk enables to 64x8-bit write strobes
for (int chunk = 0; chunk < 16; chunk++) begin
    if (chunk_enable[chunk]) begin
        w_strb[chunk*4 +: 4] = 4'hF; // 4 bytes per chunk
    end
end
```

end
end

Alignment Transfer Examples:

Flexible Mode Strobe Generation

Transfer Size	Chunk Pattern	Strobe Pattern	Description
4 bytes	16'h0001	64'h000000000 000000F	First 4 bytes
16 bytes	16'h000F	64'h000000000 00000FF	First 16 bytes
32 bytes	16'h00FF	64'h000000000 0FFFFFF	First 32 bytes
64 bytes	16'hFFFF	64'hFFFFFFFFF FFFFFFFF	All 64 bytes

23.12 Response Codes

23.12.1 Response Code Specification

Response Code Specification

Value	Name	Description	Simplified Mode Usage	Flexible Mode Usage
2'b00	OKAY	Normal access success	Bus-width aligned access	4-byte aligned access
2'b01	EXOKAY	Exclusive access success	Bus-width aligned exclusive	4-byte aligned exclusive
2'b10	SLVERR	Slave error	Slave-specific error	Slave-specific error
2'b11	DECERR	Decode error	Bus-width misalignment	4-byte misalignment

23.13 Implementation Benefits

23.13.1 Simplified Mode Benefits

Simplified Mode Benefits

Benefit Area	Simplification	Impact
Address Generation	Simple increment by bus width	Minimal logic complexity
Size Checking	No dynamic size validation	No validation logic needed
Strobe Generation	All strobes always high	Trivial implementation
Timing	Predictable single-size transfers	Optimal timing closure

23.13.2 Flexible Mode Benefits

Flexible Mode Benefits

Benefit Area	Capability	Impact
Data Placement	Arbitrary 4-byte aligned placement	Maximum flexibility
Bus Utilization	Progressive alignment optimization	High efficiency achieved
Chunk Control	Precise byte-level validity	Optimal memory utilization
Alignment Strategy	Automatic alignment to boundaries	Performance optimization

23.13.3 Mode Selection Guidelines

Mode Selection Guidelines

Interface Type	Recommended Mode	Rationale
High-bandwidth data	Flexible	Maximize throughput, handle arbitrary alignment
Control/status	Simplified	Predictable

Interface Type	Recommended Mode	Rationale
Descriptors	Simplified	timing, minimal complexity Fixed-size structures, simple implementation
Programs	Simplified	Single-word writes, minimal overhead
Flags	Simplified	Fixed-size status, predictable behavior

23.14 Validation Requirements

23.14.1 Simplified Mode Validation

Simplified Mode Validation

Validation Area	Requirements
Address Alignment	Verify all addresses aligned to full bus width
Fixed Size	Verify AxSIZE always matches DATA_WIDTH
Full Strobes	Verify w_strb is always all 1's
Burst Type	Verify AxBURST is always 2'b01

23.14.2 Flexible Mode Validation

Flexible Mode Validation

Validation Area	Requirements
Address Alignment	Verify all addresses are 4-byte aligned
Size Validation	Verify AxSIZE matches actual transfer size
Chunk Consistency	Verify chunk enables match transfer size

Validation Area	Requirements
Strobe Generation	Verify strobes generated correctly from chunks
Progressive Alignment	Verify alignment strategy optimization
Boundary Checking	Verify no 4KB boundary crossings

23.14.3 Common Validation

Common Validation

Validation Area	Requirements
No Wraparound	Verify addresses never wrap around
Incrementing Only	Verify AxBURST is always 2'b01
Response Handling	Verify proper response generation
Error Conditions	Verify alignment violation responses

23.15 Performance Characteristics

23.15.1 Simplified Mode Performance

Simplified Mode Performance

Metric	Typical Value	Description
Latency	3 cycles	Address + Data + Response
Throughput	1 transfer per clock	Sustained rate
Efficiency	100%	Perfect bus utilization
Complexity	Minimal	Simple implementation

23.15.2 Flexible Mode Performance

Flexible Mode Performance

Metric	Alignment Phase	Optimized Phase	Description
Latency	3-15 cycles	3 cycles	Variable based on alignment
Throughput	Variable	1 transfer per clock	Depends on alignment pattern
Efficiency	25-100%	100%	Improves with alignment
Complexity	Moderate	Minimal	Progressive optimization

23.15.3 Performance Optimization Strategy

Flexible Mode Alignment Strategy: 1. **Initial Phase:** Use largest possible transfer size for current alignment 2. **Progressive Phase:** Incrementally align to larger boundaries 3. **Optimized Phase:** Use full bus-width transfers once aligned 4. **Result:** Achieve maximum efficiency while handling arbitrary starting addresses

23.16 This dual-mode approach provides the best of both worlds: simplified, predictable operation for control interfaces and flexible, high-performance operation for data interfaces.

23.17 Revision History

AXI4 Interface Specification for STREAM Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

24 AXI4-Lite Interface Specification and Assumptions

24.1 Overview

This document defines the formal specification and assumptions for an AXI4-Lite interface implementation. AXI4-Lite is a subset of AXI4 optimized for simple, lightweight control register interfaces with inherent protocol simplifications.

24.2 Interface Summary

24.2.1 Number of Interfaces

- **2 Master Read Interface:** Single read channel for Monitor Packets (one for each Source and Sink)
- **2 Master Write Interface:** Single write channel for Monitor Packets plus a timestamp (one for each Source and Sink)

24.2.2 Interface Parameters

Interface Parameters

Parameter	Description	Valid Values	Default
DATA_WIDTH	AXI data bus width in bits	32, 64	32, 64
ADDR_WIDTH	AXI address bus width in bits	32, 64	37
STRB_WIDTH	Write strobe width	DATA_WIDTH/8	8

24.3 Core Protocol Assumptions

24.3.1 Inherent AXI4-Lite Simplifications

AXI4-Lite protocol inherently provides the following constraints:

Inherent AXI4-Lite Simplifications

Constraint	Description
Single Transfers Only	No burst transactions supported
No Transaction IDs	All transactions are in-order
Fixed Transfer Size	Always uses full data bus width
No User Signals	Simplified interface without user-defined extensions

24.3.2 Implementation Assumptions

24.3.2.1 *Assumption 1: Address Alignment to Data Bus Width*

Assumption 1: Address Alignment to Data Bus Width

Aspect	Requirement
Alignment Rule	All AXI4-Lite transactions aligned to data bus width
32-bit bus alignment	Address[1:0] must be 2'b00 (4-byte aligned)
64-bit bus alignment	Address[2:0] must be 3'b000 (8-byte aligned)
Rationale	Maximizes bus efficiency and eliminates unaligned access complexity
Benefit	Simplifies address decode and data steering logic

24.3.2.2 *Assumption 2: Fixed Transfer Size*

Assumption 2: Fixed Transfer Size

Aspect	Requirement
Transfer Size Rule	All transfers use maximum size equal to bus width
32-bit bus	AxSIZE = 3'b010 (4 bytes)
64-bit bus	AxSIZE = 3'b011 (8 bytes)
Rationale	Maximizes bus utilization and simplifies control logic
Benefit	No size decode logic required

24.3.2.3 Assumption 3: No Address Wraparound

Assumption 3: No Address Wraparound

Aspect	Requirement
Wraparound Rule	Transactions never wrap around top of address space
Rationale	Control register accesses never require wraparound behavior
Benefit	Simplified address boundary checking

24.3.2.4 Assumption 4: Standard Protection Attributes

Assumption 4: Standard Protection Attributes

Access Type	AxPROT Value	Description
Normal Access	3'b000	Data, secure, unprivileged
Privileged Access	3'b001	Data, secure, privileged
Rationale		Covers the majority of control register access patterns

24.4 Master Read Interface Specification

24.4.1 Read Address Channel (AR)

Read Address Channel

Signal	Width	Direction	Required Values	Description
ar_addr	ADDR_WIDTH	Master→Slave	8-byte aligned	Read address
ar_prot	3	Master→Slave	Implementation specific	Protection attributes
ar_valid	1	Master→Slave	0 or 1	Address valid
ar_ready	1	Slave→Master	0 or 1	Address ready

24.4.2 Read Data Channel (R)

Read Data Channel

Signal	Width	Direction	Description
r_data	64	Slave→Master	Read data
r_resp	2	Slave→Master	Read response
r_valid	1	Slave→Master	Read data valid
r_ready	1	Master→Slave	Read data ready

24.4.3 AXI4-Lite Simplifications (Read)

AXI4-Lite Simplifications

Removed Signal	AXI4 Usage	AXI4-Lite Reason
ar_id	Transaction ID	Single transfers, no transaction IDs
ar_len	Burst length	Single transfers only
ar_size	Transfer size	Fixed to bus width
ar_burst	Burst type	Single transfers only
ar_lock	Lock type	Simplified access model
ar_cache	Cache attributes	Simplified memory model
ar_qos	Quality of Service	Simplified priority model
ar_region	Region identifier	Simplified address space
ar_user	User-defined	Simplified interface
r_id	Transaction ID	No transaction IDs needed
r_last	Last transfer	Single transfers only
r_user	User-defined	Simplified interface

24.5 Master Write Interface Specification

24.5.1 Write Address Channel (AW)

Write Address Channel

Signal	Width	Direction	Required Values	Description
aw_addr	ADDR_WIDTH	Master→Slave	8-byte aligned	Write address
aw_prot	3	Master→Slave	Implementation specific	Protection attributes
aw_valid	1	Master→Slave	0 or 1	Address valid
aw_ready	1	Slave→Master	0 or 1	Address ready

24.5.2 Write Data Channel (W)

Write Data Channel

Signal	Width	Direction	Description
w_data	32	Master→Slave	Write data
w_strb	4	Master→Slave	Write strobes (byte enables)
w_valid	1	Master→Slave	Write data valid
w_ready	1	Slave→Master	Write data ready

24.5.3 Write Response Channel (B)

Write Response Channel

Signal	Width	Direction	Description
b_resp	2	Slave→Master	Write response
b_valid	1	Slave→Master	Response valid
b_ready	1	Master→Slave	Response ready

24.5.4 AXI4-Lite Simplifications (Write)

AXI4-Lite Simplifications

Removed Signal	AXI4 Usage	AXI4-Lite Reason
aw_id	Transaction ID	Single transfers, no transaction IDs
aw_len	Burst length	Single transfers only
aw_size	Transfer size	Fixed to bus width
aw_burst	Burst type	Single transfers only
aw_lock	Lock type	Simplified access model
aw_cache	Cache attributes	Simplified memory model
aw_qos	Quality of Service	Simplified priority model
aw_region	Region identifier	Simplified address space
aw_user	User-defined	Simplified interface
w_last	Last transfer	Single transfers only
w_user	User-defined	Simplified interface
b_id	Transaction ID	No transaction IDs needed
b_user	User-defined	Simplified interface

24.6 Address Requirements

24.6.1 Address Alignment Rules

Address Alignment Rules

Alignment Type	Formula	Description
Valid Address	$(\text{Address} \% 4) == 0$	Must be 8-byte aligned
Mandatory Alignment	Address[2:0] must be 3'b000	Per Assumption 1

24.6.2 Address Validation Examples

Address Validation Examples

Address Category	Examples	Status
Valid (84byte aligned)	0x1000, 0x1004, 0x1008, 0x100C	Accepted
Invalid (unaligned)	0x1001, 0x1002, 0x1003	DECERR response

24.7 Response Codes

24.7.1 Response Code Specification

Response Code Specification

Value	Name	Description	Usage in Control Registers
2'b00	OKAY	Normal access success	Successful register access
2'b01	EXOK AY	Exclusive access success	Not used in AXI4-Lite
2'b10	SLVER R	Slave error	Invalid register access
2'b11	DECER R	Decode error	Address decode failure or misalignment

24.7.2 Response Usage Guidelines

Response Usage Guidelines

Response Type	Usage	Description
OKAY	Normal completion	Successful register access
EXOKAY	Not applicable	AXI4-Lite doesn't support exclusive accesses
SLVERR	Register error	Invalid register operation
DECERR	Address error	Misalignment or decode failure per Assumption 1

24.8 Protection Signal Usage

24.8.1 Protection Signal Encoding

Protection Signal Encoding

Bit	Name	Description	Recommended Usage
[0]	Privileged	0=Normal, 1=Privileged	Set based on processor mode
[1]	Non-secure	0=Secure, 1=Non-secure	Set based on security domain
[2]	Instruction	0=Data, 1=Instruction	Always 0 for control registers

24.8.2 Common Protection Patterns

Common Protection Patterns

Pattern	AxPROT Value	Description
Normal Data Access	3'b000	Standard register access
Privileged Data Access	3'b001	Privileged register access
Debug Access	3'b010	Debug register access
Privileged Debug	3'b011	Privileged debug access

24.9 Implementation Benefits

24.9.1 Simplified Control Register Interface

Simplified Control Register Interface

Benefit Area	Simplification	Impact
Address Decode	Simple 8-byte aligned address comparison	Reduced decode logic
Transaction Handling	No burst or ID tracking required	Simplified state machines
Flow Control	Straightforward valid-ready	Reduced

Benefit Area	Simplification	Impact
Response Generation	handshakes	complexity
	Simple OKAY/SLVERR/DECERR responses	Minimal response logic
Size Handling	Fixed 64-bit transfers only	No size decode needed

24.9.2 Address Decode Implementation

Address Decode Implementation

Implementation Aspect	Method	Benefit
4-byte Alignment Check	addr[1:0] == 2'b00	Simple bit masking
Address Range Check	addr >= base && addr <= limit	Simple comparisons
Combined Check	alignment_ok && range_ok	Single decode decision

24.9.3 Error Generation Logic

Error Generation Logic

Error Condition	Check	Response
Address Misalignment	addr[1:0] != 2'b00	Generate DECERR
Address Out of Range	!addr_in_range(addr)	Generate DECERR
Register Error	register_error_condition	Generate SLVERR
Normal Access	All checks pass	Generate OKAY

24.10 Timing Requirements

24.10.1 Handshake Protocol

Handshake Protocol

Protocol Rule	Requirement	Description
Valid-Ready Transfer	Transfer occurs when both VALID and READY are high	Standard AXI handshake
Valid Independence	VALID can be asserted independently of READY	Master controls valid
Ready Dependency	READY can depend on VALID state	Slave controls ready
Signal Stability	Once VALID asserted, all signals stable until READY	Data integrity

24.10.2 Channel Dependencies

Channel Dependencies

Dependency	Requirement	Description
Write Channels	AW and W channels are independent	Can be presented in any order
Write Response	B channel waits for both AW and W completion	Response dependency
Read Channels	R channel waits for AR channel completion	Response dependency
Transaction Ordering	Multiple outstanding transactions not supported	Inherent AXI4-Lite limitation

24.10.3 Reset Behavior

Reset Behavior

Reset Phase	Requirement	Description
Active Reset	aresetn is active-low reset signal	Standard AXI reset

Reset Phase	Requirement	Description
Reset Requirements	All VALID signals deasserted during reset	Clean reset state
Reset Recovery	All VALID signals low after reset deassertion	Proper startup

24.11 Validation Requirements

24.11.1 Functional Validation

Functional Validation

Validation Area	Requirements
Address Alignment	Verify all accesses are 8-byte aligned per Assumption 1
Fixed Size	Verify all transfers are full 64-bit width per Assumption 2
Response Correctness	Verify appropriate response codes (DECERR for misaligned access)
Handshake Compliance	Verify all valid-ready handshakes
Register Behavior	Verify read/write register functionality
No Wraparound	Verify no address wraparound scenarios per Assumption 3

24.11.2 Timing Validation

Timing Validation

Validation Area	Requirements
Setup/Hold	Verify signal timing requirements
Reset Behavior	Verify proper reset sequence
Back-pressure	Verify ready signal behavior under load

24.11.3 Error Injection Testing

Error Injection Testing

Test Type	Injection Method	Expected Response
Misaligned Address	Inject addresses with addr[2:0] != 0	DECERR response
Out of Range	Inject addresses outside valid range	DECERR response
Register Errors	Inject register-specific errors	SLVERR response

24.12 Example Transactions

24.12.1 64-bit Register Write

64-bit Register Write

Parameter	Value	Description
Bus Width	64 bits (8 bytes)	Data bus configuration
Target Address	0x1000 (8-byte aligned)	Valid aligned address
Write Data	0xDEADBEEFCA FEBABE	64-bit data value
Required Settings	aw_addr=0x1000, aw_prot=3'b000, w_data=0xDEADBEEFCAFEFABE, w_strb=8'b11111111	Transaction configuration

24.12.2 AW Transaction Flow

AW Transaction Flow

Step	Action	Signal States
1	Assert aw_valid with address	aw_valid=1, aw_addr=0x1000
2	Assert w_valid with	w_valid=1,

Step	Action	Signal States
3	data	w_data=0xDEADBEEFCAFEBA
	Wait for handshakes	aw_ready=1, w_ready=1
4	Wait for response	b_valid=1, b_resp=OKAY
5	Complete transaction	b_ready=1

24.12.3 64-bit Register Read

64-bit Register Read

Parameter	Value	Description
Bus Width	64 bits (8 bytes)	Data bus configuration
Target Address	0x1008 (8-byte aligned)	Valid aligned address
Required Settings	ar_addr=0x1008, ar_prot=3'b000	Transaction configuration

24.12.4 AR Transaction Flow

AR Transaction Flow

Step	Action	Signal States
1	Assert ar_valid with address	ar_valid=1, ar_addr=0x1008
2	Wait for address handshake	ar_ready=1
3	Wait for data response	r_valid=1, r_resp=OKAY
4	Complete transaction	r_ready=1
5	Capture data	r_data (64 bits)

24.12.5 Misaligned Address Example

Misaligned Address Example

Parameter	Value	Description
Bus Width	64 bits (8 bytes)	Data bus configuration

Parameter	Value	Description
Target Address	0x1004 (misaligned)	Invalid address
Expected Behavior	Address decode detects misalignment → DECERR response → No register access	Error handling

24.13 Common Use Cases

24.13.1 Typical Applications

Typical Applications

Application	Description
Control/Status Registers	64-bit device configuration and monitoring
Memory-Mapped Peripherals	Simple register-based devices
Debug Interfaces	Debug and trace control registers
Configuration Space	PCIe configuration space access
Performance Counters	64-bit performance monitoring registers

24.13.2 Performance Considerations

Performance Considerations —

Consideration	Impact	Description
Latency	Single-cycle responses preferred	Simple registers
Throughput	Limited by single outstanding transaction	AXI4-Lite constraint
Efficiency	64-bit transfers maximize data efficiency	Modern system optimization

24.14 Revision History

AXI4-Lite Interface Specification and Assumptions Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

25 APB Programming Interface Specification for STREAM

25.1 Overview

This document defines the APB programming interface for the STREAM DMA engine.

Important: STREAM uses a **simplified programming interface** for descriptor kick-off, NOT a full APB slave. The interface uses standard valid/ready handshaking per channel to initiate descriptor-based transfers.

This interface is separate from the optional APB configuration interface that may be added in future implementations for runtime configuration of STREAM parameters.

25.2 Interface Summary

25.2.1 STREAM Programming Interface

STREAM provides a **simplified per-channel descriptor programming interface**:

STREAM Programming Interface

Signal Group	Per-Channel Signals	Protocol	Purpose
Programming	apb_valid[ch], apb_ready[ch], apb_addr[ch]	Valid/Ready handshake	Descriptor address input

Note: Despite the “apb_” prefix, this is NOT a full AMBA APB interface. It’s a simplified programming interface using APB naming convention for historical reasons.

25.2.2 Interface Parameters

Interface Parameters

Parameter	Description	Valid Values	Default
DATA_WIDTH	APB data bus width in bits	8, 16, 32	32
ADDR_WIDTH	APB address bus width in bits	16, 24, 32	32
STRB_WIDTH	Write strobe width (APB4 only)	DATA_WIDTH/8	4
NUM_SLAVES	Number of peripheral slaves	1-32	1
DEPTH	Internal buffer depth	2+	2

25.3 Core Protocol Assumptions

25.3.1 Inherent APB Simplifications

APB protocol inherently provides the following constraints:

1. **Single Master Only:** Only one bus master supported
2. **Non-Pipelined:** One transaction completes before next begins
3. **Simple 2-Phase Protocol:** Setup phase followed by access phase
4. **No Burst Transfers:** Only single transfers supported
5. **In-Order Completion:** No out-of-order transaction capability

25.3.2 Implementation Assumptions

25.3.2.1 Assumption 1: Word-Aligned Access Only

Assumption 1: Word-Aligned Access Only

Aspect	Requirement
Alignment Rule	All APB transfers are aligned to natural word boundaries
8-bit transfers	Byte aligned (no restriction)
16-bit transfers	2-byte aligned (PADDR[0] = 0)
32-bit transfers	4-byte aligned (PADDR[1:0] = 2'b00)
Rationale	Simplifies peripheral decode logic and ensures efficient access

25.3.2.2 Assumption 2: Standard Transfer Sizes

Assumption 2: Standard Transfer Sizes

Bus Width	Supported Transfer Sizes	Rationale
32-bit bus	8-bit, 16-bit, 32-bit	Covers typical register access patterns
16-bit bus	8-bit, 16-bit	Keeps decode logic simple
8-bit bus	8-bit only	Minimal complexity

25.3.2.3 Assumption 3: Single-Cycle Default Operation

Assumption 3: Single-Cycle Default Operation

Aspect	Requirement
Default Behavior	Most peripherals respond in a single cycle (PREADY tied high)
Optimization	PREADY optimization for simple registers
Exception	Complex peripherals may use PREADY for wait states
Rationale	Minimizes latency for control register access

25.4 Interface Signal Specification

25.4.1 Clock and Reset Signals

Clock and Reset Signals ### Address and Control Signals

Signal	IO	Description
src_clk	I	Source APB clock signal
snk_rdata[31:0]	I	Read data
src_wdata[31:0]	O	Write data

Address and Control Signals

Signal	Width	Direction	Required Values	Description
src_addr	13 bits	Master→Slave	Aligned per transfer size	Peripheral address (8KB space)
src_sel	1	Master→Slave	0 or 1	Peripheral select
src_enable	1	Master→Slave	0 or 1	Enable signal
src_write	1	Master→Slave	0 or 1	Write enable (1=write, 0=read)
snk_addr	13 bits	Master→Slave	Aligned per transfer size	Peripheral address (8KB space)
snk_sel	1	Master→Slave	0 or 1	Peripheral select
snk_enable	1	Master→Slave	0 or 1	Enable signal
snk_write	1	Master→Slave	0 or 1	Write enable (1=write, 0=read)

25.4.2 Data Signals

Data Signals

Signal	Width	Direction	Description
src_wdata	32	Master→Slave	Write data
src_rdata	32	Slave→Master	Read data
snk_wdata	32	Master→Slave	Write data
snk_rdata	32	Slave→Master	Read data

25.4.3 Response Signals

Response Signals

Signal	Width	Direction	Description
src_ready	1	Slave→Master	Transfer ready
src_slverr	1	Slave→Master	Slave error
snk_ready	1	Slave→Master	Transfer ready
snk_slverr	1	Slave→Master	Slave error

25.5 Address Space Configuration

25.5.1 Address Range Specification

Address Range Specification

Parameter	Value	Description
Address Width	13 bits (src_addr[12:0])	Full address space
Address Space	8KB (8192 bytes)	Total addressable space
Register Alignment	32-bit aligned (4-byte boundaries)	Address constraints
Usable Addresses	0x000 to 0x1FFF	Valid address range

Note: The address decode is **13 bits (8 KB)** so the STREAM monitor register block — instantiated in a separate `stream_mon_regs` regfile at **0x1000** on the same APB slave — is addressable. An 11-bit / 2 KB decode would not reach the monitor registers. See `stream_mas/ch04_registers/register_map.md`.

25.5.2 Address Decode Implementation

Address Decode Implementation

Register	Address Bits	Address Value	Description
reg0	src_addr[12:2] == 11'h000	0x000	First register
reg1	src_addr[12:2] == 11'h001	0x004	Second register
reg2	src_addr[12:2] == 11'h002	0x008	Third register
...	...	...	Up to 2048 32-bit registers

25.6 Transaction Protocol

25.6.1 2-Phase Transaction States

2-Phase Transaction States

Phase	State	Duration	Signal Requirements
Setup (T1)	SETUP	1 clock cycle	src_sel=1, src_enable=0, address/data stable
Access (T2+)	ACCESS	1+ clock cycles	src_sel=1, src_enable=1, wait for src_ready
Idle	IDLE	Variable	src_sel=0, src_enable=0

25.6.2 State Transitions

State Transitions

Current State	Next State	Condition	Description
IDLE	SETUP	Transaction start	Master drives address and control
SETUP	ACCESS	Clock edge	Master asserts src_enable
ACCESS	IDLE	src_ready=1	Transaction completes
ACCESS	ACCESS	src_ready=0	Wait state

Current State	Next State	Condition	Description
			continues

25.6.3 Transaction Timing Requirements

Transaction Timing Requirements

Timing Parameter	Requirement	Description
Setup Time	All master signals stable before rising src_clk	Signal stability
Hold Time	All master signals held after rising src_clk	Signal stability
Output Delay	Slave outputs valid after rising src_clk	Response timing

25.7 Advanced Features

25.7.1 Dynamic Clock Gating

Dynamic Clock Gating

Feature	Description	Benefit
Automatic Power Reduction	Clocks gated when no activity detected	Significant power savings
Configurable Idle Threshold	Programmable idle count before gating	Tunable power/performance
Multi-Domain Support	Independent gating for APB (PCLK) and backend (ACLK)	Flexible power management
Graceful Handoff	Ready signals forced to zero during gating	Protocol compliance
Activity Detection	Monitors valid signals across interfaces	Intelligent gating control

25.7.2 Clock Domain Crossing (CDC) Handshaking

Clock Domain Crossing (CDC) Handshaking

Feature	Description	Benefit
Arbitrary	Works across any clock	Design flexibility

Feature	Description	Benefit
Frequency Support	frequency ratios	
Robust Reliability	Proven handshaking with proper synchronization	Zero data loss
Command/Response Separation	Independent CDC paths	Maximum throughput
Skid Buffer Integration	Internal buffering prevents backpressure	Smooth operation
Deterministic Latency	Predictable timing characteristics	System predictability

25.7.3 Buffering and Flow Control

Buffering and Flow Control

Feature	Description	Benefit
Internal Skid Buffers	Configurable depth buffering	Improved performance
Backpressure Handling	Proper ready/valid handshaking	Flow control
Command Pipelining	Backend processes while APB handles responses	Efficiency
Response Queuing	Responses buffered for varying latencies	Latency tolerance

25.8 Error Handling

25.8.1 Error Response Specification

Error Response Specification

Condition	src_slverr	Description
Successful access	0	Normal completion
Address decode error	1	Invalid address
Protection violation	1	Insufficient privilege
Backend error	1	Downstream error

Condition	src_slverr	Description
		condition

25.8.2 Error Response Timing

Error Response Timing

Timing Requirement	Description
src_slverr Validity	Must be valid when src_ready is asserted
Error Completion	Error response completes transaction normally
Master Responsibility	Master must check src_slverr status

25.9 Reset Behavior

25.9.1 Reset Requirements

Reset Requirements

Reset Phase	Signal Requirements	Description
Reset Assertion	All outputs to known states	Deterministic reset
Reset Deassertion	src_sel=0, src_enable=0 in first cycle	Clean startup
Reset Values	src_ready=0, src_slverr=0, src_rdata=0	Default states

25.10 Implementation Variants

25.10.1 Available Implementations

Available Implementations

Variant	Features	Use Case
Basic APB Slave	Single clock domain, internal buffering	Simple peripherals
CDC-Enabled APB Slave	Dual clock domain (PCLK/ACLK)	Mixed-frequency systems
Power-Optimized APB	Dynamic clock gating	Power-sensitive designs

Variant	Features	Use Case
Slave		

25.11 Performance Characteristics

25.11.1 Latency Specifications

Latency Specifications

Metric	Value	Description
Minimum Read Latency	2 clock cycles	Setup + access phases
Minimum Write Latency	2 clock cycles	Setup + access phases
CDC Additional Latency	2-6 clock cycles	Depends on clock relationships
Buffer Latency	Minimal	Skid buffer design

25.11.2 Throughput Specifications

Throughput Specifications

Metric	Value	Description
Single Transaction	2 clocks minimum	Basic transaction time
Back-to-back Transactions	Limited by src_ready	Depends on peripheral
CDC Throughput	Maintained across domains	Proper buffering

25.11.3 Power Consumption

Power Consumption

Power Category	Characteristics	Description
Active Power	Standard CMOS logic	Normal operation
Idle Power	Significantly reduced with clock gating	Power optimization
Gating Overhead	Minimal additional logic	Efficient implementation

25.12 Validation Requirements

25.12.1 Functional Validation

Functional Validation

Validation Area	Requirements
Protocol Compliance	Verify 2-phase setup/access sequence
Address Decode	Verify 13-bit address handling and alignment
Error Response	Verify src_slverr generation and handling
Wait States	Verify src_ready functionality
CDC Operation	Verify cross-clock domain transfers
Clock Gating	Verify power management behavior
Buffer Operation	Verify skid buffer and flow control

25.12.2 Timing Validation

Timing Validation

Validation Area	Requirements
Setup/Hold	Verify signal timing requirements
Reset Sequence	Verify proper reset behavior
Multi-Clock	Verify CDC timing across frequency ranges
Gating Transitions	Verify clock enable/disable timing

25.13 Example Transactions

25.13.1 32-bit Register Write

32-bit Register Write

Clock Cycle	src_sel	src_enable	src_write	src_addr	src_wdata	src_ready	Phase
1	1	0	1	0x100	0xABCD	0	Setup
2	1	1	1	0x100	0xABCD	1	Access
3	0	0	-	-	-	0	Idle

25.13.2 32-bit Register Read with Wait State

32-bit Register Read with Wait State —

Clock Cycle	src_sel	src_enable	src_write	src_addr	src_rdata	src_ready	Phase
1	1	0	0	0x104	-	0	Setup
2	1	1	0	0x104	-	0	Access
3	1	1	0	0x104	0x1234	1	Complete
4	0	0	-	-	-	0	Idle

25.14 Revision History

APB Programming Interface Specification for STREAM Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers
0.93	2026-07-02	seang	Address decode widened to 13 bits / 8 KB so the monitor register block at 0x1000 (separate

Version	Date	Author	Description
			stream_mon_regs regfile, same APB slave) is addressable

Last Updated: 2026-07-02

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

26 Monitor Bus Architecture and Event Code Organization

26.1 STREAM-Specific Context

For STREAM DMA Engine: This document describes the generic Monitor Bus (MonBus) protocol used across all AMBA-based components. STREAM uses MonBus for unified event reporting from:

- Descriptor engines (8 sources, agent IDs 16-23) - Schedulers (8 sources, agent IDs 48-55) - Descriptor AXI monitor (agent ID 8) - Read/Write AXI monitors (configurable agent IDs)

All STREAM events use Unit ID = 1 and follow the standard 64-bit packet format defined below.

26.2 Overview

The Monitor Bus architecture provides a unified, scalable framework for monitoring and error reporting across multiple bus protocols in complex SoC designs. This system supports AXI, APB, MNOC (Mesh Network on Chip), ARB (Arbiter), CORE, and custom protocols through a standardized **128-bit packet format** paired with **64-bit side-band timestamps**, both carried atomically through a single handshake interface.

STREAM Integration: STREAM uses this unified MonBus for all event reporting (descriptor fetch, scheduler state, AXI transaction monitoring). See `docs/markdown/RTLAmba/includes/monitor_package_spec.md` for the canonical 128-bit packet layout.

26.3 Interface Summary

26.3.1 Number of Interfaces

- **1 Monitor Bus Output Interface:** Unified 64-bit packet stream
- **Multiple Protocol Input Interfaces:** AXI, APB, MNOC, ARB, CORE, Custom protocol monitors
- **Local Memory Interface:** Error/interrupt packet storage
- **External Memory Interface:** Bulk packet storage

26.3.2 Interface Parameters

Interface Parameters

Parameter	Description	Valid Values	Default
PACKET_WIDTH	Monitor bus packet width (FIXED)	128	128
TIMESTAMP_WIDTH	Side-band timestamp width (FIXED)	64	64
PROTOCOL_WIDTH	Protocol identifier width	3	3
EVENT_CODE_WIDTH	Event code width	4	4
PACKET_TYPE_WIDTH	Packet type width	4	4
CHANNEL_ID_WIDTH	Channel identifier width	9	9
UNIT_ID_WIDTH	Unit identifier width	8	8
AGENT_ID_WIDTH	Agent identifier width	16	16
EVENT_DATA_WIDTH	Event data width	64	64

Note: PACKET_WIDTH, TIMESTAMP_WIDTH, and other core dimensions are now **locked** across the codebase (see rtl/amba/includes/monitor_common_pkg.sv). Do not override these with per-module parameters.

26.4 Core Design Assumptions

26.4.1 Assumption 1: Hierarchical Event Organization

Assumption 1: Hierarchical Event Organization

Aspect	Requirement
Organization Rule	Protocol → Packet Type → Event Code hierarchy
Event Space	Each protocol × packet type combination has exactly 16 event codes
Mapping	1:1 mapping between packet types and event codes
Rationale	Provides clear, scalable event organization

26.4.2 Assumption 2: Protocol Isolation

Assumption 2: Protocol Isolation

Aspect	Requirement
Isolation Rule	Each protocol owns its event space
Conflict Prevention	No cross-protocol event conflicts
Independent Evolution	Protocols can evolve independently
Rationale	Prevents interference and enables protocol-specific optimization

26.4.3 Assumption 3: Two-Tier Memory Architecture

Assumption 3: Two-Tier Memory Architecture

Aspect	Requirement
Local Storage	Critical events (errors/interrupts) stored locally
External Storage	Non-critical events routed to external memory
Routing Decision	Based on packet type configuration
Rationale	Balances immediate access with bulk storage needs

26.4.4 Assumption 4: Configurable Packet Routing

Assumption 4: Configurable Packet Routing

Aspect	Requirement
Routing Rule	Different packet types can route to different destinations
Configuration	Base/limit registers define routing per packet type
Priority Support	Configurable priority levels per packet type
Rationale	Enables flexible memory allocation and access patterns

26.5 Interface Signal Specification

26.5.1 Monitor Bus Output Interface

Monitor Bus Output Interface

Signal	Width	Direction	Description
mon_packet	64	Monitor→System	Monitor packet data
mon_valid	1	Monitor→System	Packet valid signal
mon_ready	1	System→Monitor	Ready to accept packet
mon_error	1	Monitor→System	Monitor error condition

26.5.2 Protocol Input Interfaces

Protocol Input Interfaces

Signal	Width	Direction	Description
axi_event	64	AXI Monitor→Bus	AXI event packet
axi_event_valid	1	AXI Monitor→Bus	AXI event valid
axi_event_ready	1	Bus→AXI Monitor	Ready for AXI event

Signal	Width	Direction	Description
apb_event	64	APB Monitor→Bus	APB event packet
apb_event_val id	1	APB Monitor→Bus	APB event valid
apb_event_rea dy	1	Bus→APB Monitor	Ready for APB event
mnoc_event	64	MNOC Monitor→Bus	MNOC event packet
mnoc_event_va lid	1	MNOC Monitor→Bus	MNOC event valid
mnoc_event_re ady	1	Bus→MNOC Monitor	Ready for MNOC event
arb_event	64	ARB Monitor→Bus	ARB event packet
arb_event_val id	1	ARB Monitor→Bus	ARB event valid
arb_event_rea dy	1	Bus→ARB Monitor	Ready for ARB event
core_event	64	CORE Monitor→Bus	CORE event packet
core_event_va lid	1	CORE Monitor→Bus	CORE event valid
core_event_re ady	1	Bus→CORE Monitor	Ready for CORE event

26.5.3 Control and Status Signals

Control and Status Signals

Signal	Width	Direction	Description
clk	1	Input	System clock
resetrn	1	Input	Active-low reset
monitor_enabl e	1	Input	Global monitor enable

Signal	Width	Direction	Description
packet_type_enables	16	Input	Per-type enable bits
local_memory_full	1	Output	Local memory full flag
external_memory_error	1	Output	External memory error

26.6 Packet Format and Field Allocation

26.6.1 128-bit Monitor Bus Packet Structure

The 128-bit packet is the standard across all AMBA/STREAM monitoring. Full field definitions are in docs/markdown/RTLAmba/includes/monitor_package_spec.md and the SystemVerilog typedef monitor_packet_t in rtl/amba/includes/monitor_common_pkg.sv.

Quick Reference:

128-bit Monitor Bus Packet Structure

Field	Bits	Width	Description
Packet Type	[127:124]	4	Event category (Error=0x0, Completion=0x1, etc.)
Reserved	[123:109]	15	Forward-compatibility slack
Protocol	[108:105]	4	Bus protocol (AXI=0, AXIS=1, APB=2, ARB=3, CORE=4)
Event Code	[104:97]	8	Specific events within category
Channel ID	[96:88]	9	Transaction/channel identifier
Agent ID	[87:72]	16	Module identifier (descriptor fetch=8, scheduler CH0=48, etc.)
Unit ID	[71:64]	8	Subsystem identifier (STREAM=0x01)
Event Data	[63:0]	64	Event-specific payload (addresses, counts, etc.)

26.6.2 64-bit Side-Band Timestamp

The timestamp is carried in parallel with the packet (not within it) through the same valid/ready handshake:

64-bit Side-Band Timestamp

Field	Bits	Width	Description
Timestamp	[63:0]	64	Cycle count at packet capture (hardware clock counter)

26.6.3 Packet Type Definitions

Packet Type Definitions

Value	Name	Purpose	Applicable Protocols
0x0	Error	Protocol violations, response errors	All
0x1	Completion	Successful transaction completion	All
0x2	Threshold	Threshold crossed events	All
0x3	Timeout	Timeout conditions	All
0x4	Performance	Performance metrics	All
0x5	Credit	Credit management	MNOC only
0x6	Channel	Channel status	MNOC only
0x7	Stream	Stream events	MNOC only
0x8	Address Match	Address matching	AXI only
0x9	APB Specific	APB protocol events	APB only
0xA-0xE	Reserved	Future	-

Value	Name	Purpose	Applicable Protocols
0xF	Debug	expansion Debug and trace events	All

26.7 Protocol-Specific Event Codes

26.7.1 AXI Protocol Events

26.7.1.1 Error Events (*PktTypeError* + *PROTOCOL_AXI*)

Error Events

Code	Event Name	Description
0x0	AXI_ERR_RESP_SLVE RR	Slave error response
0x1	AXI_ERR_RESP_DECE RR	Decode error response
0x2	AXI_ERR_DATA_ORP HAN	Data without command
0x3	AXI_ERR_RESP_ORP HAN	Response without transaction
0x4	AXI_ERR_PROTOCOL	Protocol violation
0x5	AXI_ERR_BURST_LE NGTH	Invalid burst length
0x6	AXI_ERR_BURST_SIZ E	Invalid burst size
0x7	AXI_ERR_BURST_TYP E	Invalid burst type
0x8	AXI_ERR_ID_COLLISI ON	ID collision detected
0x9	AXI_ERR_WRITE_BEFORE_ADDR	Write data before address
0xA	AXI_ERR_RESP_BEFORE_DATA	Response before data complete
0xB	AXI_ERR_LAST_MISS	Missing LAST signal

Code	Event Name	Description
	ING	
0xC	AXI_ERR_STROBE_ERROR	Write strobe error
0xD	AXI_ERR_RESERVED_D	Reserved
0xE	AXI_ERR_RESERVED_E	Reserved
0xF	AXI_ERR_USER_DEFINED	User-defined error

26.7.1.2 Timeout Events (*PktTypeTimeout* + *PROTOCOL_AXI*)

Timeout Events

Code	Event Name	Description
0x0	AXI_TIMEOUT_CMD	Command/Address timeout
0x1	AXI_TIMEOUT_DATA	Data timeout
0x2	AXI_TIMEOUT_RESP	Response timeout
0x3	AXI_TIMEOUT_HANDSHAKE	Handshake timeout
0x4	AXI_TIMEOUT_BURST	Burst completion timeout
0x5	AXI_TIMEOUT_EXCLUSIVE	Exclusive access timeout
0x6-0xE	Reserved	Future expansion
0xF	AXI_TIMEOUT_USER_DEFINED	User-defined timeout

26.7.1.3 Performance Events (*PktTypePerf* + *PROTOCOL_AXI*)

Performance Events

Code	Event Name	Description
0x0	AXI_PERF_ADDR_LATENCY	Address phase latency
0x1	AXI_PERF_DATA_LATENCY	Data phase latency

Code	Event Name	Description
	TENCY	
0x2	AXI_PERF_RESP_LATENCY	Response phase latency
0x3	AXI_PERF_TOTAL_LATENCY	Total transaction latency
0x4	AXI_PERF_THROUGHPUT	Transaction throughput
0x5	AXI_PERF_ERROR_RATE	Error rate
0x6	AXI_PERF_ACTIVE_COUNT	Active transaction count
0x7	AXI_PERF_BANDWIDTH_UTIL	Bandwidth utilization
0x8	AXI_PERF_QUEUE_DEPTH	Average queue depth
0x9	AXI_PERF_BURST_EFFICIENCY	Burst efficiency metric
0xA-0xE	Reserved	Future expansion
0xF	AXI_PERF_USER_DEFINED	User-defined performance

26.7.2 APB Protocol Events

26.7.2.1 Error Events (*PktTypeError* + *PROTOCOL_APB*)

Error Events

Code	Event Name	Description
0x0	APB_ERR_PSLVERR	Peripheral slave error
0x1	APB_ERR_SETUP_VIOLATION	Setup phase protocol violation
0x2	APB_ERR_ACCESS_VIOLATION	Access phase protocol violation
0x3	APB_ERR_STROBE_ERROR	Write strobe error

Code	Event Name	Description
0x4	APB_ERR_ADDR_DEC ODE	Address decode error
0x5	APB_ERR_PROT_VIO LATION	Protection violation (PPROT)
0x6	APB_ERR_ENABLE_E RROR	Enable phase error
0x7	APB_ERR_READY_ER ROR	PREADY protocol error
0x8-0xE	Reserved	Future expansion
0xF	APB_ERR_USER_DEF INED	User-defined error

26.7.2.2 Timeout Events (*PktTypeTimeout* + *PROTOCOL_APB*)

Timeout Events

Code	Event Name	Description
0x0	APB_TIMEOUT_SETU P	Setup phase timeout
0x1	APB_TIMEOUT_ACCE SS	Access phase timeout
0x2	APB_TIMEOUT_ENA BLE	Enable phase timeout (PREADY stuck)
0x3	APB_TIMEOUT_PREA DY_STUCK	PREADY stuck low
0x4	APB_TIMEOUT_TRA NSFER	Overall transfer timeout
0x5-0xE	Reserved	Future expansion
0xF	APB_TIMEOUT_USER _DEFINED	User-defined timeout

26.7.2.3 Completion Events (*PktTypeCompletion* + *PROTOCOL_APB*)

Completion Events

Code	Event Name	Description
0x0	APB_COMPL_TRANS	Transaction

Code	Event Name	Description
	_COMPLETE	completed
0x1	APB_COMPL_READ_COMPLETE	Read transaction complete
0x2	APB_COMPL_WRITE_COMPLETE	Write transaction complete
0x3-0xE	Reserved	Future expansion
0xF	APB_COMPL_USER_DEFINED	User-defined completion

26.7.2.4 Threshold Events (*PktTypeThreshold* + *PROTOCOL_APB*)

Threshold Events

Code	Event Name	Description
0x0	APB_THRESH_LATE_NCY	APB latency threshold
0x1	APB_THRESH_ERROR_RATE	APB error rate threshold
0x2	APB_THRESH_ACCESS_COUNT	Access count threshold
0x3	APB_THRESH_BANDWIDTH	Bandwidth threshold
0x4-0xE	Reserved	Future expansion
0xF	APB_THRESH_USER_DEFINED	User-defined threshold

26.7.2.5 Performance Events (*PktTypePerf* + *PROTOCOL_APB*)

Performance Events

Code	Event Name	Description
0x0	APB_PERF_READ_LATENCY	Read transaction latency
0x1	APB_PERF_WRITE_LATENCY	Write transaction latency
0x2	APB_PERF_THROUGHPUT	Transaction throughput

Code	Event Name	Description
0x3	APB_PERF_ERROR_RATE	Error rate
0x4	APB_PERF_ACTIVE_COUNT	Active transaction count
0x5	APB_PERF_COMPLETED_COUNT	Completed transaction count
0x6-0xE	Reserved	Future expansion
0xF	APB_PERF_USER_DEFINED	User-defined performance

26.7.2.6 *Debug Events (PktTypeDebug + PROTOCOL_APB)*

Debug Events

Code	Event Name	Description
0x0	APB_DEBUG_STATE_CHANGE	APB state changed
0x1	APB_DEBUG_SETUP_PHASE	Setup phase event
0x2	APB_DEBUG_ACCESS_PHASE	Access phase event
0x3	APB_DEBUG_ENABLE_PHASE	Enable phase event
0x4	APB_DEBUG_PSEL_TRACE	PSEL trace
0x5	APB_DEBUG_PENABLE_TRACE	PENABLE trace
0x6	APB_DEBUG_PREADY_TRACE	PREADY trace
0x7	APB_DEBUG_PPROT_TRACE	PPROT trace
0x8	APB_DEBUG_PSTRB_TRACE	PSTRB trace
0x9-0xE	Reserved	Future expansion
0xF	APB_DEBUG_USER_DEFINED	User-defined debug

Code	Event Name	Description
	EFINED	

26.7.3 MNOC Protocol Events

26.7.3.1 Error Events (*PktTypeError* + *PROTOCOL_MNOC*)

Error Events

Code	Event Name	Description
0x0	MNOC_ERR_PARITY	Parity error
0x1	MNOC_ERR_PROTOCOL	Protocol violation
0x2	MNOC_ERR_OVERFLOW	Buffer/Credit overflow
0x3	MNOC_ERR_UNDERFLOW	Buffer/Credit underflow
0x4	MNOC_ERR_ORPHAN	Orphaned packet/ACK
0x5	MNOC_ERR_INVALID	Invalid type/channel/payload
0x6	MNOC_ERR_HEADER_CRC	Header CRC error
0x7	MNOC_ERR_PAYLOAD_CRC	Payload CRC error
0x8	MNOC_ERR_SEQUENCE	Sequence number error
0x9	MNOC_ERR_ROUTE	Routing error
0xA	MNOC_ERR_DEADLOCK	Deadlock detected
0xB-0xE	Reserved	Future expansion
0xF	MNOC_ERR_USER_DEFINED	User-defined error
	EFINED	

26.7.3.2 Credit Events (*PktTypeCredit* + *PROTOCOL_MNOC*)

Credit Events

Code	Event Name	Description
0x0	MNOC_CREDIT_ALL OCATED	Credits allocated
0x1	MNOC_CREDIT_CON SUMED	Credits consumed
0x2	MNOC_CREDIT_RET URNED	Credits returned
0x3	MNOC_CREDIT_OVE RFLOW	Credit overflow detected
0x4	MNOC_CREDIT_UND ERFLOW	Credit underflow detected
0x5	MNOC_CREDIT_EXH AUSTED	All credits exhausted
0x6	MNOC_CREDIT_REST ORED	Credits restored
0x7	MNOC_CREDIT_EFFI CIENCY	Credit efficiency metric
0x8	MNOC_CREDIT_LEA K	Credit leak detected
0x9-0xE	Reserved	Future expansion
0xF	MNOC_CREDIT_USE R_DEFINED	User-defined credit event

26.7.3.3 Channel Events (*PktTypeChannel* + *PROTOCOL_MNOC*)

Channel Events

Code	Event Name	Description
0x0	MNOC_CHANNEL_O PEN	Channel opened
0x1	MNOC_CHANNEL_CL OSE	Channel closed
0x2	MNOC_CHANNEL_ST ALL	Channel stalled

Code	Event Name	Description
0x3	MNOC_CHANNEL_RESUME	Channel resumed
0x4	MNOC_CHANNEL_CONGESTION	Channel congestion detected
0x5	MNOC_CHANNEL_PRIORITY	Channel priority change
0x6-0xE	Reserved	Future expansion
0xF	MNOC_CHANNEL_USER_DEFINED	User-defined channel event

26.7.3.4 Stream Events (*PktTypeStream* + *PROTOCOL_MNOC*)

Stream Events

Code	Event Name	Description
0x0	MNOC_STREAM_START	Stream started
0x1	MNOC_STREAM_END	Stream ended (EOS)
0x2	MNOC_STREAM_PAUSE	Stream paused
0x3	MNOC_STREAM_RESUME	Stream resumed
0x4	MNOC_STREAM_OVERFLOW	Stream buffer overflow
0x5	MNOC_STREAM_UNDERFLOW	Stream buffer underflow
0x6-0xE	Reserved	Future expansion
0xF	MNOC_STREAM_USER_DEFINED	User-defined stream event

26.7.4 ARB Protocol Events

26.7.4.1 Error Events (*PktTypeError* + *PROTOCOL_ARB*)

Error Events

Code	Event Name	Description
0x0	ARB_ERR_STARVATION	Client request starvation
0x1	ARB_ERR_ACK_TIMEOUT	Grant ACK timeout
0x2	ARB_ERR_PROTOCOL_VIOLATION	ACK protocol violation
0x3	ARB_ERR_CREDIT_VIOLATION	Credit system violation
0x4	ARB_ERR_FAIRNESS_VIOLATION	Weighted fairness violation
0x5	ARB_ERR_WEIGHT_UNDERFLOW	Weight credit underflow
0x6	ARB_ERR_CONCURRENT_GRANTS	Multiple simultaneous grants
0x7	ARB_ERR_INVALID_GRANT_ID	Invalid grant ID detected
0x8	ARB_ERR_ORPHAN_ACK	ACK without pending grant
0x9	ARB_ERR_GRANT_OVERLAP	Overlapping grant periods
0xA	ARB_ERR_MASK_ERROR	Round-robin mask error
0xB	ARB_ERR_STATE_MACHINE	FSM state error
0xC	ARB_ERR_CONFIGURATION	Invalid configuration
0xD-0xE	Reserved	Future expansion
0xF	ARB_ERR_USER_DEFINED	User-defined error

26.7.4.2 Timeout Events (*PktTypeTimeout* + *PROTOCOL_ARB*)

Timeout Events

Code	Event Name	Description
0x0	ARB_TIMEOUT_GRANT_ACK	Grant ACK timeout
0x1	ARB_TIMEOUT_REQUEST_HOLD	Request held too long
0x2	ARB_TIMEOUT_WEIGHT_UPDATE	Weight update timeout
0x3	ARB_TIMEOUT_BLOCK_RELEASE	Block release timeout
0x4	ARB_TIMEOUT_CREDIT_UPDATE	Credit update timeout
0x5	ARB_TIMEOUT_STATE_MACHINE_CHANGE	State machine timeout
0x6-0xE	Reserved	Future expansion
0xF	ARB_TIMEOUT_USER_DEFINED	User-defined timeout

26.7.4.3 Completion Events (*PktTypeCompletion* + *PROTOCOL_ARB*)

Completion Events

Code	Event Name	Description
0x0	ARB_COMPL_GRANT_ISSUED	Grant successfully issued
0x1	ARB_COMPL_ACK_RECEIVED	ACK successfully received
0x2	ARB_COMPL_TRANSACTION	Complete transaction (grant+ack)
0x3	ARB_COMPL_WEIGHT_UPDATE	Weight update completed
0x4	ARB_COMPL_CREDIT_CYCLE	Credit cycle completed
0x5	ARB_COMPL_FAIRNESS	Fairness analysis

Code	Event Name	Description
	ESS_PERIOD	period
0x6	ARB_COMPL_BLOCK_PERIOD	Block period completed
0x7-0xE	Reserved	Future expansion
0xF	ARB_COMPL_USER_DEFINED	User-defined completion

26.7.4.4 Threshold Events (*PktTypeThreshold* + *PROTOCOL_ARB*)

Threshold Events

Code	Event Name	Description
0x0	ARB_THRESH_REQUEST_LATENCY	Request-to-grant latency threshold
0x1	ARB_THRESH_ACK_LATENCY	Grant-to-ACK latency threshold
0x2	ARB_THRESH_FAIRNESS_DEVIATION	Fairness deviation threshold
0x3	ARB_THRESH_ACTIVE_REQUESTS	Active request count threshold
0x4	ARB_THRESH_GRANT_RATE	Grant rate threshold
0x5	ARB_THRESH_EFFICIENCY	Grant efficiency threshold
0x6	ARB_THRESH_CREDIT_LOW	Low credit threshold
0x7	ARB_THRESH_WEIGHT_IMBALANCE	Weight imbalance threshold
0x8	ARB_THRESH_STARVATION_TIME	Starvation time threshold
0x9-0xE	Reserved	Future expansion
0xF	ARB_THRESH_USER_DEFINED	User-defined threshold

26.7.4.5 Performance Events (*PktTypePerf* + *PROTOCOL_ARB*)

Performance Events

Code	Event Name	Description
0x0	ARB_PERF_GRANT_ISSUED	Grant issued event
0x1	ARB_PERF_ACK_RECEIVED	ACK received event

Code	Event Name	Description
0x2	ARB_PERF_GRANT_EFFICIENCY	Grant completion efficiency
0x3	ARB_PERF_FAIRNESS_METRIC	Fairness compliance metric
0x4	ARB_PERF_THROUGHPUT	Arbitration throughput
0x5	ARB_PERF_LATENCY_AVG	Average latency measurement
0x6	ARB_PERF_WEIGHT_COMPLIANCE	Weight compliance metric
0x7	ARB_PERF_CREDIT_UTILIZATION	Credit utilization efficiency
0x8	ARB_PERF_CLIENT_ACTIVITY	Per-client activity metric
0x9	ARB_PERF_STARVATION_COUNT	Starvation event count
0xA	ARB_PERF_BLOCK_EFFICIENCY	Block/unblock efficiency
0xB-0xE	Reserved	Future expansion
0xF	ARB_PERF_USER_DEFINED	User-defined performance

26.7.4.6 *Debug Events (PktTypeDebug + PROTOCOL_ARB)*

Debug Events

Code	Event Name	Description
0x0	ARB_DEBUG_STATE_CHANGE	Arbiter state machine change
0x1	ARB_DEBUG_MASK_UPDATE	Round-robin mask update
0x2	ARB_DEBUG_WEIGHT_CHANGE	Weight configuration change
0x3	ARB_DEBUG_CREDIT_UPDATE	Credit level update
0x4	ARB_DEBUG_CLIENT_MASK	Client enable/disable mask
0x5	ARB_DEBUG_PRIORITY	Priority level change

Code	Event Name	Description
	TY_CHANGE	
0x6	ARB_DEBUG_BLOCK_EVENT	Block/unblock event
0x7	ARB_DEBUG_QUEUE_STATUS	Request queue status
0x8	ARB_DEBUG_COUNTER_SNAPSHOT	Counter values snapshot
0x9	ARB_DEBUG_FIFO_STATUS	FIFO status change
0xA	ARB_DEBUG_FAIRNESS_STATE	Fairness tracking state
0xB	ARB_DEBUG_ACK_PROTOCOL_STATE	ACK protocol state
0xC-0xE	Reserved	Future expansion
0xF	ARB_DEBUG_USER_DEFINED	User-defined debug

26.7.5 CORE Protocol Events

26.7.5.1 Error Events (*PktTypeError* + *PROTOCOL_CORE*)

Error Events

Code	Event Name	Description
0x0	CORE_ERR_DESCRIPTOR_MALFORMED	Missing magic number (0x900dc0de)
0x1	CORE_ERR_DESCRIPTOR_BAD_ADDR	Invalid descriptor address
0x2	CORE_ERR_DATA_BAD_ADDR	Invalid data address (fetch or runtime)
0x3	CORE_ERR_FLAG_COMPARISON	Flag mask/compare mismatch
0x4	CORE_ERR_CREDIT_UNDERFLOW	Credit system violation
0x5	CORE_ERR_STATE_MACHINE	Invalid FSM state transition
0x6	CORE_ERR_DESCRIPTOR_ENGINE	Descriptor engine FSM error

Code	Event Name	Description
	INE	
0x7	CORE_ERR_FLAG_ENGINE	Flag engine FSM error
0x8	CORE_ERR_PROGRAM_ENGIN E	Program engine FSM error
0x9	CORE_ERR_DATA_ENGINE	Data engine error
0xA	CORE_ERR_CHANNEL_INVAL ID	Invalid channel ID
0xB	CORE_ERR_CONTROL_VIOLA TION	Control register violation
0xC-0xE	Reserved	Future expansion
0xF	CORE_ERR_USER_DEFINED	User-defined error

26.7.5.2 Timeout Events (*PktTypeTimeout* + *PROTOCOL_CORE*)

Timeout Events

Code	Event Name	Description
0x0	CORE_TIMEOUT_DESCRIPTOR_FETCH	Descriptor fetch timeout
0x1	CORE_TIMEOUT_FLAG_RETRY	Flag comparison retry timeout
0x2	CORE_TIMEOUT_PROGRAM_WRITE	Program write timeout
0x3	CORE_TIMEOUT_DATA_TRANSFER	Data transfer timeout
0x4	CORE_TIMEOUT_CREDIT_WAIT	Credit wait timeout
0x5	CORE_TIMEOUT_CONTROL_WAIT	Control enable wait timeout
0x6	CORE_TIMEOUT_ENGINE_RESPONSE	Sub-engine response timeout
0x7	CORE_TIMEOUT_STATE_TRANSITION	FSM state transition timeout
0x8-0xE	Reserved	Future expansion
0xF	CORE_TIMEOUT_USER_DEFI	User-defined timeout

Code	Event Name	Description
	NED	

26.7.5.3 Completion Events (*PktTypeCompletion* + *PROTOCOL_CORE*)

Completion Events

Code	Event Name	Description
0x0	CORE_COMPL_DESCRIPTOR_LOADED	Descriptor successfully loaded
0x1	CORE_COMPL_DESCRIPTOR_CHAIN	Descriptor chain completed
0x2	CORE_COMPL_FLAG_MATCHED	Flag comparison successful
0x3	CORE_COMPL_PROGRAM_COMPLETED	Post-programming completed
0x4	CORE_COMPL_DATA_TRANSFER	Data transfer completed
0x5	CORE_COMPL_CREDIT_CYCLE	Credit cycle completed
0x6	CORE_COMPL_CHANNEL_COMPLETE	Channel processing complete
0x7	CORE_COMPL_ENGINE_READY	Sub-engine ready
0x8-0xE	Reserved	Future expansion
0xF	CORE_COMPL_USER_DEFINED	User-defined completion

26.7.5.4 Threshold Events (*PktTypeThreshold* + *PROTOCOL_CORE*)

Threshold Events

Code	Event Name	Description
0x0	CORE_THRESH_DESCRIPTOR_QUEUE	Descriptor queue depth threshold
0x1	CORE_THRESH_CREDIT_LOW	Credit low threshold
0x2	CORE_THRESH_FLAG_RETRY_COUNT	Flag retry count threshold

Code	Event Name	Description
0x3	CORE_THRESH_LATENCY	Processing latency threshold
0x4	CORE_THRESH_ERROR_RATE	Error rate threshold
0x5	CORE_THRESH_THROUGHPUT	Throughput threshold
0x6	CORE_THRESH_ACTIVE_CHANNELS	Active channel count threshold
0x7	CORE_THRESH_PROGRAM_LATENCY	Program write latency threshold
0x8	CORE_THRESH_DATA_RATE	Data transfer rate threshold
0x9-0xE	Reserved	Future expansion
0xF	CORE_THRESH_USER_DEFINED	User-defined threshold

26.7.5.5 Performance Events (PktTypePerf + PROTOCOL_CORE)

Performance Events

Code	Event Name	Description
0x0	CORE_PERF_END_OF_DATA	Stream continuation signal
0x1	CORE_PERF_END_OF_STREAM	Stream termination signal
0x2	CORE_PERF_ENTERING_IDLE	FSM returning to idle
0x3	CORE_PERF_CREDIT_INCREMENTED	Credit added by software
0x4	CORE_PERF_CREDIT_EXHAUSTED	Credit blocking execution
0x5	CORE_PERF_STATE_TRANSITION	FSM state change
0x6	CORE_PERF_DESCRIPTOR_ACTIVE	Data processing started
0x7	CORE_PERF_FLAG_RETRY	Flag comparison retry
0x8	CORE_PERF_CHANNEL	Channel enabled by

Code	Event Name	Description
	EL_ENABLE	software
0x9	CORE_PERF_CHANN EL_DISABLE	Channel disabled by software
0xA	CORE_PERF_CREDIT_ UTILIZATION	Credit utilization metric
0xB	CORE_PERF_PROCES SING_LATENCY	Total processing latency
0xC	CORE_PERF_QUEUE_ DEPTH	Current queue depth
0xD-0xE	Reserved	Future expansion
0xF	CORE_PERF_USER_D EFINED	User-defined performance

26.7.5.6 *Debug Events (PktTypeDebug + PROTOCOL_CORE)*

Debug Events

Code	Event Name	Description
0x0	CORE_DEBUG_FSM_STATE_C HANGE	Descriptor FSM state change
0x1	CORE_DEBUG_DESCRIPTOR_ CONTENT	Descriptor content trace
0x2	CORE_DEBUG_FLAG_ENGINE _STATE	Flag engine state trace
0x3	CORE_DEBUG_PROGRAM_EN GINE_STATE	Program engine state trace
0x4	CORE_DEBUG_CREDIT_OPER ATION	Credit system operation
0x5	CORE_DEBUG_CONTROL_REG ISTER	Control register access
0x6	CORE_DEBUG_ENGINE_HAN DSHAKE	Engine handshake trace
0x7	CORE_DEBUG_QUEUE_STATU S	Queue status change
0x8	CORE_DEBUG_COUNTER_SNA	Counter values snapshot

Code	Event Name	Description
	PSHOT	
0x9	CORE_DEBUG_ADDRESS_TRACE	Address progression trace
0xA	CORE_DEBUG_PAYLOAD_TRACE	Payload content trace
0xB-0xE	Reserved	Future expansion
0xF	CORE_DEBUG_USER_DEFINED	User-defined debug

26.8 Memory Architecture and Packet Routing

26.8.1 Two-Tier Memory Architecture

26.8.1.1 Local Error/Interrupt Memory

Local Error/Interrupt Memory

Characteristic	Description
Storage Types	Error Packets (Type 0x0) and Timeout Packets (Type 0x3)
Access Method	Immediate CPU access without memory subsystem delays
Capacity	Large enough to prevent overflow during error bursts
Priority	Critical events requiring immediate attention
Indexing	Fast search and retrieval mechanisms

26.8.1.2 Configurable External Memory

Configurable External Memory

Characteristic	Description
Storage Types	Performance, Completion, Threshold, Debug packets
Access Method	Base and limit registers define memory regions

Characteristic	Description
Capacity	Bulk storage for non-critical events
DMA Support	Can be accessed via DMA for efficient transfer
Time Stamping	32-bit timestamp appended when routing externally

26.8.2 Routing Configuration

26.8.2.1 Base and Limit Registers

Base and Limit Registers

Register Set	Purpose	Configuration
Completion Config	Type 0x1 routing	base_addr, limit_addr, enable, priority
Threshold Config	Type 0x2 routing	base_addr, limit_addr, enable, priority
Performance Config	Type 0x4 routing	base_addr, limit_addr, enable, priority
Debug Config	Type 0xF routing	base_addr, limit_addr, enable, priority

26.8.2.2 Routing Decision Logic

Routing Decision Logic

Packet Type	Destination	Address Calculation
Error (0x0)	Local Memory	local_error_write_pointer
Timeout (0x3)	Local Memory	local_error_write_pointer
Completion (0x1)	External Memory	completion_config.base_addr + offset
Performance (0x4)	External Memory	performance_config.base_addr + offset
Debug (0xF)	External Memory	debug_config.base_addr + offset

26.8.3 Address Space Management

26.8.3.1 Memory Layout Example

Memory Layout Example

Address Range	Usage	Description
0x1000_0000 - 0x1000_FFFF	Local Error Memory	Immediate access storage
0x2000_0000 - 0x2001_FFFF	Performance Packets	External bulk storage
0x2010_0000 - 0x2011_FFFF	Completion Packets	External bulk storage
0x2020_0000 - 0x202F_FFFF	Debug Packets	External bulk storage

26.8.4 Transaction State and Bus Transaction Structure

26.8.4.1 Transaction State Enumeration

Transaction State Enumeration

State	Value	Description	Usage
TRANS_EMPTY	3'b000	Unused entry	Available slot
TRANS_ADDR_PHASE	3'b001	Address phase active (AXI) / Packet sent (MNOC) / Setup phase (APB)	Initial phase
TRANS_DATA_PHASE	3'b010	Data phase active (AXI) / Waiting for ACK (MNOC) / Access phase (APB)	Data transfer
TRANS_RESP_PHASE	3'b011	Response phase active (AXI) / ACK received (MNOC) / Enable phase (APB)	Response handling
TRANS_COMPLETED	3'b100	Transaction complete	Successful completion
TRANS_ERROR	3'b101	Transaction has error	Error condition
TRANS_ORPHANED	3'b110	Orphaned transaction	Missing components

State	Value	Description	Usage
TRANS_CRED IT_STALL	3'b111	Credit stall (MNOC only)	MNOC-specific stall

26.8.4.2 Enhanced Transaction Structure

Enhanced Transaction Structure

Field	Width	Description	Protocol Usage
valid	1	Entry is valid	All protocol's
protocol	3	Protocol type (AXI/MNOC/APB/ARB/CORE)	All protocols
state	3	Transaction state	All protocols
id	32	Transaction ID (AXI) / Sequence (MNOC) / PSEL encoding (APB)	All protocols
addr	64	Transaction address / Channel addr / PADDR	All protocols
len	8	Burst length (AXI) / Packet count (MNOC) / Always 0 (APB)	AXI, MNOC
size	3	Access size (AXI) / Reserved (MNOC) / Transfer size (APB)	AXI, APB
burst	2	Burst type (AXI) / Payload type (MNOC) / PPROT[1:0] (APB)	All protocols

26.8.4.3 Phase Completion Flags

Phase Completion Flags

Flag	Description	Protocol Usage
cmd_receiv ed	Address phase received / Packet sent / Setup phase	All protocols
data_starte d	Data phase started / ACK expected / Access phase	All protocols
data_comp	Data phase completed / ACK	All protocols

Flag	Description	Protocol Usage
leted	received / Enable phase	
resp_recei ved	Response received / Final ACK / PREADY asserted	All protocols

26.8.4.4 Protocol-Specific Tracking Fields

Protocol-Specific Tracking Fields

Field	Width	Description	Protocol
channel	6	Channel ID (AXI ID / MNOC channel / PSEL bit position)	All protocols
eos_seen	1	EOS marker seen	MNOC only
parity_error	1	Parity error detected	MNOC only
credit_at_sta rt	8	Credits available at start	MNOC only
retry_count	3	Number of retries	MNOC only
desc_addr_ match	1	Descriptor address match detected	AXI only
data_addr_ match	1	Data address match detected	AXI only
apb_phase	2	Current APB phase	APB only
pslverr_seen	1	PSLVERR detected	APB only
pprot_value	3	PPROT value	APB only
pstrb_value	4	PSTRB value for writes	APB only
arb_grant_id	8	Current grant ID	ARB only
arb_weight	8	Current weight value	ARB only
core_fsm_st ate	3	Current CORE FSM state	CORE only
core_channe l_id	6	CORE channel identifier	CORE only

26.8.5 APB Transaction Phases

APB Transaction Phases

Phase	Value	Description
APB_PHASE_IDLE	2'b00	Bus idle
APB_PHASE_SETUP	2'b01	Setup phase (PSEL asserted)
APB_PHASE_ACCESS	2'b10	Access phase (PENABLE asserted)
APB_PHASE_ENABLE	2'b11	Enable phase (waiting for PREADY)

26.8.6 APB Protection Types

APB Protection Types

Protection	Value	Description
APB_PROT_NORMAL	3'b000	Normal access
APB_PROT_PRIVILEGED	3'b001	Privileged access
APB_PROT_SECURE	3'b010	Secure access
APB_PROT_INSTRUCTION	3'b100	Instruction access

26.8.7 MNOC Payload Types

MNOC Payload Types

Payload	Value	Description
MNOC_PAYLOAD_CONFIG	2'b00	CONFIG_PKT
MNOC_PAYLOAD_TS	2'b01	TS_PKT
MNOC_PAYLOAD_RAW	2'b11	RAW_PKT

26.8.8 MNOC ACK Types

MNOC ACK Types

ACK Type	Value	Description
MNOC_ACK_STOP	2'b00	MSAP_STOP
MNOC_ACK_START	2'b01	MSAP_START
MNOC_ACK_CREDIT_ON	2'b10	MSAP_CREDIT_ON
MNOC_ACK_STOP_AT_EOS	2'b11	MSAP_STOP_AT_EOS

26.8.9 ARB State Types

ARB State Types

State	Value	Description
ARB_STATE_IDLE	3'b000	Idle state
ARB_STATE_ARBITRATE	3'b001	Performing arbitration
ARB_STATE_GRANT	3'b010	Grant issued, waiting for ACK
ARB_STATE_BLOCKED	3'b011	Arbitration blocked
ARB_STATE_WEIGHT_UPDATE	3'b100	Weight update in progress
ARB_STATE_ERROR	3'b101	Error state

26.8.10 CORE State Types

CORE State Types

State	Value	Description
CORE_STATE_IDLE	3'b000	Idle state
CORE_STATE_DESCRIPTOR_FETCH	3'b001	Fetching descriptor
CORE_STATE_FLAG_CHECK	3'b010	Checking flag condition
CORE_STATE_PROGRAM	3'b011	Writing program

State	Value	Description
RAM_WRITE		
CORE_STATE_DATA_TRANSFER	3'b100	Transferring data
CORE_STATE_CREDIT_WAIT	3'b101	Waiting for credits
CORE_STATE_ERROR	3'b110	Error state

26.9 Configuration and Control

26.9.1 Monitor Configuration Registers

26.9.1.1 Global Configuration

Global Configuration

Field	Width	Description
monitor_enable	1	Global monitor enable
error_local_enable	1	Enable local error storage
external_route_enable	1	Enable external routing
unit_id	4	Unit identifier
agent_id	8	Agent identifier
packet_type_enable	16	Per-type enable bits

26.9.1.2 Packet Type Enable Mapping

Packet Type Enable Mapping

Bit	Enable	Description
0	PKT_ENABLE_ERROR	Enable error packets
1	PKT_ENABLE_COMPLETION	Enable completion packets
2	PKT_ENABLE_THRESHOLD	Enable threshold

Bit	Enable	Description
	HOLD	packets
3	PKT_ENABLE_TIME OUT	Enable timeout packets
4	PKT_ENABLE_PERF	Enable performance packets
5	PKT_ENABLE_CREDI T	Enable credit packets (MNOG)
6	PKT_ENABLE_CHAN NEL	Enable channel packets (MNOG)
7	PKT_ENABLE_STREA M	Enable stream packets (MNOG)
8	PKT_ENABLE_ADDR_ MATCH	Enable address match (AXI)
9	PKT_ENABLE_APB	Enable APB packets
15	PKT_ENABLE_DEBU G	Enable debug packets

26.9.2 Protocol-Specific Configuration

26.9.2.1 AXI Monitor Configuration

AXI Monitor Configuration

Field	Width	Description
active_trans_thresh old	16	Active transaction threshold
latency_threshold	32	Latency threshold (cycles)
addr_timeout_cnt	4	Address timeout count
data_timeout_cnt	4	Data timeout count
resp_timeout_cnt	4	Response timeout count
burst_boundary_ch eck	1	Enable burst boundary checking
address_match_ena	1	Enable address

Field	Width	Description
ble		matching
desc_addr_match_base	64	Descriptor address match base
desc_addr_match_mask	64	Descriptor address match mask
data_addr_match_base	64	Data address match base
data_addr_match_mask	64	Data address match mask

26.9.2.2 MNOC Monitor Configuration

MNOC Monitor Configuration

Field	Width	Description
credit_low_threshold	8	Credit low threshold
packet_rate_threshold	16	Packet rate threshold
max_route_hops	8	Maximum routing hops
enable_credit_tracking	1	Enable credit tracking
enable_deadlock_detection	1	Enable deadlock detection
deadlock_timeout	4	Deadlock detection timeout

26.9.2.3 ARB Monitor Configuration

ARB Monitor Configuration

Field	Width	Description
grant_timeout_cnt	16	Grant ACK timeout count
fairness_window	32	Fairness analysis window
weight_update_enable	1	Enable weight

Field	Width	Description
ble		tracking
starvation_threshold	16	Starvation detection threshold
efficiency_threshold	8	Grant efficiency threshold

26.9.2.4 CORE Monitor Configuration

CORE Monitor Configuration

Field	Width	Description
descriptor_timeout_cnt	16	Descriptor fetch timeout count
flag_retry_limit	8	Maximum flag retry count
credit_low_threshold	8	Credit low threshold
processing_timeout_cnt	32	Processing timeout count
enable_descriptor_trace	1	Enable descriptor content tracing
enable_fsm_trace	1	Enable FSM state tracing

26.10 Validation Requirements

26.10.1 Functional Validation

Functional Validation

Validation Area	Requirements
Packet Format	Verify 64-bit packet structure and field encoding
Event Organization	Verify hierarchical event code organization
Protocol Isolation	Verify independent protocol event spaces

Validation Area	Requirements
Routing Logic	Verify packet routing based on type and configuration
Memory Management	Verify local and external memory operations
Configuration	Verify register configuration and enable controls

26.10.2 Performance Validation

Performance Validation

Validation Area	Requirements
Throughput	Verify monitor bus can handle peak event rates
Latency	Verify low-latency path for critical events
Memory Efficiency	Verify efficient memory usage patterns
Power Consumption	Verify power-efficient operation

26.10.3 Error Handling Validation

Error Handling Validation

Validation Area	Requirements
Error Injection	Verify error detection and reporting
Overflow Handling	Verify behavior when memories fill
Configuration Errors	Verify invalid configuration detection
Recovery Mechanisms	Verify error recovery procedures

26.11 Usage Examples

26.11.1 Creating Monitor Packets

Creating Monitor Packets

Packet Type	Example Usage
AXI Error	Protocol=AXI, Type=Error, Code=AXI_ERR_RESP_SLVERR
MNOC Credit	Protocol=MNOC, Type=Credit, Code=MNOC_CREDIT_EXHAUSTED
APB Performance	Protocol=APB, Type=Performance, Code=APB_PERF_TOTAL_LATENCY
ARB Threshold	Protocol=ARB, Type=Threshold, Code=ARB_THRESH_FAIRNESS_DEV
CORE Completion	Protocol=CORE, Type=Completion, Code=CORE_COMPL_DESCRIPTOR_LOAD ED

26.11.2 Packet Decoding

Packet Decoding

Decoding Step	Method
Extract Type	packet[63:60]
Extract Protocol	packet[59:57]
Extract Event Code	packet[56:53]
Extract Channel ID	packet[52:47]
Extract Event Data	packet[34:0]

26.11.3 Monitor Bus Packet Helper Functions

26.11.3.1 Packet Field Extraction

Packet Field Extraction

Function	Return Type	Description
get_packet_type(pk t)	logic [3:0]	Extract packet type [63:60]
get_protocol_type(pkt)	protocol_type_t	Extract protocol [59:57]

Function	Return Type	Description
get_event_code(pkt))	logic [3:0]	Extract event code [56:53]
get_channel_id(pkt))	logic [5:0]	Extract channel ID [52:47]
get_unit_id(pkt)	logic [3:0]	Extract unit ID [46:43]
get_agent_id(pkt)	logic [7:0]	Extract agent ID [42:35]
get_event_data(pkt))	logic [34:0]	Extract event data [34:0]

26.11.3.2 Packet Creation Function

Packet Creation Function

Function	Parameters	Description
create_monitor_packet()	packet_type, protocol, event_code, channel_id, unit_id, agent_id, event_data	Create complete 64-bit packet

26.11.3.3 Event Code Creation Functions

Event Code Creation Functions

Function	Parameter	Description
create_axi_error_event()	axi_error_code_t	Create AXI error event code
create_axi_timeout_event()	axi_timeout_code_t	Create AXI timeout event code
create_axi_completion_event()	axi_completion_code_t	Create AXI completion event code
create_axi_threshold_event()	axi_threshold_code_t	Create AXI threshold event code
create_axi_performance_event()	axi_performance_code_t	Create AXI performance event code
create_axi_addr_match_event()	axi_addr_match_code_t	Create AXI address match event code
create_axi_debug_event()	axi_debug_code_t	Create AXI debug event code

Function	Parameter	Description
create_apb_error_event()	apb_error_code_t	Create APB error event code
create_apb_timeout_event()	apb_timeout_code_t	Create APB timeout event code
create_apb_completion_event()	apb_completion_code_t	Create APB completion event code
create_mnoc_error_event()	mnoc_error_code_t	Create MNOC error event code
create_mnoc_timeout_event()	mnoc_timeout_code_t	Create MNOC timeout event code
create_mnoc_completion_event()	mnoc_completion_code_t	Create MNOC completion event code
create_mnoc_credit_event()	mnoc_credit_code_t	Create MNOC credit event code
create_mnoc_channel_event()	mnoc_channel_code_t	Create MNOC channel event code
create_mnoc_stream_event()	mnoc_stream_code_t	Create MNOC stream event code
create_arb_error_event()	arb_error_code_t	Create ARB error event code
create_arb_timeout_event()	arb_timeout_code_t	Create ARB timeout event code
create_arb_completion_event()	arb_completion_code_t	Create ARB completion event code
create_arb_threshold_event()	arb_threshold_code_t	Create ARB threshold event code
create_arb_performance_event()	arb_performance_code_t	Create ARB performance event code
create_arb_debug_event()	arb_debug_code_t	Create ARB debug event code
create_core_error_event()	core_error_code_t	Create CORE error event code
create_core_timeout_event()	core_timeout_code_t	Create CORE timeout event code

Function	Parameter	Description
_event()		code
create_core_completion_event()	core_completion_code_t	Create CORE completion event code
create_core_threshold_event()	core_threshold_code_t	Create CORE threshold event code
create_core_performance_event()	core_performance_code_t	Create CORE performance event code
create_core_debug_event()	core_debug_code_t	Create CORE debug event code

26.11.3.4 Validation Functions

Validation Functions

Function	Parameters	Description
is_valid_event_for_packet_type()	packet_type, protocol, event_code	Validate event code for packet type and protocol

26.11.3.5 String Functions for Debugging

String Functions for Debugging

Function	Parameter	Description
get_axi_error_name()	axi_error_code_t	Get human-readable AXI error name
get_arb_error_name()	arb_error_code_t	Get human-readable ARB error name
get_core_error_name()	core_error_code_t	Get human-readable CORE error name
get_packet_type_name()	logic [3:0]	Get packet type name string
get_protocol_name()	protocol_type_t	Get protocol name string
get_event_name()	packet_type, protocol, event_code	Get comprehensive event name

26.12 Debug and Monitoring Signals

26.12.1 Essential Debug Signals

Essential Debug Signals

Signal	Width	Purpose
debug_packet_counts	32 × 16	Packet count per type
debug_protocol_counts	32 × 5	Packet count per protocol
debug_error_counts	32	Total error packet count
debug_local_memory_level	16	Local memory usage level
debug_external_memory_level	16	External memory usage level

26.12.2 Performance Counters

Performance Counters

Counter	Width	Purpose
total_packets_processed	32	Total packets processed
packets_dropped	32	Packets dropped due to overflow
routing_errors	32	Routing configuration errors
memory_full_events	32	Memory full occurrences

26.13 Protocol Coverage Summary

26.13.1 Complete Protocol Event Matrix

Complete Protocol Event Matrix

Protocol	Error	Timeout	Completion	Threshold	Performance	Debug	Protocol-Specific
AXI	YES	YES	YES 16	YES 16	YES 16	YES	AddrMatch

Protocol	Error	Timeout	Completion	Threshold	Performance	Debug	Protocol-Specific
	16	16				16	YES 16
MNOC	YES 16	YES 16	YES 16	NO 0	NO 0	NO 0	Credit/ Channel/ Stream YES 48
APB	YES 16	YES 16	YES 16	YES 16	YES 16	YES 16	None
ARB	YES 16	YES 16	YES 16	YES 16	YES 16	YES 16	None
CORE	YES 16	YES 16	YES 16	YES 16	YES 16	YES 16	None

26.14 Total Event Codes: 544 defined across all protocols and packet types.

26.15 Revision History

Monitor Bus Architecture and Event Code Organization Revision History

Version	Date	Author	Description
0.90	2025-11-22	seang	Initial block specification
0.91	2026-01-02	seang	Added table captions and figure numbers

Last Updated: 2026-01-02

27 STREAM Register Map

27.1 Overview

The STREAM DMA engine register interface consists of three distinct regions:

1. **Channel Kick-off Registers** (0x000 - 0x03F) - Direct routing to descriptor engines
2. **Configuration and Status Registers** (0x100 - 0x3FF) - PeakRDL-generated base register file
3. **Monitor / Performance Registers** (0x1000+) - Separate stream_mon_regs regfile instantiated at 0x1000 under the same addrmap (one APB slave)

Note: The monitor block lives in a relocatable stream_mon_regs regfile instantiated at offset **0x1000** within the same PeakRDL addrmap and served by the same APB slave. The APB address decode must therefore be **at least 13 bits (8 KB address space)** so the monitor registers at 0x1000+ are reachable (the generated register file uses s_cpuiif_addr[12:0]).

27.2 Address Space Layout

Base Address (configurable parameter)

—	0x000 - 0x03F: Channel Kick-off (apbtodescr.sv routing)
—	— 0x000: CH0_CTRL_LOW - Channel 0 descriptor address [31:0]
—	— 0x004: CH0_CTRL_HIGH - Channel 0 descriptor address [63:32]
—	— 0x008: CH1_CTRL_LOW - Channel 1 descriptor address [31:0]
—	— 0x00C: CH1_CTRL_HIGH - Channel 1 descriptor address [63:32]
—	— 0x010: CH2_CTRL_LOW - Channel 2 descriptor address [31:0]
—	— 0x014: CH2_CTRL_HIGH - Channel 2 descriptor address [63:32]
—	— 0x018: CH3_CTRL_LOW - Channel 3 descriptor address [31:0]
—	— 0x01C: CH3_CTRL_HIGH - Channel 3 descriptor address [63:32]
—	— 0x020: CH4_CTRL_LOW - Channel 4 descriptor address [31:0]
—	— 0x024: CH4_CTRL_HIGH - Channel 4 descriptor address [63:32]
—	— 0x028: CH5_CTRL_LOW - Channel 5 descriptor address [31:0]
—	— 0x02C: CH5_CTRL_HIGH - Channel 5 descriptor address [63:32]
—	— 0x030: CH6_CTRL_LOW - Channel 6 descriptor address [31:0]
—	— 0x034: CH6_CTRL_HIGH - Channel 6 descriptor address [63:32]
—	— 0x038: CH7_CTRL_LOW - Channel 7 descriptor address [31:0]
—	— 0x03C: CH7_CTRL_HIGH - Channel 7 descriptor address [63:32]
—	0x040 - 0x0FF: Reserved
—	0x100 - 0x3FF: Configuration and Status Registers (base regfile)
—	— 0x100 - 0x11F: Global Control and Status
—	— 0x120 - 0x13F: Per-Channel Control
—	— 0x140 - 0x16F: Per-Channel Status

- | | 0x170 - 0x17F: Engine Completion and Error Status
- | | 0x200 - 0x21F: Scheduler Configuration (incl. SCHED_TIMEOUT_LIMIT @ 0x208)
- | | 0x220 - 0x23F: Descriptor Engine Configuration
- | | 0x2A0 - 0x2AF: AXI Transfer Configuration
- | | 0x2B0 - 0x2BF: Performance Profiler Configuration
- | | 0x2C0 - 0x2CF: Channel Observation (OBS_CTRL/OBS_FLAGS/OBS_DATA0/1)
- | 0x1000+: Monitor / Performance Registers (stream_mon_regs regfile, same APB slave)
 - | 0x1000 - 0x1007: Monitor FIFO Status / Count
 - | 0x10C0 - 0x10DF: Descriptor AXI Monitor Configuration
 - | 0x10E0 - 0x10FF: Read Engine AXI Monitor Configuration
 - | 0x1100 - 0x111F: Write Engine AXI Monitor Configuration
 - | 0x1150 - 0x11F4: Per-Monitor Performance Counters

27.3 Register Details

27.3.1 Channel Kick-off Registers (0x000 - 0x03F)

These registers are **NOT** traditional registers. Writes are routed directly to descriptor engine APB ports via `apbtodescr.sv`.

Note: Descriptor addresses are 64-bit (ADDR_WIDTH parameter, default 64). On 32-bit APB bus, each channel requires TWO registers (LOW/HIGH).

Channel Kick-off Registers

Offset	Register	Type	Reset	Description
0x000	CH0_CTRL_LOW	WO	N/A	Channel 0 descriptor address [31:0]
0x004	CH0_CTRL_HIG	WO	N/A	Channel 0 descriptor address [63:32]
0x008	CH1_CTRL_LOW	WO	N/A	Channel 1 descriptor address [31:0]
0x00C	CH1_CTRL_HIG	WO	N/A	Channel 1 descriptor address [63:32]
0x010	CH2_CTRL_LOW	WO	N/A	Channel 2 descriptor address [31:0]
0x014	CH2_CTRL_HIG	WO	N/A	Channel 2 descriptor address [63:32]

Offset	Register	Type	Reset	Description
0x018	CH3_CTRL_LOW	WO	N/A	Channel 3 descriptor address [31:0]
0x01C	CH3_CTRL_HIGH	WO	N/A	Channel 3 descriptor address [63:32]
0x020	CH4_CTRL_LOW	WO	N/A	Channel 4 descriptor address [31:0]
0x024	CH4_CTRL_HIGH	WO	N/A	Channel 4 descriptor address [63:32]
0x028	CH5_CTRL_LOW	WO	N/A	Channel 5 descriptor address [31:0]
0x02C	CH5_CTRL_HIGH	WO	N/A	Channel 5 descriptor address [63:32]
0x030	CH6_CTRL_LOW	WO	N/A	Channel 6 descriptor address [31:0]
0x034	CH6_CTRL_HIGH	WO	N/A	Channel 6 descriptor address [63:32]
0x038	CH7_CTRL_LOW	WO	N/A	Channel 7 descriptor address [31:0]
0x03C	CH7_CTRL_HIGH	WO	N/A	Channel 7 descriptor address [63:32]

Write Behavior: - Descriptor address is 64-bit, split across LOW and HIGH registers - Write to HIGH register triggers descriptor engine kick-off - LOW register write is buffered, HIGH register write initiates transfer - Both registers must be written in order (LOW then HIGH) - Write blocks until descriptor engine accepts (back-pressure) - Read not supported (returns error)

Example:

```
// Start DMA transfer on channel 0
// Descriptor at physical address 0x0000_0001_8000_0000 (64-bit)

// Write lower 32 bits first (buffered)
write32(BASE + CH0_CTRL_LOW, 0x8000_0000);

// Write upper 32 bits second (triggers kick-off)
write32(BASE + CH0_CTRL_HIGH, 0x0000_0001); // Blocks until accepted
```



```
// For descriptors in lower 4GB (typical case):
write32(BASE + CH0_CTRL_LOW, 0x8000_0000);
write32(BASE + CH0_CTRL_HIGH, 0x0000_0000); // Upper 32 bits = 0
```

27.3.2 Global Control and Status (0x100 - 0x11F)

27.3.2.1 GLOBAL_CTRL (0x100)

Master control register for entire STREAM engine.

GLOBAL_CTRL

Bits	Field	Type	Reset	Description
31:2	Reserved	RO	0x0	Reserved
1	GLOBAL_RST	RW	0	Global reset (self-clearing)
0	GLOBAL_EN	RW	0	Global enable (1=enabled, 0=disabled)

Usage:

```
// Enable STREAM engine
write32(BASE + GLOBAL_CTRL, 0x1);

// Reset all channels
write32(BASE + GLOBAL_CTRL, 0x3); // Set both EN and RST
// RST self-clears after one cycle
```

27.3.2.2 GLOBAL_STATUS (0x104)

Overall system status.

GLOBAL_STATUS

Bits	Field	Type	Description
31:1	Reserved	RO	Reserved
0	SYSTEM_IDLE	RO	System idle (all channels idle)

27.3.2.3 VERSION (0x108)

Version and configuration information (read-only).

VERSION

Bits	Field	Type	Value	Description
31:24	Reserved	RO	0x00	Reserved
23:16	NUM_CHANNE LS	RO	0x08	Number of channels (8)
15:8	MAJOR	RO	0x00	Major version (0)
7:0	MINOR	RO	0x5A	Minor version (90 decimal = 0.90)

27.3.3 Per-Channel Control and Status (0x120 - 0x17F)

27.3.3.1 CHANNEL_ENABLE (0x120)

Per-channel enable control (bit vector).

CHANNEL_ENABLE

Bits	Field	Type	Reset	Description
31:8	Reserved	RO	0x0	Reserved
7:0	CH_EN	RW	0x00	Channel enable [7:0] (1=enabled)

Usage:

```
// Enable channels 0, 1, 2
write32(BASE + CHANNEL_ENABLE, 0x07);

// Disable channel 1, keep others
uint32_t val = read32(BASE + CHANNEL_ENABLE);
val &= ~(1 << 1); // Clear bit 1
write32(BASE + CHANNEL_ENABLE, val);
```

27.3.3.2 CHANNEL_RESET (0x124)

Per-channel reset control (bit vector, self-clearing).

CHANNEL_RESET

Bits	Field	Type	Reset	Description
31:8	Reserved	RO	0x0	Reserved
7:0	CH_RST	RW	0x00	Channel reset [7:0] (write 1 to reset)

Usage:

```
// Reset channels 0 and 3
write32(BASE + CHANNEL_RESET, 0x09); // Bits 0 and 3
// Self-clears after reset completes
```

27.3.3.3 CHANNEL_IDLE (0x140)

Per-channel idle status (bit vector, read-only).

CHANNEL_IDLE

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved
7:0	CH_IDLE	RO	Channel idle [7:0] (1=idle, 0=active)

27.3.3.4 DESC_ENGINE_IDLE (0x144)

Per-channel descriptor engine idle status.

DESC_ENGINE_IDLE

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved
7:0	DESC_IDLE	RO	Descriptor engine idle [7:0] (1=idle, 0=active)

27.3.3.5 SCHEDULER_IDLE (0x148)

Per-channel scheduler idle status.

SCHEDULER_IDLE

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved
7:0	SCHED_IDL E	RO	Scheduler idle [7:0] (1=idle, 0=active)

27.3.3.6 CH_STATE[0..7] (0x150 - 0x16C)

Per-channel scheduler FSM state (8 registers, stride 0x4).

CH_STATE[0..7]

Offset	Register	Bits	Field	Type	Description
0x150	CH0_STAT E	31:7	Reserve d	RO	Reserved
		6:0	STATE	RO	Channel 0 scheduler state (one-

Offset	Register	Bits	Field	Type	Description
					hot)
0x154	CH1_STAT E	6:0	STATE	RO	Channel 1 scheduler state (one-hot)
0x158	CH2_STAT E	6:0	STATE	RO	Channel 2 scheduler state (one-hot)
0x15C	CH3_STAT E	6:0	STATE	RO	Channel 3 scheduler state (one-hot)
0x160	CH4_STAT E	6:0	STATE	RO	Channel 4 scheduler state (one-hot)
0x164	CH5_STAT E	6:0	STATE	RO	Channel 5 scheduler state (one-hot)
0x168	CH6_STAT E	6:0	STATE	RO	Channel 6 scheduler state (one-hot)
0x16C	CH7_STAT E	6:0	STATE	RO	Channel 7 scheduler state (one-hot)

State Encoding (One-Hot):

Bit 0 (0x01) = CH_IDLE	- Channel idle, waiting for descriptor
Bit 1 (0x02) = CH_FETCH_DESC	- Fetching descriptor from memory
Bit 2 (0x04) = CH_XFER_DATA	- Concurrent read AND write transfer
Bit 3 (0x08) = CH_COMPLETE	- Transfer complete
Bit 4 (0x10) = CH_NEXT_DESC	- Fetching next chained descriptor
Bit 5 (0x20) = CH_ERROR	- Error state
Bit 6 (0x40) = CH_RESERVED	- Reserved for future use

Note: Only ONE bit should be set at a time (one-hot encoding). Multiple bits set indicates a logic error.

27.3.4 Engine Completion and Error Status (0x170 - 0x17F)

27.3.4.1 SCHED_ERROR (0x170)

Per-channel scheduler error flags.

SCHED_ERROR

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved

Bits	Field	Type	Description
7:0	SCHED_ER R	RO	Scheduler error bits [7:0] (1=error, 0=no error)

27.3.4.2 AXI_RD_COMPLETE (0x174)

Per-channel AXI read engine completion status.

AXI_RD_COMPLETE

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved
7:0	RD_COMPL ETE	RO	Read completion bits [7:0] (1=complete, 0=pending)

27.3.4.3 AXI_WR_COMPLETE (0x178)

Per-channel AXI write engine completion status.

AXI_WR_COMPLETE

Bits	Field	Type	Description
31:8	Reserved	RO	Reserved
7:0	WR_COMP LETE	RO	Write completion bits [7:0] (1=complete, 0=pending)

27.3.5 Monitor FIFO Status (0x1000 - 0x1007)

The **heavy** monitor registers in this block — FIFO status, the DAXMON / RDMON / WRMON enable / mask / timeout configuration, and the latency-histogram readback — are active only when USE_AXI_MONITORS=1. The **datapath perf-window registers** (DAXMON / RDMON / WRMON PERF_CTRL plus their PROD/BP/STARV/IDLE_CYCLES, BEAT/BYTE/BURST_COUNT, and WINDOW_* readbacks) are **always-on** — populated by the lightweight in-core axi_bus_meter regardless of USE_AXI_MONITORS, so a monitors-off production build still measures datapath utilisation.

Monitor register block relocation (v0.93): All monitor and per-monitor performance registers now live in a separate stream_mon_regs regfile instantiated at **0x1000** within the same PeakRDL addrmap (served by the same APB slave). Addresses below reflect the new base; the previous layout placed these registers at 0x180-0x2FF. The mapping is $\text{new} = \text{old} - 0x180 + 0x1000$.

27.3.5.1 MON_FIFO_STATUS (0x1000)

Monitor bus FIFO status indicators.

MON_FIFO_STATUS

Bits	Field	Type	Description
31:4	Reserved	RO	Reserved
3	MON_FIFO_UNF L	RO	FIFO underflow detected (1=error)
2	MON_FIFO_OVF L	RO	FIFO overflow detected (1=error)
1	MON_FIFO_EMPTY	RO	FIFO empty (1=empty, 0=data available)
0	MON_FIFO_FULL	RO	FIFO full (1=full, 0=space available)

27.3.5.2 *MON_FIFO_COUNT (0x1004)*

Monitor bus FIFO entry count.

MON_FIFO_COUNT

Bits	Field	Type	Description
31:16	Reserved	RO	Reserved
15:0	FIFO_COUNT	RO	Number of entries in FIFO [15:0]

27.3.6 Scheduler Configuration (0x200 - 0x21F)

27.3.6.1 *SCHED_TIMEOUT_CYCLES (0x200)*

Timeout threshold for scheduler (global for all channels).

SCHED_TIMEOUT_CYCLES

Bits	Field	Type	Reset	Description
31:0	TIMEOUT_CYCLES	RW	1000	Timeout in clock cycles (32-bit)

27.3.6.2 *SCHED_CONFIG (0x204)*

Scheduler feature enables (global for all channels).

SCHED_CONFIG

Bits	Field	Type	Reset	Description
31:6	Reserved	RO	0x0	Reserved
5	RD_PREFETCH_EN	R W	1	Read-ahead descriptor prefetch (cross-descriptor streaming)
4	PERF_EN	R W	0	Performance monitoring enable
3	COMPL_EN	R W	1	Completion reporting enable
2	ERR_EN	R W	1	Error reporting enable
1	TIMEOUT_EN	R W	1	Timeout detection enable
0	SCHED_EN	R W	1	Scheduler enable

RD_PREFETCH_EN (default **enabled**) turns on read-ahead descriptor prefetch: on a chained legacy descriptor the scheduler read side loads the next descriptor from the descriptor-FIFO head and keeps filling the SRAM while the write side drains the current descriptor, collapsing the per-descriptor boundary bubble to zero. Clear it for lockstep behaviour (useful for an on-silicon A/B on one bitstream). Ignored for EXT (row/col-major) descriptors, which always run lockstep. See the Scheduler block (Section 2.4) and HAS Section 5.1.

27.3.6.3 SCHED_TIMEOUT_LIMIT (0x208)

Consecutive write-progress-timeout windows before the scheduler escalates a channel to a sticky CH_ERROR (global for all channels).

SCHED_TIMEOUT_LIMIT

Bits	Field	Type	Reset	Description
31:8	Reserved	RO	0x0	Reserved
7:0	LIMIT	R W	0x4	Consecutive timeout windows before fatal escalation (0 = never escalate)

Behavior: The write-progress timeout is a recoverable liveness fault. It re-arms each window and only escalates to a fatal, sticky CH_ERROR after LIMIT consecutive windows with no write

progress. LIMIT = 0 disables escalation (pure soft timeout). Any write progress clears the strike counter. See `stream_mas/ch02_blocks/04_scheduler.md`.

27.3.7 Descriptor Engine Configuration (0x220 - 0x23F)

27.3.7.1 *DESCENG_CONFIG (0x220)*

Descriptor engine feature enables (global for all channels).

DESCENG_CONFIG

Bits	Field	Type	Reset	Description
31:6	Reserved	RO	0x0	Reserved
5:2	FIFO_THRES H	RW	0x8	Prefetch FIFO threshold (4 bits)
1	PREFETCH_E N	RW	0	Prefetch enable
0	DESCENG_EN	RW	1	Descriptor engine enable

27.3.7.2 *DESCENG_ADDR0_BASE (0x224)*

Base address for descriptor address range 0 (lower 32 bits).

DESCENG_ADDR0_BASE

Bits	Field	Type	Reset	Description
31:0	ADDR0_BA SE	RW	0x00000000 0	Address range 0 base

27.3.7.3 *DESCENG_ADDR0_LIMIT (0x228)*

Limit address for descriptor address range 0 (lower 32 bits).

DESCENG_ADDR0_LIMIT

Bits	Field	Type	Reset	Description
31:0	ADDR0_LIM IT	RW	0xFFFFFFFF FF	Address range 0 limit

27.3.7.4 *DESCENG_ADDR1_BASE (0x22C)*

Base address for descriptor address range 1 (lower 32 bits).

DESCENG_ADDR1_BASE

Bits	Field	Type	Reset	Description
31:0	ADDR1_BA SE	RW	0x0000000 0	Address range 1 base

27.3.7.5 *DESCENG_ADDR1_LIMIT (0x230)*

Limit address for descriptor address range 1 (lower 32 bits).

DESCENG_ADDR1_LIMIT

Bits	Field	Type	Reset	Description
31:0	ADDR1_LIM IT	RW	0xFFFFFFFF FF	Address range 1 limit

27.3.8 Descriptor AXI Monitor Configuration (0x10C0 - 0x10DF)

27.3.8.1 *DAXMON_ENABLE (0x10C0)*

Descriptor AXI master monitor enable controls.

DAXMON_ENABLE

Bits	Field	Type	Reset	Description
31:5	Reserved	RO	0x0	Reserved
4	PERF_EN	RW	0	Performance packet enable
3	TIMEOUT_E N	RW	1	Timeout detection enable
2	COMPL_EN	RW	0	Completion packet enable
1	ERR_EN	RW	1	Error detection enable
0	MON_EN	RW	1	Master enable for descriptor monitor

27.3.8.2 *DAXMON_TIMEOUT (0x10C4)*

Descriptor AXI monitor timeout threshold.

DAXMON_TIMEOUT

Bits	Field	Type	Reset	Description
31:0	TIMEOUT_CYCL ES	RW	10000	Timeout in clock cycles

27.3.8.3 *DAXMON_LATENCY_THRESH (0x10C8)*

Descriptor AXI monitor latency threshold.

DAXMON_LATENCY_THRESH

Bits	Field	Type	Reset	Description
31:0	LATENCY_THR ESH	RW	5000	Latency threshold in clock cycles

27.3.8.4 *DAXMON_PKT_MASK (0x10CC)*

Descriptor AXI monitor packet type filtering.

DAXMON_PKT_MASK

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:0	PKT_MAS K	RW	0xFFFF	Packet type mask (1=enable, 0=disable)

27.3.8.5 *DAXMON_ERR_CFG (0x10D0)*

Descriptor AXI monitor error selection and filtering.

DAXMON_ERR_CFG

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	ERR_MASK	RW	0xFF	Error type filtering mask
7:4	Reserved	RO	0x0	Reserved
3:0	ERR_SELE CT	RW	0x0	Error type selection

27.3.8.6 *DAXMON_MASK1 (0x10D4)*

Descriptor AXI monitor timeout and completion masks.

DAXMON_MASK1

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	COMPL_MASK	RW	0x00	Completion mask
7:0	TIMEOUT_MA	RW	0xFF	Timeout mask

Bits	Field	Type	Reset	Description
	SK			

27.3.8.7 *DAXMON_MASK2 (0x10D8)*

Descriptor AXI monitor threshold and performance masks.

DAXMON_MASK2

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	PERF_MAS K	RW	0x00	Performanc e mask
7:0	THRESH_M ASK	RW	0xFF	Threshold mask

27.3.8.8 *DAXMON_MASK3 (0x10DC)*

Descriptor AXI monitor address and debug masks.

DAXMON_MASK3

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	DEBUG_MA SK	RW	0x00	Debug mask
7:0	ADDR_MAS K	RW	0xFF	Address mask

27.3.9 Read Engine AXI Monitor Configuration (0x10E0 - 0x10FF)

27.3.9.1 *RDMON_ENABLE (0x10E0)*

Read engine AXI master monitor enable controls.

RDMON_ENABLE

Bits	Field	Type	Reset	Description
31:5	Reserved	RO	0x0	Reserved
4	PERF_EN	RW	0	Performance packet enable
3	TIMEOUT_E	RW	1	Timeout detection enable

Bits	Field	Type	Reset	Description
	N			
2	COMPL_EN	RW	0	Completion packet enable
1	ERR_EN	RW	1	Error detection enable
0	MON_EN	RW	1	Master enable for read monitor

27.3.9.2 *RDMON_TIMEOUT (0x10E4)*

Read engine AXI monitor timeout threshold.

RDMON_TIMEOUT

Bits	Field	Type	Reset	Description
31:0	TIMEOUT_CYCL ES	RW	10000	Timeout in clock cycles

27.3.9.3 *RDMON_LATENCY_THRESH (0x10E8)*

Read engine AXI monitor latency threshold.

RDMON_LATENCY_THRESH

Bits	Field	Type	Reset	Description
31:0	LATENCY_THR ESH	RW	5000	Latency threshold in clock cycles

27.3.9.4 *RDMON_PKT_MASK (0x10EC)*

Read engine AXI monitor packet type filtering.

RDMON_PKT_MASK

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:0	PKT_MAS K	RW	0xFFFF F	Packet type mask (1=enable, 0=disable)

27.3.9.5 *RDMON_ERR_CFG (0x10F0)*

Read engine AXI monitor error selection and filtering.

RDMON_ERR_CFG

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved

Bits	Field	Type	Reset	Description
15:8	ERR_MASK	RW	0xFF	Error type filtering mask
7:4	Reserved	RO	0x0	Reserved
3:0	ERR_SELE CT	RW	0x0	Error type selection

27.3.9.6 RDMON_MASK1 (0x10F4)

Read engine AXI monitor timeout and completion masks.

RDMON_MASK1

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	COMPL_MASK	RW	0x00	Completion mask
7:0	TIMEOUT_MA SK	RW	0xFF	Timeout mask

27.3.9.7 RDMON_MASK2 (0x10F8)

Read engine AXI monitor threshold and performance masks.

RDMON_MASK2

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	PERF_MAS K	RW	0x00	Performanc e mask
7:0	THRESH_M ASK	RW	0xFF	Threshold mask

27.3.9.8 RDMON_MASK3 (0x10FC)

Read engine AXI monitor address and debug masks.

RDMON_MASK3

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	DEBUG_MA SK	RW	0x00	Debug mask
7:0	ADDR_MAS	RW	0xFF	Address

Bits	Field	Type	Reset	Description
	K			mask

27.3.10 Write Engine AXI Monitor Configuration (0x1100 - 0x111F)

27.3.10.1 WRMON_ENABLE (0x1100)

Write engine AXI master monitor enable controls.

WRMON_ENABLE

Bits	Field	Type	Reset	Description
31:5	Reserved	RO	0x0	Reserved
4	PERF_EN	RW	0	Performance packet enable
3	TIMEOUT_EN	RW	1	Timeout detection enable
2	COMPL_EN	RW	0	Completion packet enable
1	ERR_EN	RW	1	Error detection enable
0	MON_EN	RW	1	Master enable for write monitor

27.3.10.2 WRMON_TIMEOUT (0x1104)

Write engine AXI monitor timeout threshold.

WRMON_TIMEOUT

Bits	Field	Type	Reset	Description
31:0	TIMEOUT_CYCLES	RW	10000	Timeout in clock cycles

27.3.10.3 WRMON_LATENCY_THRESH (0x1108)

Write engine AXI monitor latency threshold.

WRMON_LATENCY_THRESH

Bits	Field	Type	Reset	Description
31:0	LATENCY_THRESHOLD	RW	5000	Latency threshold in clock cycles

27.3.10.4 WRMON_PKT_MASK (0x110C)

Write engine AXI monitor packet type filtering.

WRMON_PKT_MASK

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:0	PKT_MAS K	RW	0xFFF F	Packet type mask (1=enable, 0=disable)

27.3.10.5 *WRMON_ERR_CFG (0x1110)*

Write engine AXI monitor error selection and filtering.

WRMON_ERR_CFG

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	ERR_MASK	RW	0xFF	Error type filtering mask
7:4	Reserved	RO	0x0	Reserved
3:0	ERR_SELE CT	RW	0x0	Error type selection

27.3.10.6 *WRMON_MASK1 (0x1114)*

Write engine AXI monitor timeout and completion masks.

WRMON_MASK1

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	COMPL_MASK	RW	0x00	Completion mask
7:0	TIMEOUT_MA SK	RW	0xFF	Timeout mask

27.3.10.7 *WRMON_MASK2 (0x1118)*

Write engine AXI monitor threshold and performance masks.

WRMON_MASK2

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	PERF_MAS K	RW	0x00	Performanc e mask

Bits	Field	Type	Reset	Description
7:0	THRESH_M ASK	RW	0xFF	Threshold mask

27.3.10.8 WRMON_MASK3 (0x111C)

Write engine AXI monitor address and debug masks.

WRMON_MASK3

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	DEBUG_MA SK	RW	0x00	Debug mask
7:0	ADDR_MAS K	RW	0xFF	Address mask

27.3.11 AXI Transfer Configuration (0x2A0 - 0x2AF)

27.3.11.1 AXI_XFER_CONFIG (0x2A0)

AXI read and write transfer burst sizes.

AXI_XFER_CONFIG

Bits	Field	Type	Reset	Description
31:16	Reserved	RO	0x0	Reserved
15:8	WR_XFER_B EATS	RW	15	AXI write transfer beats (AWLEN: 0-255 = 1-256)
7:0	RD_XFER_BE ATS	RW	15	AXI read transfer beats (ARLEN: 0-255 = 1-256)

Usage:

```
// Configure for 16-beat bursts (default)
write32(BASE + AXI_XFER_CONFIG, 0x0F0F);
```

```
// Configure for 64-beat bursts
write32(BASE + AXI_XFER_CONFIG, 0x3F3F);
```



```
// Configure for maximum 256-beat bursts
write32(BASE + AXI_XFER_CONFIG, 0xFFFF);
```

27.3.12 Performance Profiler Configuration (0x2B0 - 0x2BF)

27.3.12.1 PERF_CONFIG (0x2B0)

Performance profiler enable and mode controls.

PERF_CONFIG

Bits	Field	Type	Reset	Description
31:3	Reserved	RO	0x0	Reserved
2	PERF_CLEA R	RW	0	Clear counters (write 1, self-clearing)
1	PERF_MOD E	RW	0	Mode: 0=count, 1=histogram
0	PERF_EN	RW	0	Performance profiler enable

Usage:

```
// Enable performance profiler in count mode
write32(BASE + PERF_CONFIG, 0x01);

// Enable in histogram mode
write32(BASE + PERF_CONFIG, 0x03);

// Clear counters
write32(BASE + PERF_CONFIG, 0x05); // EN + CLEAR
// PERF_CLEAR self-clears after one cycle
```

27.4 Typical Usage Flow

27.4.1 Initialization

```
// 1. Global enable
write32(BASE + GLOBAL_CTRL, 0x1);

// 2. Configure scheduler
write32(BASE + SCHED_TIMEOUT_CYCLES, 10000);
write32(BASE + SCHED_CONFIG, 0x1F); // All features enabled
```

```
// 3. Configure descriptor engine
write32(BASE + DESCENG_CONFIG, 0x01); // Enable, no prefetch
write32(BASE + DESCENG_ADDR0_BASE, 0x8000_0000);
write32(BASE + DESCENG_ADDR0_LIMIT, 0x8FFF_FFFF);
```

```
// 4. Configure monitors (minimal reporting)
write32(BASE + DAXMON_CONFIG, 0x05); // Error + timeout only
write32(BASE + RDMON_CONFIG, 0x05);
write32(BASE + WRMON_CONFIG, 0x05);
```

```
// 5. Enable desired channels
write32(BASE + CHANNEL_ENABLE, 0xFF); // All 8 channels
```

27.4.2 Start Transfer

```
// Write 64-bit descriptor address to channel kick-off registers
// Descriptor at address 0x0000_0000_8000_0100 (64-bit)
write32(BASE + CH0_CTRL_LOW, 0x8000_0100); // Lower 32 bits
(buffered)
write32(BASE + CH0_CTRL_HIGH, 0x0000_0000); // Upper 32 bits
(triggers kick-off)
// Transfer starts immediately (blocks until descriptor engine ready)
```

27.4.3 Poll for Completion

```
// Check channel 0 idle status
while (!(read32(BASE + CHANNEL_IDLE) & 0x01)) {
    // Wait for channel 0 to become idle
}

// Or check scheduler state (one-hot encoding)
while ((read32(BASE + CH0_STATE) & 0x7F) != 0x01) {
    // Wait for CH_IDLE state (bit 0 = 0x01)
}
```

27.4.4 Error Handling

```
// Check all channel states for errors (one-hot encoding)
for (int ch = 0; ch < 8; ch++) {
    uint32_t state = read32(BASE + CH0_STATE + (ch * 4)) & 0x7F;
    if (state & 0x20) { // CH_ERROR (bit 5)
        // Reset channel
        write32(BASE + CHANNEL_RESET, 1 << ch);
    }
}
```

27.5 Register Summary Table

Register Summary Table

Offset Range	Description	Count	Type
0x000-0x03F	Channel kick-off registers (LOW/HIGH)	16	Write-routing
0x100-0x11F	Global control and status	3	RW/RO
0x120-0x13F	Per-channel control	2	RW
0x140-0x16F	Per-channel status	11	RO
0x170-0x17F	Engine completion and error status	3	RO
0x200-0x21F	Scheduler configuration	3	RW
0x220-0x23F	Descriptor engine configuration	5	RW
0x2A0-0x2AF	AXI transfer configuration	1	RW
0x2B0-0x2BF	Performance profiler configuration	1	RW
0x2C0-0x2CF	Channel observation	4	RW/RO
0x1000-0x1007	Monitor FIFO status (mon regfile)	2	RO
0x10C0-0x10DF	Descriptor AXI monitor configuration	8	RW
0x10E0-0x10FF	Read engine AXI monitor configuration	8	RW
0x1100-0x111F	Write engine AXI monitor configuration	8	RW

Note: The monitor block (0x1000+) resides in the separate `stream_mon_regs` regfile on the same APB slave; per-monitor performance counters (0x1150-0x11F4) are additionally present but omitted from this summary. Base config/status (0x100-0x3FF) also includes `PERF_CH_SEL` and the `HIST_*` histogram registers.

27.6 PeakRDL Generation

To generate SystemVerilog from the register definition:

```
cd projects/components/dmas/stream/rtl/stream_macro/  
peakrdl regblock stream_regs.rdl -o generated/
```

This generates: - `stream_regs_pkg.sv` - Register definitions package - `stream_regs.sv` - APB slave register interface

27.6.1 Channel Observation Registers (0x2C0 - 0x2CF)

New in v0.92 — Enables software to observe scheduler state without halting transfers (commit 4467554e).

Channel Observation Registers

Offset	Register	Type	Reset	Description
0x2C0	OBS_CTRL	RW	0x0	Channel observation mux selector [2:0] = channel index
0x2C4	OBS_FLAGS	RO	0x0	Observation flags for selected channel (error sticky, timeout)
0x2C8	OBS_DATA0	RO	0x0	Per-channel diagnostic data 0 (low 32 bits)
0x2CC	OBS_DATA1	RO	0x0	Per-channel diagnostic data 1 (high 32 bits)

Usage: 1. Write channel index (0-7) to OBS_CTRL[2:0] 2. Read OBS_FLAGS, OBS_DATA0, OBS_DATA1 (muxed views of selected channel) 3. OBS_FLAGS[0] = SCH_ERROR_STICKY (persists until `cfg_channel_reset`) 4. OBS_FLAGS[1] = SCH_TIMEOUT (current timeout state)

Implementation: `stream_config_block.sv` implements the 8:1 mux for per-channel state observation (commit 0c979c35 exposes scheduler error/timeout signals into OBS_FLAGS).

Revision History:

PeakRDL Generation

Version	Date	Author	Description
1.0	2025-10-20	sean galloway	Initial creation

Version	Date	Author	Description
1.1	2025-12-01	sean galloway	Added complete monitor registers from RDL Added engine completion/error status (0x170) Added monitor FIFO status (0x180) Added AXI transfer config (0x2A0) Added performance profiler config (0x2B0)
1.2	2026-07-02	sean galloway	Monitor block relocated into separate stream_mon_regs regfile at 0x1000 (same APB slave, 13-bit/8 KB decode); added SCHED_TIMEOUT_LIMIT @ 0x208

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

28 Chapter 5: Programming Models

This chapter provides software developer guidance for using the STREAM DMA engine.

28.1 Contents

28.1.1 01_initialization.md

- Power-on initialization sequence
- Register configuration
- Channel setup
- Configuration presets (minimal, high-performance)

28.1.2 02_single_transfer.md

- Descriptor format (256-bit)
- Kick-off address map and write sequence

- Simple single-descriptor transfer
- C code examples with complete workflow

28.1.303_chained_transfers.md

- Multi-descriptor chains
- Descriptor linking via `next_descriptor_ptr`
- Chain termination methods
- Scatter-gather operations
- Prefetch mode for performance

28.1.404_multi_channel.md

- 8-channel concurrent operations
- Priority-based scheduling
- Resource sharing strategies
- Channel pooling and load balancing
- Performance considerations

28.1.505_error_handling.md

- Error types (AXI, timeout, internal)
- Error detection registers
- Recovery procedures
- Interrupt-based error handling
- Debug monitoring

28.2 Planned (Future)

28.2.106_performance_tuning.md

- Burst size selection
- Priority tuning
- SRAM depth considerations
- Maximizing throughput

28.2.207_software_examples.md

- Complete working examples
- Linux driver skeleton
- Bare-metal usage

- Common use cases

Status: Core programming guides complete (01-05)

Target Audience: - Software engineers integrating STREAM - Driver developers - System architects - Application developers

Last Updated: 2025-12-01

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

29 Chapter 6: Configuration Reference

Version: 0.90 **Last Updated:** 2025-11-22 **Purpose:** Complete reference for all STREAM configuration signals

29.1 Overview

STREAM provides comprehensive runtime configuration through APB registers and compile-time parameters. This chapter documents all configuration signals, their valid ranges, default values, and recommended settings for different use cases.

29.1.1 Configuration Categories

Configuration Categories

Category	Signals	Purpose
Channel Control	2	Enable/reset individual channels
Scheduler	6	Transfer scheduling and timeouts
Descriptor Engine	7	Descriptor fetch behavior
AXI Monitors	45 (15 per monitor ×	Debug/trace filtering

Category	Signals	Purpose
	3)	
AXI Transfer	2	Burst configuration
Performance	3	Profiling and metrics
Total	65 configuration signals	Full system control

29.2 1. Channel Control Configuration

29.2.1 cfg_channel_enable[NUM_CHANNELS-1:0]

Type: Per-channel enable **Width:** 1 bit × NUM_CHANNELS (default 8) **Default:** 8'hFF (all enabled)
Register: CHANNEL_ENABLE @ 0x120

Description: Enables or disables individual DMA channels. When disabled, the channel: - Ignores descriptor kick-off requests - Completes current transfer if in progress - Enters idle state - Remains in idle until re-enabled

Valid Values: - 1'b1: Channel enabled (can accept transfers) - 1'b0: Channel disabled (ignores new transfers)

Use Cases:

```
// Enable only channels 0, 2, 4 (even channels)
cfg_channel_enable = 8'b01010101;
```

```
// Disable all channels (emergency stop)
cfg_channel_enable = 8'b00000000;
```

```
// Enable all channels (normal operation)
cfg_channel_enable = 8'b11111111;
```

Interaction with Global Enable: The final channel enable is: `cfg_channel_enable[i] & reg_global_ctrl_global_en`

29.2.2 cfg_channel_reset[NUM_CHANNELS-1:0]

Type: Per-channel reset **Width:** 1 bit × NUM_CHANNELS (default 8) **Default:** 8'h00 (no resets active) **Register:** CHANNEL_RESET @ 0x124

Description: Asserts reset for individual channels without affecting other channels or global state. When asserted: - Channel FSM returns to IDLE - Pending transfers aborted - SRAM buffers flushed - Descriptor engine reset

Valid Values: - 1'b1: Channel in reset (clears state) - 1'b0: Channel operating normally

Use Cases:

```
// Reset channel 3 after error
cfg_channel_reset = 8'b00001000;
wait_cycles(10);
cfg_channel_reset = 8'b00000000; // Release reset
```

```
// Reset all channels (soft reset)
cfg_channel_reset = 8'hFF;
wait_cycles(10);
cfg_channel_reset = 8'h00;
```

IMPORTANT: Assert reset for minimum 10 clock cycles to ensure complete FSM reset.

29.3 2. Scheduler Configuration

29.3.1 cfg_sched_timeout_cycles[15:0]

Type: Timeout threshold **Width:** 16 bits **Default:** 16'd1000 (1,000 cycles) **Register:** SCHED_TIMEOUT_CYCLES @ 0x200[15:0]

Description: Number of clock cycles before a channel operation times out. Applies to: - Descriptor fetch latency - AXI read/write response latency - Scheduler state transitions

Valid Range: 100 to 65535 cycles **Typical Values:** - Fast SRAM: 100-500 cycles - DDR3/DDR4: 1000-10000 cycles - High-latency: 10000-65535 cycles

Calculation:

Timeout cycles = (Expected latency × 10) + margin

Example for DDR4 @ 200 MHz:

- Expected read latency: 100 cycles (500 ns)
 - Safety margin: 10x
 - Timeout = 100 × 10 = 1000 cycles
-

29.3.2 cfg_sched_enable

Type: Global scheduler enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** SCHED_CONFIG @ 0x204[0]

Description: Master enable for all schedulers. When disabled, all channels stop scheduling new operations.

29.3.3 cfg_sched_timeout_enable

Type: Timeout detection enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** SCHED_CONFIG @ 0x204[1]

Description: Enables timeout detection for scheduler operations. Disable for simulation or known slow memory.

29.3.4 cfg_sched_err_enable

Type: Error detection enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** SCHED_CONFIG @ 0x204[2]

Description: Enables error packet generation for scheduler errors (AXI SLVERR, DECERR, timeouts).

29.3.5 cfg_sched_compl_enable

Type: Completion event enable **Width:** 1 bit **Default:** 1'b1 (enabled by default) **Register:** SCHED_CONFIG @ 0x204[3]

Description: Enables MonBus packets for transfer completion events. Generate high traffic, use sparingly.

29.3.6 cfg_sched_perf_enable

Type: Performance monitoring enable **Width:** 1 bit **Default:** 1'b0 (disabled by default) **Register:** SCHED_CONFIG @ 0x204[4]

Description: Enables performance profiling packets (latency, throughput metrics).

29.3.7 cfg_rd_prefetch_enable

Type: Read-ahead descriptor prefetch enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** SCHED_CONFIG @ 0x204[5]

Description: Enables read-ahead descriptor prefetch. On a chained legacy descriptor the scheduler read side loads the next descriptor from the descriptor-FIFO head and keeps filling the SRAM while the write side drains the current descriptor, collapsing the per-descriptor boundary bubble to zero (cross-descriptor streaming). Clear it for lockstep behaviour — useful for an on-silicon A/B on a single bitstream. Ignored for EXT (row/col-major) descriptors, which always run lockstep. See MAS Section 2.4 (Scheduler) and HAS Section 5.1.

29.4 3. Descriptor Engine Configuration

29.4.1 cfg_desceng_enable

Type: Descriptor engine global enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** DESCENG_CTRL @ 0x220[0]

Description: Global enable for all descriptor engines. When disabled, descriptor fetch stops.

29.4.2 cfg_desceng_prefetch

Type: Prefetch enable **Width:** 1 bit **Default:** 1'b0 (disabled) **Register:** DESCENG_CTRL @ 0x220[1]

Description: Enables descriptor prefetching to hide fetch latency.

Prefetch Behavior: - Enabled: Fetch next descriptor while current transfer executes - Disabled: Wait for current transfer completion before fetching next

Performance Impact: - Prefetch ON: +15-30% throughput for chained descriptors - Prefetch OFF: Simpler logic, lower area

29.4.3 cfg_desceng_fifo_thresh[3:0]

Type: FIFO threshold **Width:** 4 bits **Default:** 4'h8 (8 entries) **Register:** DESCENG_CTRL @ 0x220[7:4]

Description: Number of entries in descriptor FIFO before asserting backpressure.

Valid Range: 1-15 entries **Typical Values:** - Low latency: 2-4 entries - Balanced: 8 entries (default) - High throughput: 12-15 entries

29.4.4 cfg_desceng_addr0_base[31:0]

Type: Base address for descriptor region 0 **Width:** 32 bits **Default:** 32'h0000_0000 **Register:** DESCENG_ADDR0_BASE @ 0x224

Description: Base address of first descriptor memory region. Descriptors fetched from this region if within range.

Alignment: Must be aligned to descriptor size (256 bits = 32 bytes)

29.4.5 cfg_desceng_addr0_limit[31:0]

Type: Limit address for descriptor region 0 **Width:** 32 bits **Default:** 32'hFFFF_FFFF (no limit) **Register:** DESCENG_ADDR0_LIMIT @ 0x228

Description: Upper limit of first descriptor region. Descriptors beyond this address use region 1.

29.4.6 cfg_desceng_addr1_base[31:0]

Type: Base address for descriptor region 1 **Width:** 32 bits **Default:** 32'h0000_0000 **Register:** DESCENG_ADDR1_BASE @ 0x22C

Description: Base address of second descriptor memory region (optional).

29.4.7 cfg_desceng_addr1_limit[31:0]

Type: Limit address for descriptor region 1 **Width:** 32 bits **Default:** 32'hFFFF_FFFF **Register:** DESCENG_ADDR1_LIMIT @ 0x230

Description: Upper limit of second descriptor region.

29.5 4. AXI Monitor Configuration

STREAM includes three independent AXI monitors with identical configuration sets:

1. **Descriptor AXI Monitor** (cfg_desc_mon_*) - Monitors descriptor fetch AXI master
2. **Read Engine Monitor** (cfg_rdeng_mon_*) - Monitors data read AXI master

3. Write Engine Monitor (cfg_wreng_mon_*) - Monitors data write AXI master

Each monitor has 15 configuration signals with the same structure.

29.5.1 4.1 Descriptor AXI Monitor (cfg_desc_mon_*)

Register Base: 0x240-0x25F

29.5.1.1 *cfg_desc_mon_enable*

Type: Monitor master enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** DAXMON_ENABLE @ 0x240[0]

Description: Master enable for descriptor AXI monitor. All monitor packets disabled when this is 0.

29.5.1.2 *cfg_desc_mon_err_enable*

Type: Error packet enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** DAXMON_ENABLE @ 0x240[1]

Description: Enables error packet generation (SLVERR, DECERR, protocol violations).

29.5.1.3 *cfg_desc_mon_perf_enable*

Type: Performance packet enable **Width:** 1 bit **Default:** 1'b0 (disabled) **Register:** DAXMON_ENABLE @ 0x240[2]

Description: Enables performance monitoring packets (latency, bandwidth).

WARNING: High packet rate - use only during debug.

29.5.1.4 *cfg_desc_mon_timeout_enable*

Type: Timeout detection enable **Width:** 1 bit **Default:** 1'b1 (enabled) **Register:** DAXMON_ENABLE @ 0x240[3]

Description: Enables timeout packet generation when transactions exceed threshold.

29.5.1.5 *cfg_desc_mon_timeout_cycles[31:0]*

Type: Timeout threshold **Width:** 32 bits **Default:** 32'd10000 **Register:** DAXMON_TIMEOUT @ 0x244

Description: Number of cycles before transaction times out.

29.5.1.6 *cfg_desc_mon_latency_thresh[31:0]*

Type: Latency threshold **Width:** 32 bits **Default:** 32'd1000 **Register:** DAXMON_LATENCY @ 0x248

Description: Latency threshold for performance warnings.

29.5.1.7 *cfg_desc_mon_pkt_mask[15:0]*

Type: Packet type filter **Width:** 16 bits (1 bit per packet type) **Default:** 16'h00FF (errors + completions) **Register:** DAXMON_PKT_MASK @ 0x24C[15:0]

Description: Bit mask to filter packet types. Only packet types with corresponding bit set are generated.

Packet Type Mapping:

Bit	Packet Type
-----	-----
0	AR command
1	AW command
2	R completion
3	W data
4	B response
5	Error
6	Timeout
7	Completion
8-15	Reserved

Examples:

```
// Only errors
cfg_desc_mon_pkt_mask = 16'h0020; // Bit 5

// Errors + timeouts
cfg_desc_mon_pkt_mask = 16'h0060; // Bits 5-6

// All packets
cfg_desc_mon_pkt_mask = 16'hFFFF;
```

29.5.1.8 *cfg_desc_mon_err_select[3:0]*

Type: Error type selector **Width:** 4 bits **Default:** 4'hF (all errors) **Register:** DAXMON_ERR_SELECT @ 0x24C[19:16]

Description: Selects which error types to monitor.

Error Type Bits:

Bit	Error Type
----	-----
0	SLVERR
1	DECERR
2	Protocol violation
3	Reserved

29.5.1.9 *cfg_desc_mon_err_mask[7:0]*

Type: Error event filter **Width:** 8 bits **Default:** 8'hFF (all errors) **Register:** DAXMON_MASK1 @ 0x250[7:0]

Description: Bit mask for error packet generation (channel-specific filtering).

29.5.1.10 *cfg_desc_mon_timeout_mask[7:0]*

Type: Timeout channel filter **Width:** 8 bits **Default:** 8'hFF (all channels) **Register:** DAXMON_MASK1 @ 0x250[15:8]

Description: Channel mask for timeout packets. Set bit enables timeout detection for corresponding channel.

29.5.1.11 *cfg_desc_mon_compl_mask[7:0]*

Type: Completion channel filter **Width:** 8 bits **Default:** 8'h00 (no channels) **Register:** DAXMON_MASK1 @ 0x250[23:16]

Description: Channel mask for completion packets.

WARNING: Completion packets are high volume - enable only for specific channels.

29.5.1.12 *cfg_desc_mon_thresh_mask[7:0]*

Type: Threshold event filter **Width:** 8 bits **Default:** 8'hFF (all channels) **Register:** DAXMON_MASK2 @ 0x254[7:0]

Description: Channel mask for latency threshold exceedance packets.

29.5.1.13 *cfg_desc_mon_perf_mask[7:0]*

Type: Performance packet filter **Width:** 8 bits **Default:** 8'h00 (no channels) **Register:** DAXMON_MASK2 @ 0x254[15:8]

Description: Channel mask for performance monitoring packets.

29.5.1.14 *cfg_desc_mon_addr_mask[7:0]*

Type: Address-based filter **Width:** 8 bits **Default:** 8'hFF (all addresses) **Register:** DAXMON_MASK2 @ 0x254[23:16]

Description: Channel mask for address-range-based packet filtering.

29.5.1.15 *cfg_desc_mon_debug_mask[7:0]*

Type: Debug event filter **Width:** 8 bits **Default:** 8'h00 (no debug packets) **Register:** DAXMON_MASK2 @ 0x254[31:24]

Description: Channel mask for debug-level packets (verbose trace).

29.5.24.2 Read Engine Monitor (*cfg_rdeng_mon_**)

Register Base: 0x260-0x27F

All signals identical to Descriptor Monitor (Section 4.1), but applied to data read AXI master.

Key Differences: - Monitors data transfers (not descriptors) - Higher throughput → more packets - Recommended: Keep `cfg_rdeng_mon_compl_enable = 0` unless debugging

29.5.34.3 Write Engine Monitor (*cfg_wreng_mon_**)

Register Base: 0x280-0x29F

All signals identical to Descriptor Monitor (Section 4.1), but applied to data write AXI master.

Key Differences: - Monitors write transactions (AW/W/B channels) - B response timing critical for performance - Recommended: Enable only error packets by default

29.6 5. AXI Transfer Configuration

29.6.1 `cfg_axi_rd_xfer_beats[7:0]`

Type: Read transfer size **Width:** 8 bits **Default:** 8'd16 (16 beats) **Register:** AXI_RD_XFER @ 0x2A0[7:0]

Description: Default number of beats per AXI read burst. Actual burst size may be less to respect 4KB boundaries.

Valid Range: 1-256 beats (AXI4 standard) **Typical Values:** - Small transfers: 4-8 beats - Balanced: 16 beats (default) - Large transfers: 32-64 beats - Maximum throughput: 128-256 beats

Calculation:

Transfer size (bytes) = beats × (DATA_WIDTH / 8)

Example for DATA_WIDTH = 512 bits:

- 16 beats = 16 × 64 = 1024 bytes (1 KB)
 - 64 beats = 64 × 64 = 4096 bytes (4 KB)
-

29.6.2 `cfg_axi_wr_xfer_beats[7:0]`

Type: Write transfer size **Width:** 8 bits **Default:** 8'd16 (16 beats) **Register:** AXI_WR_XFER @ 0x2A0[15:8]

Description: Default number of beats per AXI write burst.

Same constraints as `cfg_axi_rd_xfer_beats`

29.7 6. Performance Profiler Configuration

29.7.1 `cfg_perf_enable`

Type: Profiler enable **Width:** 1 bit **Default:** 1'b0 (disabled) **Register:** PERF_CTRL @ 0x2B0[0]

Description: Enables performance profiling for all channels. When enabled, profiler captures: - Transfer start/end timestamps - Latency per channel - Throughput measurements - Channel utilization

29.7.2 cfg_perf_mode

Type: Profiling mode **Width:** 1 bit **Default:** 1'b0 (timestamp mode) **Register:** PERF_CTRL @ 0x2B0[1]

Description: Selects profiling mode: - 1'b0: Timestamp mode - Record absolute timestamps - 1'b1: Elapsed time mode - Record delta times

Use Cases: - Timestamp: Correlate events across multiple blocks - Elapsed: Measure operation latencies

29.7.3 cfg_perf_clear

Type: Clear profiler state **Width:** 1 bit (write-only) **Default:** 1'b0 **Register:** PERF_CTRL @ 0x2B0[2]

Description: Write 1'b1 to clear profiler FIFOs and counters. Self-clearing (automatically returns to 0).

29.8 7. Configuration Presets

29.8.17.1 Minimal Configuration (Tutorial/Embedded)

Use Case: Educational, minimal logic, single-channel operation

```
// Channel control
cfg_channel_enable = 8'b00000001; // Only channel 0

// Scheduler
cfg_sched_enable = 1'b1;
cfg_sched_timeout_cycles = 16'd500; // Short timeout (SRAM)
cfg_sched_timeout_enable = 1'b1;
cfg_sched_err_enable = 1'b1;
cfg_sched_compl_enable = 1'b0; // Disable completion packets
cfg_sched_perf_enable = 1'b0; // Disable performance packets

// Descriptor engine
cfg_desceng_enable = 1'b1;
```

```

cfg_desceng_prefetch = 1'b0;           // No prefetch (simpler)
cfg_desceng_fifo_thresh = 4'h4;        // Small FIFO

// All monitors DISABLED (reduce logic)
cfg_desc_mon_enable = 1'b0;
cfg_rdeng_mon_enable = 1'b0;
cfg_wreng_mon_enable = 1'b0;

// AXI transfer
cfg_axi_rd_xfer_beats = 8'd8;          // Small bursts
cfg_axi_wr_xfer_beats = 8'd8;

// Performance profiler
cfg_perf_enable = 1'b0;                // Disabled

```

29.8.27.2 Balanced Configuration (Typical FPGA)

Use Case: General-purpose DMA, moderate channels, balanced performance/area

```

// Channel control
cfg_channel_enable = 8'hFF;            // All 8 channels

// Scheduler
cfg_sched_enable = 1'b1;
cfg_sched_timeout_cycles = 16'd5000;   // DDR4 timeout
cfg_sched_timeout_enable = 1'b1;
cfg_sched_err_enable = 1'b1;
cfg_sched_compl_enable = 1'b0;         // Errors only
cfg_sched_perf_enable = 1'b0;

// Descriptor engine
cfg_desceng_enable = 1'b1;
cfg_desceng_prefetch = 1'b1;          // Enable prefetch
cfg_desceng_fifo_thresh = 4'h8;       // Balanced FIFO

// Descriptor monitor (errors only)
cfg_desc_mon_enable = 1'b1;
cfg_desc_mon_err_enable = 1'b1;
cfg_desc_mon_perf_enable = 1'b0;
cfg_desc_mon_timeout_enable = 1'b1;
cfg_desc_mon_timeout_cycles = 32'd10000;
cfg_desc_mon_pkt_mask = 16'h0060;     // Errors + timeouts

// Read/write monitors (errors only)
cfg_rdeng_mon_enable = 1'b1;

```

```

cfg_rdeng_mon_err_enable = 1'b1;
cfg_rdeng_mon_perf_enable = 1'b0;

cfg_wreng_mon_enable = 1'b1;
cfg_wreng_mon_err_enable = 1'b1;
cfg_wreng_mon_perf_enable = 1'b0;

// AXI transfer
cfg_axi_rd_xfer_beats = 8'd32;           // Moderate bursts
cfg_axi_wr_xfer_beats = 8'd32;

// Performance profiler
cfg_perf_enable = 1'b1;                 // Enable profiling
cfg_perf_mode = 1'b1;                   // Elapsed time mode

```

29.8.37.3 High-Performance Configuration (ASIC/Datacenter)

Use Case: Maximum throughput, all channels active, full monitoring

```

// Channel control
cfg_channel_enable = 8'hFF;             // All channels

// Scheduler
cfg_sched_enable = 1'b1;
cfg_sched_timeout_cycles = 16'd20000;   // High latency tolerance
cfg_sched_timeout_enable = 1'b1;
cfg_sched_err_enable = 1'b1;
cfg_sched_compl_enable = 1'b1;          // Full monitoring
cfg_sched_perf_enable = 1'b1;

// Descriptor engine
cfg_desceng_enable = 1'b1;
cfg_desceng_prefetch = 1'b1;
cfg_desceng_fifo_thresh = 4'hF;         // Max FIFO depth

// All monitors ENABLED with full profiling
cfg_desc_mon_enable = 1'b1;
cfg_desc_mon_err_enable = 1'b1;
cfg_desc_mon_perf_enable = 1'b1;        // Performance monitoring
cfg_desc_mon_timeout_enable = 1'b1;
cfg_desc_mon_timeout_cycles = 32'd20000;
cfg_desc_mon_pkt_mask = 16'hFFFF;      // All packet types

// Same for read/write monitors
cfg_rdeng_mon_enable = 1'b1;

```

```

cfg_rdeng_mon_err_enable = 1'b1;
cfg_rdeng_mon_perf_enable = 1'b1;

cfg_wreng_mon_enable = 1'b1;
cfg_wreng_mon_err_enable = 1'b1;
cfg_wreng_mon_perf_enable = 1'b1;

// AXI transfer
cfg_axi_rd_xfer_beats = 8'd128;           // Large bursts
cfg_axi_wr_xfer_beats = 8'd128;

// Performance profiler
cfg_perf_enable = 1'b1;
cfg_perf_mode = 1'b0;                     // Timestamp mode
(correlation)

```

29.8.47.4 Debug Configuration (Verbose Monitoring)

Use Case: Debugging integration issues, detailed trace analysis

```

// Enable specific channel for debug
cfg_channel_enable = 8'b00000001;       // Channel 0 only

// Scheduler
cfg_sched_enable = 1'b1;
cfg_sched_timeout_cycles = 16'd65535;   // Long timeout for debug
cfg_sched_timeout_enable = 1'b1;
cfg_sched_err_enable = 1'b1;
cfg_sched_compl_enable = 1'b1;          // All events
cfg_sched_perf_enable = 1'b1;

// Descriptor engine
cfg_desceng_enable = 1'b1;
cfg_desceng_prefetch = 1'b0;            // Simpler for debug

// All monitors ENABLED with verbose trace
cfg_desc_mon_enable = 1'b1;
cfg_desc_mon_err_enable = 1'b1;
cfg_desc_mon_perf_enable = 1'b1;
cfg_desc_mon_timeout_enable = 1'b1;
cfg_desc_mon_pkt_mask = 16'hFFFF;       // ALL packets
cfg_desc_mon_compl_mask = 8'h01;        // Channel 0 completions
cfg_desc_mon_debug_mask = 8'h01;       // Channel 0 debug packets

// Same for read/write monitors

```

```

cfg_rdeng_mon_enable = 1'b1;
cfg_rdeng_mon_pkt_mask = 16'hFFFF;           // Verbose
cfg_rdeng_mon_compl_mask = 8'h01;

cfg_wreng_mon_enable = 1'b1;
cfg_wreng_mon_pkt_mask = 16'hFFFF;
cfg_wreng_mon_compl_mask = 8'h01;

// AXI transfer (small for debug)
cfg_axi_rd_xfer_beats = 8'd4;
cfg_axi_wr_xfer_beats = 8'd4;

// Performance profiler
cfg_perf_enable = 1'b1;
cfg_perf_mode = 1'b0;                       // Timestamps

```

29.9 8. Configuration Best Practices

29.9.18.1 Monitor Configuration Guidelines

General Rules:

1. **Start with errors only:** Enable only `cfg_*_mon_err_enable` initially. Add other packets as needed.
 2. **Completion packets are expensive:** Only enable `cfg_*_mon_compl_enable` for specific channels during debug.
 3. **Performance packets flood MonBus:** Enable `cfg_*_mon_perf_enable` sparingly (1-2 channels maximum).
 4. **Use masks aggressively:** Set channel masks to enable monitoring only on channels of interest.
-

29.9.28.2 Timeout Configuration

Calculation Method:

Recommended timeout = (Expected latency × Safety factor) + Margin

Safety factor:

- SRAM: 2-5x
- DDR3/DDR4: 5-10x

- High-latency: 10-20x

Margin: +100 cycles minimum

Examples:

SRAM @ 200 MHz:

- Expected: 20 cycles (100 ns)
- Safety: 5x
- Timeout: $20 \times 5 + 100 = 200$ cycles

DDR4 @ 200 MHz:

- Expected: 100 cycles (500 ns)
 - Safety: 10x
 - Timeout: $100 \times 10 + 100 = 1100$ cycles
-

29.9.38.3 Prefetch Configuration

Enable prefetch when: - Descriptor chains > 2 descriptors - Memory latency > 50 cycles - Throughput is priority

Disable prefetch when: - Area is constrained - Single descriptors only - Simplicity is priority

29.9.48.4 Burst Size Selection

Read Burst Size:

```
Optimal burst size = min(  
    Memory controller page size,  
    4KB (AXI limit),  
    SRAM FIFO depth / 2  
)
```

Example for DDR4 (8KB page), FIFO depth 512 entries:

- Page size: 8192 bytes = 128 beats (512-bit)
- AXI limit: 4096 bytes = 64 beats
- FIFO limit: $512/2 = 256$ beats
- Optimal: $\min(128, 64, 256) = 64$ beats

Write Burst Size: - Usually same as read burst size - May be smaller if write FIFO depth is limited

29.10 9. Configuration Register Map Summary

9. Configuration Register Map Summary

Address	Register Name	Fields	Section
0x100	GLOBAL_CTRL	global_en, global_rst	-
0x120	CHANNEL_ENABLE	ch_en[7:0]	1
0x124	CHANNEL_RESET	ch_rst[7:0]	1
0x200	SCHED_TIMEOUT_CYCLES	timeout_cycle s[15:0]	2
0x204	SCHED_CONFIG	enable, timeout_en, err_en, compl_en, perf_en	2
0x220	DESCENG_CONFIG	enable, prefetch, fifo_thresh	3
0x224	DESCENG_ADDR0_BASE	addr0_base[3 1:0]	3
0x228	DESCENG_ADDR0_LIMIT	addr0_limit[3 1:0]	3
0x22C	DESCENG_ADDR1_BASE	addr1_base[3 1:0]	3
0x230	DESCENG_ADDR1_LIMIT	addr1_limit[3 1:0]	3
0x240-0x25F	DAXMON_*	Descriptor monitor (15 signals)	4.1
0x260-0x27F	RDMON_*	Read engine monitor (15 signals)	4.2
0x280-0x29F	WRMON_*	Write engine monitor (15 signals)	4.3

Address	Register Name	Fields	Section
0x2A0	AXI_XFER_CFG	rd_xfer_beats , wr_xfer_beats	5
0x2B0	PERF_CTRL	perf_enable, perf_mode, perf_clear	6

Total Address Space: 0x000-0x3FF (1KB)

29.11 10. Software Configuration Examples

29.11.1 10.1 C/C++ Initialization (Minimal)

```
// Minimal configuration for single-channel operation
void stream_init_minimal(volatile uint32_t *base_addr) {
    // Global enable
    base_addr[0x100/4] = 0x00000001; // GLOBAL_CTRL.global_en

    // Enable channel 0 only
    base_addr[0x120/4] = 0x00000001; // CHANNEL_ENABLE.ch_en

    // Scheduler config
    base_addr[0x200/4] = 500; // SCHED_TIMEOUT_CYCLES (SRAM)
    base_addr[0x204/4] = 0x00000007; // SCHED_CONFIG: enable |
    timeout_en | err_en

    // Descriptor engine
    base_addr[0x220/4] = 0x00000041; // enable | fifo_thresh=4

    // Disable all monitors (minimal)
    base_addr[0x240/4] = 0x00000000; // DAXMON_ENABLE
    base_addr[0x260/4] = 0x00000000; // RDMON_ENABLE
    base_addr[0x280/4] = 0x00000000; // WRMON_ENABLE

    // AXI transfer config
    base_addr[0x2A0/4] = 0x00000808; // 8 beats read + write
}
```

29.11.2 10.2 C/C++ Initialization (Balanced)

```
// Balanced configuration for typical FPGA
void stream_init_balanced(volatile uint32_t *base_addr) {
    // Global enable
    base_addr[0x100/4] = 0x00000001;

    // Enable all 8 channels
    base_addr[0x120/4] = 0x000000FF;

    // Scheduler config
    base_addr[0x200/4] = 5000;           // SCHED_TIMEOUT_CYCLES (DDR4)
    base_addr[0x204/4] = 0x00000007;    // SCHED_CONFIG: enable |
    timeout_en | err_en

    // Descriptor engine
    base_addr[0x220/4] = 0x00000083;    // enable | prefetch |
    fifo_thresh=8

    // Descriptor AXI monitor (errors only)
    base_addr[0x240/4] = 0x0000000B;    // enable | err_en | timeout_en
    base_addr[0x244/4] = 10000;         // timeout_cycles
    base_addr[0x24C/4] = 0x00000060;    // pkt_mask: errors + timeouts

    // Read/write monitors (errors only)
    base_addr[0x260/4] = 0x0000000B;    // RDMON: enable | err_en |
    timeout_en
    base_addr[0x280/4] = 0x0000000B;    // WRMON: enable | err_en |
    timeout_en

    // AXI transfer config
    base_addr[0x2A0/4] = 0x00002020;    // 32 beats read + write

    // Enable performance profiler
    base_addr[0x2B0/4] = 0x00000003;    // enable | elapsed mode
}
```

29.12 11. Troubleshooting Configuration Issues

29.12.1 Problem: No transfers occurring

Check: 1. `cfg_channel_enable[n]` set for channel `n`? 2. `cfg_sched_enable` = 1? 3. `cfg_desceng_enable` = 1? 4. Global enable set?

29.12.2 Problem: Timeout errors

Check: 1. `cfg_sched_timeout_cycles` too small? 2. Memory latency higher than expected? 3. AXI backpressure not handled?

Solution: - Increase timeout: `cfg_sched_timeout_cycles = 20000` - Disable temporarily: `cfg_sched_timeout_enable = 0`

29.12.3 Problem: MonBus overflow

Check: 1. Too many completion packets enabled? 2. Performance packets enabled on all channels? 3. Debug packets enabled?

Solution: - Disable completion: `cfg_*_mon_compl_enable = 0` - Use channel masks: `cfg_*_mon_compl_mask = 8'h01` (channel 0 only) - Reduce packet types: `cfg_*_mon_pkt_mask = 16'h0060` (errors + timeouts)

29.12.4 Problem: Low throughput

Check: 1. Prefetch disabled? 2. Burst size too small? 3. FIFO threshold too conservative?

Solution: - Enable prefetch: `cfg_desceng_prefetch = 1` - Increase bursts: `cfg_axi_rd_xfer_beats = 64` - Increase FIFO: `cfg_desceng_fifo_thresh = 12`

29.13 Related Documentation

- [Register Map](#) - Complete APB register specification
 - [Clocks and Reset](#) - Timing requirements
 - [Programming Guide](#) - Software API examples
-

Last Updated: 2025-12-01 **Maintained By:** STREAM Architecture Team

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · MIT License

30 Product Requirements Document (PRD)

30.1 STREAM - Scatter-gather Transfer Rapid Engine for AXI Memory

Version: 1.0 **Date:** 2025-10-17 **Status:** Nearly Complete - Final Integration Pending **Owner:** RTL Design Sherpa Project **Parent Document:** /PRD.md

30.2 1. Executive Summary

The **STREAM** (Scatter-gather Transfer Rapid Engine for AXI Memory) is a simplified DMA-style accelerator designed for efficient memory-to-memory data movement with descriptor-based scatter-gather support. STREAM serves as a beginner-friendly tutorial demonstrating descriptor-based DMA engine design patterns while maintaining production-quality RTL practices.

30.2.1 1.1 Quick Stats

- **Modules:** ~8-10 SystemVerilog files (estimated)
- **Channels:** Maximum 8 independent channels
- **Interfaces:** APB (config), AXI4 (descriptor fetch + data read/write), MonBus (monitoring)
- **Architecture:** Simplified from RAPIDS - pure memory-to-memory
- **Tutorial Focus:** Aligned addresses only, straightforward data flow
- **Status:** Nearly complete - config interface and top-level wrapper pending

30.2.2 1.2 Project Goals

- **Primary:** Educational DMA engine demonstrating scatter-gather descriptor chains
 - **Secondary:** Production-quality RTL suitable for FPGA/ASIC implementation
 - **Tertiary:** Foundation for understanding more complex DMA architectures (e.g., RAPIDS)
-

30.3 2. Key Design Principles

30.3.1 2.1 Simplifications from RAPIDS

STREAM is intentionally simplified for tutorial purposes:

Feature	RAPIDS	STREAM
Network Interfaces	Yes (Network master/slave)	✗ No (pure memory-to-memory)
Address Alignment	Complex fixup logic	✓ Aligned addresses only
Credit Management	Exponential encoding	✓ Simple transaction limits
Control Engines	Control read/write engines	✗ No (direct APB config)
Descriptor Length	Chunks (4-byte)	✓ Beats (data width)
Program Engine	Complex alignment FSM	✓ Simplified coordination

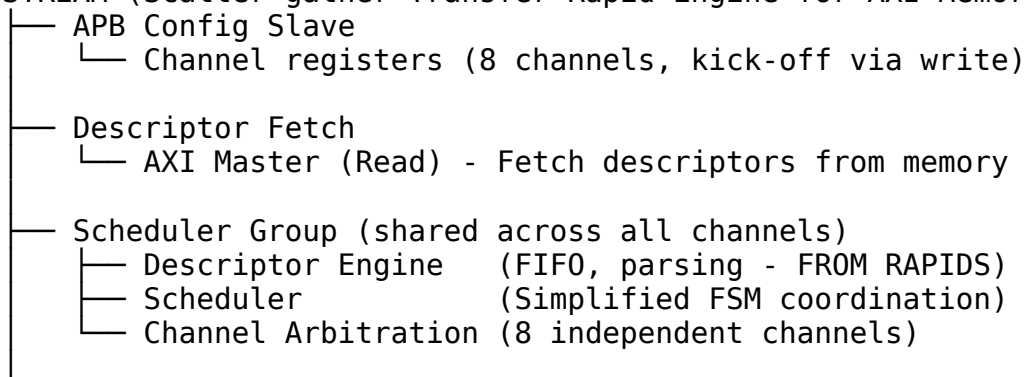
30.3.22.2 Tutorial-Friendly Features

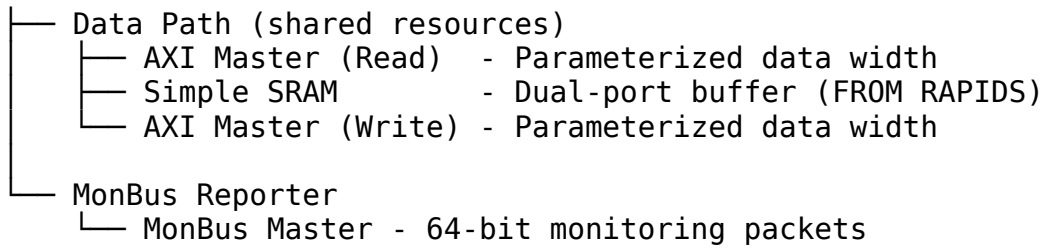
- **Aligned Addresses:** Source and destination addresses must be aligned to data width
- **Length in Beats:** Descriptor length specified in data beats (not bytes/chunks)
- **Single APB Write:** One APB register write kicks off entire descriptor chain
- **No Circular Buffers:** Chained descriptors with explicit termination
- **Parameterized Engines:** Multiple AXI engine versions (compile-time selection)

30.4 3. Architecture Overview

30.4.13.1 Top-Level Block Diagram

STREAM (Scatter-gather Transfer Rapid Engine for AXI Memory)





30.4.23.2 Data Flow

Descriptor-Based Transfer Sequence:

1. Software writes to APB channel register
↓
2. Descriptor fetch via AXI descriptor master
↓
3. Descriptor Engine parses descriptor fields
↓
4. Scheduler coordinates data transfer:
 - a. AXI Read Engine fetches source data → SRAM buffer
 - b. AXI Write Engine writes SRAM → destination
 ↓
5. Check for chained descriptor (next_descriptor_ptr != 0)
↓ (if chained)
6. Fetch next descriptor, repeat from step 3
↓ (if last)
7. Generate MonBus completion packet

Channel Independence: - 8 channels operate independently - All channels share: SRAM, AXI data masters, descriptor fetch master - Arbitration required for shared resources

30.5 4. Interfaces

30.5.1 4.1 External Interfaces

Interface	Type	Width	Purpose	Notes
APB Slave	Slave	32-bit	Configuration, channel kick-off	Write to channel register starts transfer
AXI Master (Descriptor)	Master	256-bit	Fetch descriptors from memory	Dedicated descriptor fetch path

Interface	Type	Width	Purpose	Notes
AXI Master (Data Read)	Master	Parameterizable	Read source data	Multiple engine versions (compile-time)
AXI Master (Data Write)	Master	Parameterizable	Write destination data	Multiple engine versions (compile-time)
MonBus Master	Master	64-bit	Monitor packet output	Standard AMBA format

30.5.24.2 Descriptor Format

256-bit Descriptor Structure:

Bits	Field	Description
[63:0]	src_addr	Source address (64-bit, must be aligned to data width)
[127:64]	dst_addr	Destination address (64-bit, must be aligned to data width)
[159:128]	length	Transfer length in BEATS (not bytes!)
[191:160]	next_descriptor_ptr	Address of next descriptor (0 = last in chain)
[192]	valid	Descriptor is valid
[193]	interrupt	Generate interrupt on completion
[194]	last	Last descriptor in chain (explicit flag)
[195]	error	Error status (used for reporting)
[199:196]	channel_id	Channel ID (0-7)
[207:200]	priority	Transfer priority (for arbitration)
[255:208]	reserved	Reserved for future use

Key Descriptor Features: - ✓ **Chained descriptors:** next_descriptor_ptr links to next descriptor - ✗ **No circular buffers:** Explicit termination (last flag or ptr=0) - ✓ **Length in beats:**

Simplified for tutorial (no byte/chunk conversion) - ✓ **Aligned addresses:** Tutorial constraint (performance hidden for now)

30.6 5. Key Components

30.6.1 5.1 Descriptor Engine (APB-Only for STREAM)

Source: Adapted from RAPIDS descriptor_engine.sv

Purpose: - Autonomous descriptor fetch and chaining - APB interface for initial descriptor address - AXI read interface for descriptor memory fetches - Descriptor FIFO storage and distribution

Key Features: - ✓ **Autonomous chaining:** Automatically fetches next descriptor if next_descriptor_ptr != 0 AND last == 0 - ✓ **Address validation:** Validates next descriptor addresses against cfg_addr0/1_base/limit - ✓ **APB blocking:** APB blocked until channel_idle == 1 (channel fully idle) - ✓ **Error handling:** AXI errors stop chaining, set descriptor_error, block descriptor_valid

Adaptations from RAPIDS: - ✓ **APB-only:** Single APB write kicks off entire descriptor chain - ✓ **Descriptor Read Address FIFO:** 2-deep FIFO stores addresses for AXI fetch (APB + chaining) - ✓ **Chaining logic:** Descriptor engine autonomously manages next_descriptor_ptr chaining

Idle Signal: - descriptor_engine_idle asserted when: - FSM in RD_IDLE state - No pending descriptor fetches (address FIFO empty) - No active AXI transactions

30.6.2 5.2 Scheduler Group (Integration Wrapper)

Purpose: Wraps descriptor engine and scheduler into a single channel processing unit

Architecture:

```
scheduler_group (  
    // APB interface (from APB config slave)  
    .apb_valid          (apb_valid),  
    .apb_ready          (apb_ready),    // Blocked when channel not idle  
    .apb_addr           (descriptor_addr),  
  
    // Channel idle signal composition (CRITICAL!)  
    .channel_idle       (channel_idle),  
  
    // Descriptor → Scheduler flow  
    .descriptor_valid   (desc_valid),  
    .descriptor_ready   (desc_ready),  
    .descriptor_packet   (desc_packet),
```



```

// Data engine interfaces
.datard_*      (datard_*),    // Read engine
.datawr_*      (datawr_*),    // Write engine

// Status
.scheduler_idle (sched_idle),
.descriptor_idle (desc_idle)
);

```

Channel Idle Signal Composition:

```

// Channel is idle ONLY when BOTH sub-blocks are idle
assign channel_idle = scheduler_idle && descriptor_engine_idle;

```

Why Both Signals Matter:

Signal	Indicates	Used For
scheduler_idle	No active data transfers, all descriptors processed	Prevents new APB request during active transfer
descriptor_engine_idle	No pending descriptor fetches (FIFO empty)	Prevents new APB request during chaining
channel_idle (AND of both)	Channel fully quiescent	Gates APB interface

APB Blocking Logic:

```

// Descriptor engine blocks APB when channel not idle
assign apb_ready = apb_skid_ready_in &&
                  !r_channel_reset_active &&
                  w_desc_addr_fifo_empty &&
                  channel_idle;
// No pending fetches
// Scheduler +
descriptor_idle

```

Example Scenario:

1. Software writes APB → descriptor_addr = 0x1000
 - channel_idle = 1 (both idle)
 - APB accepted
2. Descriptor engine fetches descriptor @ 0x1000
 - descriptor_engine_idle = 0 (fetch in progress)
 - channel_idle = 0
 - APB BLOCKED
3. Descriptor pushed to scheduler
 - descriptor_engine_idle = 1 (fetch complete)
 - scheduler_idle = 0 (transfer starting)

- channel_idle = 0
 - APB BLOCKED
4. Scheduler completes data transfer
 - Descriptor has next_descriptor_ptr = 0x1100 (chained!)
 - Descriptor engine autonomously fetches @ 0x1100
 - descriptor_engine_idle = 0 (autonomous fetch)
 - channel_idle = 0
 - APB BLOCKED
 5. Final descriptor completes (last = 1 OR next_ptr = 0)
 - scheduler_idle = 1 (transfer done)
 - descriptor_engine_idle = 1 (no more fetches)
 - channel_idle = 1
 - APB UNBLOCKED (ready for next transfer!)

Key Insight: The AND gate ensures software cannot interrupt a descriptor chain in progress!

30.6.35.3 Scheduler (Simplified from RAPIDS)

Purpose: - Coordinate descriptor-to-data-transfer flow - Manage 8 independent channels - Arbitrate shared resources (SRAM, AXI masters)

FSM States:

```
typedef enum logic [7:0] {
    SCHED_IDLE = 8'b00000001, // Idle, waiting for
channel activation
    SCHED_FETCH_DESCRIPTOR = 8'b00000010, // Fetch descriptor via
AXI master
    SCHED_PARSE_DESCRIPTOR = 8'b00000100, // Parse descriptor fields
    SCHED_READ_PHASE = 8'b00001000, // Coordinate read engine
    SCHED_WRITE_PHASE = 8'b00010000, // Coordinate write engine
    SCHED_CHAIN_CHECK = 8'b00100000, // Check for next
descriptor
    SCHED_COMPLETE = 8'b01000000, // Transfer complete,
report status
    SCHED_ERROR = 8'b10000000 // Error state
} scheduler_state_t;
```

Key Differences from RAPIDS: - ✗ No credit management (just simple transaction limits) - ✗ No program engine coordination (no alignment fixup) - ✓ Simplified FSM (no control read/write phases)

30.6.45.3 AXI Read Engine (Streaming Pipeline - NO FSM)

Purpose: High-performance streaming reads from memory to SRAM buffer

Architecture: Pipelined streaming design (NO FSM for performance)

Key Insight: FSMs are horrible for performance! Instead, use: - **Arbiter** selects which channel's datard_* interface gets access - **Streaming pipeline** continuously moves data when granted - **Data interface** (datard_valid, datard_ready, datard_beats_remaining) controls flow

Data Read Interface (per channel):

```
// Channel requests read access
input logic      datard_valid;           // Channel has read request
output logic     datard_ready;           // Engine ready for request
input logic [63:0] datard_addr;          // Source address (aligned)
input logic [31:0] datard_beats_remaining; // Beats left to read
input logic [7:0]  datard_burst_len;     // Preferred burst length
input logic [3:0]  datard_channel_id;    // Channel ID for tracking
```

Multiple Versions (Compile-Time Selection): 1. **Version 1 - Basic:** Single outstanding read, fixed burst length 2. **Version 2 - Pipelined:** Multiple outstanding reads, configurable bursts 3. **Version 3 - Adaptive:** Dynamic burst sizing based on remaining beats

Operation:

1. Arbiter grants channel access based on priority
2. Engine accepts datard_* request (continuous streaming)
3. AXI AR channel issues read burst
4. AXI R channel streams data → SRAM (no FSM stalls!)
5. Engine updates beats_remaining, accepts next request
6. Different channels can have different burst lengths (e.g., CH0: 8 beats, CH1: 16 beats)

Example: Different Burst Lengths per Channel

```
// Channel 0 prefers 8-beat bursts
datard_burst_len[0] = 8'd8;

// Channel 1 prefers 16-beat bursts
datard_burst_len[1] = 8'd16;

// Engine adapts to requested burst length (within MAX_BURST_LEN)
```

30.6.55.4 AXI Write Engine (Streaming Pipeline - NO FSM)

Purpose: High-performance streaming writes from SRAM buffer to memory

Architecture: Pipelined streaming design (NO FSM for performance)

Key Insight: Same as read engine - no FSMs! Use streaming pipeline with arbiter.

Data Write Interface (per channel):

```
// Channel requests write access
input logic      datawr_valid;           // Channel has write
```

```

request
output logic          datawr_ready;           // Engine ready for
request
input logic [63:0] datawr_addr;               // Destination address
(aligned)
input logic [31:0] datawr_beats_remaining;    // Beats left to write
input logic [7:0] datawr_burst_len;          // Preferred burst length
input logic [3:0] datawr_channel_id;         // Channel ID for tracking

```

Multiple Versions (Compile-Time Selection): 1. **Version 1 - Basic:** Single outstanding write, fixed burst length 2. **Version 2 - Pipelined:** Multiple outstanding writes, configurable bursts 3. **Version 3 - Adaptive:** Dynamic burst sizing based on remaining beats

Operation:

1. Arbiter grants channel access based on priority
2. Engine accepts datawr_* request (continuous streaming)
3. Engine reads data from SRAM
4. AXI AW channel issues write address
5. AXI W channel streams data (no FSM stalls!)
6. AXI B channel receives response, updates beats_remaining
7. Different channels can have different burst lengths (e.g., CH0: 8 beats, CH2: 32 beats)

Read/Write Asymmetry Example:

```

// Channel can use different burst lengths for read vs write
// Example: Read in small bursts, write in large bursts
datard_burst_len[0] = 8'd8;    // Read: 8 beats
datawr_burst_len[0] = 8'd16;   // Write: 16 beats

// Engine handles asymmetry via SRAM buffering

```

30.6.65.5 Simple SRAM

Source: Direct copy from RAPIDS simple_sram.sv

Purpose: - Dual-port SRAM buffer - Decouples read and write engines - Shared across all channels (arbitration required)

Why Reuse: - Standard dual-port SRAM design - Proven in RAPIDS integration tests - Parameterizable depth and width

30.7 6. Configuration and Control

30.7.1 6.1 APB Register Map

Offset	Register	Access	Description
0x0000	GLOBAL_CTRL	RW	Global enable, reset
0x0004	GLOBAL_STATUS	RO	Global status, error flags
0x0100	CH0_CTRL	WO	Channel 0 kick-off (write descriptor address)
0x0104	CH0_STATUS	RO	Channel 0 status
0x0108	CH0_DESC_ADDR	RO	Channel 0 current descriptor address
0x010C	CH0_BYTES_XFER	RO	Channel 0 bytes transferred
...	...	...	... (repeat for channels 1-7)
0x0200	CH1_CTRL	WO	Channel 1 kick-off
...			
0x0700	CH7_CTRL	WO	Channel 7 kick-off

Kick-Off Sequence: 1. Software writes descriptor address to CHx_CTRL register 2. STREAM fetches descriptor from memory 3. Transfer begins automatically 4. If chained, STREAM follows next_descriptor_ptr automatically 5. Completion reported via MonBus packet

30.7.2 6.2 Channel Configuration

Each channel independently configurable: - **Descriptor start address:** Written to CHx_CTRL - **Priority:** Encoded in descriptor (arbitration) - **Interrupt enable:** Per-descriptor flag - **Status monitoring:** Read CHx_STATUS

30.8 7. Resource Sharing and Arbitration

30.8.1 7.1 Shared Resources

All channels share: 1. **Descriptor Fetch AXI Master** - Fetches descriptors for all channels 2. **Data Read AXI Master** - Reads source data for all channels 3. **Data Write AXI Master** - Writes

destination data for all channels 4. **SRAM Buffer** - Shared buffer (dual-port, but still arbitrated) 5. **MonBus Reporter** - Single monitor output

30.8.27.2 Arbitration Strategy

Priority-Based Round-Robin: - Channels have priority field in descriptor - Higher priority = serviced first - Within same priority: round-robin - Prevents starvation with timeout

Example Arbitration:

Channel 0: Priority 7 (highest)
Channel 1: Priority 5
Channel 2: Priority 5
Channel 3: Priority 3

Service order: CH0 → CH1 → CH2 (round-robin) → CH0 → CH1 → CH2 → CH3 ...

30.9 8. Error Detection and Recovery

30.9.18.1 Error Types

Error Type	Detection	Response
Invalid descriptor	Valid bit = 0	Skip, move to next
Alignment error	Address not aligned	Set error flag, halt channel
AXI SLVERR	AXI response	Set error flag, halt channel
AXI DECERR	AXI response	Set error flag, halt channel
Timeout	Transaction timeout	Set error flag, halt channel
SRAM overflow	Buffer full	Backpressure, wait

30.9.28.2 Error Recovery

Per-Channel Error Handling: - Error sets channel to CH_ERROR state - Channel halts, does not affect other channels - Software must: 1. Read CHx_STATUS to identify error 2. Clear error condition 3. Re-kick channel with new descriptor

No Automatic Retry: - Tutorial design keeps error handling simple - Software responsible for retry logic

30.10 9. MonBus Integration

30.10.1 9.1 Standard MonBus Format

Uses standard 64-bit MonBus packet format: - [63:60] Packet Type (0=ERROR, 1=COMPL, etc.) - [59:57] Protocol (custom STREAM protocol) - [56:53] Event Code (STREAM-specific events) - [52:47] Channel ID (0-7) - [46:43] Unit ID (unused for STREAM) - [42:35] Agent ID (unused for STREAM) - [34:0] Event Data (address, byte count, etc.)

30.10.2 9.2 STREAM Event Codes

Code	Event	Description
0x0	DESC_START	Descriptor started
0x1	DESC_COMPLETE	Descriptor completed
0x2	READ_START	Read phase started
0x3	READ_COMPLETE	Read phase completed
0x4	WRITE_START	Write phase started
0x5	WRITE_COMPLETE	Write phase completed
0x6	CHAIN_FETCH	Chained descriptor fetch
0xF	ERROR	Error occurred

30.10.3 9.3 Default Configuration

Tutorial-Friendly Defaults: - **Errors only:** `cfg_error_enable = 1`, all others = 0 - **Interrupts:** Descriptor flag controls per-transfer interrupt - **Reduces MonBus traffic** for beginner understanding

30.11 10. Design Constraints

30.11.1 10.1 Tutorial Constraints (Intentional Simplifications)

Constraint	Rationale
Aligned addresses only	Simplify data path, hide alignment

Constraint	Rationale
	complexity
Length in beats	Avoid byte/chunk conversion math
No circular buffers	Explicit termination easier to understand
Single APB kick-off	Simple software model
No credit management	Avoid exponential encoding complexity

30.11.2 10.2 Implementation Constraints

Parameter	Value	Notes
Max channels	8	Compile-time parameter
Max burst length	256	AXI4 spec limit
Descriptor size	256 bits	4 × 64-bit words
SRAM depth	Parameterizable	Typical: 1024-4096 entries
Data width	Parameterizable	Typical: 512-bit

30.12 11. Verification Strategy

30.12.1 11.1 Test Organization

```

projects/components/dmas/stream/dv/tests/
├── fub_tests/                # Functional Unit Block tests
│   ├── descriptor_engine/    # Copy from RAPIDS (adapt imports)
│   ├── scheduler/           # Simplified scheduler tests
│   ├── axi_engines/         # Read/write engine tests
│   └── sram/                 # SRAM tests
├── integration_tests/        # Multi-block scenarios
│   ├── single_channel/      # Single channel transfers
│   ├── multi_channel/       # 8-channel concurrent
│   ├── chained_descriptors/ # Descriptor chain tests
│   └── error_handling/      # Error recovery tests

```


30.12.2 11.2 Test Levels

Basic (Quick Smoke Tests): - Single descriptor, single channel - Aligned addresses, simple transfers - ~30 seconds runtime

Medium (Typical Scenarios): - Multiple descriptors, 2-4 channels - Chained descriptors (2-3 deep) - ~90 seconds runtime

Full (Comprehensive Validation): - All 8 channels concurrent - Long descriptor chains (10+ deep) - Error injection, stress testing - ~180 seconds runtime

30.13 12. Performance Characteristics

30.13.1 12.1 Throughput by Engine Version

V1 (Low Performance - Tutorial Mode): - **Throughput:** 0.14 beats/cycle (DDR4), 0.40 beats/cycle (SRAM) - **Architecture:** Single outstanding transaction, blocks on completion - **Use Case:** Tutorial examples, embedded systems, low-latency SRAM

V2 (Medium Performance - Command Pipelined): - **Throughput:** 0.94 beats/cycle (DDR4), 0.85 beats/cycle (SRAM) - **Improvement:** 6.7x over V1 (DDR4), 2.1x over V1 (SRAM) - **Architecture:** Command queue (4-8 deep), hides memory latency - **Use Case:** General-purpose FPGA, DDR3/DDR4, best area efficiency

V3 (High Performance - Out-of-Order): - **Throughput:** 0.98 beats/cycle (DDR4), 0.92 beats/cycle (SRAM) - **Improvement:** 7.0x over V1 (DDR4), 2.3x over V1 (SRAM) - **Architecture:** OOO command selection, maximizes memory controller efficiency - **Use Case:** Datacenter, ASIC, HBM2, high-performance memory

Key Insight: Benefit scales with memory latency. V1 throughput degrades from 40% (SRAM) to 14% (DDR4), while V2/V3 maintain 94-98% throughput regardless of latency.

Configuration Parameters: - `ENABLE_CMD_PIPELINE = 0`: V1 (default, tutorial mode) - `ENABLE_CMD_PIPELINE = 1`: V2 (command pipelined) - `ENABLE_CMD_PIPELINE = 1`, `ENABLE_OOO_DRAIN = 1` (write) or `ENABLE_OOO_READ = 1` (read): V3

Factors Affecting Throughput: - Memory latency (V2/V3 hide latency via pipelining) - SRAM buffer size (limits burst pipelining) - Channel arbitration overhead - Descriptor fetch latency - Engine version (V1/V2/V3 configuration)

30.13.2 12.2 Latency

Transfer Latency Breakdown: - Descriptor fetch: ~10-50 cycles (depends on memory) - Read phase: $(\text{length} / \text{burst_len}) \times \text{burst_latency}$ - Write phase: $(\text{length} / \text{burst_len}) \times \text{burst_latency}$ - End-to-end: Typically <200 cycles for small transfers (V1), <100 cycles (V2/V3 pipelined)

V2/V3 Latency Hiding: - Command pipelining overlaps descriptor fetch, read, write operations - Multiple outstanding transactions hide memory latency - OOO completion (V3) reduces head-of-line blocking

30.13.3 12.3 Resource Utilization by Engine Version

V1 Configuration (Tutorial - Minimum Area): - Total: ~9,500 LUTs + 64 KB SRAM - Descriptor Engine (8×): ~2,400 LUTs - Scheduler (8×): ~3,200 LUTs - AXI Read Engine: ~1,250 LUTs - AXI Write Engine: ~1,250 LUTs - SRAM Controller: ~1,600 LUTs - APB Config: ~350 LUTs - MonBus AXIL Group: ~1,000 LUTs

V2 Configuration (Balanced - Best Area Efficiency): - Total: ~11,000 LUTs + 64 KB SRAM (1.16x area) - AXI Read Engine: ~2,000 LUTs (1.6x increase, 6.7x throughput) - AXI Write Engine: ~2,500 LUTs (2.0x increase, 6.7x throughput) - Other blocks: Same as V1

V3 Configuration (Maximum Performance): - Total: ~14,000 LUTs + 64 KB SRAM (1.47x area) - AXI Read Engine: ~3,500 LUTs (2.8x increase, 7.0x throughput) - AXI Write Engine: ~4,000 LUTs (3.2x increase, 7.0x throughput) - Other blocks: Same as V1

Area Efficiency Comparison: - V1: 1.00 throughput / 1.00 area = 1.00 - V2: 6.70 throughput / 1.16 area = 5.78 (best efficiency) - V3: 7.00 throughput / 1.47 area = 4.76

Recommendation: V2 provides best area efficiency for most use cases. V3 justified only for high-performance memory controllers that support OOO responses.

30.14 13. Development Roadmap

30.14.1 13.1 Phase 1: Foundation (Current)

- ✓ Directory structure
- ✓ Package definitions (`stream_pkg.sv`)
- ✓ Imports header (`stream_imports.svh`)
- 🕒 Documentation (this PRD)

30.14.2 13.2 Phase 2: Core Blocks

- Adapt descriptor engine from RAPIDS
- Design simplified scheduler
- Create APB config interface
- Copy simple SRAM from RAPIDS

30.14.3 13.3 Phase 3: Data Path

- AXI read engine (version 1 - basic)

- AXI write engine (version 1 - basic)
- SRAM integration
- Channel arbitration

30.14.4 13.4 Phase 4: Integration

- Top-level module
- MonBus reporter
- Integration testbench
- Single-channel validation

30.14.5 13.5 Phase 5: Multi-Channel

- 8-channel support
- Arbiter implementation
- Multi-channel tests
- Performance tuning

30.14.6 13.6 Phase 6: Advanced Engines (Future - V2/V3)

Goal: Add parameterized high-performance engine variants

V2 - Command Pipelined (Medium Performance): - Command queue implementation (4-8 deep) - W drain FSM for write engine - B response scoreboard (write) or in-order R reception (read) - Per-command SRAM pointer tracking - Parameter: `ENABLE_CMD_PIPELINE = 1` - Expected: 6.7x throughput improvement over V1

V3 - Out-of-Order Completion (High Performance): - OOO command selection logic - Transaction ID matching (AXI ID to queue entry) - SRAM data availability checking (write engine) - R beat matching to queue entry (read engine) - Parameters: `ENABLE_CMD_PIPELINE = 1`, `ENABLE_OOO_DRAIN = 1` (write) or `ENABLE_OOO_READ = 1` (read) - Expected: 7.0x throughput improvement over V1

Deliverables: - Updated RTL with parameterization - Performance comparison tests (V1 vs V2 vs V3) - Tutorial documentation explaining trade-offs - Area/throughput measurements on target FPGA

30.15 14. Educational Value

30.15.1 14.1 Learning Objectives

STREAM teaches: 1. **Descriptor-based DMA design patterns** - Descriptor structure and parsing - Chained descriptors (scatter-gather) - Descriptor fetch via AXI

2. AXI4 Memory Interface Usage

- Burst transactions
- Address/data/response channels
- Outstanding transactions

3. Resource Sharing and Arbitration

- Multiple channels sharing AXI masters
- Priority-based arbitration
- Conflict resolution

4. FSM Design and Coordination

- Multiple interconnected FSMs
- State machine composition
- Error handling

5. Buffer Management

- SRAM-based buffering
- Rate matching
- Flow control

30.15.2 14.2 Progression Path

Learning Sequence: 1. **STREAM (this project):** Memory-to-memory DMA, aligned addresses 2. **STREAM Extended:** Add alignment fixup, more complex scenarios 3. **RAPIDS:** Add network interfaces, credit management, full complexity

30.16 15. Success Criteria

30.16.1 15.1 Functional

- ✓ Single descriptor transfer working
- ✓ Chained descriptors (2-3 deep)
- ✓ Multi-channel operation (8 channels concurrent)
- ✓ Error detection and reporting
- ✓ MonBus packet generation
- ✓ >90% functional test coverage

30.16.2 15.2 Quality

- ✓ Verilator compiles with 0 warnings

- ✓ All tests passing (100% success rate)
- ✓ Comprehensive documentation
- ✓ Tutorial-quality code comments

30.16.3 15.3 Performance

- ✓ Achieves >80% of theoretical AXI bandwidth
 - ✓ <5K LUT utilization (excluding SRAM)
 - ✓ <200 cycle end-to-end latency for small transfers
-

30.17 16. Open Questions (For Review)

30.17.1 16.1 Descriptor Engine Adaptation

- **Q:** Should descriptor engine use APB-only, or support additional modes?
- **A (pending):** TBD - depends on software use case preference

30.17.2 16.2 AXI Descriptor Master

- **Q:** Fixed 256-bit width, or parameterizable?
- **A (pending):** Propose fixed 256-bit for simplicity

30.17.3 16.3 Channel Arbitration

- **Q:** Fixed priority, or dynamic priority based on age/fairness?
- **A (pending):** Propose fixed priority with round-robin for tutorial simplicity

30.17.4 16.4 SRAM Partitioning

- **Q:** Single shared SRAM, or per-channel SRAMs?
 - **A (pending):** Propose single shared SRAM with arbitration (matches RAPIDS pattern)
-

30.18 16. Attribution and Contribution Guidelines

30.18.1 16.1 Git Commit Attribution

When creating git commits for STREAM documentation or implementation:

Use:

Documentation and implementation support by Claude.

Do NOT use:

Co-Authored-By: Claude <noreply@anthropic.com>

Rationale: STREAM documentation and organization receives AI assistance for structure and clarity, while design concepts and architectural decisions remain human-authored.

30.19 16.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

/mnt/data/github/rtldesignsherpa/projects/components/dmas/stream/docs/

Quick Command: Use the provided shell script:

```
cd
/mnt/data/github/rtldesignsherpa/projects/components/dmas/stream/docs
./generate_pdf.sh
```

The shell script will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the stream_spec index file 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

 **See:** bin/md_to_docx.py for complete implementation details

30.20 17. References

30.20.1 17.1 Internal Documentation

- **RAPIDS PRD:** projects/components/dmas/rapids/PRD.md - Parent architecture
- **RAPIDS Descriptor Engine:** projects/components/dmas/rapids/rtl/rapids_fub/descriptor_engine.sv
- **RAPIDS Simple SRAM:** projects/components/dmas/rapids/rtl/rapids_fub/simple_sram.sv
- **AMBA PRD:** rtl/amba/PRD.md - MonBus integration
- **Repository Guide:** /CLAUDE.md - Design patterns and conventions

30.20.2 17.2 External References

- **AXI4 Specification:** ARM IHI0022E
- **APB Specification:** ARM IHI0024C

- CocoTB Documentation: <https://docs.cocotb.org/>
 - Verilator Manual: <https://verilator.org/guide/latest/>
-

Document Version: 1.0 **Last Updated:** 2025-10-17 **Review Cycle:** Weekly during initial design
Next Review: TBD (after user feedback) **Owner:** RTL Design Sherpa Project

30.21 Navigation

- ← **Back to Root:** /PRD.md
- **Parent Architecture:** projects/components/dmas/rapids/PRD.md
- **AI Guidance:** CLAUDE.md (to be created)
- **Quick Start:** README.md (to be created)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

31 Claude Code Guide: STREAM Subsystem

Version: 1.0 **Last Updated:** 2025-10-17 **Purpose:** AI-specific guidance for working with STREAM subsystem

31.1 Quick Context

What: STREAM - Scatter-gather Transfer Rapid Engine for AXI Memory **Status:** 🟡 Initial design - tutorial-focused DMA engine **Your Role:** Help users build a beginner-friendly descriptor-based DMA engine

📖 **Complete Specification:** projects/components/dmas/stream/PRD.md ← **Always reference this for technical details**

31.2 📖 Global Requirements Reference

IMPORTANT: Review /GLOBAL_REQUIREMENTS.md for all mandatory requirements

All mandatory requirements are consolidated in the global requirements document: - **See:** /GLOBAL_REQUIREMENTS.md - Repository-wide mandatory requirements - **STREAM-Specific:** Attribution format, tutorial focus (intentional simplifications) - **Universal:** TB location, three methods, TBBase inheritance, 100% success

This CLAUDE.md provides STREAM-specific guidance. Also review: - Root /CLAUDE.md - Repository-wide patterns - projects/components/CLAUDE.md - Project area standards (reset macros, FPGA attributes) - bin/TBClasses/CLAUDE.md - Framework usage patterns

31.3 Critical Rules for This Subsystem

31.3.1 Rule #0: Attribution Format for Git Commits

IMPORTANT: When creating git commit messages for STREAM documentation or code:

Use:

Documentation and implementation support by Claude.

Do NOT use:

Co-Authored-By: Claude <noreply@anthropic.com>

Rationale: STREAM receives AI assistance for structure and clarity, while design concepts and architectural decisions remain human-authored.

31.3.2 Rule #0.1: TUTORIAL FOCUS - Intentional Simplifications

⚠️ **STREAM is INTENTIONALLY SIMPLIFIED for educational purposes** ⚠️

Key Simplifications (DO NOT “fix” these): 1. ✓ **Aligned addresses only** - No alignment fixup logic (kept for RAPIDS) 2. ✓ **Length in beats** - Not bytes or chunks (simplifies math) 3. ✓ **No circular buffers** - Explicit chain termination only 4. ✓ **No credit management** - Simple transaction limits 5. ✓ **Pure memory-to-memory** - No network interfaces

When users ask “Can we add alignment fixup?”: - ✓ **Correct answer:** “STREAM intentionally keeps addresses aligned for tutorial simplicity. For complex alignment, see RAPIDS.” - ✗ **Wrong answer:** “Sure, let me add alignment logic...” (defeats tutorial purpose!)

31.3.3 Rule #0.1: Testbench Location and Test Structure (MANDATORY)

📖 **See:** /GLOBAL_REQUIREMENTS.md Section 2.1 for complete requirement

STREAM-Specific Directory Structure:

```
projects/components/dmas/stream/dv/  
├── tbclasses/                # ★ STREAM TB classes (project  
area!)
```



```

├── scheduler_tb.py           # Scheduler testbench
├── descriptor_engine_tb.py   # Descriptor engine testbench
├── axi_engine_tb.py         # AXI engine testbenches
└── tests/fub_tests/         # Test runners import from
    tbclasses/

```

STREAM Import Pattern:

```

# Import STREAM TB from project area
from projects.components.dmas.stream.dv.tbclasses.scheduler_tb import
StreamSchedulerTB

```

```

# Shared utilities from framework
from TBClasses.shared.tbbase import TBBBase

```

📖 **Complete Pattern:** projects/components/dmas/rapids/CLAUDE.md Rule #0.1

31.3.4 Rule #0.2: Three Mandatory TB Methods (MANDATORY)

📖 **See:** /GLOBAL_REQUIREMENTS.md Section 2.2 for complete requirement

STREAM-Specific Context:

STREAM has simpler config requirements than RAPIDS - most config can be set after reset.

Standard STREAM Pattern:

```

class StreamSchedulerTB(TBBBase):
    async def setup_clocks_and_reset(self):
        """Standard STREAM initialization"""
        await self.start_clock('clk', freq=10, units='ns')
        await self.assert_reset()
        await self.wait_clocks('clk', 10)
        await self.deassert_reset()
        await self.wait_clocks('clk', 5)

    async def assert_reset(self):
        self.dut.rst_n.value = 0

    async def deassert_reset(self):
        self.dut.rst_n.value = 1

```

Note: Unlike RAPIDS, STREAM typically doesn't need config set before reset.

31.3.5 Rule #1: REUSE from RAPIDS Where Appropriate

Direct Reuse (No Modification): - ✓ descriptor_engine.sv - Works with STREAM descriptors -
 ✓ simple_sram.sv - Standard dual-port SRAM - ✓ Descriptor engine tests - Adapt imports only

Adapt from RAPIDS: - ⚠ scheduler.sv - **Simplify FSM** (no credit management, no control engines) - ⚠ AXI engines - **Create simplified versions** (no alignment fixup)

Create New for STREAM: - 📝 APB config interface - PeakRDL-generated (like HPET), 8 channels, kick-off registers - 📝 Top-level integration - Different interface set

Always Ask Yourself: “Can I reuse from RAPIDS instead of creating new?”

31.3.6 Rule #2: Descriptor Format is DIFFERENT from RAPIDS

STREAM Descriptor (256-bit): - src_addr (64-bit), dst_addr (64-bit), length (**beats**), next_descriptor_ptr (32-bit) - **Length is in BEATS, not chunks!**

RAPIDS Descriptor: - Uses chunks (4-byte units) - Has alignment metadata

When comparing/referencing RAPIDS: - ✓ “RAPIDS uses chunks, STREAM uses beats for tutorial simplicity” - ✗ Don’t assume RAPIDS descriptor format applies to STREAM

31.3.7 Rule #3: Know the Shared Resources

All 8 channels share: 1. Descriptor fetch AXI master 2. Data read AXI master 3. Data write AXI master 4. SRAM buffer 5. MonBus reporter

Arbitration is required! - Never assume exclusive access - All shared resources need arbiter logic - Priority-based round-robin (descriptor priority field)

31.4 Architecture Quick Reference

31.4.1 Block Organization

STREAM Architecture (Estimated: 8-10 modules)

— APB Config	
— apb_config_slave.sv	(To be created)
— Scheduler Group	
— descriptor_engine.sv	(FROM RAPIDS - adapt imports)
— scheduler.sv	(Simplified from RAPIDS)
— channel_arbiter.sv	(To be created)
— Data Path	
— axi_read_engine_v1.sv	(Version 1: Basic)
— axi_read_engine_v2.sv	(Version 2: Pipelined)
— axi_write_engine_v1.sv	(Version 1: Basic)
— axi_write_engine_v2.sv	(Version 2: Pipelined)
— simple_sram.sv	(FROM RAPIDS - no changes)

└─ MonBus Reporter
 └─ monbus_reporter.sv (To be created)

31.4.2 Module Status

Module	Source	Status	Notes
descriptor_engine.sv	RAPIDS (copy)	✓ Copied	Adapt #include only
simple_sram.sv	RAPIDS (copy)	✓ Copied	No changes needed
stream_pkg.sv	New	✓ Created	Descriptor format defined
stream_imports.svh	New	✓ Created	Package imports
scheduler.sv	RAPIDS (simplify)	⌚ Pending	Remove credit mgmt, control engines
apb_config_slave.sv	New	⌚ Pending	8 channel registers
axi_read_engine_v1.sv	New	⌚ Pending	Basic version
axi_write_engine_v1.sv	New	⌚ Pending	Basic version
channel_arbiter.sv	New	⌚ Pending	Priority-based round-robin
monbus_reporter.sv	New	⌚ Pending	STREAM event codes
stream_top.sv	New	⌚ Pending	Top-level integration

31.5 Common User Questions and Responses

31.5.1 Q: “How is STREAM different from RAPIDS?”

A: STREAM is intentionally simplified for tutorial purposes:

Simplifications: | Feature | RAPIDS | STREAM | |———|———|———| | **Interfaces** | APB + AXI + Network | APB + AXI only | | **Address Alignment** | Complex fixup | Aligned only | | **Credit**

Management | Exponential encoding | Simple limits | | **Descriptor Length** | Chunks (4-byte) | Beats (data width) | | **Control Engines** | Control read/write | None (direct APB) |

Learning Path: 1. **STREAM** - Basic DMA with scatter-gather 2. **STREAM Extended** - Add alignment fixup 3. **RAPIDS** - Full complexity with network + credit management

 **See:** PRD.md Section 2.1 for complete comparison table

31.5.2 Q: “How do I kick off a transfer?”

A: Single APB write starts descriptor chain:

```
// Software writes descriptor address to channel register
write_apb(ADDR_CH0_CTRL, 0x1000_0000); // Start address of descriptor

// STREAM automatically:
// 1. Fetches descriptor from 0x1000_0000
// 2. Parses src_addr, dst_addr, length
// 3. Reads source data → SRAM
// 4. Writes SRAM → destination
// 5. Checks next_descriptor_ptr
//     - If != 0: Fetch next descriptor, repeat
//     - If == 0 or last flag set: Complete
```

Chained Descriptors:

```
Descriptor 0 @ 0x1000_0000:
  src_addr = 0x2000_0000
  dst_addr = 0x3000_0000
  length = 64 beats
  next_descriptor_ptr = 0x1000_0100 ← Points to next descriptor
```

```
Descriptor 1 @ 0x1000_0100:
  src_addr = 0x2000_1000
  dst_addr = 0x3000_1000
  length = 32 beats
  next_descriptor_ptr = 0x0000_0000 ← Last descriptor (0 = stop)
```

 **See:** PRD.md Section 3.2 for complete data flow

31.5.3 Q: “How many channels can I use?”

A: Maximum 8 independent channels:

Channel Operation: - Each channel has own FSM state - Each channel can have separate descriptor chain - All channels share: AXI masters, SRAM, MonBus

Arbitration: - Priority-based (descriptor priority field) - Round-robin within same priority - Prevents starvation with timeout

Example:

```
// Kick off 3 channels concurrently
write_apb(ADDR_CH0_CTRL, desc0_addr); // Channel 0: Priority 7
write_apb(ADDR_CH1_CTRL, desc1_addr); // Channel 1: Priority 5
write_apb(ADDR_CH2_CTRL, desc2_addr); // Channel 2: Priority 5

// Service order: CH0 → CH1 → CH2 → CH0 → CH1 → CH2 ...
```

 **See:** PRD.md Section 7 for arbitration details

31.5.4 Q: “What’s the descriptor format?”

A: 256-bit descriptor (4 × 64-bit words):

```
typedef struct packed {
    logic [63:0] reserved;           // [255:192] Reserved
    logic [7:0] priority;             // [207:200] Priority
    logic [3:0] channel_id;          // [199:196] Channel ID
    logic error;                     // [195] Error flag
    logic last;                       // [194] Last in chain
    logic interrupt;                 // [193] Generate interrupt
    logic valid;                     // [192] Valid descriptor
    logic [31:0] next_descriptor_ptr; // [191:160] Next descriptor
    address
    logic [31:0] length;              // [159:128] Length in BEATS
    ★
    logic [63:0] dst_addr;            // [127:64] Destination
    address
    logic [63:0] src_addr;            // [63:0] Source address
} descriptor_t;
```

★ **CRITICAL:** length is in **BEATS**, not bytes or chunks!

Example:

Data width = 512 bits = 64 bytes
Length = 16 beats
Total transfer = 16 × 64 = 1024 bytes

 **See:** PRD.md Section 4.2 for complete descriptor specification

31.5.5 Q: “How do I run STREAM tests?”

A: Multi-layered test approach (same as RAPIDS):

```
# 1. FUB (Functional Unit Block) Tests - Individual blocks
pytest projects/components/dmas/stream/dv/tests/fub_tests/scheduler/ -
v
pytest
```

```
projects/components/dmas/stream/dv/tests/fub_tests/descriptor_engine/  
-v  
pytest projects/components/dmas/stream/dv/tests/fub_tests/ -v # All  
FUB tests
```

2. Integration Tests - Multi-block scenarios

```
pytest projects/components/dmas/stream/dv/tests/integration_tests/ -v
```

Run with waveforms for debugging

```
pytest projects/components/dmas/stream/dv/tests/fub_tests/scheduler/  
--vcd=debug.vcd  
gtkwave debug.vcd
```

Test Organization: - **FUB tests:** Focus on individual block functionality - **Integration tests:** Verify block-to-block interfaces and complete data flows

31.6 Integration Patterns

31.6.1 Pattern 1: Basic STREAM Instantiation

```
stream_top #(  
    .NUM_CHANNELS(8),  
    .DATA_WIDTH(512),  
    .ADDR_WIDTH(64),  
    .SRAM_DEPTH(4096)  
) u_stream (  
    // Clock and Reset  
    .aclk          (system_clk),  
    .aresetn       (system_rst_n),  
  
    // APB Configuration Interface  
    .s_apb_paddr   (apb_paddr),  
    .s_apb_psel    (apb_psel),  
    .s_apb_penable (apb_penable),  
    .s_apb_pwrite  (apb_pwrite),  
    .s_apb_pwdata  (apb_pwdata),  
    .s_apb_pready  (apb_pready),  
    .s_apb_prdata  (apb_prdata),  
    .s_apb_pslverr (apb_pslverr),  
  
    // AXI Master - Descriptor Fetch (256-bit)  
    .m_axi_desc_araddr (desc_araddr),  
    .m_axi_desc_arlen  (desc_arlen),  
    .m_axi_desc_arsize (desc_arsize),  
    .m_axi_desc_arvalid (desc_arvalid),
```

```

.m_axi_desc_arready (desc_arready),
.m_axi_desc_rdata   (desc_rdata),
.m_axi_desc_rresp   (desc_rresp),
.m_axi_desc_rlast   (desc_rlast),
.m_axi_desc_rvalid  (desc_rvalid),
.m_axi_desc_rready  (desc_rready),

// AXI Master - Data Read (parameterizable width)
.m_axi_rd_araddr    (rd_araddr),
// ... (full AXI4 AR + R channels)

// AXI Master - Data Write (parameterizable width)
.m_axi_wr_awaddr    (wr_awaddr),
// ... (full AXI4 AW + W + B channels)

// MonBus Output
.monbus_pkt_valid   (stream_mon_valid),
.monbus_pkt_ready   (stream_mon_ready),
.monbus_pkt_data    (stream_mon_data)
);

```

31.6.2 Pattern 2: Descriptor Creation (Software Model)

```

// C/C++ software model for descriptor creation
typedef struct {
    uint64_t src_addr;           // Source address (must be aligned!)
    uint64_t dst_addr;           // Destination address (must be
aligned!)
    uint32_t length;             // Transfer length in BEATS
    uint32_t next_descriptor_ptr; // Next descriptor address (0 =
last)
    uint8_t control;             // valid | interrupt | last | error |
channel_id | priority
} stream_descriptor_t;

// Create descriptor chain
stream_descriptor_t desc[2];

// Descriptor 0
desc[0].src_addr = 0x80000000ULL; // Aligned to 64B (512-bit data)
desc[0].dst_addr = 0x90000000ULL;
desc[0].length = 64; // 64 beats × 64 bytes = 4KB transfer
desc[0].next_descriptor_ptr = (uint32_t)&desc[1]; // Chain to next
desc[0].control = 0x01; // valid = 1

// Descriptor 1 (last)
desc[1].src_addr = 0x80001000ULL;

```

```

desc[1].dst_addr = 0x90001000ULL;
desc[1].length = 32; // 32 beats × 64 bytes = 2KB transfer
desc[1].next_descriptor_ptr = 0; // Last descriptor
desc[1].control = 0x45; // valid | last | interrupt

// Kick off transfer
write_apb_reg(CH0_CTRL, (uint32_t)&desc[0]);

```

31.6.3 Pattern 3: MonBus Integration

```

// Always add downstream FIFO for MonBus
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(256)
) u_stream_mon_fifo (
    .i_clk      (aclk),
    .i_rst_n    (aresetn),
    .i_data      (monbus_pkt_data),
    .i_valid     (monbus_pkt_valid),
    .o_ready     (monbus_pkt_ready),
    .o_data      (fifo_mon_data),
    .o_valid     (fifo_mon_valid),
    .i_ready     (consumer_ready)
);

```

31.7 Anti-Patterns to Catch

31.7.1 X Anti-Pattern 1: Adding Alignment Fixup

x WRONG:

"Let me add alignment fixup logic to handle unaligned addresses..."

✓ CORRECTED:

"STREAM intentionally requires aligned addresses for tutorial simplicity."

If you need unaligned transfers, that's a great learning exercise **for** extending STREAM, **or use** RAPIDS which has full alignment support."

31.7.2 X Anti-Pattern 2: Using Length in Bytes

x WRONG:

```
descriptor.length = 4096; // Thinking this is 4096 bytes
```

✓ CORRECTED:

// Length is in BEATS, not bytes!

// For 512-bit data width (64 bytes per beat):

```
descriptor.length = 4096 / 64; // = 64 beats for 4096 bytes
```


31.7.3 X Anti-Pattern 3: Circular Buffer Descriptors

x WRONG:

```
// Descriptor chain that loops back
desc[9].next_descriptor_ptr = &desc[0]; // Circular!
```

✓ CORRECTED:

```
"STREAM does not support circular buffers (no stop condition).
Always terminate chains explicitly:
desc[last].next_descriptor_ptr = 0; // Stop
desc[last].last = 1; // Explicit last flag
```

31.7.4 X Anti-Pattern 4: Assuming Exclusive Channel Access

x WRONG:

```
// Assume channel has exclusive AXI master access
assign m_axi_arvalid = channel_request; // No arbitration!
```

✓ CORRECTED:

```
// All channels share AXI masters - arbitration required
channel_arbiter u_arbiter (
    .ch_requests (channel_requests[7:0]),
    .ch_grant     (channel_grant[7:0]),
    .axi_master_available (axi_master_idle)
);
```

31.8 Debugging Workflow

31.8.1 Issue: Descriptor Not Fetched

Check in order: 1. ✓ APB write to CHx_CTRL register successful? 2. ✓ Descriptor address valid? 3. ✓ AXI descriptor master not stalled? 4. ✓ Descriptor memory accessible? 5. ✓ Arbiter granting descriptor fetch?

Debug commands:

```
pytest projects/components/dmas/stream/dv/tests/fub_tests/scheduler/ -
v -s # Verbose test
pytest projects/components/dmas/stream/dv/tests/fub_tests/scheduler/
--vcd=debug.vcd
gtkwave debug.vcd # Inspect FSM state transitions
```

31.8.2 Issue: Data Transfer Stalls

Check in order: 1. ✓ SRAM buffer depth sufficient? 2. ✓ Source/destination addresses aligned? 3. ✓ AXI read/write engines getting grants? 4. ✓ Downstream AXI ready signals asserted? 5. ✓ Channel not in error state?

Waveform Analysis: - Check SRAM read/write pointers - Verify AXI AR/AW/R/W/B handshakes - Inspect scheduler FSM state - Check arbiter grant signals

31.8.3 Issue: Chained Descriptors Not Following

Check in order: 1. ✓ `next_descriptor_ptr != 0`? 2. ✓ last flag not set prematurely? 3. ✓ Descriptor fetch completing successfully? 4. ✓ Scheduler transitioning to CHAIN_CHECK state? 5. ✓ MonBus showing CHAIN_FETCH event?

31.9 Testing Guidance

31.9.1 Test Organization

```
projects/components/dmas/stream/dv/tests/
├── fub_tests/                                # Individual block tests
│   ├── descriptor_engine/                    # Adapt from RAPIDS tests
│   ├── scheduler/                           # Simplified scheduler tests
│   ├── axi_engines/                         # Read/write engine tests
│   └── sram/                                # SRAM tests (from RAPIDS)
├── integration_tests/                       # Multi-block scenarios
│   ├── single_channel/                     # Single channel transfers
│   ├── multi_channel/                     # 8-channel concurrent
│   ├── chained_descriptors/               # Descriptor chain tests
│   └── error_handling/                     # Error recovery tests
```

31.9.2 Test Levels

Basic (Quick Smoke Tests): - Single descriptor, single channel - Aligned addresses, simple transfers - ~30 seconds runtime

Medium (Typical Scenarios): - Multiple descriptors, 2-4 channels - Chained descriptors (2-3 deep) - ~90 seconds runtime

Full (Comprehensive Validation): - All 8 channels concurrent - Long descriptor chains (10+ deep) - Error injection, stress testing - ~180 seconds runtime

31.10 Key Documentation Links

31.10.1 Always Reference These

This Subsystem: - `projects/components/dmas/stream/PRD.md` - **Complete specification** - `projects/components/dmas/stream/README.md` - Quick start guide (to be created) - `projects/components/dmas/stream/known_issues/` - Bug tracking

Related: - projects/components/dmas/rapids/PRD.md - Parent architecture (for comparison)
- rtl/amba/PRD.md - MonBus integration - /PRD.md - Master requirements - /CLAUDE.md -
Repository guide

31.11 Quick Commands

View complete specification

```
cat projects/components/dmas/stream/PRD.md
```

Check package definition

```
cat projects/components/dmas/stream/rtl/includes/stream_pkg.sv
```

Run tests (once created)

```
pytest projects/components/dmas/stream/dv/tests/fub_tests/ -v  
pytest projects/components/dmas/stream/dv/tests/integration_tests/ -v
```

Lint (once RTL created)

```
verilator --lint-only  
projects/components/dmas/stream/rtl/stream_fub/scheduler.sv
```

Search for modules

```
find projects/components/dmas/stream/rtl/ -name "*.sv" -exec grep -H  
"^module" {} \;
```

31.12 Remember

1. 📖 **Tutorial focus** - Intentional simplifications for learning
 2. ↺ **Reuse from RAPIDS** - Descriptor engine, SRAM, patterns
 3. 📏 **Length in beats** - Not bytes or chunks!
 4. 🔗 **Aligned addresses** - No fixup logic
 5. 🔗 **Chained descriptors** - No circular buffers
 6. 🎯 **8 channels** - Shared resources, arbitration required
 7. 🔍 **MonBus standard** - Same format as AMBA/RAPIDS
 8. 🏗️ **Testbench reuse** - Always create TB classes in bin/TBClasses/stream/
-

31.13 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

/mnt/data/github/rtldesignsherpa/projects/components/dmas/stream/docs/

Quick Command: Use the provided shell script:

```
cd
/mnt/data/github/rtldesignsherpa/projects/components/dmas/stream/docs
./generate_pdf.sh
```

The shell script will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the stream_spec index file 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

 **See:** bin/md_to_docx.py for complete implementation details

Version: 1.0 **Last Updated:** 2025-10-17 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

32 STREAM Register Definitions

Purpose: PeakRDL register files and generated RTL for STREAM configuration

32.1 Directory Structure

```
regs/
├── README.md                # This file
├── stream_regs.rdl          # Register definition (to be created)
├── generated/               # PeakRDL-generated outputs (to be
created)
│   ├── rtl/
│   │   ├── stream_regs.sv
│   │   └── stream_regs_pkg.sv
│   └── docs/
│       └── stream_regs.html
```

32.2 Register Generation Workflow

32.2.1 Phase 1: Define Register Map (Future)

Create `stream_regs.rdl` following the same pattern as `apb_hpet`.

32.2.2 Phase 2: PeakRDL-Generated Registers (Future)

Following the same pattern as `projects/components/apb_hpet/`:

1. Define Register Map (`stream_regs.rdl`)

```
addrmap stream_regs {  
    name = "STREAM Configuration Registers";  
  
    // Global control  
    reg {  
        field { sw=rw; hw=r; } enable;  
        field { sw=rw; hw=r; } reset;  
    } GLOBAL_CTRL @ 0x00;  
  
    // Channel registers (8 channels × 16 bytes)  
    regfile channel_regs {  
        reg { ... } CH_CTRL;  
        reg { ... } CH_STATUS;  
        reg { ... } CH_RD_BURST;  
        reg { ... } CH_WR_BURST;  
    };  
  
    channel_regs CH[8] @ 0x10 += 0x10;  
};
```

2. Generate RTL

```
cd projects/components/dmas/stream/regs  
../../bin/peakrdl_generate.py stream_regs.rdl --copy-rtl  
../rtl/stream_macro
```

3. Create APB Config Wrapper

- Instantiate generated register block (similar to `apb_hpet.sv`)
 - Add `apb_slave_cdc` wrapper if clock domain crossing needed
 - Wire register outputs to STREAM control signals
-

32.3 Register Map (Planned)

32.3.1 Global Registers

Address	Name	Access	Description
0x00	GLOBAL_CTRL	RW	Global enable, reset
0x04	GLOBAL_STAT US	RO	Global status, channel idle/error
0x08	GLOBAL_CONF IG	RW	Global configuration
0x0C	Reserved	-	-

32.3.2 Channel Registers (8 × 0x10 bytes)

Each channel (0-7) has a 16-byte register block starting at $0x10 + (\text{channel_id} \times 0x10)$:

Offset	Name	Access	Description
+0x00	CHx_CTRL	WO	Descriptor address (write to kick off)
+0x04	CHx_STATUS	RO	Channel status, idle, error
+0x08	CHx_RD_BURS T	RW	Read burst length (for engine config)
+0x0C	CHx_WR_BURS T	RW	Write burst length (for engine config)

Example: - Channel 0 registers: 0x10, 0x14, 0x18, 0x1C - Channel 1 registers: 0x20, 0x24, 0x28, 0x2C - Channel 7 registers: 0x80, 0x84, 0x88, 0x8C

32.4 Integration Pattern

32.4.1 PeakRDL-Generated Implementation (Future):

```
// rtl/stream_macro/apb_config.sv
module apb_config (
    // APB interface
    input logic [ADDR_WIDTH-1:0] paddr,
    // ...
);
    // Instantiate PeakRDL-generated registers
    stream_regs u_regs (
        .pclk      (pclk),
        .presetn   (presetn),
        .paddr     (paddr),
        .psel      (psel),
        // ... APB signals

        // Register field outputs
        .global_ctrl_enable (ch_enable_internal),
        .ch0_ctrl_desc_addr (ch_desc_addr[0]),
        .ch0_rd_burst       (ch_read_burst_len[0]),
        // ... generated field outputs
    );

    // Optional: Add CDC wrapper if crossing clock domains
    // (like HPET's apb_slave_cdc)
endmodule
```

32.5 Reference Examples

HPET PeakRDL Implementation: -

projects/components/apb_hpet/peakrdl/hpet_regs.rdl - Register definition -

projects/components/apb_hpet/peakrdl/generated/ - Generated outputs -

projects/components/apb_hpet/rtl/apb_hpet.sv - Wrapper with CDC

HPET Generation Command:

```
../../../../bin/peakrdl_generate.py hpet_regs.rdl --copy-rtl ..
```

32.6 Status

-  **Phase 1:** Manual apb_config.sv (placeholder)

- 🕒 **Phase 2:** Create `stream_regs.rdl` (deferred to after Phase 2 RTL implementation)
- 🕒 **Phase 3:** Generate register RTL with PeakRDL
- 🕒 **Phase 4:** Update `apb_config.sv` wrapper to use generated registers

Note: Per user feedback, “The apb config will be the last thing done. They will have configs done along the lines of how they are done for the hpet.”

Last Updated: 2025-10-17 **Related Documentation:** - `../docs/ARCHITECTURAL_NOTES.md`
Section 6 - APB Configuration (Deferred) - `../PRD.md` Section 3.1 - APB Configuration Interface -
`../..../apb_hpet/peakrdl/README.md` - HPET PeakRDL example